
PYTHON · INTERVIEW PREPARATION

LeetCode Patterns

The Recitation Method

A pattern book for engineers who have no time to grind. 93 memorisable templates, 1151 high-frequency problems reduced to their smallest kernels.

93 patterns · **1151** interview problems · **Python**

380 Easy · **641** Medium · **130** Hard

Frequency data drawn from **70** company interview sets

About This Book

LeetCode Patterns: The Recitation Method is organised around *reduction*, not around data structures. Every chapter answers one question: *what is the smallest thing you must remember, such that a whole family of problems becomes a mechanical variation of it?*

The organising principle is deliberately not the textbook taxonomy. A chapter called “BFS” tells you nothing you can memorise. A chapter called “Binary Search — The Boundary Rules” tells you that there are exactly two loop shapes and two midpoint formulas, that `hi = mid` implies the lower midpoint and `lo = mid` implies the upper one, and that violating this pairing is an infinite loop rather than a wrong answer. That is memorisable. That is what this book contains.

Where the problem set comes from

The index is built from 1151 problems that appear in at least one of 70 company interview sets, weighted so that tier-one companies (Google, Amazon, Meta, Microsoft, Apple, Netflix, Bloomberg, ByteDance, Tencent, Alibaba, Baidu) count triple. Difficulty, acceptance rate and topic tags come from LeetCode’s own problem catalogue. The frequency score printed beside each problem is a rough proxy for how often it is asked — not a probability.

How the numbers were derived

Problem identifiers are LeetCode’s *frontend* numbers, the ones that appear in the URL. This distinction matters more than it sounds: LeetCode’s public problem API returns an internal database id in its `question_id` field which diverges from the frontend number (the internal id of problem 704 is 792). Every identifier in this book was resolved through the frontend field and cross-checked against the official title and difficulty, and the build fails loudly if a reference does not resolve.

A note on premium problems

Problems marked with a dagger (†) are LeetCode Premium and are not freely accessible. They are included because they still appear in real interviews and the pattern is worth knowing, but you should not treat them as required practice.

This edition contains all 93 chapters across 17 parts, followed by the complete problem index in Appendix A.

Contents

PART I • THE METHOD

1	How to Use This Book	1
2	The Pattern Recognition Decision Tree	6
3	The Python Toolkit and Complexity Sheet	7
4	The Master Template Sheet	8

PART II • SEQUENTIAL SCANS

5	Two Pointers – Converging	10
6	Two Pointers – Same Direction	36
7	Sliding Window – Fixed Size	38
8	Sliding Window – Variable Size	40
9	Sliding Window – At Most K Reduces to Exactly K	43

PART III • BINARY SEARCH

10	Binary Search – The Boundary Rules	46
11	Binary Search – Lower and Upper Bound	77
12	Binary Search on Rotated, Peak, and Bitonic Arrays	80
13	Binary Search on the Answer	83
14	Binary Search Plus Counting	86

PART IV • PREFIX SUMS & INTERVALS

15	Prefix Sum and the Subarray-Sum Hash	89
16	2D Prefix Sum	93
17	Difference Array	95
18	Intervals – Merge, Insert, Intersect	97
19	Sweep Line and Event Counting	101

PART V • HASHING

20	Hash Map – Complement Lookup	104
21	Hash Map – Frequency and Anagrams	107
22	Hash Map – Canonical-Key Grouping	110

23	Hash Set - Dedup, Consecutive Runs, and Cycles	112
24	Hash-Based Design (LRU, LFU, O(1) Random)	115

PART VI • STACKS & QUEUES

25	Stack - Matching and Parsing	120
26	Monotonic Stack - Next Greater / Smaller	123
27	Monotonic Stack - The Histogram Family	127
28	Monotonic Queue - Sliding Window Extremes	129
29	Stack - Simulation, Paths, and Collapse	131

PART VII • HEAPS & PRIORITY QUEUES

30	Heap - Top-K	135
31	Two Heaps - Running Median and Balancing	138
32	Heap - K-Way Merge and Scheduling	140
33	Bucket and Counting Sort as a Heap Replacement	143

PART VIII • LINKED LISTS

34	Linked List - The Reversal Family	147
35	Linked List - Fast and Slow Pointers	150
36	Linked List - Merge, Sort, and Reorder	152
37	Linked List - Dummy Head, Deletion, and Design	155

PART IX • TREES

38	Tree Traversal - The Recursive Skeleton	158
39	Tree Traversal - Iterative and Morris	162
40	Tree BFS - The Level Order Family	165
41	Tree DFS - Top-Down vs Bottom-Up	167
42	BST - Exploiting the Ordering	170
43	Tree - Path Problems	173
44	Tree - LCA, Structure, and Serialization	176

PART X • BACKTRACKING

45	Backtracking - The Universal Template	180
46	Backtracking - Subsets and Combinations	183
47	Backtracking - Permutations and the Dedup Rule	186

48	Backtracking – Target Sum and Partitioning	189
49	Backtracking – Grid Search	192
50	Backtracking – Constraint Satisfaction	195

PART XI • GRAPHS & GRIDS

51	Grid Flood Fill – The Islands Family	199
52	Graph BFS – Unweighted Shortest Path	202
53	Multi-Source BFS and 0-1 BFS	206
54	Topological Sort	208
55	Union-Find (Disjoint Set Union)	211
56	Weighted Shortest Path – Dijkstra, Bellman-Ford, Floyd	215
57	Bipartite Graphs and Colouring	218
58	Matrix Manipulation – Rotate, Spiral, and In-Place Marks	220

PART XII • DYNAMIC PROGRAMMING

59	DP – How to Think About It	225
60	DP – 1D Linear Recurrence	227
61	DP – Kadane and the Maximum Subarray Family	231
62	DP – Longest Increasing Subsequence and Friends	234
63	DP – Two-Sequence DP (LCS and Edit Distance)	237
64	DP – 0/1 Knapsack	240
65	DP – Unbounded Knapsack and Coin Change	243
66	DP – Subset Sum and Partitioning	246
67	DP – Grid and Path DP	249
68	DP – Interval DP	252
69	DP – State Machine (The Stock Family)	255
70	DP – Tree DP	258
71	DP – Palindrome DP	261
72	DP – Game Theory and Minimax	264
73	DP – Bitmask DP	267
74	DP – Digit DP	270

PART XIII • GREEDY

75	Greedy – Sort First	274
76	Greedy – Reachability and Exchange Arguments	279
77	Greedy – Heap-Assisted Choice	284

PART XIV • BITS, MATHS & STRINGS

78	Bit Manipulation	288
79	Bitmask Enumeration	292
80	Number Theory Essentials	295
81	Math Simulation and Big Numbers	301
82	String – Palindromes, Pointers, and Parsing	306
83	String – KMP and the Z-Algorithm	313
84	String – Rolling Hash and Substring Search	316

PART XV • THE HARD CORE

85	Divide and Conquer	320
86	Quickselect and Partitioning	323
87	Randomized Algorithms and Reservoir Sampling	326
88	Hard Design Problems – The Composite Round	329

PART XVI • SPECIALIST PATTERNS

89	Trie (Prefix Tree)	336
90	Eulerian Paths, Bridges, and Advanced Graph Construction	340
91	Computational Geometry Basics	343
92	Concurrency (Where It Still Appears)	346

PART XVII • APPENDICES

93	Appendix A – The Complete High-Frequency Index	349
-----------	--	-----

PART I

The Method

Four short chapters that install the method: how to read this book, how to choose a pattern from the problem statement, the Python toolkit you should not have to re-derive, and the twenty templates that cover most of the thousand problems that follow.

How to Use This Book

THE KERNEL

Read the kernel, type the template from memory, then only the delta matters. Never read a solution before you can write the template blind.

This book is written for a specific reader: someone who already writes code professionally, who has an interview in three to eight weeks, and who does not have two hundred hours to spend on a problem set. It assumes you can read Python and reason about complexity. It does not assume you have solved three hundred problems, and it does not spend pages on what an array is.

The organising bet is this: **interview problems are not a thousand problems, they are about ninety patterns, and most of those patterns are three to ten lines of code plus a rule about boundaries.** If you learn the rules, you can re-derive the code. If you only learn the code, you will be lost the moment the problem is dressed differently.

So every chapter in this book has the same five parts, in the same order, and each one has a job.

The Method

- 1 Pass 1 (recall): read only the chapter's Kernel and Boundary Rules, then close the book and type the template from memory.
- 2 Pass 2 (recognition): read the Reduction Table. For each row, say out loud which single line of the template changes. If you cannot, you do not yet own the pattern.
- 3 Pass 3 (drill): work the Index problems under a 25-minute timer each, writing only the changed lines, not the whole solution.

- 4 Never memorise solutions. Memorise the kernel plus the boundary rule. Every worked example in this book is a re-derivation, not a recitation.

The five parts of a chapter

The Kernel is one sentence stating what every problem in the chapter collapses to. If you read nothing else, read this. A chapter whose kernel you cannot restate is a chapter you have not learned.

The Boundary Rules are the memorisable content. They are numbered because you should be able to recite them: *rule three says `hi = mid` implies the lower midpoint*. If you find yourself reasoning from first principles during an interview, you are spending attention you need for the problem itself.

The Template is the canonical code, written once, in the form you should be able to type from memory. It is deliberately not golfed — a template you cannot reproduce under pressure is worthless.

Worked Solutions show the template being applied, with the reasoning made explicit. Read these as derivations, not as answers to memorise.

The Index lists every remaining problem in the chapter's family with a single sentence naming what changes in the template. This is the part that makes the book cover a thousand problems without becoming a thousand pages.

The three-pass method

Do not read this book front to back like a novel. Do not do the problems in chapter order either. Each chapter gets three passes, and the passes are deliberately separated in time.

- **Pass 1 — recall.** Read only the Kernel and the Boundary Rules. Close the book. Type the template from memory. If you cannot, re-read and try again tomorrow; there is no point moving on.
- **Pass 2 — recognition.** Read the Index table. For each row, say out loud which line of the template changes. You are not solving anything; you are training the recognition step, which is what actually fails in interviews.

- **Pass 3 — drill.** Work the Drill Set with a timer. Write only the changed lines, not the whole solution, unless the problem is genuinely new. Twenty-five minutes per problem is the budget; past that, read the chapter again.

What to do when you are stuck

Being stuck has three distinct causes, and they have three distinct fixes. Diagnosing which one you have is most of the work.

- **You do not recognise the pattern.** You read the problem and have no idea where to start. Fix: go back to chapter 2's decision tree and work the four questions explicitly, out loud. Recognition is a skill and it is trained by Pass 2, not by solving more problems.
- **You recognise the pattern but cannot write the code.** Fix: you learned the code instead of the rules. Re-read the Boundary Rules and re-derive the template from them.
- **You can write the code but it fails on edge cases.** Fix: this is a boundary problem, and it is the most common failure of all. Before submitting, test the empty input, the single element, the two elements, and the all-equal input. In the binary search part that list is mandatory; in the rest of the book it is a habit worth building.

On memorising

There is a taboo against memorising in interview preparation, and it is misplaced. What is worth memorising is not solutions — it is **rules**. The difference matters: a memorised solution is worthless the moment the problem changes, whereas a memorised rule (*if the branch moves `lo` to `mid`, the midpoint must round up*) applies to every problem in the family and can be checked in two seconds.

The test of whether you have memorised a rule rather than a solution is whether you can state the *failure mode* it prevents. If you can say 'this rule exists because otherwise the loop does not terminate on a two-element range', you own it. If you can only say 'this is what the template looks like', you have memorised the wrong thing.

THE PRINCIPLE

Memorise rules, derive code. A rule you can state with its failure mode is worth ten solutions you can only reproduce.

How the problem index was built

Every problem in this book appears in at least one of seventy company interview sets. The frequency score printed beside each problem weights tier-one companies (Google, Amazon, Meta, Microsoft, Apple, Netflix, Bloomberg, ByteDance, Tencent, Alibaba, Baidu) triple, so a problem asked by four tier-one companies outranks one asked by six smaller ones. Difficulty, acceptance rate and topic tags come from LeetCode's own catalogue.

A dagger (†) marks LeetCode Premium problems. They are included because they genuinely appear in interviews, but you should not treat them as required practice.

The index table that ends every chapter has five columns. **#** is the LeetCode problem number, so you can find the problem instantly. **Problem** is the title as LeetCode spells it. **Level** is Easy, Medium or Hard. **Asked** is how many of the seventy company sets contain that problem — a rough proxy for how often it actually comes up, and the reason the rows appear in the order they do. **What changes** is the column that matters: it is the one or two edits that turn the chapter's template into a solution for that specific problem. Read the index with a finger over that last column and try to predict the edit before you read it. That prediction, not the reading, is the practice.

Appendix A lists all of them in frequency order. Use it as a checklist, not a reading list.

Pitfalls

- 1 **Reading the book cover to cover.** It is a reference with a reading order, not a narrative. If you read all ninety-three chapters without doing Pass 1 on any of them, you will finish with a warm feeling and no new capability.
- 2 **Doing Pass 3 first.** Solving problems before you can state the rules teaches you the problems, not the pattern. It feels productive and it is the most common way to spend two hundred hours and still fail the interview.
- 3 **Skipping the Drill Set because you ‘already know’ a pattern.** The patterns you already know are exactly the ones whose boundary cases you have quietly wrong.
- 4 **Treating the index tables as filler.** They are the compression mechanism of the whole book: one sentence per problem is what lets a thousand problems fit in a book you can actually finish.

The Pattern Recognition Decision Tree

THE KERNEL

Interview pattern choice is a 4-question funnel: what is the input shape, what is being asked, is there an ordering or monotonicity to exploit, and what is the constraint that forces the complexity class.

The Method

- 1 Q1 — Input shape: array / string / linked list / tree / graph / matrix / interval set / nothing-but-a-number.
- 2 Q2 — Ask shape: find one, count all, find the best, enumerate all, decide yes-no, or design a data structure.
- 3 Q3 — Monotonicity: is the input sorted, or can a predicate be made monotone over a candidate answer? If yes, binary search is on the table.
- 4 Q4 — Constraint: read the limits before you design. $n \leq 20$ means bitmask or exponential. $n \leq 500$ means $O(n^2)$ DP or interval DP. $n \leq 10^4$ means $O(n \log n)$. $n \leq 10^6$ means $O(n)$ or $O(n \log n)$ with a small constant. $n \leq 10^9$ means $O(\log n)$ or $O(\sqrt{n})$ — almost certainly binary search or maths.

The Python Toolkit and Complexity Sheet

THE KERNEL

Python gives you a standard library that already implements half the patterns. Know which lines you never have to write again.

The Method

- 1 Sorted insert / search: `bisect_left`, `bisect_right` — never hand-roll binary search when the array is already sorted.
- 2 Top-K: `heapq.nlargest(k, a)` / `heapq.nsmallest(k, a)` for $k \ll n$; `sorted(a)[-k:]` for k close to n .
- 3 Frequency: `collections.Counter`; ordered counts: `defaultdict(int)`.
- 4 Sliding-window maxima: `collections.deque` with `maxlen` where applicable.
- 5 Cache: `functools.lru_cache` for memoised DP; `functools.cache` when the key space is small and unhashable args are not used.
- 6 Iteration tools: `itertools.accumulate` for prefix sums, `itertools.combinations` / `permutations` / `product` for brute-force verification of your DP.
- 7 Never mutate a list while iterating it; never use `list.pop(0)` in a loop (it is $O(n)$) — use a deque.
- 8 Default arguments are evaluated once: never write `def f(path=[])` in a backtracking signature.

The Master Template Sheet

THE KERNEL

Twenty code shapes cover roughly nine hundred of the thousand problems in this book. This chapter is the whole book compressed.

The Method

- 1 Each template is printed on one page with its boundary rule, its complexity, and the problems that reduce to it.
- 2 Photocopy this chapter. It is the last thing you read before an interview.

PART II

Sequential Scans

One-dimensional scans. Pointers and windows, and the boundary conditions that decide whether they are correct.

Two Pointers - Converging

THE KERNEL

One pointer at each end, both moving inward. Every problem here is the same `while l < r:` loop; only the move rule and the update rule differ.

This is the cheapest pattern in the book. There is one loop, two pointers, and a rule that says which one moves. Once the array is sorted, a pair is no longer something you search for — it is something you walk toward, and the walk is $O(n)$ instead of $O(n^2)$.

The reason it works is a monotonicity argument, and you should be able to state it out loud in an interview. Put the smallest element on the left and the largest on the right. If their sum is too small, then the smallest element cannot be part of any valid pair — it is too small to pair with anything, including the largest value available. So discard it and move on. If the sum is too large, by the same argument the largest element cannot be in any valid pair. Every iteration removes one candidate permanently, which is what makes a single linear pass enough.

Everything else — 3Sum, 4Sum, closest pair, container with most water, palindrome checking — is that argument wearing a different costume. The chapter is short because there is genuinely only one idea in it.

32PROBLEMS
MAPPED**14**

EASY

17

MEDIUM

1

HARD

The Boundary Rules

Seven rules. Rule 1 and rule 2 are the loop itself; rule 3 is the precondition that makes the loop valid; rules 4 through 7 are the traps that separate a working answer from an accepted one.

- 1 Loop condition is always `while l < r` (never `<=`) — a converging pair must have two distinct indices.
- 2 Move exactly one pointer per iteration, decided by a comparison against the target. Moving both is the classic bug that silently skips candidates.
- 3 If the array is not sorted and the problem is about pairs/triplets, sort first — the $O(n \log n)$ sort is what buys you the $O(n)$ scan.
- 4 After sorting, the index you get back is an index into the *sorted* array. If the problem asks for original indices, you must carry them.
- 5 Dedup rule: skip a value only when it equals the previous *kept* value, and only while `l < r`. `if l > start and a[l] == a[l-1]: continue`.
- 6 3Sum is `for i in range(n-2)` wrapping a converging pair; the outer loop carries its own dedup and its own early exit `if a[i] > 0: break`.
- 7 Container-with-most-water never needs sorting: the greedy move is `move the shorter side`, because the shorter side is the bottleneck.

The Template

There is one frame and one shell. The frame is the converging loop. The shell is the deduplication wrapper you need whenever the problem asks for a *set* of tuples rather than a single pair, which is where most submissions fail.

Memorise the frame. The shell you can rebuild each time from rule 5, as long as you remember that duplicates are skipped at the anchor and at the pointer separately — they are two different loops and they are both required.

```
# ----- the frame
# Sorted array, one pointer at each end, both moving inward.
```

```

def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return [l, r]
        if s < target:
            l += 1
        else:
            r -= 1
    return []

# ----- the dedup shell
# Use whenever the answer is a SET of tuples, not one pair.

def three_sum(a):
    a.sort()
    out = []
    for i in range(len(a) - 2):
        if a[i] > 0:
            break
        if i > 0 and a[i] == a[i - 1]:
            continue
        l, r = i + 1, len(a) - 1
        while l < r:
            s = a[i] + a[l] + a[r]
            if s == 0:
                out.append([a[i], a[l], a[r]])
                l += 1
                while l < r and a[l] == a[l - 1]:
                    l += 1
            elif s < 0:
                l += 1
            else:
                r -= 1
    return out

```

Why moving both pointers at once is wrong

It is tempting to think that when the sum is too small you should advance `l` and retreat `r`, since neither endpoint looked right. That is a bug, and it is a silent one: it skips candidates without ever failing a test you would notice on a small example.

The monotonicity argument is what tells you the truth. When `a[l] + a[r]` is too small, `a[l]` is too small to pair with `a[r]` — the largest remaining value. That tells you nothing about `a[r]`. `a[r]` may well be the correct partner for some *other* element further in. Moving `r` discards it on no evidence.

So the rule is mechanical: exactly one pointer moves per iteration, and the comparison decides which. If you find yourself writing both, you have lost the invariant and the linear bound with it.

The two deduplication loops are not the same loop

In the `three_sum` shell there are two places duplicates are skipped, and they exist for different reasons. The outer check `if i > 0 and a[i] == a[i - 1]: continue` skips a repeated **anchor** — if you have already produced every triple starting with the value 2, starting again at another 2 produces the same set.

The inner loop after a hit skips repeated **partners** — once you have emitted `[-4, 2, 2]`, advancing `l` onto a second `2` would emit it again.

The trap is that the inner loop must be guarded by `l < r`, not by a fixed bound. Without it, a long run of equal values can walk `l` past `r` and produce a pair that reuses the same element twice, which is not a pair at all.

Worked Solutions

Eight problems. Each one is the frame with one decision changed, and the decision is named in the heading so you can find it again under pressure.

167 Two Sum II - Input Array Is Sorted

LC 167 · Medium · asked by 6 companies · AC 65.9% · Array, Two Pointers, Binary Search

THE PROBLEM

You are given a **1-indexed** array of integers `numbers` that is already **sorted in non-decreasing order**.

Find **two** numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index}_1 < \text{index}_2 \leq \text{numbers.length}$.

Return the indices of the two numbers `index1` and `index2` as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: `numbers = [2,7,11,15]`, `target = 9`
Output: `[1,2]`

Explanation The sum of 2 and 7 is 9. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Example 2:

Input: `numbers = [2,3,4]`, `target = 6`
Output: `[1,3]`

Explanation The sum of 2 and 4 is 6. Therefore `index1 = 1`, `index2 = 3`. We return `[1, 3]`.

Example 3:

Input: `numbers = [-1,0]`, `target = -1`
Output: `[1,2]`

Explanation The sum of -1 and 0 is -1. Therefore `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- `numbers` is sorted in **non-decreasing order**.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is **exactly one solution**.

SIGNATURE

```
def twoSum(numbers, target): # -> integer[]
```

IN ONE LINE In a sorted array, return the 1-indexed positions of the two numbers that add to `target`.

THE INSIGHT The frame in its purest form. The array is already sorted, so the only work is the comparison.

`l` starts at the smallest value and `r` at the largest. If the sum is too small, the smallest value cannot participate in any valid pair, so `l` advances. If too large, the largest value cannot, so `r` retreats. Each step discards one index for good.

The only thing the problem adds over the template is the off-by-one in the return: the statement is 1-indexed while Python is 0-indexed. That single `+ 1` on each end is the most commonly lost point on this problem.

```
def twoSum(numbers, target):
    l, r = 0, len(numbers) - 1
    while l < r:
        s = numbers[l] + numbers[r]
        if s == target:
            return [l + 1, r + 1] # the statement is 1-indexed
        if s < target:
            l += 1
        else:
            r -= 1
    return []
```

Complexity. $O(n)$ time, $O(1)$ space. Compare the hash-map solution for `Two Sum` (1): $O(n)$ time but $O(n)$ space. Sortedness is what lets you drop the memory.

PITFALL

Returning `[l, r]` instead of `[l + 1, r + 1]`.

PITFALL

Writing `while l <= r`. With `<=` the loop can settle on a single index and the function reports a number paired with itself.

Follow-up. “What if the array were not sorted?” Then either sort first – $O(n \log n)$ – or use the hash map from chapter 20. The converging frame requires order.

125 Valid Palindrome

LC 125 · Easy · asked by 6 companies · AC 54.3% · Two Pointers, String

THE PROBLEM

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a **palindrome**, or `false` otherwise.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
```

Explanation “amanaplanacanalpanama” is a palindrome.

Example 2:

```
Input: s = "race a car"
Output: false
```

Explanation “raceacar” is not a palindrome.

Example 3:

```
Input: s = " "
Output: true
```

Explanation `s` is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- `1 <= s.length <= 2 * 105`
- `s` consists only of printable ASCII characters.

SIGNATURE

```
def isPalindrome(s): # -> boolean
```


IN ONE LINE Decide whether a string reads the same forwards and backwards, ignoring case and non-alphanumeric characters.

THE INSIGHT Same frame, but the pointers must skip characters that do not count, and the skip loops need the same `l < r` guard as the outer loop.

The frame compares `s[l]` with `s[r]` and moves both inward — note that this is one of the few places both pointers move, because a palindrome match consumes both ends at once.

The new ingredient is the inner skip loops. A character that is not alphanumeric is invisible to the problem, so each pointer walks over it. The subtlety is that these loops must be guarded by `l < r` themselves. Without the guard, a string like `".,"` walks `l` off the end of the string and the next comparison reads out of range.

Normalise with `.lower()` at the point of comparison rather than rebuilding the string. Filtering into a new string is also correct but allocates $O(n)$ memory for no reason.

```
def isPalindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1
        r -= 1
    return True
```

Complexity. $O(n)$ time, $O(1)$ space. Each character is visited at most once by each pointer.

PITFALL

Omitting `l < r` inside the skip loops. This is the single most common failure on this problem.

PITFALL

Using `s[l].isalpha()` when the statement says alphanumeric — digits count, so `"0P"` is not a palindrome but `"0P0"` is.

Follow-up. "What if you may delete at most one character?" That is problem 680 below, and it is a one-line change with a trap.

15 3Sum

LC 15 · **Medium** · asked by 19 companies · AC 39.9% · Array, Two Pointers, Sorting

THE PROBLEM

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`
Output: `[[-1,-1,2], [-1,0,1]]`

Explanation `nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`. `nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`. `nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`. The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`. Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: `nums = [0,1,1]`
Output: `[]`

Explanation The only possible triplet does not sum up to 0.

Example 3:

Input: `nums = [0,0,0]`
Output: `[[0,0,0]]`

Explanation The only possible triplet sums up to 0.

Constraints:

- `3 <= nums.length <= 3000`

- `-105 <= nums[i] <= 105`

SIGNATURE

```
def threeSum(nums): # -> list<list<integer>>
```

IN ONE LINE Return every triple of distinct positions whose values sum to zero, with no repeated triples.

THE INSIGHT Sort, then wrap the converging frame in a loop over the anchor. The whole difficulty is the two deduplication loops, not the search.

Sorting does two things at once: it makes the inner converging scan valid, and it makes duplicates adjacent, which is what makes them skippable at all. Without the sort you cannot dedup in $O(1)$ space per candidate.

The outer loop fixes `nums[i]` as the anchor and asks the inner frame for a pair summing to `-nums[i]`. Two early exits matter. `if nums[i] > 0: break` is valid because once the anchor is positive, every later anchor is larger and no triple can reach zero. The duplicate-anchor check `if i > 0 and nums[i] == nums[i - 1]: continue` is what stops the same triple being emitted from different starting positions.

After a hit, `l` must skip past equal values. This is the second dedup loop, and it is guarded by `l < r` so it cannot overrun.

```
def threeSum(nums):
    nums.sort()
    out = []
    for i in range(len(nums) - 2):
        if nums[i] > 0:
            break # sorted: no later triple reaches 0
        if i > 0 and nums[i] == nums[i - 1]:
            continue # duplicate anchor
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s == 0:
                out.append([nums[i], nums[l], nums[r]])
                l += 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1
```

```

        l += 1          # duplicate partner
    elif s < 0:
        l += 1
    else:
        r -= 1
return out

```

Complexity. $O(n^2)$ time – $O(n \log n)$ to sort plus $O(n)$ for each of n anchors. $O(1)$ extra space beyond the output.

PITFALL

Keeping only one of the two dedup checks. With only the anchor check, a triple like `[-1, 0, 1]` is emitted once per duplicate partner; with only the partner check, it is emitted once per duplicate anchor.

PITFALL

Using a `set` of tuples instead of the two loops. It works, but it hides whether you understand the invariant, and interviewers ask.

PITFALL

Dropping the `l < r` guard inside the partner skip loop, which lets `l` run past `r` and reuse an element twice.

Follow-up. “And 4Sum?” Add a second anchor loop around the same shell – $O(n^3)$. The pattern does not change; only the nesting does.

11 Container With Most Water

LC 11 · **Medium** · asked by 9 companies · AC 60.9% · Array, Two Pointers, Greedy

THE PROBLEM

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:

[diagram omitted — see leetcode.com/problems/container-with-most-water/]

Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49

Explanation The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]
Output: 1

Constraints:

- `n == height.length`
- `2 <= n <= 105`
- `0 <= height[i] <= 104`

SIGNATURE

```
def maxArea(height): # -> integer
```

IN ONE LINE Given vertical lines at each index, choose two that with the x-axis hold the most water.

THE INSIGHT The only problem in the chapter that must **not** be sorted, because the answer depends on the original positions. The move rule is different too: move the shorter side.

The area is `min(height[l], height[r]) * (r - l)`. Sorting would destroy the widths, so the array stays as it is and the frame runs on the original indices.

The move rule follows from the bottleneck argument. The shorter wall caps the area. Moving the taller wall inward can only reduce the width while the cap stays the same, so it can never improve. Moving the shorter wall is the only move that has any chance of finding a larger cap. This is the same “discard the endpoint that

cannot participate” argument as the sorted case, applied to a different monotonicity.

Note what you do **not** do: you never look ahead. The greedy move is locally justified and globally optimal here, and the proof is the bottleneck argument above.

```
def maxArea(height):
    l, r = 0, len(height) - 1
    best = 0
    while l < r:
        h = min(height[l], height[r])
        best = max(best, h * (r - l))
        if height[l] < height[r]:
            l += 1          # the shorter wall is the bottleneck
        else:
            r -= 1
    return best
```

Complexity. $O(n)$ time, $O(1)$ space. One comparison and one move per iteration.

PITFALL

Sorting the array first. The area depends on index distance, so sorting changes the answer — this is the trap that catches people who have just learned 3Sum.

PITFALL

Moving the taller wall. It looks harmless and is wrong on inputs where the optimum needs both walls from one half.

PITFALL

Computing $h * (r - l)$ with $h = \text{height}[l]$ only, instead of the minimum of the two.

Follow-up. “Prove the greedy move.” This is the standard exchange argument: any solution that keeps the shorter wall and discards the taller one is dominated by the move the algorithm makes.

16 3Sum ClosestLC 16 · **Medium** · asked by 7 companies · AC 47.4% · Array, Two Pointers, Sorting**THE PROBLEM**

You are given an integer array `nums` of length `n` and an integer `target`.

Find three integers at **distinct indices** in `nums` such that the sum is **closest** to `target`.

Return the sum of the three integers.

You may assume that each input would have **exactly** one solution.

Example 1:

Input: `nums = [-1,2,1,-4]`, `target = 1`
Output: 2

Explanation The sum that is closest to the target is 2. $(-1 + 2 + 1 = 2)$.

Example 2:

Input: `nums = [0,0,0]`, `target = 1`
Output: 0

Explanation The sum that is closest to the target is 0. $(0 + 0 + 0 = 0)$.

Constraints:

- `3 <= nums.length <= 500`
- `-1000 <= nums[i] <= 1000`
- `-104 <= target <= 104`

SIGNATURE

```
def threeSumClosest(nums, target): # -> integer
```

IN ONE LINE Find the triple whose sum is closest to a target.

THE INSIGHT The 3Sum shell with the equality test replaced by a running best. No deduplication is needed, because the answer is a single number rather than a set of tuples.

This is the clearest demonstration that the two dedup loops in 3Sum are not part of the frame — they belong to the *output shape*. Here the output is one integer, so both loops vanish and the code gets shorter.

The frame is unchanged: too small advances `l`, too large retreats `r`. The only addition is that every candidate is compared against the best so far, since the exact target may never be hit.

The early return when `s == target` is worth keeping: an exact hit is optimal and no further work can improve it.

```
def threeSumClosest(nums, target):
    nums.sort()
    best = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if abs(s - target) < abs(best - target):
                best = s
            if s < target:
                l += 1
            elif s > target:
                r -= 1
            else:
                return target          # exact hit: cannot be beaten
    return best
```

Complexity. $O(n^2)$ time, $O(1)$ space. The early return makes the common case much faster than the bound suggests.

PITFALL

Seeding `best` with a sentinel like `float('inf')` and then returning it when the array is short. Seed from the first triple instead — the array always has at least three elements.

PITFALL

Forgetting the exact-hit early return, which is not a correctness bug but is the difference between passing and looking like you know the problem.

Follow-up. “Closest pair in a sorted array?” Same running-best idea with the plain frame, no anchor loop.

42 Trapping Rain Water

LC 42 · **Hard** · asked by 18 companies · AC 68.2% · Array, Two Pointers, Dynamic Programming, Stack

THE PROBLEM

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:

[diagram omitted — see leetcode.com/problems/trapping-rain-water/]

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6

Explanation The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]
Output: 9

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

SIGNATURE

```
def trap(height): # -> integer
```

IN ONE LINE Given an elevation map, compute how much rain it traps.

THE INSIGHT The hardest problem in the chapter, and it is still the same frame. The water above any bar is `min(left_max, right_max) - height`, and the converging scan can supply that minimum without ever building either array.

The straightforward solution computes two prefix arrays — the tallest bar to the left of each index and the tallest to the right — then sums `min(left, right) - height`. That is $O(n)$ time and $O(n)$ space, and it is a perfectly good answer.

The two-pointer version removes the arrays. The key observation: when `height[l] < height[r]`, the water above `l` is determined *entirely* by `left_max`, because there is a bar at `r` at least as tall as `height[l]` somewhere to the right, which is enough to guarantee `right_max >= height[l]`. So `min(left_max, right_max) == left_max` and you can settle `l` immediately without knowing `right_max` at all.

That is the whole trick, and it is worth stating precisely: you never need both maxima, only the one on the side of the shorter bar. The loop advances whichever side is shorter and settles that index permanently.

```
def trap(height):
    if not height:
        return 0
    l, r = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

Complexity. $O(n)$ time, $O(1)$ space. Strictly better than the prefix-array solution on memory, and the version an interviewer is hoping you reach.

PITFALL

Comparing `height[l]` with `height[r]` and then using the wrong maximum. The rule is: whichever bar is shorter, its side's maximum is the binding constraint. Swap them and the answer is wrong on asymmetric inputs.

PITFALL

Updating `left_max` after accumulating instead of before, which adds a negative term at every new maximum.

PITFALL

Forgetting the empty-input guard, which makes `l` and `r` invalid on an empty array.

Follow-up. “What about the 2-D version?” Problem 407. It needs a heap rather than two pointers – see chapter 32 – because there is no single pair of maxima that bounds the water.

680 Valid Palindrome II

LC 680 · **Easy** · asked by 7 companies · AC 44.8% · Two Pointers, String, Greedy

THE PROBLEM

Given a string `s`, return `true` if the `s` can be palindrome after deleting **at most one** character from it.

Example 1:

Input: `s = "aba"`
Output: `true`

Example 2:

Input: `s = "abca"`
Output: `true`

Explanation You could delete the character ‘c’.

Example 3:

Input: `s = "abc"`
Output: `false`

Constraints:

- `1 <= s.length <= 105`
- `s` consists of lowercase English letters.

SIGNATURE

```
def validPalindrome(s):    # -> boolean
```

IN ONE LINE Decide whether a string is a palindrome after deleting at most one character.

THE INSIGHT Run the frame; at the first mismatch, the answer is the OR of the two possible deletions. Only one branch can be right, and you must try both.

The frame advances while the characters match. The moment they differ, exactly one of two things must be true: the left character is the extra one, or the right character is. So you check the remaining substring both ways and accept if either works.

The trap is thinking you can decide *which* one to delete from the local comparison. You cannot. Given "abca" mismatching at the outer pair, both branches must be tested, because the surrounding context decides it and you have not looked at the surroundings yet.

Factor the plain check into a helper. The outer loop runs once and calls the helper at most twice, which is why the total work is still linear — each helper call scans a substring, and there are only two of them.

```
def validPalindrome(s):
    def ok(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return ok(l + 1, r) or ok(l, r - 1)
        l += 1
        r -= 1
    return True
```

Complexity. $O(n)$ time – the outer loop plus at most two linear scans. $O(1)$ space beyond the recursion-free helper.

PITFALL

Trying only one branch, e.g. always skipping `s[l]`. Which character is the extra one depends on the rest of the string, not on the mismatch itself.

PITFALL

Recurring instead of using a helper and accidentally making it exponential, because each mismatch can spawn two more.

PITFALL

Returning `False` after both branches fail but forgetting that the loop must also return `True` when no mismatch is found at all.

Follow-up. "At most k deletions?" That is the two-sequence DP of chapter 62 – the greedy two-branch trick works only for $k = 1$.

977 Squares of a Sorted Array

LC 977 · **Easy** · asked by 5 companies · AC 74.2% · Array, Two Pointers, Sorting

THE PROBLEM

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`

Explanation After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`
Output: `[4,9,9,49,121]`

Constraints:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`

- `nums` is sorted in **non-decreasing** order.

SIGNATURE

```
def sortedSquares(nums): # -> integer[]
```

IN ONE LINE Given a sorted array, return the squares of its elements in sorted order.

THE INSIGHT The largest square is at one of the two ends, so fill the output from the back while converging inward.

The array is sorted, but its *squares* are not: a large negative number at the left can square to more than a large positive at the right. So the usual “smallest first” order is broken.

The fix is to reverse the direction of the fill. The largest square is always at one of the two ends, because that is where the largest magnitudes live. Compare `abs(nums[l])` with `abs(nums[r])`, take the bigger square, place it at the current back position, and advance that pointer. The output fills from the last index down to zero.

Note that both pointers move inward but only one per iteration, exactly as rule 2 requires. Writing into a preallocated list from the back avoids the `insert(0, ...)` that would make the solution quadratic.

```
def sortedSquares(nums):
    n = len(nums)
    out = [0] * n
    l, r = 0, n - 1
    for k in range(n - 1, -1, -1):
        if abs(nums[l]) > abs(nums[r]):
            out[k] = nums[l] * nums[l]
            l += 1
        else:
            out[k] = nums[r] * nums[r]
            r -= 1
    return out
```

Complexity. $O(n)$ time, $O(n)$ space for the output. Beats the obvious sort-the-squares approach, which is $O(n \log n)$.

PITFALL

Using `nums[l] > nums[r]` without `abs`. The squares of negatives compare wrongly if you forget.

PITFALL

Writing to the front with `out.append(...)` and then reversing, which is correct but obscures the direction of the fill.

PITFALL

Using `insert(0, x)`, which is $O(n)$ per call and quietly turns a linear solution into a quadratic one.

Follow-up. “Why is the maximum always at an end?” Because x^2 is monotone in $|x|$, and the extremes of $|x|$ are at the ends of a sorted array.

The Index – What Changes in the Template

Every remaining problem in this chapter, sorted by how often it is asked. The last column names the single thing that changes in the frame — read that column and nothing else on a first pass.

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
349	Intersection of Two Arrays	Easy	7	Sort both, then converge on equality; advance the side that is smaller
18	4Sum	Medium	8	Two anchor loops around the ch 5 frame; the dedup rules apply at every level
344	Reverse String	Easy	6	The frame with both pointers always moving – one <code>swap</code> per iteration
844	Backspace String Compare	Easy	7	Converge from the back , skipping characters cancelled by <code>#</code>
881	Boats to Save People	Medium	7	Sort; heaviest with lightest – both board if they fit, otherwise the heavy one alone
532	K-diff Pairs in an Array	Medium	7	Sort then converge, or hash the counts; dedup on the value , not the index

TWO POINTERS - CONVERGING

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
392	Is Subsequence	Easy	3	Two pointers moving the same direction, not converging – chapter 6
1099	Two Sum Less Than K †	Easy	2	The frame with < instead of ==; track the best valid sum
611	Valid Triangle Number	Medium	3	Sort; fix the largest side as r, then count pairs with <code>a[l] + a[mid] > a[r]</code>
923	3Sum With Multiplicity	Medium	2	Same frame; count combinations instead of emitting, over <code>i < j < k</code>
345	Reverse Vowels of a String	Easy	1	The frame plus skip loops, like 125, with a swap on a match
1471	The k Strongest Values in an Array	Medium	1	Sort by distance from the median, which is <code>a[(n - 1) // 2]</code>
2149	Rearrange Array Elements by Sign	Medium	1	Fill a preallocated output from two write pointers, one per parity
259	3Sum Smaller †	Medium	0	Sorted + converging: when <code>s < target</code> , all <code>r - l</code> pairs count at once – add them and move l
905	Sort Array By Parity	Easy	0	Converging partition: swap when l is odd and r is even
948	Bag of Tokens	Medium	0	Sort; spend tokens at the cheap end, gain at the expensive end
1498	Number of Subsequences That Satisfy the Given Sum Condition	Medium	0	Sort; for each l find the largest r with <code>a[l] + a[r] <= t</code> , then add <code>2^(r-l-1)</code>
2105	Watering Plants II	Medium	0	Two pointers over the plants , one from each side – not the array ends
2330	Valid Palindrome IV †	Medium	0	Count mismatched pairs in the frame; at most two may differ
2441	Largest Positive Integer That Exists With Its Negative	Easy	0	A hash set, not two pointers – there is no ordering to exploit
2562	Find the Array Concatenation Value	Easy	0	The frame; concatenate <code>str(a[l]) + str(a[r])</code> and accumulate
2824	Count Pairs Whose Sum is Less than Target	Easy	0	<code>n <= 50</code> so brute force passes, but sorted + converging counts in one pass

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
2825	Make String a Subsequence Using Cyclic Increments	Medium	0	Two pointers with a cyclic-increment budget carried on each character
2932	Maximum Strong Pair XOR I	Easy	0	<code>n <= 50</code> : brute force. The fast version needs a binary trie – chapter 90

Pitfalls

- Sorting a problem that depends on original positions.** Rule 3 tells you to sort for pair problems. It does not apply to `Container With Most Water` (11), `Trapping Rain Water` (42) or `Boats to Save People` (881) — in the first two the answer is the distance between indices, and sorting destroys it. Ask whether the answer depends on positions before you call `.sort()`.
- Moving both pointers in one iteration.** The comparison justifies discarding exactly one endpoint. Moving both discards a candidate on no evidence and silently skips valid answers.
- Writing `while l <= r`.** A pair needs two distinct indices. With `<=`, the loop can settle on `l == r` and pair an element with itself.
- Keeping only one of the two dedup loops.** In 3Sum they are separate for a reason: one skips duplicate anchors, the other skips duplicate partners. Removing either produces repeated triples.
- The partner skip loop without its own `l < r` guard.** A long run of equal values walks `l` past `r`, and the next comparison reads the same element twice or goes out of range.
- Forgetting that sorting costs $O(n \log n)$.** The frame is $O(n)$, but the problem is not linear unless the input arrives sorted. Say the bound as $O(n \log n)$ when you sort.
- Reaching for two pointers when `n` is tiny.** `Maximum Strong Pair XOR` (2932) has `n <= 50`; brute force is the intended answer and the converging frame does not apply at all.
- Assuming the frame needs a `for` loop.** The frame is a `while l < r`. If you find yourself indexing with `for i in range(...)`, you are probably in a different pattern.

Drill Set

Thirty problems, ordered so that each adds exactly one idea to the previous. The first eight are the frame alone; 15 and 16 introduce the dedup shell; 42 is the only one that needs an argument rather than a substitution.

Do them without looking at the template. If a problem takes more than eight minutes, stop, re-read the boundary rule it violates, and restart from the frame.

Target: thirty minutes for the first twelve, forty for the rest. If you finish faster you are guessing rather than applying the rules, and the mistakes will surface under interview pressure.

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
680	Valid Palindrome II	Easy	7	
42	Trapping Rain Water	Hard	18	
15	3Sum	Medium	19	
125	Valid Palindrome	Easy	6	
11	Container With Most Water	Medium	9	
977	Squares of a Sorted Array	Easy	5	
16	3Sum Closest	Medium	7	
349	Intersection of Two Arrays	Easy	7	Sort both, then converge on equality; advance the side that is smaller
18	4Sum	Medium	8	Two anchor loops around the ch 5 frame; the dedup rules apply at every level
167	Two Sum II - Input Array Is Sorted	Medium	6	
344	Reverse String	Easy	6	The frame with both pointers always moving – one <code>swap</code> per iteration
844	Backspace String Compare	Easy	7	Converge from the back , skipping characters cancelled by <code>#</code>
881	Boats to Save People	Medium	7	Sort; heaviest with lightest – both board if they fit, otherwise the heavy one alone
532	K-diff Pairs in an Array	Medium	7	Sort then converge, or hash the counts; dedup on the value , not the index

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
392	Is Subsequence	Easy	3	Two pointers moving the same direction, not converging – chapter 6
1099	Two Sum Less Than K †	Easy	2	The frame with < instead of ==; track the best valid sum
611	Valid Triangle Number	Medium	3	Sort; fix the largest side as r, then count pairs with <code>a[l] + a[mid] > a[r]</code>
923	3Sum With Multiplicity	Medium	2	Same frame; count combinations instead of emitting, over <code>i < j < k</code>
345	Reverse Vowels of a String	Easy	1	The frame plus skip loops, like 125, with a swap on a match
1471	The k Strongest Values in an Array	Medium	1	Sort by distance from the median, which is <code>a[(n - 1) // 2]</code>
2149	Rearrange Array Elements by Sign	Medium	1	Fill a preallocated output from two write pointers, one per parity
259	3Sum Smaller †	Medium	0	Sorted + converging: when <code>s < target</code> , all <code>r - l</code> pairs count at once – add them and move l
905	Sort Array By Parity	Easy	0	Converging partition: swap when l is odd and r is even
948	Bag of Tokens	Medium	0	Sort; spend tokens at the cheap end, gain at the expensive end
1498	Number of Subsequences That Satisfy the Given Sum Condition	Medium	0	Sort; for each l find the largest r with <code>a[l] + a[r] <= t</code> , then add <code>2^(r-l-1)</code>
2105	Watering Plants II	Medium	0	Two pointers over the plants , one from each side – not the array ends
2330	Valid Palindrome IV †	Medium	0	Count mismatched pairs in the frame; at most two may differ
2441	Largest Positive Integer That Exists With Its Negative	Easy	0	A hash set, not two pointers – there is no ordering to exploit
2562	Find the Array Concatenation Value	Easy	0	The frame; concatenate <code>str(a[l]) + str(a[r])</code> and accumulate
2824	Count Pairs Whose Sum is Less than Target	Easy	0	<code>n <= 50</code> so brute force passes, but sorted + converging counts in one pass

Two Pointers – Same Direction

THE KERNEL

One slow write-pointer, one fast read-pointer. The slow pointer delineates the answer prefix; the fast pointer scans everything else. This is the in-place filtering pattern.

22PROBLEMS
MAPPED**14**

EASY

8

MEDIUM

0

HARD

The Boundary Rules

- 1 The write pointer only advances when you *keep* an element. Everything before `w` is finished output; everything at or after `f` is unread.
- 2 `for f in range(n) + if keep: a[w] = a[f]; w += 1` is the whole shape. Never write a second loop.
- 3 Deleting duplicates in place: compare against `a[w-1]` (the last kept value), not `a[f-1]` — that is the single most common mistake.
- 4 When at most two copies are allowed, the guard becomes `if w < 2 or a[f] != a[w-2]`.
- 5 Merge from the back when the destination array has spare capacity at the end (88): `w = m + n - 1`, and always copy the *larger* of the two tails so you never overwrite unread data.
- 6 Fast/slow pointers on a linked list are the same idea applied to a non-indexable structure; see the Linked List part.

- 7 Cycle detection on an array-as-functional-graph uses Floyd's tortoise and hare: `slow = f(slow); fast = f(f(fast))` and the meeting point is not the entry point — you need a second pass to find the entry.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
977	Squares of a Sorted Array	Easy	5
202	Happy Number	Easy	6
443	String Compression	Medium	7
80	Remove Duplicates from Sorted Array II	Medium	1
922	Sort Array By Parity II	Easy	1
1768	Merge Strings Alternately	Easy	1
457	Circular Array Loop	Medium	1
1385	Find the Distance Value Between Two Arrays	Easy	1
1826	Faulty Sensor †	Easy	1
2200	Find All K-Distant Indices in an Array	Easy	1
905	Sort Array By Parity	Easy	0
905	Sort Array By Parity	Easy	0
1089	Duplicate Zeros	Easy	0
1089	Duplicate Zeros	Easy	0
1750	Minimum Length of String After Deleting Similar Ends	Medium	1
2460	Apply Operations to an Array	Easy	0
2486	Append Characters to String to Make Subsequence	Medium	0
2938	Separate Black and White Balls	Medium	0

Sliding Window - Fixed Size

THE KERNEL

A window of exactly k elements slides right by one. Add the entering element, remove the leaving element, evaluate the invariant. Nothing else ever happens.

23PROBLEMS
MAPPED**7**

EASY

15

MEDIUM

1

HARD

The Boundary Rules

- 1 Initialise by consuming the first k elements, then loop `for r in range(k, n)`. Do not try to fold the first window into the loop — it costs you a branch on every iteration.
- 2 The leaving index is always `r - k`. If you ever find yourself writing `r - k + 1` you have mixed up a fixed and a variable window.
- 3 For ‘anagram in a string’, compare two Counters of size 26, or keep a single `matches` counter and increment it only when a bucket hits zero. Rebuilding the Counter each step turns $O(n)$ into $O(26n)$ with a big constant.
- 4 For ‘at most k distinct in a fixed window’ use the variable-window template instead — a fixed window whose validity can fail mid-window is a variable window in disguise.
- 5 Fixed-window maxima are NOT a sliding window chapter problem; they need a monotonic deque (chapter 28). Recognise the difference: sum/count invariants are $O(1)$ to update, max/min invariants are not.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1151	Minimum Swaps to Group All 1's Together †	Medium	1
246	Strobogrammatic Number †	Easy	2
30	Substring with Concatenation of All Words	Hard	5
187	Repeated DNA Sequences	Medium	2
413	Arithmetic Slices	Medium	2
1100	Find K-Length Substrings With No Repeated Characters †	Medium	1
2090	K Radius Subarray Averages	Medium	1
2134	Minimum Swaps to Group All 1's Together II	Medium	1
1176	Diet Plan Performance †	Easy	1
1343	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Medium	1
1876	Substrings of Size Three with Distinct Characters	Easy	1
1052	Grumpy Bookstore Owner	Medium	0
1652	Defuse the Bomb	Easy	0
2269	Find the K-Beauty of a Number	Easy	0
2379	Minimum Recolors to Get K Consecutive Black Blocks	Easy	0
2461	Maximum Sum of Distinct Subarrays With Length K	Medium	0
2653	Sliding Subarray Beauty	Medium	0
3254	Find the Power of K-Size Subarrays I	Medium	0

Sliding Window - Variable Size

THE KERNEL

Grow the right edge unconditionally; shrink the left edge only while the window is invalid. The window is always the maximal valid window ending at `r`.

29PROBLEMS
MAPPED**2**

EASY

23

MEDIUM

4

HARD

The Boundary Rules

- 1 The loop is `for r in range(n):` then `while <invalid>: remove(a[l]); l += 1`. The inner `while` is what makes it $O(n)$ — each element enters and leaves exactly once.
- 2 Never write `if` where the inner loop needs `while`. An `if` silently allows an invalid window to persist and is the number-one cause of wrong answers in this chapter.
- 3 Decide up front which of the two variants you need: (a) *longest* valid window — shrink until valid, then record `r - l + 1`; (b) *shortest* window satisfying a target — shrink while valid, recording before each shrink, and grow only when invalid.
- 4 For 'longest', the answer is recorded *after* the shrink loop. For 'shortest', the answer is recorded *inside* the shrink loop, before the pointer moves. Swapping these two is the second most common bug.

- 5 The state you maintain must be incrementally updatable in $O(1)$: a count, a sum, a frequency map, a count of distinct keys, or a count of ‘already satisfied’ constraints. If it is not, you have the wrong window definition.
- 6 Longest substring without repeating characters is the archetype: the window is invalid exactly when `freq[c] > 1`, and the fix is to move `l` until that stops being true — not to move `l` to `last[c] + 1` unless you also clear the intermediate characters.
- 7 At-most-K distinct problems are ‘longest’ problems. Exactly-K distinct problems are `at_most(K) - at_most(K-1)`; see chapter 9.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
713	Subarray Product Less Than K	Medium	3
159	Longest Substring with At Most Two Distinct Characters †	Medium	2
395	Longest Substring with At Least K Repeating Characters	Medium	2
487	Max Consecutive Ones II †	Medium	2
1658	Minimum Operations to Reduce X to Zero	Medium	2
340	Longest Substring with At Most K Distinct Characters †	Medium	2
727	Minimum Window Subsequence †	Hard	1
2024	Maximize the Confusion of an Exam	Medium	1
930	Binary Subarrays With Sum	Medium	0
992	Subarrays with K Different Integers	Hard	0
1208	Get Equal Substrings Within Budget	Medium	0
1248	Count Number of Nice Subarrays	Medium	0
1695	Maximum Erasure Value	Medium	0
1838	Frequency of the Most Frequent Element	Medium	0
2401	Longest Nice Subarray	Medium	0
2779	Maximum Beauty of an Array After Applying Operation	Medium	0
2958	Length of Longest Subarray With at Most K Frequency	Medium	0

SLIDING WINDOW - VARIABLE SIZE

#	PROBLEM	LEVEL	ASKED
2962	Count Subarrays Where Max Element Appears at Least K Times	Medium	0
3090	Maximum Length Substring With Two Occurrences	Easy	0
3095	Shortest Subarray With OR at Least K I	Easy	0
3208	Alternating Groups II	Medium	0
3298	Count Substrings That Can Be Rearranged to Contain a String II	Hard	0

Sliding Window - At Most K Reduces to Exactly K

THE KERNEL

Counting windows with an *exact* property is hard; counting windows with an *at most* property is a five-line scan. So compute the easy one twice.

14

PROBLEMS
MAPPED

2

EASY

8

MEDIUM

4

HARD

The Boundary Rules

- 1 $\text{exactly}(K) = \text{at_most}(K) - \text{at_most}(K-1)$. This identity is the whole chapter; recognise it and the problem collapses.
- 2 $\text{at_most}(K)$ is a pure variable window: grow r , shrink l while the count of distinct keys exceeds K , and add $r - l + 1$ to the answer. $r - l + 1$ is the number of valid windows *ending at* r .
- 3 Binary subarrays with sum K (930) uses the same identity on the numeric sum, with a *non-negative* array. Non-negativity is what makes the window monotone; with negatives you must use prefix sums + hash (chapter 15).
- 4 Subarrays with K Different Integers (992) and Count Number of Nice Subarrays (1248) are the same two-call shape. For 1248 the 'distinct keys' count becomes 'count of odd numbers'.

- 5** Do not try to track the number of windows with exactly K in a single pass with a third pointer unless you can prove it — the two-call identity is always correct and always fast enough.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
713	Subarray Product Less Than K	Medium	3
1512	Number of Good Pairs	Easy	3
1358	Number of Substrings Containing All Three Characters	Medium	3
1759	Count Number of Homogenous Substrings	Medium	0
1839	Longest Substring Of All Vowels in Order	Medium	0
2062	Count Vowel Substrings of a String	Easy	0
2302	Count Subarrays With Score Less Than K	Hard	0
2537	Count the Number of Good Subarrays	Medium	0
2953	Count Complete Substrings	Hard	0
3116	Kth Smallest Amount With Single Denomination Combination	Hard	0

PART III

Binary Search

Two loop shapes and two midpoint formulas. Everything else is bookkeeping.

Binary Search - The Boundary Rules

THE KERNEL

There are only two loop shapes and two midpoint formulas. Pick the shape by asking ‘does the answer live in the closed range [lo, hi] or at the boundary where the range collapses?’

Binary search is the most commonly failed easy problem in technical interviews, and it fails for one reason: candidates memorise *an* implementation rather than the *rule that chooses between* implementations. They write `while lo <= hi`, then pair it with `hi = mid`, and the loop hangs. Or they write `lo = mid` with the lower midpoint and the loop hangs in the other direction. Both bugs come from the same missing piece of knowledge, and both are trivially avoidable once you have it.

This chapter contains no clever ideas. It contains two loop shapes, two midpoint formulas, and one pairing rule that connects them. Everything else in the binary search family — bounds, rotated arrays, peak finding, binary search on the answer — is a substitution into that frame.

Read the boundary rules below, then close the book and write the template from memory. If you cannot, you have not yet learned this chapter, regardless of how many problems you have solved.

55PROBLEMS
MAPPED**21**

EASY

31

MEDIUM

3

HARD

The Boundary Rules

These ten rules are the whole chapter. Rules 1 through 5 are the framework; rules 6 through 10 are the traps.

- 1 **Shape A — closed range.** `while lo <= hi:` with `hi = mid - 1` and `lo = mid + 1`. Exits with `lo == hi + 1`. Use when a target may be absent and you must report 'not found'.
- 2 **Shape B — half-open, answer is the collapse point.** `while lo < hi:` with `hi = mid` and `lo = mid + 1`. Exits with `lo == hi == answer`. Use for lower bound, upper bound, 'smallest feasible', and binary search on the answer.
- 3 **Midpoint rule.** `mid = lo + (hi - lo) // 2` is the lower mid.
`mid = lo + (hi - lo + 1) // 2` is the upper mid.
- 4 **The pairing rule — memorise this one line.** If the branch that moves `lo` can leave `lo == mid` (i.e. `lo = mid`), you MUST use the upper mid. If the branch that moves `hi` can leave `hi == mid` (i.e. `hi = mid`), you MUST use the lower mid. Violating this is an infinite loop, not a wrong answer — which is why it is easy to spot under test.
- 5 Equivalently: `hi = mid` implies lower mid; `lo = mid` implies upper mid.
- 6 **Return rule.** After Shape A the candidate is `lo` and it is valid only if `lo < n` and `a[lo] == target`. After Shape B `lo` is the answer unconditionally — Shape B never returns 'not found', it returns the insertion point, which is why it is the shape you want for bounds.
- 7 **Overflow.** `lo + (hi - lo) // 2` never overflows; `(lo + hi) // 2` can in a fixed-width language. Python has arbitrary precision, but write the safe form anyway — interviewers read it as a signal.
- 8 **Never `while lo <= hi` with `hi = mid`.** The range does not shrink when `hi == mid == lo + 1`, and you loop forever.
- 9 **Never `lo = mid` without `+1`.** Same failure, other side.
- 10 Searching a 2D matrix that is sorted row-wise and column-wise (240) is NOT a binary search over the whole matrix — start at the top-right corner and walk, because that corner is the only cell that is a minimum in one direction and a maximum in the other.

The Template

There are exactly three code shapes. Shape A is for ‘find this exact value and tell me if it is missing’. Shape B is for ‘find the boundary where a predicate flips’. Shape C is Shape B’s mirror, and it exists only because of the midpoint rule: when the branch that moves `lo` can leave `lo == mid`, the midpoint must round up.

Memorise Shape B and Shape C as a pair. If you can write both from memory and state why Shape C needs the `+ 1`, you can solve every problem in this chapter and the next four.

```
# ----- Shape A
# Closed range [lo, hi]. Exits with lo == hi + 1.
# Use when the target may be ABSENT and you must report that.

def search_closed(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:                # <= : the range is closed
        mid = lo + (hi - lo) // 2  # lower mid
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1            # +1 : mid is excluded
        else:
            hi = mid - 1           # -1 : mid is excluded
    return -1                      # absent; lo is the insert point

# ----- Shape B
# Half-open [lo, hi). The answer is where the range collapses.
# "Smallest i such that predicate(i) is True."

def lower_bound(a, x):
    lo, hi = 0, len(a)            # hi is one PAST the end
    while lo < hi:                # < : the range is half-open
        mid = lo + (hi - lo) // 2  # lower mid
        if a[mid] >= x:
            hi = mid               # mid may be the answer: keep it
        else:
            lo = mid + 1           # mid is not: exclude it
    return lo                     # lo == hi == the answer

# ----- Shape C
```



```

# Shape B's mirror. "Largest x with feasible(x) True."
# The midpoint MUST round up: lo = mid has to make progress.

def largest_feasible(lo, hi, feasible):
    while lo < hi:
        mid = lo + (hi - lo + 1) // 2  # UPPER mid (lo = mid)
        if feasible(mid):
            lo = mid  # mid may be the answer: keep it
        else:
            hi = mid - 1  # mid is not: exclude it
    return lo  # the largest feasible value

# ----- the one-line form
import bisect

def count_equal(a, x):
    """How many elements equal x? Two Shape-B calls."""
    return bisect.bisect_right(a, x) - bisect.bisect_left(a, x)

```

Why Shape B cannot return “not found”

This surprises people, so it is worth being explicit. Shape B narrows a half-open range until `lo == hi`, and returns `lo`. That value always exists — it is an index in `[0, n]`. When the target is absent, Shape B returns the *insertion point*, which is the index where the target would go to keep the array sorted. That is not a failure; it is extra information, and it is exactly what `Search Insert Position` (35) asks for.

So Shape B is strictly more informative than Shape A, and you should prefer it whenever the problem asks for a position rather than a yes/no. When the problem genuinely needs ‘not found’, compute the lower bound and then verify: `i = bisect_left(a, x)` and check `i < len(a)` and `a[i] == x`.

THE REFRAMING

Shape B never fails. It returns the insertion point, which is the answer to a different question than the one you thought you were asking.

The pairing rule, stated three ways

The infinite-loop bug always has the same cause: a branch that assigns `mid` to a bound without moving past it, combined with a midpoint that rounds the wrong way. Three equivalent statements of the fix:

- **By the branch:** if a branch assigns `lo = mid`, use the upper mid. If a branch assigns `hi = mid`, use the lower mid.
- **By the loop:** `while lo < hi` with `hi = mid` is the lower-mid shape. `while lo < hi` with `lo = mid` is the upper-mid shape. Never mix them.
- **By the symptom:** if your search hangs, your midpoint rounds down and your `lo` branch does not advance. Add `+ 1` to the midpoint — do not add `+ 1` to `lo = mid`.

Choosing between the shapes in an interview

Say the following sentence out loud before you write code: *the answer is either a value I am looking for, in which case I use the closed shape and check for absence; or it is the boundary where a monotone predicate flips, in which case I use the half-open shape and return the collapse point.*

That sentence takes four seconds and it converts the most error-prone part of the interview into a decision you have already made.

Worked Solutions

Ten problems, in increasing order of how much the boundary rules matter. Each one is annotated with the line of the template that changes.

704 Binary Search

LC 704 · Easy · asked by 9 companies · AC 61.4% · Array, Binary Search

THE PROBLEM

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
Output: `4`

Explanation 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
Output: `-1`

Explanation 2 does not exist in `nums` so return -1

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

SIGNATURE

```
def search(nums, target): # -> integer
```

IN ONE LINE Return the index of `target` in a sorted array, or `-1`.

THE INSIGHT This is the pure Shape A. The only decision is that the range is closed, so the loop uses `<=` and both branches move past `mid`.

The closed range `[lo, hi]` contains the candidates. Each iteration examines `mid` and discards it by moving a bound past it. When `lo > hi` the range is empty and the target is absent.

The invariant to state: **every index outside `[lo, hi]` has been ruled out**. That is what justifies returning `-1` at the end.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Complexity. $O(\log n)$ time, $O(1)$ space. The loop runs at most `ceil(log2(n + 1))` times.

PITFALL

Writing `hi = mid` instead of `hi = mid - 1`. With the lower midpoint this hangs on two-element ranges: `lo = 0, hi = 1` gives `mid = 0`, and `hi = 0` repeats forever.

PITFALL

Writing `while lo < hi`. That drops the last candidate and silently returns `-1` for a single-element array.

Follow-up. “What if the array has duplicates and you want the first occurrence?” That is chapter 11 – `bisect_left`, which is Shape B with the predicate `a[i] >= x`.

35 Search Insert Position

LC 35 · Easy · asked by 9 companies · AC 52.3% · Array, Binary Search

THE PROBLEM

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
Output: 4

Constraints:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`
- `nums` contains **distinct** values sorted in **ascending** order.
- `-104 <= target <= 104`

SIGNATURE

```
def searchInsert(nums, target): # -> integer
```

IN ONE LINE Return the index where `target` should be inserted to keep the array sorted.

THE INSIGHT This is Shape B with no modification at all. The insertion point *is* the collapse point of the half-open range.

Compare with 704: there, an absent target was a failure. Here it is the answer. The same search returns both.

`bisect.bisect_left(nums, target)` is the standard library version. Write the loop by hand in an interview, then mention that `bisect_left` does exactly this.

```
def search_insert(nums, target):
    lo, hi = 0, len(nums)          # half-open: hi may be len(nums)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] >= target:    # predicate: a[mid] >= target
            hi = mid               # mid may be the answer
        else:
            lo = mid + 1           # mid is not
    return lo                      # == bisect_left(nums, target)
```

Complexity. $O(\log n)$ time, $O(1)$ space. Note `hi` starts at `len(nums)`, which is why the range is half-open.

PITFALL

Starting `hi = len(nums) - 1`. Then a target larger than every element returns `len(nums) - 1` instead of `len(nums)`.

PITFALL

Using `nums[mid] > target` as the predicate. That computes `bisect_right`, which is off by one when duplicates exist.

Follow-up. “And if you want the position *after* all equal elements?” Change `>=` to `>`. The two functions differ by exactly one character – which is why `count_equal` in the template is a subtraction of two `bisect` calls rather than a custom loop.

34 Find First and Last Position of Element in Sorted Array

LC 34 · **Medium** · asked by 0 companies · AC 49.7% · Array, Binary Search

THE PROBLEM

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]

Example 2:

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]

Example 3:

Input: nums = [], target = 0
Output: [-1,-1]

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- `nums` is a non-decreasing array.
- $-10^9 \leq \text{target} \leq 10^9$

SIGNATURE

```
def searchRange(nums, target): # -> integer[]
```

IN ONE LINE Return `[first, last]` index of `target`, or `[-1, -1]`.

THE INSIGHT Two `bisect` calls with predicates `>=` and `>`. No custom binary search is needed, and no post-processing beyond one validity check.

`first = bisect_left(a, x)` is the smallest index with `a[i] >= x`. `last = bisect_right(a, x) - 1` is the largest index with `a[i] <= x`.

The existence check is `first < len(a)` and `a[first] == x`. Without it, an absent target returns `[n, n-1]`, which is nonsense that some test cases will catch.

```
from bisect import bisect_left, bisect_right

def search_range(nums, target):
    first = bisect_left(nums, target) # smallest index, a[i] >= x
    if first == len(nums) or nums[first] != target:
        return [-1, -1] # absent; Shape B cannot tell
    last = bisect_right(nums, target) - 1 # largest index, a[i] <= x
```

```

    return [first, last]

# By hand, to show it is the same shape twice:
def search_range_manual(nums, target):
    def bound(x, strict):
        lo, hi = 0, len(nums)
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if nums[mid] > x or (not strict and nums[mid] == x):
                hi = mid
            else:
                lo = mid + 1
        return lo
    first = bound(target, False)
    if first == len(nums) or nums[first] != target:
        return [-1, -1]
    return [first, bound(target, True) - 1]

```

Complexity. $O(\log n)$ time, $O(1)$ space – two searches, so about twice the constant of 704.

PITFALL

Returning `[first, bisect_right(nums, target)]`. The right bound is *exclusive*, so it needs the `- 1`.

PITFALL

Skipping the validity check. `[n, n-1]` is not a valid answer and will fail the empty-result test.

Follow-up. “What if you must not use `bisect`?” Write `bound` once with a `strict` flag, as above. The flag is the only difference between the two calls.

278 First Bad Version

LC 278 · Easy · asked by 7 companies · AC 47.7% · Binary Search, Interactive

THE PROBLEM

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each

version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: $n = 5$, $\text{bad} = 4$
Output: 4

Explanation

```
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

Example 2:

Input: $n = 1$, $\text{bad} = 1$
Output: 1

Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

SIGNATURE

```
def firstBadVersion(n, bad): # -> integer
```

IN ONE LINE Given `isBadVersion(v)`, find the first bad version.

THE INSIGHT The predicate `isBadVersion` is monotone by construction (false, false, ..., true, true), so this is Shape B with no array at all.

This is the cleanest possible demonstration of the pattern: there is no data structure, only a monotone predicate. The search space is the version numbers, not an index into anything.

Set `lo = 1`, `hi = n` and let `hi` be *inclusive* of the answer candidate. `while lo < hi` with `hi = mid` and `lo = mid + 1` converges on the first `true`.

```
def first_bad_version(n):
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if isBadVersion(mid):      # monotone predicate
            hi = mid                # mid may be the first bad one
        else:
            lo = mid + 1            # mid is definitely good
    return lo
```

Complexity. $O(\log n)$ calls to `isBadVersion`, $O(1)$ space. The number of calls is the cost model here – say so, because the problem statement asks for it.

PITFALL

Using `Shape A` and a `last_bad` variable. That is $O(n)$ in the worst case and misses the point of the exercise.

PITFALL

Writing `hi = mid - 1`. With `hi` inclusive of a candidate, subtracting 1 discards the answer.

Follow-up. “How many calls does your solution make?” `ceil(log2 n)`. For `n = 231 - 1` that is 31 calls. Being able to answer this precisely is worth as much as the code.

69 Sqrt(x)

LC 69 · **Easy** · asked by 7 companies · AC 42.3% · Math, Binary Search, Newton's Method

THE PROBLEM

Given a non-negative integer `x`, return the square root of `x` rounded down to the nearest integer. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: `x = 4`
Output: `2`

Explanation The square root of 4 is 2, so we return 2.

Example 2:

Input: `x = 8`

Output: `2`

Explanation The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

Constraints:

- $0 \leq x \leq 2^{31} - 1$

SIGNATURE

```
def mySqrt(x): # -> integer
```

IN ONE LINE Integer square root of `x`, rounded down.

THE INSIGHT This is Shape C — the *largest feasible* form — and it is the problem that exposes the midpoint rule.

The predicate is `mid * mid <= x`. It is monotone: if `mid` works, every smaller value works. So the answer is the largest feasible `mid`, which is Shape C.

Because the feasible branch assigns `lo = mid`, the midpoint must round **up**. If you use the lower midpoint here, `lo = mid` makes no progress on a two-element range and the loop hangs. This is the bug the whole chapter is about, in its purest form.

Note `hi = x // 2 + 1` rather than `x`. For `x >= 4` the answer is at most `x // 2`, so this halves the search range; the `+ 1` keeps `x = 1` working.

```
def my_sqrt(x):
    if x < 2:
        return x
    lo, hi = 1, x // 2 + 1          # answer <= x // 2 for x >= 4
    while lo < hi:
        mid = lo + (hi - lo + 1) // 2 # UPPER mid: required by lo = mid
        if mid * mid <= x:           # feasible
            lo = mid                  # keep it; mid may be the answer
        else:
            hi = mid - 1              # discard it
```

```

    return lo

# Same thing without the midpoint trap; Shape B on first failure:
def my_sqrt_shapeb(x):
    lo, hi = 1, x // 2 + 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if mid * mid > x:                # "too big" is the predicate
            hi = mid
        else:
            lo = mid + 1
    return lo - 1                        # one before first too-big value

```

Complexity. $O(\log x)$ time, $O(1)$ space. Newton's method is faster in practice but is not the expected answer here.

PITFALL

Using the lower midpoint with `lo = mid`. **Infinite loop.** If you remember only one thing from this chapter, remember this line.

PITFALL

Forgetting the `x < 2` guard. `x = 0` gives `hi = 1` and `x = 1` gives `hi = 1`, both of which happen to work, but the guard makes it obvious rather than accidental.

Follow-up. "Can you avoid the midpoint rule entirely?" Yes – search for the first `mid` with `mid * mid > x` using Shape B and return `lo - 1`, as in `my_sqrt_shapeb`. **Choosing the predicate so that Shape B applies is a legitimate way to dodge the upper midpoint**, and offering it shows you understand the trade.

162 Find Peak Element

LC 162 · Medium · asked by 8 companies · AC 47.2% · Array, Binary Search

THE PROBLEM

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `2`

Explanation 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: `5`

Explanation Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Constraints:

- `1 <= nums.length <= 1000`
- `-231 <= nums[i] <= 231 - 1`
- `nums[i] != nums[i + 1]` for all valid `i`.

SIGNATURE

```
def findPeakElement(nums): # -> integer
```

IN ONE LINE Find any peak element in an array where `a[i] != a[i+1]`.

THE INSIGHT The array is not sorted, but the *comparison with the right neighbour* is a monotone predicate. Where the slope turns from up to down, there is a peak.

Define `predicate(mid) = a[mid] > a[mid + 1]`. If `mid` is on a downward slope, a peak exists at or to the left. If `mid` is on an upward slope, a peak exists at or to the right.

That makes the predicate monotone in the search sense — false, false, ..., true, true — so Shape B applies directly. `hi` starts at `len(a) - 1` because `mid + 1` must be in range.

This is the template for a whole family: **binary search works whenever you can decide which half contains the answer, even if the array is not sorted.**

```
def find_peak_element(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] > nums[mid + 1]: # descending -> peak at or left
            hi = mid
        else: # ascending -> peak at or right
            lo = mid + 1
    return lo # a peak index
```

Complexity. $O(\log n)$ time, $O(1)$ space. The problem asks for $O(\log n)$, which is the hint that this is a binary search despite the unsorted input.

PITFALL

Comparing with `a[mid - 1]` instead. Both work, but mixing the two comparisons across the two branches is how off-by-one bugs get in. Pick one reference and stay consistent.

PITFALL

Reading `nums[mid + 1]` when `mid` could be `len(nums) - 1`. With `hi = len(nums) - 1` and the lower midpoint, `mid < hi` always holds, so `mid + 1` is safe — but you must be able to say why.

Follow-up. “What if there are duplicates?” [Find Peak Element II](#) (1901) does it in 2-D. With duplicates in 1-D the predicate stops being monotone and you need [Find Minimum in Rotated Sorted Array II](#) (154)'s trick of shrinking both ends.

33 Search in Rotated Sorted Array

LC 33 · **Medium** · asked by 21 companies · AC 45.6% · Array, Binary Search

THE PROBLEM

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is

`[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: `4`

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: `-1`

Example 3:

Input: `nums = [1]`, `target = 0`
Output: `-1`

Constraints:

- `1 <= nums.length <= 5000`
- `-104 <= nums[i] <= 104`
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- `-104 <= target <= 104`

SIGNATURE

```
def search(nums, target): # -> integer
```

IN ONE LINE Search for `target` in a rotated sorted array with distinct values.

THE INSIGHT Exactly one of the two halves `[lo, mid]` and `[mid, hi]` is sorted. Find which, then ask whether the target lies inside that sorted half.

The array is not sorted, but it is *piecewise* monotone, and one piece always contains `mid`. The test `a[lo] <= a[mid]` tells you whether the left half is the sorted one.

If the left half is sorted, the target is in it exactly when `a[lo] <= target < a[mid]`; otherwise it must be in the right half. The two branches are mirror images.

This is Shape A with a two-step decision replacing the three-way comparison. The bounds still move past `mid`.

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:      # left half [lo, mid] sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1          # target lies inside it
            else:
                lo = mid + 1          # so it is in the right half
        else:                         # right half [mid, hi] sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1          # target lies inside it
            else:
                hi = mid - 1          # so it is in the left half
    return -1
```

Complexity. $O(\log n)$ time, $O(1)$ space. The rotation costs nothing because the sorted-half test is $O(1)$.

PITFALL

Using `nums[lo] < nums[mid]` instead of `<=`. With two elements `lo == mid`, and the strict comparison misclassifies the half.

PITFALL

Testing `nums[lo] <= target <= nums[mid]` with `<=` on the right end. `nums[mid] == target` was already handled, and including it again makes the `else` branch wrong.

PITFALL

Deciding which half is sorted by comparing with `a[hi]` in one branch and `a[lo]` in the other. Both are valid references, but you must be consistent — and the `a[hi]` reference is the one you need for problem 153, so pick that and reuse it.

Follow-up. "What if values can repeat?" Problem 81. When `a[lo] == a[mid] == a[hi]` you cannot tell which half is sorted, so you must shrink by one from each end and accept $O(n)$ worst-case time. Say that the guarantee is lost – that is the expected answer.

153 Find Minimum in Rotated Sorted Array

LC 153 · Medium · asked by 7 companies · AC 55.3% · Array, Binary Search

THE PROBLEM

Suppose an array of length `n` sorted in ascending order is **rotated** between `1` and `n` times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated `4` times.
- `[0,1,2,4,5,6,7]` if it was rotated `7` times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`
Output: `1`

Explanation The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`
Output: `0`

Explanation The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`
Output: `11`

Explanation The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- `n == nums.length`

- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between `1` and `n` times.

SIGNATURE

```
def findMin(nums):    # -> integer
```

IN ONE LINE Find the minimum in a rotated sorted array with distinct values.

THE INSIGHT Compare `a[mid]` with `a[hi]`. If `a[mid] < a[hi]` the right half is sorted, so the minimum is at or left of `mid`. Otherwise it is strictly to the right.

This is the cleanest rotated-array search because it needs no nested decision: the single comparison with `a[hi]` partitions the array correctly every time.

Note that `hi = mid` (not `mid - 1`) — `mid` may be the minimum, so it must stay in the range. That makes this Shape B.

The comparison reference matters here. Comparing with `a[lo]` instead does not work for the minimum, because when `a[lo] < a[mid]` you still cannot tell whether the rotation point is before or after `mid`. **`a[hi]` is the right reference for minimum-finding; remember that as a separate fact from problem 33.**

```
def find_min(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] < nums[hi]:    # right half is sorted
            hi = mid                # minimum is at or left of mid
        else:                      # minimum is strictly right of mid
            lo = mid + 1
    return nums[lo]
```

Complexity. $O(\log n)$ time, $O(1)$ space.

PITFALL

Writing `hi = mid - 1` in the first branch. If `mid` is the minimum you have just discarded it.

PITFALL

Returning `nums[hi]` at the end. Both work since `lo == hi`, but pick one and be consistent with problem 154.

Follow-up. "With duplicates?" Problem 154 adds `elif nums[mid] == nums[hi]: hi -= 1`. The `elif` must come *after* the `<` test, and the `-= 1` is what degrades the worst case to $O(n)$ when the array is all equal values.

540 Single Element in a Sorted Array

LC 540 · **Medium** · asked by 6 companies · AC 59.4% · Array, Binary Search

THE PROBLEM

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in $O(\log n)$ time and $O(1)$ space.

Example 1:

Input: `nums = [1,1,2,3,3,4,4,8,8]`
Output: `2`

Example 2:

Input: `nums = [3,3,7,7,10,11,11]`
Output: `10`

Constraints:

- `1 <= nums.length <= 105`
- `0 <= nums[i] <= 105`

SIGNATURE

```
def singleNonDuplicate(nums): # -> integer
```

IN ONE LINE In a sorted array where every element appears twice except one, find the single element.

THE INSIGHT Binary search on the *parity of the index*. Before the single element, pairs start at even indices; after it, at odd indices. The index XOR trick makes the pair test one expression.

For any index `i`, its pair partner is `i ^ 1`. Before the single element, `a[i] == a[i ^ 1]` holds for every `i`; after it, the parity is flipped and the equality fails.

So the predicate `a[mid] == a[mid ^ 1]` is monotone: true, true, ..., false, false. Shape B finds the first `false`.

`mid` must be even for the test to mean what you think. `mid -= mid % 2` forces that, and it is the line that makes the solution robust rather than lucky.

```
def single_non_duplicate(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        mid -= mid % 2           # force mid to be even
        if nums[mid] == nums[mid + 1]: # pair intact -> single right
            lo = mid + 2
        else:                     # broken pair -> single at or left
            hi = mid
    return nums[lo]
```

Complexity. $O(\log n)$ time, $O(1)$ space. The XOR form `mid ^ 1` is equivalent and slightly more elegant; both are accepted.

PITFALL

Advancing `lo = mid + 1`. Because pairs move in steps of two, the correct advance is `mid + 2` — a `+ 1` leaves `lo` pointing at the second half of a pair.

PITFALL

Forgetting the `mid -= mid % 2` normalisation. On odd `mid` the comparison tests the wrong pair and the search goes to the wrong half.

Follow-up. “Where else does index-parity binary search appear?” Any problem where a *structural property* flips at a point — `Find the Duplicate Number` (287) has a similar flavour, though its expected answer is Floyd’s cycle detection from chapter 35.

74 Search a 2D MatrixLC 74 · **Medium** · asked by 10 companies · AC 54.3% · Array, Binary Search, Matrix**THE PROBLEM**

You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

[diagram omitted — see leetcode.com/problems/search-a-2d-matrix/]

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`
Output: `true`

Example 2:

[diagram omitted — see leetcode.com/problems/search-a-2d-matrix/]

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 13`
Output: `false`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 100`
- `-104 <= matrix[i][j], target <= 104`

SIGNATURE

```
def searchMatrix(matrix, target): # -> boolean
```

IN ONE LINE Search a matrix whose rows are sorted and whose first element of each row is greater than the last element of the previous row.

THE INSIGHT The matrix is a sorted array that has been wrapped. Treat it as a flat array of length `m * n` and convert the midpoint with `divmod`.

The problem statement guarantees a *total* order, which is stronger than the sorted-matrix condition. That total order is exactly what lets you index the matrix as a flat array.

`row, col = divmod(mid, n)` converts a flat index to a cell. Everything else is problem 704 verbatim.

Contrast with problem 240, whose matrix is sorted only row-wise and column-wise. There the flat trick fails and you must walk from the top-right corner instead. Knowing which of the two matrices you have is the entire decision.

```
def search_matrix(matrix, target):
    m, n = len(matrix), len(matrix[0])
    lo, hi = 0, m * n - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        row, col = divmod(mid, n)      # flat index -> cell
        if matrix[row][col] == target:
            return True
        if matrix[row][col] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False
```

Complexity. $O(\log(mn))$ time, $O(1)$ space.

PITFALL

Writing `matrix[mid // n][mid % n]` inline four times. `divmod` once per iteration is clearer and no slower.

PITFALL

Using this solution on problem 240's matrix. It produces wrong answers, not an error — which is why the distinction is worth stating explicitly in an interview.

Follow-up. "And if the matrix is only row-wise and column-wise sorted?" Start at the top-right corner. If the value is larger than the target, move left; if smaller, move down. $O(m + n)$, and it is the only cell from which both moves are informative.

The Index – What Changes in the Template

Every remaining problem in this chapter, sorted by how often it is asked. The final column names the single thing that changes in the template — read that column and nothing else on a first pass.

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
33	Search in Rotated Sorted Array	Medium	21	
4	Median of Two Sorted Arrays	Hard	13	Shape B on the partition of the shorter array; <code>±inf</code> sentinels remove every edge case (ch 14)
74	Search a 2D Matrix	Medium	10	Flatten: <code>row, col = divmod(mid, n)</code> – total order makes it 1-D
1146	Snapshot Array	Medium	2	Per-index <code>((snap_id, value))</code> history + <code>bisect_right</code> (ch 24)
1539	Kth Missing Positive Number	Easy	5	Shape B: missing count at <code>mid</code> is <code>a[mid] - mid - 1 >= k</code>
153	Find Minimum in Rotated Sorted Array	Medium	7	
240	Search a 2D Matrix II	Medium	6	Not binary search – walk from the top-right corner, $O(m+n)$
1428	Leftmost Column with at Least a One †	Medium	3	240's walk, but only left/down moves – never right
81	Search in Rotated Sorted Array II	Medium	4	Shape A, but <code>a[lo] == a[mid] == a[hi]</code> means shrink both ends; $O(n)$ worst case
1287	Element Appearing More Than 25% In Sorted Array	Easy	2	Binary search the boundary of the majority run
744	Find Smallest Letter Greater Than Target	Easy	3	<code>bisect_right</code> ; wrap to <code>a[0]</code> when the result is past the end
2089	Find Target Indices After Sorting Array	Easy	2	Sort, then <code>bisect_left / bisect_right</code> to get the range
888	Fair Candy Swap	Easy	2	Set difference on the two sums – no search needed
154	Find Minimum in Rotated Sorted Array II	Hard	1	Shape B vs <code>a[hi]</code> ; add <code>elif a[mid] == a[hi]: hi -= 1</code>

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
1237	Find Positive Integer Solution for a Given Equation	Medium	1	Monotone 2-D walk, same as 240
1346	Check If N and Its Double Exist	Easy	1	Hash set; or sort and search for <code>2x</code>
1351	Count Negative Numbers in a Sorted Matrix	Easy	1	<code>bisect_left</code> per row, or the 240 staircase walk
275	H-Index II	Medium	1	Shape B: first <code>i</code> with <code>citations[i] >= n - i</code>
1385	Find the Distance Value Between Two Arrays	Easy	1	Sort <code>b</code> , <code>bisect_left</code> , then check both neighbours
1337	The K Weakest Rows in a Matrix	Easy	1	<code>bisect_left</code> per row for the soldier count, then sort by <code>(count, row)</code>
1608	Special Array With X Elements Greater Than or Equal X	Easy	1	Sort, then Shape B over <code>x</code> , or a bucket count
2080	Range Frequency Queries	Medium	1	<code>bisect_left / bisect_right</code> over per-value index lists
441	Arranging Coins	Easy	0	Shape B on <code>k(k+1)/2 <= n</code> , or close the quadratic form
702	Search in a Sorted Array of Unknown Size †	Medium	0	Shape A with <code>hi</code> doubling until it exceeds the target, then search
1064	Fixed Point †	Easy	0	Shape B: first <code>i</code> with <code>a[i] >= i</code>
1292	Maximum Side Length of a Square with Sum Less than or Equal to Threshold	Medium	0	2-D prefix sums + binary search the side length (ch 13, 16)
1631	Path With Minimum Effort	Medium	0	Binary search the effort + BFS feasibility (ch 13)
1855	Maximum Distance Between a Pair of Values	Medium	0	Two pointers, not binary search – <code>i</code> and <code>j</code> both advance
1885	Count Pairs in Two Arrays †	Medium	0	Sort + <code>bisect</code> per element
1898	Maximum Number of Removable Characters	Medium	0	Binary search the number removed + a subsequence check (ch 13)
1979	Find Greatest Common Divisor of Array	Easy	0	<code>math.gcd(min(a), max(a))</code>

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
2048	Next Greater Numerically Balanced Number	Medium	0	Enumerate digit patterns – no search involved
2141	Maximum Running Time of N Computers	Hard	0	Binary search the runtime + a feasibility sum (ch 13)
2226	Maximum Candies Allocated to K Children	Medium	0	Binary search the pile size + greedy count (ch 13)
2300	Successful Pairs of Spells and Potions	Medium	0	Sort potions, <code>bisect_left</code> per spell
2387	Median of a Row Wise Sorted Matrix †	Medium	0	Binary search the value + per-row count (ch 14)
2476	Closest Nodes Queries in a Binary Search Tree	Medium	0	Inorder to a sorted array, then <code>bisect</code>
2529	Maximum Count of Positive Integer and Negative Integer	Easy	0	<code>bisect_left(0)</code> and <code>n - bisect_right(0)</code>
2540	Minimum Common Value	Easy	0	Two pointers over two sorted arrays
2563	Count the Number of Fair Pairs	Medium	0	Sort + <code>bisect</code> per element
2592	Maximize Greatness of an Array	Medium	0	Sort + two pointers
2772	Apply Operations to Make All Array Elements Equal to Zero	Medium	0	Difference array – no binary search (ch 17)
2845	Count of Interesting Subarrays	Medium	0	Prefix sums + hash (ch 15)
2981	Find Longest Special Substring That Occurs Thrice I	Medium	0	Enumerate candidate lengths and count
3020	Find the Maximum Number of Elements in Subset	Medium	0	Enumerate powers of two and count

Pitfalls

- 1 **The infinite loop.** `lo = mid` with the lower midpoint, or `hi = mid` with the upper midpoint. Both hang. The fix is always to change the midpoint, never to change the assignment.
- 2 **while `lo <= hi` with `hi = mid`.** The range does not shrink when `hi == mid == lo + 1`, so this hangs even though both individual lines look correct. The loop condition and the bound assignments are a matched pair; change them together.
- 3 **Forgetting that `bisect_right` is exclusive.** `bisect_right(a, x) - 1` is the last index of `x`, not `bisect_right(a, x)`.
- 4 **Assuming `bisect` means 'found'.** Both `bisect_left` and `bisect_right` return a valid index even when the value is absent. The existence check `i < n` and `a[i] == x` is always your responsibility.
- 5 **Comparing with the wrong reference in a rotated array.** `a[lo]` is the right reference for *searching* (problem 33); `a[hi]` is the right reference for *finding the minimum* (problem 153). Mixing them produces wrong answers on specific rotations, not on all inputs — which is why they are easy to ship broken.
- 6 **Reading `a[mid + 1]` without proving `mid < hi`.** It is safe when `hi` starts at `n - 1` and the midpoint rounds down, but you should be able to say why rather than hope.
- 7 **Computing the midpoint as `(lo + hi) // 2`.** Python is safe, but every interviewer reads `lo + (hi - lo) // 2` as a signal that you know why the naive form is a bug in other languages. Write the safe form.
- 8 **Using `while lo < hi` when the target may be absent.** Shape B cannot report absence. If the problem needs a `-1`, either use Shape A or add an explicit validity check after the loop.

Drill Set

Twenty-two problems, ordered so that each one adds exactly one idea to the previous. Do them without looking at the template. If a problem takes more than eight

minutes, stop, re-read the boundary rule it violates, and start again from the template.

Target: forty minutes for the first twelve, thirty for the rest. If you finish faster than that you are guessing rather than applying the rules, and the mistakes will surface under interview pressure.

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
33	Search in Rotated Sorted Array	Medium	21	
162	Find Peak Element	Medium	8	
278	First Bad Version	Easy	7	
74	Search a 2D Matrix	Medium	10	Flatten: <code>row, col = divmod(mid, n)</code> – total order makes it 1-D
1539	Kth Missing Positive Number	Easy	5	Shape B: missing count at <code>mid</code> is <code>a[mid] - mid - 1 >= k</code>
704	Binary Search	Easy	9	
35	Search Insert Position	Easy	9	
69	Sqrt(x)	Easy	7	
153	Find Minimum in Rotated Sorted Array	Medium	7	
540	Single Element in a Sorted Array	Medium	6	
240	Search a 2D Matrix II	Medium	6	Not binary search – walk from the top-right corner, $O(m+n)$
852	Peak Index in a Mountain Array	Medium	4	
744	Find Smallest Letter Greater Than Target	Easy	3	<code>bisect_right</code> ; wrap to <code>a[0]</code> when the result is past the end
367	Valid Perfect Square	Easy	2	
374	Guess Number Higher or Lower	Easy	2	
1351	Count Negative Numbers in a Sorted Matrix	Easy	1	<code>bisect_left</code> per row, or the 240 staircase walk
275	H-Index II	Medium	1	Shape B: first <code>i</code> with <code>citations[i] >= n - i</code>

#	PROBLEM	LEVEL	ASKED	WHAT CHANGES
34	Find First and Last Position of Element in Sorted Array	Medium	0	
441	Arranging Coins	Easy	0	Shape B on <code>k(k+1)/2 <= n</code> , or close the quadratic form
1064	Fixed Point †	Easy	0	Shape B: first <code>i</code> with <code>a[i] >= i</code>
2300	Successful Pairs of Spells and Potions	Medium	0	Sort potions, <code>bisect_left</code> per spell
2529	Maximum Count of Positive Integer and Negative Integer	Easy	0	<code>bisect_left(0)</code> and <code>n - bisect_right(0)</code>

Binary Search - Lower and Upper Bound

THE KERNEL

`bisect_left` is Shape B with predicate `a[i] >= x`. `bisect_right` is Shape B with predicate `a[i] > x`. Every ‘first/last position’, ‘insert point’, and ‘count of elements $\leq x$ ’ problem is one of these two lines.

35

 PROBLEMS
MAPPED

15

EASY

18

MEDIUM

2

HARD

The Boundary Rules

- 1 `bisect_left(a, x)` returns the first index `i` with `a[i] >= x`; `bisect_right(a, x)` returns the first index with `a[i] > x`.
- 2 The count of elements equal to `x` is `bisect_right(a, x) - bisect_left(a, x)`. Memorise this — it answers ‘first and last position’ (34) in two lines with no custom loop.
- 3 `bisect_left` and `bisect_right` differ by exactly one character in the predicate: `>=` versus `>`. Write the predicate as a lambda first, then mechanically translate it.
- 4 When the answer must be a *value* rather than an index, and the value space is huge, binary search the value with Shape B and write the predicate as a helper `feasible(x)`.
- 5 Returning `-1` when absent: Shape B cannot do it. Compute `i = bisect_left(a, x)` and then check `i < len(a)` and `a[i] == x`.

- 6 Rotated arrays break the global ordering but preserve it locally: exactly one of `[lo, mid]` and `[mid, hi]` is sorted. Test which, then decide whether the target lies inside the sorted half.
- 7 Duplicate-heavy rotated arrays (81, 154) cannot always decide which half is sorted — when `a[lo] == a[mid] == a[hi]` you must shrink by one from each end and lose the $O(\log n)$ guarantee on those inputs.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
4	Median of Two Sorted Arrays	Hard	13
162	Find Peak Element	Medium	8
278	First Bad Version	Easy	7
1539	Kth Missing Positive Number	Easy	5
240	Search a 2D Matrix II	Medium	6
852	Peak Index in a Mountain Array	Medium	4
81	Search in Rotated Sorted Array II	Medium	4
1287	Element Appearing More Than 25% In Sorted Array	Easy	2
744	Find Smallest Letter Greater Than Target	Easy	3
2089	Find Target Indices After Sorting Array	Easy	2
374	Guess Number Higher or Lower	Easy	2
888	Fair Candy Swap	Easy	2
154	Find Minimum in Rotated Sorted Array II	Hard	1
1237	Find Positive Integer Solution for a Given Equation	Medium	1
1351	Count Negative Numbers in a Sorted Matrix	Easy	1
275	H-Index II	Medium	1
1385	Find the Distance Value Between Two Arrays	Easy	1
1337	The K Weakest Rows in a Matrix	Easy	1
1608	Special Array With X Elements Greater Than or Equal X	Easy	1
2080	Range Frequency Queries	Medium	1

#	PROBLEM	LEVEL	ASKED
441	Arranging Coins	Easy	0
702	Search in a Sorted Array of Unknown Size †	Medium	0
1064	Fixed Point †	Easy	0
1885	Count Pairs in Two Arrays †	Medium	0
2300	Successful Pairs of Spells and Potions	Medium	0
2387	Median of a Row Wise Sorted Matrix †	Medium	0
2476	Closest Nodes Queries in a Binary Search Tree	Medium	0
2529	Maximum Count of Positive Integer and Negative Integer	Easy	0
2563	Count the Number of Fair Pairs	Medium	0

Binary Search on Rotated, Peak, and Bitonic Arrays

THE KERNEL

These arrays are not sorted, but they are *piecewise monotone*. Find the piece, then the ordinary binary search still works.

34

 PROBLEMS
MAPPED

4

EASY

26

MEDIUM

4

HARD

The Boundary Rules

- 1 Rotated sorted array, distinct: compare `a[mid]` with `a[hi]` to decide which side is sorted. `a[mid] < a[hi]` means the right half is sorted; otherwise the left half is sorted. Comparing with `a[lo]` instead works for the search variant but not for the minimum variant — pick one reference and stay consistent.
- 2 Rotated sorted array, minimum (153): `while lo < hi, if a[mid] < a[hi]: hi = mid else: lo = mid + 1, return a[lo]`. With duplicates (154) add `elif a[mid] == a[hi]: hi -= 1`.
- 3 Peak index in a mountain array (852) and find peak element (162) are the same line: `if a[mid] < a[mid+1]: lo = mid + 1 else: hi = mid`.
- 4 Bitonic / mountain array search: binary search the peak, then binary search each monotone half. Two Shape A calls.
- 5 Single element in a sorted array (540): binary search on the *parity of the index*. For index `i`, the pair partner is `i ^ 1`. If `a[mid] == a[mid ^ 1]` the single element is to the right, else to the left. XOR-indexing is the trick worth memorising.

- 6 2D sorted matrix (74 , fully sorted): treat it as a flat array of length $m * n$;
`row, col = divmod(mid, n)` .
- 7 2D sorted matrix (240 , rows and columns sorted): no binary search — start at the top-right and walk left/down. $O(m + n)$.
- 8 Kth smallest in a sorted matrix (378) and kth smallest pair distance (719) are binary searches on the *value* with a counting predicate. They belong to chapter 14.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
4	Median of Two Sorted Arrays	Hard	13
74	Search a 2D Matrix	Medium	10
378	Kth Smallest Element in a Sorted Matrix	Medium	8
1146	Snapshot Array	Medium	2
1891	Cutting Ribbons †	Medium	1
1428	Leftmost Column with at Least a One †	Medium	3
1283	Find the Smallest Divisor Given a Threshold	Medium	2
1150	Check If a Number Is Majority Element in a Sorted Array †	Easy	2
1237	Find Positive Integer Solution for a Given Equation	Medium	1
1300	Sum of Mutated Array Closest to Target	Medium	1
1182	Shortest Distance to Target Color †	Medium	1
702	Search in a Sorted Array of Unknown Size †	Medium	0
1064	Fixed Point †	Easy	0
1095	Find in Mountain Array	Hard	0
1631	Path With Minimum Effort	Medium	0
1855	Maximum Distance Between a Pair of Values	Medium	0
1979	Find Greatest Common Divisor of Array	Easy	0
2048	Next Greater Numerically Balanced Number	Medium	0
2141	Maximum Running Time of N Computers	Hard	0

#	PROBLEM	LEVEL	ASKED
2226	Maximum Candies Allocated to K Children	Medium	0
2592	Maximize Greatness of an Array	Medium	0
2772	Apply Operations to Make All Array Elements Equal to Zero	Medium	0
2845	Count of Interesting Subarrays	Medium	0
2981	Find Longest Special Substring That Occurs Thrice I	Medium	0
3020	Find the Maximum Number of Elements in Subset	Medium	0
3263	Convert Doubly Linked List to Array I †	Easy	0

Binary Search on the Answer

THE KERNEL

The array is not sorted — the *answer space* is. If you can write a predicate `feasible(x)` that is monotone (false, false, ..., true, true), then the smallest feasible `x` is a binary search, and you never have to be clever.

25

 PROBLEMS
MAPPED

1

EASY

19

MEDIUM

5

HARD

The Boundary Rules

- 1 Step 1 — name the answer variable. ‘Minimum capacity’, ‘minimum speed’, ‘minimum days’, ‘minimum largest sum’. If the question says *minimise the maximum* or *maximise the minimum*, it is almost always this pattern.
- 2 Step 2 — write `feasible(x) -> bool`. It must be monotone: making `x` more generous never turns a feasible answer infeasible. If you cannot argue monotonicity in one sentence, the pattern does not apply.
- 3 Step 3 — choose the shape. *Smallest feasible*: Shape B with `if feasible(mid): hi = mid else: lo = mid + 1`, return `lo`. *Largest feasible*: Shape B with the upper mid and `if feasible(mid): lo = mid else: hi = mid - 1`, return `lo`.
- 4 The largest-feasible form is the one people get wrong. It needs the upper mid `(lo + hi + 1) // 2`, because `lo = mid` must be able to make progress.

- 5 Step 4 — set the search bounds to a *provably feasible* `hi` and a `lo` that is either infeasible or the trivial answer. The single most common failure is a `hi` that is not actually feasible, which makes the binary search return a wrong boundary rather than crash.
- 6 Bounds by example: ship packages -> `lo = max(weights)`, `hi = sum(weights)`. Koko bananas -> `lo = 1`, `hi = max(piles)`. Split array largest sum -> `lo = max(nums)`, `hi = sum(nums)`.
- 7 The predicate is usually a single greedy left-to-right pass that counts how many buckets of size `x` are needed. Write it as its own function; do not inline it.
- 8 Complexity is $O(n \log(\text{range}))$. Say the range out loud in the interview — it is what proves the solution is fast enough.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
644	Maximum Average Subarray II †	Hard	1
774	Minimize Max Distance to Gas Station †	Hard	0
1102	Path With Maximum Minimum Value †	Medium	0
1231	Divide Chocolate †	Hard	0
1760	Minimum Limit of Balls in a Bag	Medium	0
1802	Maximum Value at a Given Index in a Bounded Array	Medium	0
1870	Minimum Speed to Arrive on Time	Medium	0
1898	Maximum Number of Removable Characters	Medium	0
2141	Maximum Running Time of N Computers	Hard	0
2226	Maximum Candies Allocated to K Children	Medium	0
2439	Minimize Maximum of Array	Medium	0
2517	Maximum Tastiness of Candy Basket	Medium	0
2594	Minimum Time to Repair Cars	Medium	0
2616	Minimize the Maximum Difference of Pairs	Medium	0
2861	Maximum Number of Alloys	Medium	0

#	PROBLEM	LEVEL	ASKED
3007	Maximum Number That Sum of the Prices Is Less Than or Equal to K	Medium	0
3065	Minimum Operations to Exceed Threshold Value I	Easy	0
3281	Maximize Score of Numbers in Ranges	Medium	0

Binary Search Plus Counting

THE KERNEL

To find the k th smallest value without enumerating, binary search the value and count how many elements are $\leq x$. The count is monotone in x , so the pattern applies.

16

PROBLEMS
MAPPED

0

EASY

9

MEDIUM

7

HARD

The Boundary Rules

- 1 The predicate is always $\text{count}(x) \geq k$ — 'at least k elements are $\leq x$ '. Find the smallest such x with Shape B.
- 2 **Kth Smallest Element in a Sorted Matrix** (378): $\text{count}(x)$ is a staircase walk from the bottom-left corner, $O(n)$ per call, $O(n \log(\max - \min))$ total.
- 3 **Find K-th Smallest Pair Distance** (719): sort, then $\text{count}(x)$ is a two-pointer count of pairs with distance $\leq x$, $O(n)$ per call.
- 4 **Median of Two Sorted Arrays** (4) is the crown jewel of this family. Binary search the *partition point* in the shorter array; the predicate is 'the left half contains no element greater than the right half's minimum'. Always binary search the shorter array so the complexity is $O(\log \min(m, n))$.
- 5 For 4, the sentinels $-\text{inf}$ and $+\text{inf}$ on the virtual array boundaries remove every edge case. Write them as $\text{float}(-\text{inf})$ / $\text{float}(\text{inf})$.
- 6 **Kth Smallest Prime Fraction** (786) and **Ugly Number III** (1201) are the same shape with a different `count`.

- 7 If `count(x)` itself needs a binary search, you are at $O(\log^2 n)$ — still fine, and still the intended solution for several problems in this chapter.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1439	Find the Kth Smallest Sum of a Matrix With Sorted Rows	Hard	1
644	Maximum Average Subarray II †	Hard	1
1508	Range Sum of Sorted Subarray Sums	Medium	1
668	Kth Smallest Number in Multiplication Table	Hard	0
786	K-th Smallest Prime Fraction	Medium	0
1102	Path With Maximum Minimum Value †	Medium	0
1918	Kth Smallest Subarray Sum †	Medium	0
2040	Kth Smallest Product of Two Sorted Arrays	Hard	0
2387	Median of a Row Wise Sorted Matrix †	Medium	0
2517	Maximum Tastiness of Candy Basket	Medium	0
2594	Minimum Time to Repair Cars	Medium	0
3116	Kth Smallest Amount With Single Denomination Combination	Hard	0

PART IV

Prefix Sums & Intervals

Turning a range question into a lookup. Prefix sums, difference arrays, and the interval scan that solves nine interval problems out of ten.

Prefix Sum and the Subarray-Sum Hash

THE KERNEL

$\text{sum}(i..j) = \text{pre}[j+1] - \text{pre}[i]$. So ‘find a subarray whose sum is k ’ becomes ‘find two prefix sums that differ by k ’ — a one-pass hash lookup.

56

 PROBLEMS
MAPPED

15

EASY

37

MEDIUM

4

HARD

The Boundary Rules

- 1 Build the prefix array with a leading zero: `pre = [0]`, for `x` in `a`: `pre.append(pre[-1] + x)`. The leading zero is not optional — it is what makes subarrays starting at index 0 work.
- 2 The counting form is a single pass with `seen = {0: 1}`. The `{0: 1}` seed is the same leading zero expressed as a hash entry. Forgetting it loses every subarray that starts at index 0.
- 3 Loop body: `cur += x`, then `ans += seen.get(cur - k, 0)`, then `seen[cur] += 1`. The order matters: query before insert, otherwise a zero-length subarray is counted when `k == 0`.
- 4 For the *first* index rather than a count, store the earliest index (`seen.setdefault(cur, i)`). For the *longest* subarray, always keep the earliest index — later duplicates can only shorten.

- 5 Prefix sums work with negative numbers; sliding windows do not. If the array can contain negatives, this chapter is the answer, not chapter 8.
- 6 Product of array except self (238) is a prefix product and a suffix product in two passes — the same idea with a different operator, and it avoids division entirely.
- 7 Subarray sum divisible by k (974) hashes `cur % k`; normalise with `% k` (Python's `%` already returns a non-negative result for positive `k`, unlike C++).
- 8 Contiguous array (525) maps `0 -> -1` and hashes the running sum; the earliest index of each sum gives the longest balanced subarray.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
53	Maximum Subarray	Medium	20
523	Continuous Subarray Sum	Medium	7
152	Maximum Product Subarray	Medium	6
1004	Max Consecutive Ones III	Medium	7
724	Find Pivot Index	Easy	9
304	Range Sum Query 2D - Immutable	Medium	5
325	Maximum Size Subarray Sum Equals k †	Medium	2
713	Subarray Product Less Than K	Medium	3
1099	Two Sum Less Than K †	Easy	2
1588	Sum of All Odd Length Subarrays	Easy	3
1854	Maximum Population Year	Easy	3
1658	Minimum Operations to Reduce X to Zero	Medium	2
548	Split Array with Equal Sum †	Hard	1
1732	Find the Highest Altitude	Easy	1
1991	Find the Middle Index in Array	Easy	1
1171	Remove Zero Sum Consecutive Nodes from Linked List	Medium	1
1314	Matrix Block Sum	Medium	1

#	PROBLEM	LEVEL	ASKED
1413	Minimum Value to Get Positive Step by Step Sum	Easy	1
1455	Check If a Word Occurs As a Prefix of Any Word in a Sentence	Easy	1
1480	Running Sum of 1d Array	Easy	1
1664	Ways to Make a Fair Array	Medium	1
1769	Minimum Number of Operations to Move All Balls to Each Box	Medium	1
2024	Maximize the Confusion of an Exam	Medium	1
848	Shifting Letters	Medium	1
644	Maximum Average Subarray II †	Hard	1
1371	Find the Longest Substring Containing Vowels in Even Counts	Medium	1
1422	Maximum Score After Splitting a String	Easy	1
2256	Minimum Average Difference	Medium	1
1829	Maximum XOR for Each Query	Medium	1
1961	Check If String Is a Prefix of Array	Easy	1
930	Binary Subarrays With Sum	Medium	0
1074	Number of Submatrices That Sum to Target	Hard	0
1124	Longest Well-Performing Interval	Medium	0
1416	Restore The Array	Hard	0
1442	Count Triplets That Can Form Two Arrays of Equal XOR	Medium	0
1590	Make Sum Divisible by P	Medium	0
1712	Ways to Split Array Into Three Subarrays	Medium	0
1744	Can You Eat Your Favorite Candy on Your Favorite Day?	Medium	0
1838	Frequency of the Most Frequent Element	Medium	0
1983	Widest Pair of Indices With Equal Range Sum †	Medium	0
2121	Intervals Between Identical Elements	Medium	0
2207	Maximize Number of Subsequences in a String	Medium	0
2270	Number of Ways to Split Array	Medium	0
2381	Shifting Letters II	Medium	0
2389	Longest Subsequence With Limited Sum	Easy	0

PREFIX SUM AND THE SUBARRAY-SUM HASH

#	PROBLEM	LEVEL	ASKED
2485	Find the Pivot Integer	Easy	0
2574	Left and Right Sum Differences	Easy	0
2680	Maximum OR	Medium	0
2906	Construct Product Matrix	Medium	0
3026	Maximum Good Subarray Sum	Medium	0

2D Prefix Sum

THE KERNEL

A 2D prefix array turns any axis-aligned rectangle sum into four look-ups. The inclusion-exclusion formula is the entire chapter.

17

 PROBLEMS
MAPPED

2

EASY

11

MEDIUM

4

HARD

The Boundary Rules

- 1 $\text{pre}[i+1][j+1] = a[i][j] + \text{pre}[i][j+1] + \text{pre}[i+1][j] - \text{pre}[i][j]$. Note the **minus** on the last term — it is the doubly-counted overlap.
- 2 Rectangle $(r1, c1)$ to $(r2, c2)$ inclusive is $\text{pre}[r2+1][c2+1] - \text{pre}[r1][c2+1] - \text{pre}[r2+1][c1] + \text{pre}[r1][c1]$. Memorise it as ‘bottom-right, minus above, minus left, plus the corner you removed twice’.
- 3 The prefix grid is $(m+1) \times (n+1)$ so the -1 boundary never needs an `if`. Do not optimise it away.
- 4 If the input is mutated between queries, 2D prefix sums are wrong — you need a Fenwick tree in two dimensions, or the constraints will tell you the queries are offline.
- 5 `Matrix Block Sum` (1314) is a straight rectangle-sum application. `Range Sum Query 2D - Immutable` (304) is the canonical statement of the pattern.
- 6 The subtraction form (`difference array` in 2D, chapter 17) is the inverse operation: it lets you add `v` to a rectangle in $O(1)$ and recover the grid with one 2D prefix pass.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1277	Count Square Submatrices with All Ones	Medium	2
308	Range Sum Query 2D - Mutable †	Medium	1
661	Image Smoother	Easy	1
1074	Number of Submatrices That Sum to Target	Hard	0
1292	Maximum Side Length of a Square with Sum Less than or Equal to Threshold	Medium	0
1444	Number of Ways of Cutting a Pizza	Hard	0
1504	Count Submatrices With All Ones	Medium	0
1738	Find Kth Largest XOR Coordinate Value	Medium	0
2132	Stamping the Grid	Hard	0
2428	Maximum Sum of an Hourglass	Medium	0
2536	Increment Submatrices by One	Medium	0
2679	Sum in a Matrix	Medium	0
2732	Find a Good Subset of the Matrix	Hard	0
3142	Check if Grid Satisfies Conditions	Easy	0
3212	Count Submatrices With Equal Frequency of X and Y	Medium	0

Difference Array

THE KERNEL

A prefix sum answers ‘what is the sum over a range’. A difference array answers ‘add v over a range, then read every cell’ — both in $O(1)$ per range update.

21

 PROBLEMS
MAPPED

3

EASY

13

MEDIUM

5

HARD

The Boundary Rules

- 1 To add v on $[l, r]$: `diff[l] += v; diff[r+1] -= v`. Then a single prefix pass over `diff` recovers the array.
- 2 The `r + 1` slot must be allocated. Size the diff array as `n + 1` whenever a range can end at `n - 1`.
- 3 For ‘maximum overlap of intervals’ you do not even need the prefix pass: sort the `+v` and `-v` events by coordinate and sweep. That is chapter 19.
- 4 `Corporate Flight Bookings` (1109) and `Car Pooling` (1094) are the same two lines. `Range Addition` (370, paid) is the canonical statement.
- 5 2D difference arrays use the same inclusion-exclusion as 2D prefix sums, with the signs flipped: `diff[r2+1][c2+1] += v` for the four corners.
- 6 A difference array is the right tool when updates are *offline* (you know them all before you read) and queries are *after* all updates. If queries are interleaved with updates, you need a Fenwick tree.

- 7** **Minimum Number of Increments on Subarrays (1526)** is a difference-array problem in disguise: the answer is the sum of the positive jumps of the array.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1094	Car Pooling	Medium	7
370	Range Addition †	Medium	2
598	Range Addition II	Easy	1
1893	Check if All the Integers in a Range Are Covered	Easy	1
731	My Calendar II	Medium	0
798	Smallest Rotation with Highest Score	Hard	0
1109	Corporate Flight Bookings	Medium	0
1589	Maximum Sum Obtained of Any Permutation	Medium	0
1674	Minimum Moves to Make Array Complementary	Medium	0
1943	Describe the Painting	Medium	0
2021	Brightest Position on Street †	Medium	0
2132	Stamping the Grid	Hard	0
2237	Count Positions on Street With Required Brightness †	Medium	0
2251	Number of Flowers in Full Bloom	Hard	0
2381	Shifting Letters II	Medium	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2536	Increment Submatrices by One	Medium	0
2772	Apply Operations to Make All Array Elements Equal to Zero	Medium	0
2848	Points That Intersect With Cars	Easy	0
3329	Count Substrings With K-Frequency Characters II †	Hard	0

Intervals - Merge, Insert, Intersect

THE KERNEL

Sort by start. Then a single left-to-right scan where you either extend the current interval or flush it. Nine out of ten interval problems are this scan with a different flush condition.

71

 PROBLEMS
MAPPED

28

EASY

31

MEDIUM

12

HARD

The Boundary Rules

- 1 Always sort first. `intervals.sort(key=lambda x: x[0])`. Sorting by end instead is correct only for a specific family (see the greedy chapter).
- 2 The scan is: if `start <= cur_end` then `cur_end = max(cur_end, end)`; else flush `[cur_start, cur_end]` and start a new interval.
- 3 The `max(cur_end, end)` is mandatory. Without it, a nested interval (`[1,10]` then `[2,3]`) truncates the merged result.
- 4 **Overlap test.** Closed intervals overlap iff `a.start <= b.end` and `b.start <= a.end`. For half-open intervals the test is `a.start < b.end` and `b.start < a.end`. Getting this boundary wrong is the most common interval bug, so decide up front which convention the problem uses.
- 5 **Meeting rooms** (252 , 253): the minimum number of rooms is the maximum number of simultaneously open intervals — a sweep over sorted start and end events, not a merge. With a min-heap of end times it is $O(n \log n)$ and reads more naturally in an interview.

- 6 **Insert interval** (57): three phases — copy intervals that end before the new one starts, absorb every interval that overlaps the new one by expanding it, then copy the rest. Do not try to do it in one conditional.
- 7 **Non-overlapping intervals** (435) and **minimum arrows** (452) sort by *end*, because the greedy choice is ‘keep the interval that finishes earliest’. Merge problems sort by start; removal problems sort by end. That single distinction is worth memorising.
- 8 **Interval list intersections** (986) is the two-pointer scan: advance whichever interval ends first.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1094	Car Pooling	Medium	7
370	Range Addition †	Medium	2
896	Monotonic Array	Easy	2
616	Add Bold Tag in String †	Medium	1
849	Maximize Distance to Closest Person	Medium	3
729	My Calendar I	Medium	3
163	Missing Ranges †	Easy	1
414	Third Maximum Number	Easy	3
2016	Maximum Difference Between Increasing Elements	Easy	2
1313	Decompress Run-Length Encoded List	Easy	2
1470	Shuffle the Array	Easy	2
485	Max Consecutive Ones	Easy	2
674	Longest Continuous Increasing Subsequence	Easy	1
1200	Minimum Absolute Difference	Easy	2
665	Non-decreasing Array	Medium	1
1431	Kids With the Greatest Number of Candies	Easy	1
436	Find Right Interval	Medium	2

#	PROBLEM	LEVEL	ASKED
352	Data Stream as Disjoint Intervals	Hard	1
717	1-bit and 2-bit Characters	Easy	1
758	Bold Words in String †	Medium	1
941	Valid Mountain Array	Easy	1
1299	Replace Elements with Greatest Element on Right Side	Easy	1
1491	Average Salary Excluding the Minimum and Maximum Salary	Easy	1
1752	Check if Array Is Sorted and Rotated	Easy	1
1779	Find Nearest Point That Has the Same X or Y Coordinate	Easy	1
1909	Remove One Element to Make the Array Strictly Increasing	Easy	1
495	Teemo Attacking	Easy	1
1184	Distance Between Bus Stops	Easy	1
1502	Can Make Arithmetic Progression From Sequence	Easy	1
1619	Mean of Array After Removing Some Elements	Easy	1
2012	Sum of Beauty in the Array	Medium	1
2057	Smallest Index With Equal Value	Easy	1
2155	All Divisions With the Highest Score of a Binary Array	Medium	1
2210	Count Hills and Valleys in an Array	Easy	1
228	Summary Ranges	Easy	0
715	Range Module	Hard	0
731	My Calendar II	Medium	0
732	My Calendar III	Hard	0
759	Employee Free Time †	Hard	0
1109	Corporate Flight Bookings	Medium	0
1229	Meeting Scheduler †	Medium	0
1272	Remove Interval †	Medium	0
1288	Remove Covered Intervals	Medium	0
1326	Minimum Number of Taps to Open to Water a Garden	Hard	0
1851	Minimum Interval to Include Each Query	Hard	0

#	PROBLEM	LEVEL	ASKED
1871	Jump Game VII	Medium	0
1943	Describe the Painting	Medium	0
1982	Find Array Given Subset Sums	Hard	0
2054	Two Best Non-Overlapping Events	Medium	0
2184	Number of Ways to Build Sturdy Brick Wall †	Medium	0
2231	Largest Number After Digit Swaps by Parity	Easy	0
2251	Number of Flowers in Full Bloom	Hard	0
2274	Maximum Consecutive Floors Without Special Floors	Medium	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2446	Determine if Two Events Have Conflict	Easy	0
2462	Total Cost to Hire K Workers	Medium	0
2470	Number of Subarrays With LCM Equal to K	Medium	0
2509	Cycle Length Queries in a Tree	Hard	0
2589	Minimum Time to Complete All Tasks	Hard	0
2655	Find Maximal Uncovered Ranges †	Medium	0
2848	Points That Intersect With Cars	Easy	0
2953	Count Complete Substrings	Hard	0
3169	Count Days Without Meetings	Medium	0
3209	Number of Subarrays With AND Value of K	Hard	0

Sweep Line and Event Counting

THE KERNEL

Turn every interval into two events (open, close). Sort the events. Walk them once, maintaining a running count. The answer is an extremum of that count.

27

 PROBLEMS
MAPPED

4

EASY

14

MEDIUM

9

HARD

The Boundary Rules

- 1 Sort key for events: for ‘maximum concurrent’ the order of a close and an open at the *same* coordinate decides the answer. Touching intervals (`[1,2]` , `[2,3]`) do not overlap, so a close must be processed before an open at the same coordinate. Encode this by sorting closes before opens at equal coordinates, or by using half-open coordinates.
- 2 The classic trick is to encode `close` as `+1` and `open` as `-1` and sort the tuple, so that at equal coordinates the close sorts first.
- 3 **Skyline** (`218`) is a sweep with a multiset of active heights and a max-heap (or a sorted container) to find the current maximum. Emit a key point only when the maximum changes.
- 4 **Meeting rooms II** (`253`) is the simplest sweep; `My Calendar` (`729 / 731`) is the same sweep with insertion.
- 5 **Rectangle area** (`223`) and **rectangle overlap** (`836`) are inclusion-exclusion rather than sweeps — recognise them and skip the machinery.

- 6 Sweep line problems are often solvable with a heap instead. If the coordinate range is small, a difference array (chapter 17) is simpler and faster.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
435	Non-overlapping Intervals	Medium	3
252	Meeting Rooms †	Easy	2
391	Perfect Rectangle	Hard	2
452	Minimum Number of Arrows to Burst Balloons	Medium	3
223	Rectangle Area	Medium	1
352	Data Stream as Disjoint Intervals	Hard	1
495	Teemo Attacking	Easy	1
731	My Calendar II	Medium	0
759	Employee Free Time †	Hard	0
850	Rectangle Area II	Hard	0
1229	Meeting Scheduler †	Medium	0
1272	Remove Interval †	Medium	0
1851	Minimum Interval to Include Each Query	Hard	0
1943	Describe the Painting	Medium	0
2054	Two Best Non-Overlapping Events	Medium	0
2184	Number of Ways to Build Sturdy Brick Wall †	Medium	0
2251	Number of Flowers in Full Bloom	Hard	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2503	Maximum Number of Points From Grid Queries	Hard	0
2589	Minimum Time to Complete All Tasks	Hard	0
2655	Find Maximal Uncovered Ranges †	Medium	0
2848	Points That Intersect With Cars	Easy	0
3169	Count Days Without Meetings	Medium	0

PART V

Hashing

Hash maps and sets. The complement lookup, the frequency counter, the canonical key, and the design problems that are a hash plus one more structure.

Hash Map - Complement Lookup

THE KERNEL

You need `b` such that `f(a, b) == target`. Instead of searching for `b`, store every `a` you have seen and look up the one value of `b` that would close the equation. One pass, one dictionary.

31PROBLEMS
MAPPED**14**

EASY

14

MEDIUM

3

HARD

The Boundary Rules

- 1 Loop body is exactly two lines: `if target - x in seen: return ...` then `seen[x] = i`. **Query before insert.** If you insert first, `x + x == target` matches itself and you report an element pairing with itself.
- 2 Store the *index* if the problem wants indices; store the *count* if the problem counts pairs. The dictionary value is the answer payload — decide what it is before writing the loop.
- 3 For ‘count the pairs’, the loop is `ans += seen.get(x - k, 0)` then `seen[x] += 1`. For ‘does a pair exist’, it is a set, not a dict.
- 4 `Two Sum (1)` with a sorted array does not need a hash at all — it is chapter 5. Recognise which of the two the interviewer wants: the hash version is $O(n)$ but needs $O(n)$ memory; the two-pointer version is $O(n \log n)$ but $O(1)$ memory. Saying this trade-off out loud is worth as much as the code.
- 5 `Contains Duplicate II (219)` needs the *last seen index*, not just membership — so the dict maps value to index, and the test is `i - seen[x] <= k`.

- 6 Contains Duplicate III (220) cannot be solved by a plain hash; it needs value bucketing ($x // (t+1)$) or a sorted container. Bucketing is the interview answer: two values are within t if they land in the same bucket or in adjacent buckets.
- 7 Divisibility pair problems (1010 , 1497) hash the *remainder*, not the value: pair r with $(k - r) \% k$. The $\% k$ on the complement is what makes $r == 0$ work.
- 8 Any problem that says ‘pair’, ‘two numbers’, ‘complement’, ‘difference’, or ‘sum to’ should trigger this chapter before anything else.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
202	Happy Number	Easy	6
149	Max Points on a Line	Hard	4
454	4Sum II	Medium	4
697	Degree of an Array	Easy	5
1817	Finding the Users Active Minutes	Medium	1
1331	Rank Transform of an Array	Easy	2
1711	Count Good Meals	Medium	3
653	Two Sum IV - Input is a BST	Easy	2
1213	Intersection of Three Sorted Arrays †	Easy	1
1128	Number of Equivalent Domino Pairs	Easy	2
554	Brick Wall	Medium	1
1640	Check Array Formation Through Concatenation	Easy	1
1679	Max Number of K-Sum Pairs	Medium	1
170	Two Sum III - Data structure design †	Easy	1
594	Longest Harmonious Subsequence	Easy	0
1442	Count Triplets That Can Form Two Arrays of Equal XOR	Medium	0
2001	Number of Pairs of Interchangeable Rectangles	Medium	0

#	PROBLEM	LEVEL	ASKED
2196	Create Binary Tree From Descriptions	Medium	0
2342	Max Sum of a Pair With Equal Sum of Digits	Medium	0
2354	Number of Excellent Pairs	Hard	0
2610	Convert an Array Into a 2D Array With Conditions	Medium	0
2815	Max Pair Sum in an Array	Easy	0
2964	Number of Divisible Triplet Sums †	Medium	0
3162	Find the Number of Good Pairs I	Easy	0
3207	Maximum Points After Enemy Battles	Medium	0

Hash Map - Frequency and Anagrams

THE KERNEL

Two strings are anagrams iff their character counts are equal. Once you think in counts instead of sequences, half of the string problems collapse to a single Counter comparison.

40

 PROBLEMS
MAPPED

18

EASY

21

MEDIUM

1

HARD

The Boundary Rules

- 1 `Counter(a) == Counter(b)` is the whole of `Valid Anagram` (242). Do not write a sorting solution unless the interviewer asks for $O(1)$ extra space — `sorted(a) == sorted(b)` is $O(n \log n)$ and strictly worse.
- 2 For fixed-alphabet comparisons, a `list` of 26 ints beats a Counter: indexing is faster than hashing and equality is a list compare. Use `ord(c) - ord('a')`.
- 3 **Increment/decrement trick.** Instead of building two counters, build one: increment for string A, decrement for string B, then check that every bucket is zero. This halves the memory and is the shape interviewers expect.
- 4 `First Unique Character` (387) is `freq == 1` plus a second pass in order. Remember the second pass — a hash alone does not preserve order.
- 5 `Ransom Note` (383) is a one-directional containment test: every bucket must end `>= 0`. Do not use `==`.

- 6 `Isomorphic Strings` (205) and `Word Pattern` (290) are **bijective** mappings, which means you need two dicts (or one dict plus one set). One dict alone passes the forward direction and fails the reverse.
- 7 `Sort Characters By Frequency` (451) is a counting sort over buckets, not a heap: bucket by count, then emit from the highest bucket down. That is $O(n)$ and beats `heapq` on both time and code length.
- 8 `Minimum Window Substring` (76) lives in chapter 8, but its state is a frequency map plus a `formed` counter. Recognise the shared machinery.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
49	Group Anagrams	Medium	19
791	Custom Sort String	Medium	2
692	Top K Frequent Words	Medium	9
438	Find All Anagrams in a String	Medium	10
767	Reorganize String	Medium	6
1207	Unique Number of Occurrences	Easy	6
567	Permutation in String	Medium	7
1347	Minimum Number of Steps to Make Two Strings Anagram	Medium	4
1481	Least Number of Unique Integers after K Removals	Medium	4
358	Rearrange String k Distance Apart †	Hard	4
1910	Remove All Occurrences of a Substring	Medium	3
1002	Find Common Characters	Easy	1
1160	Find Words That Can Be Formed by Characters	Easy	2
811	Subdomain Visit Count	Medium	2
1297	Maximum Number of Occurrences of a Substring	Medium	1
1370	Increasing Decreasing String	Easy	1
2023	Number of Pairs of Strings With Concatenation Equal to Target	Medium	1
1189	Maximum Number of Balloons	Easy	1

#	PROBLEM	LEVEL	ASKED
760	Find Anagram Mappings †	Easy	1
1078	Occurrences After Bigram	Easy	1
1781	Sum of Beauty of All Substrings	Medium	1
2053	Kth Distinct String in an Array	Easy	1
2068	Check Whether Two Strings are Almost Equivalent	Easy	1
2085	Count Common Words With One Occurrence	Easy	1
1405	Longest Happy String	Medium	0
1657	Determine if Two Strings Are Close	Medium	0
1941	Check if All Characters Have Equal Number of Occurrences	Easy	0
2287	Rearrange Characters to Make Target String	Easy	0
2405	Optimal Partition of String	Medium	0
2423	Remove Letter To Equalize Frequency	Easy	0
2645	Minimum Additions to Make Valid String	Medium	0
2981	Find Longest Special Substring That Occurs Thrice I	Medium	0
3121	Count the Number of Special Characters II	Medium	0
3144	Minimum Substring Partition of Equal Character Frequency	Medium	0

Hash Map - Canonical-Key Grouping

THE KERNEL

Grouping problems are always the same two lines: compute a canonical key that is identical for every member of a group, then

`groups[key].append(item)`. The entire difficulty is designing the key.

15PROBLEMS
MAPPED**3**

EASY

12

MEDIUM

0

HARD

The Boundary Rules

- 1 The key must be *hashable*. A `list` is not; a `tuple` is; a `frozenset` is; a `str` is. Choose the cheapest hashable form.
- 2 `Group Anagrams` (49): the key is `tuple(sorted(word))` or a 26-length count tuple. The count tuple is $O(n)$ instead of $O(n \log n)$ and is the better answer if the alphabet is fixed.
- 3 `Group Shifted Strings` (249): the key is the *difference vector* between consecutive characters, modulo 26. A single string cannot be the key because the absolute letters differ.
- 4 `Find and Replace Pattern` (890): normalise the word to `abbc`-form (map each new character to the next unused letter) and group by the normalised form. Normalisation is the same bijection check as chapter 21, applied to a whole string.

- 5 Find Common Characters (1002): the key is not what is grouped — you take the *intersection* of 26-bucket counters. Recognise the difference between grouping and intersecting.
- 6 Longest Consecutive Sequence (128) is *not* a grouping problem even though it looks like one; it is a set-expansion problem and belongs to chapter 23.
- 7 When the key is a tuple of counts, consider encoding it as a string with a separator (`','.join(map(str, counts))`). Python tuples hash fine but string keys are easier to debug and print in an interview.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1657	Determine if Two Strings Are Close	Medium	0
2131	Longest Palindrome by Concatenating Two Letter Words	Medium	0
2342	Max Sum of a Pair With Equal Sum of Digits	Medium	0
2405	Optimal Partition of String	Medium	0
2506	Count Pairs Of Similar Strings	Easy	0
2564	Substring XOR Queries	Medium	0
2610	Convert an Array Into a 2D Array With Conditions	Medium	0
2811	Check if it is Possible to Split Array	Medium	0
2860	Happy Students	Medium	0
3006	Find Beautiful Indices in the Given Array I	Medium	0
3110	Score of a String	Easy	0

Hash Set - Dedup, Consecutive Runs, and Cycles

THE KERNEL

A set answers ‘have I seen this’ in $O(1)$. Three families fall out of that: deduplication, run-expansion from run starts, and cycle detection.

29PROBLEMS
MAPPED**17**

EASY

11

MEDIUM

1

HARD

The Boundary Rules

- 1 **Run expansion.** `Longest Consecutive Sequence` (128) only expands from a *run start*: `if x - 1 not in s`. Without that guard you re-walk the same run and degrade to $O(n^2)$. The guard is the entire solution.
- 2 128 must be solved in $O(n)$, which rules out sorting. The set plus run-start guard is what makes it $O(n)$ amortised.
- 3 **Index-as-value trick.** `Find All Numbers Disappeared in an Array` (448), `Find All Duplicates` (442), `Set Mismatch` (645) and `First Missing Positive` (41) all use the array itself as the hash table, marking `nums[abs(x) - 1]` negative. This is $O(1)$ extra space and is the answer interviewers are fishing for.
- 4 41 adds one requirement: the answer is in `[1, n+1]`, so you can ignore out-of-range values entirely. Say that sentence out loud — it is the insight the problem is testing.

- 5** **Cycle detection.** `Happy Number` (202) is Floyd's tortoise and hare on the function `f(n) = sum of squared digits`. A visited set also works and is easier to justify; Floyd's is $O(1)$ space.
- 6** `Intersection of Two Arrays` (349 / 350) — the set version returns unique values, the Counter version returns multiplicity. Read the problem statement for which one it wants.
- 7** When the universe is small and bounded (e.g. `1..n` or `'a'..'z'`), a boolean array beats a set on constant factor and is usually clearer.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1650	Lowest Common Ancestor of a Binary Tree III †	Medium	6
287	Find the Duplicate Number	Medium	7
268	Missing Number	Easy	8
136	Single Number	Easy	9
349	Intersection of Two Arrays	Easy	7
350	Intersection of Two Arrays II	Easy	5
137	Single Number II	Medium	1
260	Single Number III	Medium	1
1790	Check if One String Swap Can Make Strings Equal	Easy	1
457	Circular Array Loop	Medium	1
2215	Find the Difference of Two Arrays	Easy	1
2260	Minimum Consecutive Cards to Pick Up	Medium	1
2032	Two Out of Three	Easy	1
575	Distribute Candies	Easy	0
961	N-Repeated Element in Size 2N Array	Easy	0
1647	Minimum Deletions to Make Character Frequencies Unique	Medium	0
2367	Number of Arithmetic Triplets	Easy	0
2442	Count Number of Distinct Integers After Reverse Operations	Medium	0

HASH SET - DEDUP, CONSECUTIVE RUNS, AND CYCLES

#	PROBLEM	LEVEL	ASKED
2549	Count Distinct Numbers on Board	Easy	0
2717	Semi-Ordered Permutation	Easy	0
2965	Find Missing and Repeated Values	Easy	0
2996	Smallest Missing Integer Greater Than Sequential Prefix Sum	Easy	0
3143	Maximum Points Inside the Square	Medium	0

Hash-Based Design (LRU, LFU, O(1) Random)

THE KERNEL

Design problems are hash-map-plus-one-more-structure. The hash gives $O(1)$ lookup; the second structure gives $O(1)$ *ordering*. Pick the second structure from the eviction rule.

53

 PROBLEMS
MAPPED

9

EASY

31

MEDIUM

13

HARD

The Boundary Rules

- 1 **LRU (146)** = dict + doubly linked list. The dict maps key to node; the list holds recency with a sentinel head (most recent) and sentinel tail (least recent). Use **two sentinel nodes** — it removes every `if node.prev is None` branch and is the difference between a 40-line and an 80-line answer.
- 2 **LFU (460)** = dict + per-frequency ordered sets + a `min_freq` counter. You must maintain `min_freq` incrementally: on insert it becomes 1, and on eviction you increment it only when the evicted bucket empties. Recomputing `min(freqs)` costs $O(n)$ and fails the follow-up.
- 3 **O(1) random (380)** = list of values + dict value-to-index. Delete is `swap with last, pop, fix the dict`. That swap-with-last step is the whole trick; without it deletion is $O(n)$.
- 4 **381** (duplicates allowed) makes the dict value a *set* of indices, and deletion picks an arbitrary index from that set. Same swap-with-last idea.

- 5 **Time-based store (981)** = dict of key to parallel lists of `(timestamp, value)`, plus `bisect_right` for the lookup. Recognise this as binary search wearing a design costume.
- 6 **Snapshot array (1146)** = per-index list of `(snap_id, value)` and a global snapshot counter. Only record a value when it actually changes, otherwise the snapshots blow up.
- 7 **Twitter (355)** = per-user tweet list + follow set + heap merge for the news feed. The feed is `Merge k Sorted Lists` in disguise.
- 8 **Authentication manager (1797)** = dict of token to expiry plus a lazy-expiry sweep. Lazy expiry (check on access) beats eager expiry (a timer thread) in every interview.
- 9 The universal interview rule for this chapter: **state the two data structures and the invariant that links them before writing any code**. Candidates who start typing lose.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
346	Moving Average from Data Stream †	Easy	4
359	Logger Rate Limiter †	Easy	6
588	Design In-Memory File System †	Hard	6
716	Max Stack †	Hard	6
362	Design Hit Counter †	Medium	7
460	LFU Cache	Hard	9
706	Design HashMap	Easy	8
2034	Stock Price Fluctuation	Medium	1
1656	Design an Ordered Stream	Easy	1
1472	Design Browser History	Medium	4
432	All O'one Data Structure	Hard	3
1352	Product of the Last K Numbers	Medium	3
381	Insert Delete GetRandom O(1) - Duplicates allowed	Hard	2

#	PROBLEM	LEVEL	ASKED
355	Design Twitter	Medium	3
1429	First Unique Number †	Medium	2
705	Design HashSet	Easy	2
1823	Find the Winner of the Circular Game	Medium	2
379	Design Phone Directory †	Medium	2
635	Design Log Storage System †	Medium	1
631	Design Excel Sum Formula †	Hard	1
1670	Design Front Middle Back Queue	Medium	1
683	K Empty Slots †	Hard	2
604	Design Compressed String Iterator †	Easy	1
1804	Implement Trie II (Prefix Tree) †	Medium	1
950	Reveal Cards In Increasing Order	Medium	1
2073	Time Needed to Buy Tickets	Easy	1
933	Number of Recent Calls	Easy	1
1756	Design Most Recently Used Queue †	Medium	1
1797	Design Authentication Manager	Medium	1
641	Design Circular Deque	Medium	0
707	Design Linked List	Medium	0
1244	Design A Leaderboard †	Medium	0
1500	Design a File Sharing System †	Medium	0
1600	Throne Inheritance	Medium	0
1622	Fancy Sequence	Hard	0
1865	Finding Pairs With a Certain Sum	Medium	0
1912	Design Movie Rental System	Hard	0
2013	Detect Squares	Medium	0
2227	Encrypt and Decrypt Strings	Hard	0
2254	Design Video Sharing Platform †	Hard	0
2296	Design a Text Editor	Hard	0

#	PROBLEM	LEVEL	ASKED
2336	Smallest Number in Infinite Set	Medium	0
2353	Design a Food Rating System	Medium	0
2408	Design SQL	Medium	0
2502	Design Memory Allocator	Medium	0
2671	Frequency Tracker	Medium	0
2958	Length of Longest Subarray With at Most K Frequency	Medium	0
3062	Winner of the Linked List Game †	Easy	0
3116	Kth Smallest Amount With Single Denomination Combination	Hard	0

PART VI

Stacks & Queues

Stacks and deques. Matching, monotonicity, and the histogram family that spans seven problems.

Stack - Matching and Parsing

THE KERNEL

A stack handles anything with nested, well-formed structure. Push on an opener, pop on a closer, and the stack content at any moment is the set of still-open scopes.

35PROBLEMS
MAPPED**4**

EASY

19

MEDIUM

12

HARD

The Boundary Rules

- 1 **Valid Parentheses** (20) is three lines: push openers; on a closer, check `stack` and `stack[-1] == match[c]` then pop; at the end require `not stack`.
- 2 **Empty-stack guard.** `if not stack: return False` before the pop is mandatory. Skipping it turns a wrong answer into an `IndexError`, which looks worse in an interview than returning the wrong boolean.
- 3 Map closers to openers with a dict literal `{')': '(', ']': '[', '}': '{'}`. Writing a four-branch `if/elif` chain instead is the most common style deduction in this chapter.
- 4 **Minimum Add to Make Valid** (921) does not need a stack at all — a single counter suffices: `need_open` for unmatched closers and `open_count` for unmatched openers. Recognise when the stack collapses to a counter.
- 5 **Longest Valid Parentheses** (32) is *not* a counter problem. Use a stack of indices with `-1` pushed first, or a DP The `-1` sentinel is the trick worth memorising: `i - stack[-1]` is the current valid length.

- 6 **Expression evaluation order.** Infix (224 , 227) requires a precedence rule; postfix (150) does not. In postfix, push operands and apply the operator to the top two — and remember the operand order is `second op first`, which is the classic bug with subtraction and division.
- 7 **Basic Calculator II** (227) needs `+ - /` precedence. Two stacks (values and operators) with a precedence table, or the 'apply all pending' and `/` immediately' single-pass trick. The single pass is shorter and less error-prone.
- 8 **Decode String** (394) is two stacks — one for the repeat counts and one for the partial strings. Do not try to encode both in one stack.
- 9 **Exclusive Time of Functions** (636) is the same nesting idea on log events; the stack holds `(function_id, start_time)`.
- 10 For bracket problems with more than one bracket type, always re-check the type on pop. A stack that only counts depth passes `{ [ ] }`, which is invalid.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1249	Minimum Remove to Make Valid Parentheses	Medium	7
921	Minimum Add to Make Parentheses Valid	Medium	4
22	Generate Parentheses	Medium	12
224	Basic Calculator	Hard	9
301	Remove Invalid Parentheses	Hard	2
678	Valid Parenthesis String	Medium	5
856	Score of Parentheses	Medium	4
247	Strobogrammatic Number II †	Medium	3
1541	Minimum Insertions to Balance a Parentheses String	Medium	2
1614	Maximum Nesting Depth of the Parentheses	Easy	2
248	Strobogrammatic Number III †	Hard	1
591	Tag Validator	Hard	2
1190	Reverse Substrings Between Each Pair of Parentheses	Medium	2

#	PROBLEM	LEVEL	ASKED
1021	Remove Outermost Parentheses	Easy	1
1807	Evaluate the Bracket Pairs of a String	Medium	1
2116	Check if a Parentheses String Can Be Valid	Medium	1
385	Mini Parser	Medium	1
439	Ternary Expression Parser †	Medium	1
726	Number of Atoms	Hard	0
736	Parse Lisp Expression	Hard	0
772	Basic Calculator III †	Hard	0
1096	Brace Expansion II	Hard	0
1106	Parsing A Boolean Expression	Hard	0
1111	Maximum Nesting Depth of Two Valid Parentheses Strings	Medium	0
1739	Building Boxes	Hard	0
1963	Minimum Number of Swaps to Make the String Balanced	Medium	0
2197	Replace Non-Coprime Numbers in Array	Hard	0
2696	Minimum String Length After Removing Substrings	Easy	0
2755	Deep Merge of Two Objects †	Medium	0

Monotonic Stack – Next Greater / Smaller

THE KERNEL

Keep a stack whose values are monotonically decreasing (for ‘next greater’) or increasing (for ‘next smaller’). When a new element breaks the monotonicity, it is the answer for everything it pops. Each index is pushed once and popped once — $O(n)$.

43PROBLEMS
MAPPED**2**

EASY

29

MEDIUM

12

HARD

The Boundary Rules

- 1 Direction rule.** ‘Next greater’ -> keep the stack *decreasing* -> pop while `stack` and `a[stack[-1]] < a[i]`. ‘Next smaller’ -> keep it *increasing* -> pop while `a[stack[-1]] > a[i]`. Flip both the comparison and the mental model together; mixing them up is the standard error.
- 2 Strict or non-strict?** Use `<` when duplicates should also be popped and `<=` when they should be kept. For `Daily Temperatures` (739) and `Next Greater Element I` (496) the strict `<` is correct. For `Sum of Subarray Minimums` (907) you need *both* a strict and a non-strict pass to handle duplicates correctly.
- 3 Store indices, not values**, whenever the answer is a distance, a width, or a position. `i - stack[-1]` is the pattern; storing values loses the index and forces a second lookup.
- 4** Push the current index after the pop loop, unconditionally. Do not branch on whether the stack is empty.

- 5 **Circular arrays** (503) iterate `for i in range(2 * n)` and take `a[i % n]`, with the loop bound at `2n` and the answer filled at `i < n` only.
- 6 **Sentinel trick.** For 'largest rectangle' (84) append a virtual `0` at the end of the heights so every bar gets popped and no post-loop flush is needed. This removes an entire cleanup loop and is worth memorising.
- 7 **Contribution counting.** 907 , 2104 , 1856 and 2281 do not ask for the next-greater element — they ask for the *sum over all subarrays* of the min/max. The trick is to count, for each index, how many subarrays have it as the minimum: `left_span right_span a[i]`. Compute `left_span` and `right_span` with two monotonic passes.
- 8 For 907 the duplicate handling is subtle: one side must use strict comparison and the other non-strict, or subarrays containing equal elements get counted twice. State this explicitly — it is the follow-up question.
- 9 **Lexicographic monotonic stack.** Remove K Digits (402), Remove Duplicate Letters (316), Find the Most Competitive Subsequence (1673) and Smallest Subsequence of Distinct Characters (1081) keep the stack *increasing* and pop while 'the new digit is smaller AND we still have removals left'. The extra condition (`k > 0` , or a `last_occurrence` check) is the only difference between them.
- 10 316 and 1081 are the *same problem*. 402 and 1673 are the same problem with a different stopping condition. Four problems, one template.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1996	The Number of Weak Characters in the Game	Medium	3
901	Online Stock Span	Medium	3
85	Maximal Rectangle	Hard	3
581	Shortest Unsorted Continuous Subarray	Medium	2
1019	Next Greater Node In Linked List	Medium	2
1574	Shortest Subarray to be Removed to Make Array Sorted	Medium	2
456	132 Pattern	Medium	1

#	PROBLEM	LEVEL	ASKED
1526	Minimum Number of Increments on Subarrays to Form a Target Array	Hard	1
321	Create Maximum Number	Hard	2
654	Maximum Binary Tree	Medium	1
769	Max Chunks To Make Sorted	Medium	1
1475	Final Prices With a Special Discount in a Shop	Easy	1
1673	Find the Most Competitive Subsequence	Medium	1
1717	Maximum Score From Removing Substrings	Medium	1
1966	Binary Searchable Numbers in an Unsorted Array †	Medium	1
962	Maximum Width Ramp	Medium	0
1063	Number of Valid Subarrays †	Hard	0
1081	Smallest Subsequence of Distinct Characters	Medium	0
1124	Longest Well-Performing Interval	Medium	0
1504	Count Submatrices With All Ones	Medium	0
1856	Maximum Subarray Min-Product	Medium	0
1950	Maximum of Minimum Values in All Subarrays †	Medium	0
2030	Smallest K-Length Subsequence With Occurrences of a Letter	Hard	0
2104	Sum of Subarray Ranges	Medium	0
2281	Sum of Total Strength of Wizards	Hard	0
2334	Subarray With Elements Greater Than Varying Threshold	Hard	0
2345	Finding the Number of Visible Mountains †	Medium	0
2355	Maximum Number of Books You Can Take †	Hard	0
2454	Next Greater Element IV	Hard	0
2487	Remove Nodes From Linked List	Medium	0
2556	Disconnect Path in a Binary Matrix by at Most One Flip	Medium	0
2737	Find the Closest Marked Node †	Medium	0
2863	Maximum Length of Semi-Decreasing Subarrays †	Medium	0
2945	Find Maximum Non-decreasing Array Length	Hard	0

MONOTONIC STACK - NEXT GREATER / SMALLER

#	PROBLEM	LEVEL	ASKED
3113	Find the Number of Subarrays Where Boundary Elements Are Maximum	Hard	0
3221	Maximum Array Hopping Score II †	Medium	0

Monotonic Stack - The Histogram Family

THE KERNEL

The largest rectangle in a histogram is the parent of a whole family: maximal rectangle, sum of subarray minimums, min-product. They all ask ‘for each bar, how far can it extend before something shorter blocks it’.

14

 PROBLEMS
MAPPED

0

EASY

7

MEDIUM

7

HARD

The Boundary Rules

- 1 **Largest Rectangle in Histogram** (84): for each bar find the nearest strictly-smaller bar to the left and to the right; the area is $\text{height} * (\text{right} - \text{left} - 1)$. One monotonic pass with a 0 sentinel gives both spans at once.
- 2 The sentinel must be **strictly smaller than every bar** — 0 works because heights are non-negative. If heights can be zero, append -1.
- 3 **Maximal Rectangle** (85): reduce each row to a histogram of consecutive 1s above it, then run 84. This is the single most valuable reduction in the stack part — learn the shape, not the code.
- 4 **Trapping Rain Water** (42) has a monotonic-stack solution, but the two-pointer solution (chapter 5) is shorter and $O(1)$ space. Know both; lead with two pointers.

- 5 Sum of Subarray Minimums (907) and Sum of Subarray Ranges (2104 , which is max-sum minus min-sum) use the *counting* form: for each index, `left` `right` `value` . Use strict on one side and non-strict on the other, or equal elements are double counted.
- 6 Maximum Subarray Min-Product (1856) needs prefix sums *and* monotonic spans. Recognise it as 907 with a different aggregation.
- 7 If the problem says ‘largest’, ‘maximal’, ‘sum over all subarrays’, and the array is 1-D or 2-D, this family is the first thing to try.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
221	Maximal Square	Medium	7
304	Range Sum Query 2D - Immutable	Medium	5
363	Max Sum of Rectangle No Larger Than K	Hard	0
907	Sum of Subarray Minimums	Medium	0
1504	Count Submatrices With All Ones	Medium	0
1727	Largest Submatrix With Rearrangements	Medium	1
1856	Maximum Subarray Min-Product	Medium	0
2281	Sum of Total Strength of Wizards	Hard	0
2818	Apply Operations to Maximize Score	Hard	0
2945	Find Maximum Non-decreasing Array Length	Hard	0

Monotonic Queue - Sliding Window Extremes

THE KERNEL

A deque that is kept monotonic gives $O(1)$ access to the maximum (or minimum) of a sliding window. Pop from the back while the new element dominates; pop from the front when the element leaves the window.

13

 PROBLEMS
MAPPED

0

EASY

6

MEDIUM

7

HARD

The Boundary Rules

- 1 **The deque stores indices, never values.** You need the index to know when an element leaves the window.
- 2 Two pops, in this order, every iteration: (1) `while dq and a[dq[-1]] <= a[i]: dq.pop()` — evict dominated elements from the back; (2) `append i`; (3) `if dq[0] <= i - k: dq.popleft()` — evict the expired front.
- 3 The expiry test is `dq[0] <= i - k` (or `< i - k + 1`), and it must be `<=`, not `<`. Off-by-one here produces a window of `k+1` and is the single most common bug in this chapter.
- 4 For a *maximum* window keep the deque decreasing and pop with `<=`. For a *minimum* window keep it increasing and pop with `>=`. Using `<=` for maxima keeps the newest duplicate, which is correct for windowing; using `<` keeps the oldest, which is also correct for windowing but wrong for counting-style problems. Know which you need.

- 5 Sliding Window Maximum (239) is the archetype. Longest Continuous Subarray With Absolute Diff $\leq$ Limit (1438) is the same deque run twice, once for max and once for min.
- 6 Shortest Subarray with Sum at Least K (862) is the *prefix-sum* version: the deque holds indices with increasing prefix sums and you pop from the front whenever the constraint is met. It is the hardest problem in this chapter and the one worth re-deriving from scratch.
- 7 Constrained Subsequence Sum (1425) and Jump Game VI (1696) are 1-D DP whose transition is a windowed maximum — replace the inner $O(k)$ loop with this deque and the DP drops from $O(nk)$ to $O(n)$. Recognising ‘DP whose transition is a sliding maximum’ is the transferable skill.
- 8 If k is small and fixed, a plain $\max(\text{window})$ is fine and the interviewer will not object. Reach for the deque when k is large or when you are asked for the $O(n)$ bound.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1499	Max Value of Equation	Hard	0
1696	Jump Game VI	Medium	0
2398	Maximum Number of Robots Within Budget	Hard	0
2444	Count Subarrays With Fixed Bounds	Hard	0
2462	Total Cost to Hire K Workers	Medium	0
2762	Continuous Subarrays	Medium	0
2944	Minimum Number of Coins for Fruits	Medium	0
2969	Minimum Number of Coins for Fruits II †	Hard	0
3255	Find the Power of K-Size Subarrays II	Medium	0

Stack – Simulation, Paths, and Collapse

THE KERNEL

Whenever a stream of tokens can *cancel*, *collapse*, or *nest*, the stack is the natural machine. Push what survives; pop when it is cancelled.

26

 PROBLEMS
MAPPED

9

EASY

17

MEDIUM

0

HARD

The Boundary Rules

- 1 **Collapse rule.** `Remove All Adjacent Duplicates (1047)`, `Make The String Great (1544)`, `Removing Stars (2390)`, `Minimum String Length After Removing Substrings (2696)` and `Backspace String Compare (844)` are one template: `if stack and cancel(stack[-1], c): stack.pop() else: stack.append(c)`.
- 2 `Remove All Adjacent Duplicates II (1209)` adds a count: store `(char, count)` and pop when the count reaches `k`. Do not push `k` copies.
- 3 **Simplify Path (71)** is `split('/')`, skip `'.'` and `'..'`, pop on `'..'`, else push; then `'/' + '/'.join(stack)`. The final join must produce `'/'` for an empty stack — that is the one edge case the tests always include.
- 4 **Asteroid Collision (735)** is the collapse rule with a direction check: only a right-mover followed by a left-mover can collide, and the smaller one dies while equal ones both die. Write the `while` loop, not an `if` — a single asteroid can destroy several.

- 5 `Validate Stack Sequences` (946) is a simulation where you push from the input and pop whenever the top matches the next expected output. $O(n)$ and no extra structure beyond the stack.
- 6 `Min Stack` (155) is a design problem solved by storing `(value, current_min)` pairs — pushing the running minimum alongside the value. The alternative (a second stack of minimums) is equally good; pick one and justify it.
- 7 `Implement Queue using Stacks` (232) uses two stacks: push onto `in`, and only move to `out` when `out` is empty. **Never move elements back** — amortised $O(1)$ depends on moving in one direction only.
- 8 `Baseball Game` (682) is a pure simulation warm-up; if it takes more than ten lines you are over-thinking.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1249	Minimum Remove to Make Valid Parentheses	Medium	7
1472	Design Browser History	Medium	4
1381	Design a Stack With Increment Operation	Medium	1
388	Longest Absolute File Path	Medium	2
946	Validate Stack Sequences	Medium	3
1700	Number of Students Unable to Eat Lunch	Easy	1
1190	Reverse Substrings Between Each Pair of Parentheses	Medium	2
225	Implement Stack using Queues	Easy	1
1544	Make The String Great	Easy	2
880	Decoded String at Index	Medium	1
1441	Build an Array With Stack Operations	Medium	1
1673	Find the Most Competitive Subsequence	Medium	1
682	Baseball Game	Easy	1
1598	Crawler Log Folder	Easy	0
1653	Minimum Deletions to Make String Balanced	Medium	0

#	PROBLEM	LEVEL	ASKED
1963	Minimum Number of Swaps to Make the String Balanced	Medium	0
2696	Minimum String Length After Removing Substrings	Easy	0
3170	Lexicographically Minimum String After Removing Stars	Medium	0

PART VII

Heaps & Priority Queues

Heaps. Top-K, two-heap balancing, k-way merge, and the bounded-key cases where a bucket beats a heap.

Heap - Top-K

THE KERNEL

To find the k best of n , keep a heap of size k and evict the worst. $O(n \log k)$ time, $O(k)$ space, and you never sort the whole input.

24

 PROBLEMS
MAPPED

6

EASY

16

MEDIUM

2

HARD

The Boundary Rules

- 1 **Direction rule — memorise this.** For the k *largest*, use a *min-heap* of size k (so the smallest of your current best is on top and gets evicted). For the k *smallest*, use a *max-heap*. The heap's direction is always the opposite of what you want to keep.
- 2 Python's `heapq` is min-only. For a max-heap, push negated values (`heappush(h, -x)`) and negate on the way out. For tuples, negate the key field: `(-freq, word)`.
- 3 **Tie-breaking in tuples.** `heapq` compares tuples element by element, so `(-freq, word)` gives lexicographic ascending order on ties, which is exactly what `Top K Frequent Words` (692) wants. If the problem wants the opposite, negate the string or use a counter.
- 4 Never `sorted(...)[-k:]` unless k is a large fraction of n . State the complexity trade-off: sorting is $O(n \log n)$, heap is $O(n \log k)$.
- 5 `heapq.nlargest(k, it, key=...)` is a one-liner and is implemented with the heap trick already. Use it when the interviewer allows `stdlib` shortcuts — but be ready to write the loop by hand.

- 6 **Heapify beats repeated heappush.** `heapq.heapify(a)` is $O(n)$; `for x in a: heappush(h, x)` is $O(n \log n)$. If you already have the whole list, heapify.
- 7 **Kth Largest Element in an Array (215)** has three valid answers: sort $O(n \log n)$, heap $O(n \log k)$, and quickselect $O(n)$ average. Know all three and lead with quickselect if asked for linear time — that is chapter 86.
- 8 **K Closest Points to Origin (973)** is top-k with the key `xx + yy`. Do not compute `sqrt`; comparing squared distances is equivalent and avoids floating point.
- 9 **Find K Closest Elements (658)** is *not* a heap problem despite the name — it is binary search on the window start. That misdirection is deliberate.
- 10 **Last Stone Weight (1046)** is the canonical two-heap-pop loop and is the easiest way to demonstrate you know how to negate for a max-heap.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
23	Merge k Sorted Lists	Hard	14
378	Kth Smallest Element in a Sorted Matrix	Medium	8
658	Find K Closest Elements	Medium	7
703	Kth Largest Element in a Stream	Easy	5
451	Sort Characters By Frequency	Medium	5
1636	Sort Array by Increasing Frequency	Easy	3
1985	Find the Kth Largest Integer in the Array	Medium	1
1337	The K Weakest Rows in a Matrix	Easy	1
2208	Minimum Operations to Halve Array Sum	Medium	1
1383	Maximum Performance of a Team	Hard	0
2342	Max Sum of a Pair With Equal Sum of Digits	Medium	0
2405	Optimal Partition of String	Medium	0
2462	Total Cost to Hire K Workers	Medium	0
2542	Maximum Subsequence Score	Medium	0

#	PROBLEM	LEVEL	ASKED
2558	Take Gifts From the Richest Pile	Easy	0
2599	Make the Prefix Sum Non-negative †	Medium	0
2974	Minimum Number Game	Easy	0
3066	Minimum Operations to Exceed Threshold Value II	Medium	0
3275	K-th Nearest Obstacle Queries	Medium	0

Two Heaps - Running Median and Balancing

THE KERNEL

Split the stream into a lower half (max-heap) and an upper half (min-heap). Keep their sizes within one of each other, and the median is on a top.

13PROBLEMS
MAPPED**0**

EASY

6

MEDIUM

7

HARD

The Boundary Rules

- 1 The invariant.** `len(low) == len(high)` or `len(low) == len(high) + 1`. Always push to `low` first, then move `low`'s top to `high`, then rebalance back if `high` got bigger. This three-step order is what keeps the invariant true without branching on the value.
- 2 The rebalance is** `if len(low) > len(high) + 1: heappush(high, -heappop(low))` and `if len(high) > len(low): heappush(low, -heappop(high))`.
- 3 Median is** `low[0]` when `len(low) > len(high)`, otherwise `(low[0] - high[0]) / 2` (remember `low` stores negated values, so the subtraction becomes addition of the negatives). Writing `(low[0] + high[0]) / 2` with a negated heap is the classic sign bug.
- 4 Sliding Window Median (480)** adds *removal*, which Python's `heapq` does not support. Two options: lazy deletion with a `to_delete` counter and a `clean()` helper, or a sorted container. Lazy deletion is the interview answer.

- 5 **IP0 (502)** is two heaps in a different arrangement: sort projects by capital, push every now-affordable project into a max-heap by profit, then take the best. It is ‘top-k with a feasibility gate’.
- 6 **Find Median from Data Stream (295)** is the single most-asked design problem after LRU Cache. Be able to write it in under 20 lines.
- 7 If you find yourself wanting to peek at both the smallest and the largest of a set, that is the signal for two heaps — or for a sorted container if the language has one.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1985	Find the Kth Largest Integer in the Array	Medium	1
1825	Finding MK Average	Hard	0
1942	The Number of the Smallest Unoccupied Chair	Medium	0
2054	Two Best Non-Overlapping Events	Medium	0
2102	Sequentially Ordinal Rank Tracker	Hard	0
2336	Smallest Number in Infinite Set	Medium	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2422	Merge Operations to Turn Array Into a Palindrome †	Medium	0
2503	Maximum Number of Points From Grid Queries	Hard	0
2959	Number of Possible Sets of Closing Branches	Hard	0

Heap – K-Way Merge and Scheduling

THE KERNEL

When you have k sorted sources and want a globally sorted order, put the head of each source in a heap and repeatedly pop the minimum. When you have tasks competing for a resource, put the task times in a heap and pop what finishes first.

43PROBLEMS
MAPPED**3**

EASY

25

MEDIUM

15

HARD

The Boundary Rules

- 1 **K-way merge shape.** Heap of `(value, source_index, element_index)`. Pop, append, and push the next element from the *same* source. The source index in the tuple is mandatory — without it you cannot refill.
- 2 **Merge k Sorted Lists** (23) is the canonical version. The alternative is divide-and-conquer pairwise merging, which is $O(n \log k)$ too but recurses; the heap version is iterative and easier to get right under pressure.
- 3 **Find K Pairs with Smallest Sums** (373) is k-way merge over an implicit matrix of pair sums. Only push `(i, j+1)` (not both neighbours) or you push duplicates and blow the memory. The dedup trick is the whole problem.
- 4 **Smallest Range Covering Elements from K Lists** (632) needs a running maximum alongside the heap of minimums. Track `cur_max` on every push and the range is `cur_max - heap[0][0]`.

- 5 **Scheduling shape.** `Task Scheduler` (621) is not a heap problem — it is a counting/greedy formula ($(\text{max_freq} - 1) * (n + 1) + \text{num_max}$). Recognise it and save ten minutes.
- 6 `Reorganize String` (767) is a heap problem: max-heap by count, and hold the previous character aside for one step so it is not immediately reused. The ‘hold aside’ buffer is the transferable trick.
- 7 `Meeting Rooms II` (253) is sweep line or min-heap-of-end-times. Both are $O(n \log n)$; the heap version reads better and is the one to write.
- 8 `Number of the Smallest Unoccupied Chair` (1942) is two heaps — one of free chairs, one of occupied (`leave_time, chair`). Rebalance by popping the expired from the occupied heap into the free heap.
- 9 `Single-Threaded CPU` (1834) is the same shape: a min-heap of (`available_time, ...`) plus a heap of ready tasks.
- 10 The unifying question: *what is the resource, and what is the rule that decides who gets it next?* The heap always holds the contenders keyed by that rule.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1094	Car Pooling	Medium	7
895	Maximum Frequency Stack	Hard	7
358	Rearrange String k Distance Apart †	Hard	4
1086	High Five †	Easy	1
218	The Skyline Problem	Hard	3
313	Super Ugly Number	Medium	3
407	Trapping Rain Water II	Hard	1
1348	Tweet Counts Per Frequency	Medium	1
1464	Maximum Product of Two Elements in an Array	Easy	3
254	Factor Combinations †	Medium	1
1705	Maximum Number of Eaten Apples	Medium	2

#	PROBLEM	LEVEL	ASKED
264	Ugly Number II	Medium	1
1354	Construct Target Array With Multiple Sums	Hard	1
1439	Find the Kth Smallest Sum of a Matrix With Sorted Rows	Hard	1
1675	Minimize Deviation in Array	Hard	1
1792	Maximum Average Pass Ratio	Medium	1
506	Relative Ranks	Easy	1
632	Smallest Range Covering Elements from K Lists	Hard	0
786	K-th Smallest Prime Fraction	Medium	0
857	Minimum Cost to Hire K Workers	Hard	0
1167	Minimum Cost to Connect Sticks †	Medium	0
1383	Maximum Performance of a Team	Hard	0
1405	Longest Happy String	Medium	0
1418	Display Table of Food Orders in a Restaurant	Medium	1
1606	Find Servers That Handled Most Number of Requests	Hard	0
1834	Single-Threaded CPU	Medium	0
1882	Process Tasks Using Servers	Medium	0
2054	Two Best Non-Overlapping Events	Medium	0
2336	Smallest Number in Infinite Set	Medium	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2462	Total Cost to Hire K Workers	Medium	0
2503	Maximum Number of Points From Grid Queries	Hard	0
2542	Maximum Subsequence Score	Medium	0
2551	Put Marbles in Bags	Hard	0
2645	Minimum Additions to Make Valid String	Medium	0
2959	Number of Possible Sets of Closing Branches	Hard	0
3092	Most Frequent IDs	Medium	0

Bucket and Counting Sort as a Heap Replacement

THE KERNEL

When the key you are sorting by is a bounded integer (a count, a digit, a score in $0..n$), you do not need a heap. Bucket by the key and read the buckets in order — $O(n)$ instead of $O(n \log n)$.

19

PROBLEMS
MAPPED

4

EASY

14

MEDIUM

1

HARD

The Boundary Rules

- 1 Bucket sort applies whenever the key range is $O(n)$ or smaller. Ask ‘what is the maximum possible value of my key?’ — if the answer is $\leq n$, bucket.
- 2 Top K Frequent Elements (347) with buckets: `bucket[i]` holds the values that occur `i` times; walk from `n` down and stop after `k`. $O(n)$ time. This is the answer interviewers want when they say ‘better than $O(n \log n)$ ’.
- 3 Sort Characters By Frequency (451) is the same bucket walk with characters.
- 4 H-Index (274) is a counting sort over citation counts. Because an h-index cannot exceed `n`, every count $> n$ clamps to `n`. That clamping is what makes the bucket array size `n + 1` instead of `max(citations)`.

- 5 **Maximum Gap** (164) requires linear time, which rules out comparison sorting. The answer is pigeonhole bucketing: with `n` numbers, the maximum gap is at least $\text{ceil}((hi - lo) / (n - 1))$, so bucket widths of that size guarantee the answer crosses buckets. This is a genuinely hard idea and worth studying carefully.
- 6 **Relative Sort Array** (1122) is counting sort with a custom order — count, then emit in `arr2` order, then emit the remainder ascending.
- 7 **Radix sort** is bucket sort applied digit by digit. It is $O(d * n)$ and stable. You rarely write it in an interview, but knowing that **Maximum Gap** and **Sort an Array** (912) can be done in linear time with it is useful.
- 8 **Contains Duplicate III** (220) is bucket-by-value-range with width `t + 1`; two values within `t` land in the same or adjacent buckets. Chapter 20 lists it too — it is genuinely a hashing problem, and the bucket is the hash.
- 9 Never bucket when the key range is unbounded or unknown. `dict`-based grouping (chapter 22) is the safe fallback.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
692	Top K Frequent Words	Medium	9
2034	Stock Price Fluctuation	Medium	1
1051	Height Checker	Easy	3
1481	Least Number of Unique Integers after K Removals	Medium	4
220	Contains Duplicate III	Hard	4
1636	Sort Array by Increasing Frequency	Easy	3
912	Sort an Array	Medium	2
1365	How Many Numbers Are Smaller Than the Current Number	Easy	2
1057	Campus Bikes †	Medium	1
1338	Reduce Array Size to The Half	Medium	1
2208	Minimum Operations to Halve Array Sum	Medium	1
2542	Maximum Subsequence Score	Medium	0

#	PROBLEM	LEVEL	ASKED
2740	Find the Value of the Partition	Medium	0
3092	Most Frequent IDs	Medium	0

PART VIII

Linked Lists

Pointer surgery. Reversal, Floyd's algorithm, merging, and the dummy head that removes every head-replacement branch.

Linked List – The Reversal Family

THE KERNEL

`prev, cur = cur, cur.next` after saving `cur.next`. Every reversal problem in this book is that three-line loop with a different starting point and a different stopping point.

23

 PROBLEMS
MAPPED

4

EASY

18

MEDIUM

1

HARD

The Boundary Rules

- 1 **The loop, memorised.** `while cur: nxt = cur.next; cur.next = prev; prev = cur; cur = nxt`. Save the next pointer *before* overwriting `cur.next` — this is the only ordering that works.
- 2 **Full reversal** (206): start `prev = None, cur = head`; return `prev`.
- 3 **Partial reversal** (92): walk to node `left - 1` with a `dummy` head, then reverse exactly `right - left + 1` nodes, then re-stitch four pointers. The four-pointer re-stitch is where candidates lose points — name them out loud: `pre.next`, the new segment head, the new segment tail, and the node after the segment.
- 4 **k-group reversal** (25): before each group, first *check that k nodes exist*. If not, stop and leave the remainder untouched. Forgetting the length check is the classic failure.
- 5 **The dummy node.** `dummy = ListNode(0, head)` removes every ‘what if the head changes’ branch. Use it in every linked-list problem where the head might be replaced or removed. It costs one line and saves three branches.

- 6 **Swap in pairs** (24) and **reverse even-length groups** (2074) are 25 with $k = 2$ and a parity condition. Recognise the reduction.
- 7 **Palindrome list** (234): find the middle with fast/slow, reverse the second half, compare, then *optionally restore*. Interviewers sometimes ask you to restore the list — do it, and say that you are doing it.
- 8 **Reorder list** (143): middle, reverse second half, then interleave. It is three templates glued together, and it is a very common interview problem precisely because it tests whether you can compose.
- 9 **Rotate list** (61): compute the length, take $k \% \text{len}$, find the new tail, and cut. Do not rotate one step at a time.
- 10 Recursion is a valid alternative for 206 (`return reverse(head.next)` with the pointer fix-up on unwind) but it costs $O(n)$ stack. Say so if you use it.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
2130	Maximum Twin Sum of a Linked List	Medium	1
328	Odd Even Linked List	Medium	5
83	Remove Duplicates from Sorted List	Easy	5
445	Add Two Numbers II	Medium	4
82	Remove Duplicates from Sorted List II	Medium	3
1721	Swapping Nodes in a Linked List	Medium	3
725	Split Linked List in Parts	Medium	3
1290	Convert Binary Number in a Linked List to Integer	Easy	2
86	Partition List	Medium	1
369	Plus One Linked List †	Medium	1
2181	Merge Nodes in Between Zeros	Medium	1
2058	Find the Minimum and Maximum Number of Nodes Between Critical Points	Medium	0
2074	Reverse Nodes in Even Length Groups	Medium	0
2487	Remove Nodes From Linked List	Medium	0

#	PROBLEM	LEVEL	ASKED
2807	Insert Greatest Common Divisors in Linked List	Medium	0
2816	Double a Number Represented as a Linked List	Medium	0

Linked List – Fast and Slow Pointers

THE KERNEL

Two pointers at different speeds. If there is a cycle they meet; if not, the fast one hits the end. The offset between them also locates the middle, or the *k*th node from the end.

16PROBLEMS
MAPPED**6**

EASY

10

MEDIUM

0

HARD

The Boundary Rules

- 1 Cycle detection (141):** `while fast and fast.next: slow = slow.next; fast = fast.next.next; if slow is fast: return True`. The `fast` and `fast.next` guard is mandatory — `fast.next.next` on a one-node list raises.
- 2 Cycle entry (142):** after they meet, put one pointer back at `head` and advance both one step at a time. They meet at the entry. The maths is $2(a + b) = a + b + c + b$ giving $a = c$ — be able to state that identity, because interviewers ask *why* it works.
- 3 Middle node (876):**
`while fast and fast.next: slow = slow.next; fast = fast.next.next`.
For an even-length list this returns the *second* middle. If the problem wants the first middle, start `fast = head.next`.
- 4** The first-vs-second-middle distinction matters enormously in 143 and 234, where a wrong choice silently produces the wrong interleaving.

- 5 **Nth from end (19)**: advance `fast` by `n + 1` from a `dummy`, then move both until `fast` is `None`. `slow` is now the node *before* the target, which is what you need to delete it. Advancing by `n` instead of `n + 1` puts you on the target itself and you cannot unlink it.
- 6 **Intersection of two lists (160)**: walk both with wraparound — `a = a.next if a else headB`. After at most `m + n` steps they align at the intersection, or both become `None` together. No length computation needed.
- 7 **Sort List (148)** uses fast/slow to split, then merges. The fast/slow here must find the *first* middle to avoid infinite recursion on two-element lists.
- 8 The same technique applies to non-linked structures: **Happy Number (202)** and **Find the Duplicate Number (287)** treat an array as a functional graph. 287 is the most important non-list application — it is a cycle-entry problem with a binary-search alternative, and interviewers like hearing both.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
234	Palindrome Linked List	Easy	11
202	Happy Number	Easy	6
2130	Maximum Twin Sum of a Linked List	Medium	1
143	Reorder List	Medium	7
148	Sort List	Medium	5
457	Circular Array Loop	Medium	1
2181	Merge Nodes in Between Zeros	Medium	1
2032	Two Out of Three	Easy	1
2095	Delete the Middle Node of a Linked List	Medium	0
2487	Remove Nodes From Linked List	Medium	0

Linked List - Merge, Sort, and Reorder

THE KERNEL

Merge two sorted lists is a dummy-head while-loop. Sort a list is merge sort, which is the only $O(n \log n)$ sort that works on a linked list without random access.

17PROBLEMS
MAPPED**1**

EASY

15

MEDIUM

1

HARD

The Boundary Rules

- 1 **Merge shape.** `dummy` + `tail` pointer; while `a` and `b`: pick the smaller; `tail = tail.next`; then `tail.next = a or b` to drain the rest. The final `or` is the elegant drain and always works because at most one list is non-empty.
- 2 Use `<=` not `<` when merging to keep the merge *stable*. It rarely changes the answer, but it is the correct choice and interviewers notice.
- 3 `Merge Two Sorted Lists` (21) is the base case of `Merge k Sorted Lists` (23). `Sort List` (148) is 21 plus a fast/slow split plus recursion. Three problems, one template, increasing depth.
- 4 **Sort List** must be merge sort, not quicksort: quicksort needs random access to pick a good pivot, and a linked list does not have it. Saying this in an interview is worth more than the code.

- 5 **Add Two Numbers** (2) is a merge with carry. The carry is a third loop variable, and after the loop you must handle a trailing carry (`if carry: tail.next = ListNode(1)`).
- 6 **Add Two Numbers II** (445) reverses the inputs — either reverse the lists first, or use two stacks. Stacks are cleaner and avoid mutating the input, which the problem may forbid.
- 7 **Partition List** (86) is two dummy heads (a `less` list and a `ge` list) joined at the end. Two dummies beats trying to do it in place.
- 8 **Odd Even Linked List** (328) is also two chains, interleaved by index parity, then joined. Same shape as 86.
- 9 **Reorder List** (143) is 876 + 206 + 21 composed. If you have internalised the three templates, this problem is 15 lines.
- 10 **Copy List with Random Pointer** (138) belongs here by adjacency: the O(1)-space solution interleaves cloned nodes into the original list (`a -> a' -> b -> b'`) so the random pointers can be resolved by `node.next.random = node.random.next` . That interleaving trick is worth memorising; the dict-based O(n)-space solution is the safe fallback.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
23	Merge k Sorted Lists	Hard	14
143	Reorder List	Medium	7
1721	Swapping Nodes in a Linked List	Medium	3
147	Insertion Sort List	Medium	1
1171	Remove Zero Sum Consecutive Nodes from Linked List	Medium	1
369	Plus One Linked List †	Medium	1
2181	Merge Nodes in Between Zeros	Medium	1
1669	Merge In Between Linked Lists	Medium	0
2058	Find the Minimum and Maximum Number of Nodes Between Critical Points	Medium	0

#	PROBLEM	LEVEL	ASKED
2816	Double a Number Represented as a Linked List	Medium	0

Linked List – Dummy Head, Deletion, and Design

THE KERNEL

Almost every linked-list bug is a null-pointer or a head-replacement edge case. A dummy head removes the second category entirely; careful guard clauses remove the first.

19

PROBLEMS
MAPPED

4

EASY

14

MEDIUM

1

HARD

The Boundary Rules

- 1 **Rule: if the head can be removed or replaced, use a dummy.** (203, 82, 83, 19, 21, 86, 92, 25, 148, 138) — all of them.
- 2 **Delete by value** (203): `while cur.next: if cur.next.val == v: cur.next = cur.next.next else: cur = cur.next`. Do **not** advance `cur` in the delete branch — you would skip a consecutive duplicate.
- 3 **Delete all duplicates** (82): when a duplicate run is detected, walk to the end of the run *first*, then unlink the whole run. Do not unlink node by node.
- 4 **Delete a node given only that node** (237): copy the next node's value into the current node and unlink the next. It is a trick, and it does not work for the last node — the problem guarantees it will not be the last node.
- 5 **Design Linked List** (707) needs `addAtHead`, `addAtTail`, `addAtIndex`, `deleteAtIndex`. Use a dummy head plus a `size` field; deriving the size by walking is $O(n)$ and will fail the follow-up.

- 6 **Circular queue** (622) and **circular deque** (641) are easier with a fixed-size list and two index pointers than with actual nodes. Do not over-engineer.
- 7 **LRU Cache** (146) is the flagship linked-list design problem: dummy head and dummy tail, `add_to_head` and `remove_node` as two helpers. Write the helpers first and the rest becomes trivial.
- 8 **Flatten a Multilevel Doubly Linked List** (430) is a stack-based depth-first walk, or a recursive helper that returns the tail of the flattened segment. Both are acceptable; the iterative stack version has no recursion depth risk.
- 9 The universal debugging habit for this chapter: draw three nodes and trace your pointer updates by hand before running anything. Linked-list bugs are almost never logical — they are ordering bugs in the assignment sequence.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
146	LRU Cache	Medium	23
460	LFU Cache	Hard	9
83	Remove Duplicates from Sorted List	Easy	5
1290	Convert Binary Number in a Linked List to Integer	Easy	2
1670	Design Front Middle Back Queue	Medium	1
1474	Delete N Nodes After M Nodes of a Linked List †	Easy	1
2181	Merge Nodes in Between Zeros	Medium	1
641	Design Circular Deque	Medium	0
1669	Merge In Between Linked Lists	Medium	0
2046	Sort Linked List Already Sorted Using Absolute Values †	Medium	0
2095	Delete the Middle Node of a Linked List	Medium	0
2487	Remove Nodes From Linked List	Medium	0
2807	Insert Greatest Common Divisors in Linked List	Medium	0

PART IX

Trees

Trees. One recursive skeleton, two directions of information flow, and the hybrid that computes a diameter while returning a height.

Tree Traversal - The Recursive Skeleton

THE KERNEL

Every recursive tree function is the same six lines: base case on `None`, recurse left, recurse right, combine. The only question is *where* you put the combine step — before (preorder), between (inorder), or after (postorder).

52PROBLEMS
MAPPED**29**

EASY

22

MEDIUM

1

HARD

The Boundary Rules

- 1 The skeleton.** `def f(node): if not node: return BASE; L = f(node.left); R = f(node.right); return COMBINE(node, L, R)`. Write it once and you own a hundred tree problems.
- 2 Base case first, always.** `if not node: return <identity>` where the identity is `0` for sums and depths, `None` for nodes, `[]` for lists, `True` for boolean properties.
- 3 Where you place the work decides the order.** Work before the recursive calls = preorder (root first). Work between them = inorder (only meaningful for BSTs). Work after = postorder (children first, needed whenever a parent's answer depends on its children's answers).
- 4 Preorder** (144) is for top-down information: carrying a bound, a prefix, a depth, or a path down the tree.

- 5 **Postorder** (145) is for bottom-up information: heights, subtree sums, subtree validity. If the parent needs a value *returned* by the children, it is postorder.
- 6 **Inorder** (94) on a BST gives sorted order. That single fact answers `Validate BST` (98), `Kth Smallest` (230), `BST Iterator` (173), `Minimum Absolute Difference` (530), `Find Mode` (501), `Recover BST` (99) and `Convert BST to Greater Tree` (538).
- 7 **Invert Binary Tree** (226) is `node.left, node.right = f(node.right), f(node.left)`. The tuple assignment is not a party trick — it avoids the classic bug of overwriting `node.left` before recursing on it.
- 8 **Same Tree** (100) is the base-case pair: `if not p and not q: return True;`
`if not p or not q: return False`. Write both guards, in that order.
- 9 **Symmetric Tree** (101) is `Same Tree` with mirrored arguments: `mirror(a.left, b.right)` and `mirror(a.right, b.left)`.
- 10 **Subtree of Another Tree** (572) is `is_same(root, sub)` or `is_subtree(root.left, sub)` or `is_subtree(root.right, sub)`. $O(n*m)$ worst case; the $O(n)$ answer uses tree hashing and is a nice follow-up.
- 11 **Merge Two Binary Trees** (617) and **Leaf-Similar Trees** (872) are the same pairwise-recursion shape applied to two trees at once.
- 12 N-ary trees (589 , 590 , 559 , 429) are identical with a loop over `node.children` replacing the two explicit recursions.
- 13 Recursion depth is $O(h)$. For a degenerate tree of 10^5 nodes Python's default limit of 1000 will raise `RecursionError` — mention this, and know that `sys.setrecursionlimit` is the fix, or write it iteratively.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
938	Range Sum of BST	Easy	3
112	Path Sum	Easy	5
1110	Delete Nodes And Return Forest	Medium	4
222	Count Complete Tree Nodes	Medium	3

#	PROBLEM	LEVEL	ASKED
617	Merge Two Binary Trees	Easy	4
1448	Count Good Nodes in Binary Tree	Medium	2
257	Binary Tree Paths	Easy	5
1305	All Elements in Two Binary Search Trees	Medium	4
1382	Balance a Binary Search Tree	Medium	1
671	Second Minimum Node In a Binary Tree	Easy	1
993	Cousins in Binary Tree	Easy	3
111	Minimum Depth of Binary Tree	Easy	2
1022	Sum of Root To Leaf Binary Numbers	Easy	3
655	Print Binary Tree	Medium	2
429	N-ary Tree Level Order Traversal	Medium	1
589	N-ary Tree Preorder Traversal	Easy	1
742	Closest Leaf in a Binary Tree †	Medium	1
965	Univalued Binary Tree	Easy	1
1161	Maximum Level Sum of a Binary Tree	Medium	1
508	Most Frequent Subtree Sum	Medium	1
559	Maximum Depth of N-ary Tree	Easy	1
590	N-ary Tree Postorder Traversal	Easy	1
606	Construct String from Binary Tree	Medium	1
700	Search in a Binary Search Tree	Easy	1
872	Leaf-Similar Trees	Easy	1
897	Increasing Order Search Tree	Easy	1
1302	Deepest Leaves Sum	Medium	1
1315	Sum of Nodes with Even-Valued Grandparent	Medium	1
1379	Find a Corresponding Node of a Binary Tree in a Clone of That Tree	Easy	1
1485	Clone Binary Tree With Random Pointer †	Medium	1
1490	Clone N-ary Tree †	Medium	1
404	Sum of Left Leaves	Easy	1

#	PROBLEM	LEVEL	ASKED
1261	Find Elements in a Contaminated Binary Tree	Medium	1
1469	Find All The Lonely Nodes †	Easy	1
501	Find Mode in Binary Search Tree	Easy	0
563	Binary Tree Tilt	Easy	0
623	Add One Row to Tree	Medium	0
1080	Insufficient Nodes in Root to Leaf Paths	Medium	0
1530	Number of Good Leaf Nodes Pairs	Medium	0
1602	Find Nearest Right Node in Binary Tree †	Medium	0
2265	Count Nodes Equal to Average of Subtree	Medium	0
2331	Evaluate Boolean Binary Tree	Easy	0
2385	Amount of Time for Binary Tree to Be Infected	Medium	0
2509	Cycle Length Queries in a Tree	Hard	0

Tree Traversal - Iterative and Morris

THE KERNEL

An explicit stack replaces the call stack. Preorder is trivial; inorder is ‘go left as far as possible, then visit, then go right’; postorder is preorder with the children swapped and the result reversed.

22PROBLEMS
MAPPED**9**

EASY

12

MEDIUM

1

HARD

The Boundary Rules

- 1 Inorder iterative — memorise this one shape.** `stack = []; cur = root;`
`while stack or cur: while cur: stack.append(cur); cur = cur.left; cur`
`= stack.pop(); visit(cur); cur = cur.right.`
- 2 Preorder iterative** is the inorder shape with `visit` moved before the left descent, or the simple ‘push right then left’ stack version.
- 3 Postorder iterative** is preorder with `left` and `right` pushed in swapped order, then `result.reverse()`. That one-line reduction is far safer than the two-stack or the ‘last visited’ variants.
- 4 BST Iterator (173)** is the inorder loop frozen mid-flight: keep the stack as instance state, and `next()` pops one node and pushes its entire left spine. `has-Next()` is `bool(stack)`. This is the single most asked follow-up on the inorder template.

- 5 **Morris traversal** achieves $O(1)$ space by threading the rightmost node of the left subtree back to the current node. Know that it exists and that it is $O(n)$ time, $O(1)$ space — you will almost never be asked to write it, but ‘Morris traversal gives $O(1)$ space’ is a strong answer to ‘can you do better on space?’.
- 6 **Kth Smallest in a BST (230)** is inorder with a counter and an early exit. Iterative is better than recursive here precisely because you can stop early — recursion cannot.
- 7 **Validate BST (98)** has three solutions: inorder-must-be-increasing (cleanest), recursion with `(low, high)` bounds (most instructive), and Morris with $O(1)$ space. The bounds version is the one to write under pressure: `f(node, lo, hi)` with `if not (lo < node.val < hi): return False`.
- 8 Never validate a BST by comparing only with the immediate children. `[5,1,4,null,null,3,6]` passes the naive test and is not a BST. This is the trap the problem is built on.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
99	Recover Binary Search Tree	Medium	6
450	Delete Node in a BST	Medium	5
272	Closest Binary Search Tree Value II †	Hard	2
1305	All Elements in Two Binary Search Trees	Medium	4
1382	Balance a Binary Search Tree	Medium	1
538	Convert BST to Greater Tree	Medium	3
701	Insert into a Binary Search Tree	Medium	3
783	Minimum Distance Between BST Nodes	Easy	2
1038	Binary Search Tree to Greater Sum Tree	Medium	2
653	Two Sum IV - Input is a BST	Easy	2
530	Minimum Absolute Difference in BST	Easy	1
606	Construct String from Binary Tree	Medium	1
700	Search in a Binary Search Tree	Easy	1

TREE TRAVERSAL - ITERATIVE AND MORRIS

#	PROBLEM	LEVEL	ASKED
897	Increasing Order Search Tree	Easy	1
426	Convert Binary Search Tree to Sorted Doubly Linked List †	Medium	0
501	Find Mode in Binary Search Tree	Easy	0

Tree BFS - The Level Order Family

THE KERNEL

A queue plus a ‘process the whole level before moving on’ loop. The level boundary is what makes BFS different from DFS, and it is what every level-based question actually asks for.

18

PROBLEMS
MAPPED

2

EASY

15

MEDIUM

1

HARD

The Boundary Rules

- 1 **The level-boundary idiom.** `while q: for _ in range(len(q)): node = q.popleft(); ...`. Capturing `len(q)` *before* the inner loop is what separates levels. Iterating `while q` without it flattens the tree and breaks every level question.
- 2 `Level Order` (102) collects each level into its own list; `Level Order II` (107) is the same list reversed; `Zigzag` (103) reverses the level when `level % 2 == 1`.
- 3 **Right Side View** (199) is the last node of each level — `level[-1]`. **Average of Levels** (637) is `sum(level) / len(level)`. **Largest Value in Each Row** (515) is `max(level)`. **Bottom Left** (513) is the first node of the last level.
- 4 **Populating Next Right Pointers** (116 , 117) can be done with a queue in $O(n)$ space, but the $O(1)$ -space solution uses the already-built `next` pointers of the level above as a linked list. The $O(1)$ version is the follow-up and is worth knowing.

- 5 **Maximum Width of Binary Tree** (662) requires *position indices*: left child `2i`, right child `2i+1`. Without indices, nulls between real nodes are invisible and the width is wrong. Beware of integer overflow in other languages; Python is safe.
- 6 **Vertical Order Traversal** (314 , 987) needs a column index carried through the BFS (or DFS) plus a sort. 314 groups by column only; 987 also sorts by row and value, which is why it is the harder of the two.
- 7 **Minimum Depth** (111) must use BFS, not DFS, if you want the early exit — and it must test `if not node.left and not node.right` rather than `if not node.left or not node.right`, or a node with one child terminates the search incorrectly.
- 8 **All Nodes Distance K** (863) is BFS on the tree *after* adding parent pointers — an undirected graph traversal in disguise. That reframing is the whole insight.
- 9 Use BFS when the question mentions a *level*, a *depth*, a *shortest* path, or *nearest*. Use DFS when it mentions a *path*, a *subtree*, or an *aggregate*.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
314	Binary Tree Vertical Order Traversal †	Medium	4
987	Vertical Order Traversal of a Binary Tree	Hard	5
515	Find Largest Value in Each Tree Row	Medium	1
637	Average of Levels in Binary Tree	Easy	3
958	Check Completeness of a Binary Tree	Medium	1
429	N-ary Tree Level Order Traversal	Medium	1
107	Binary Tree Level Order Traversal II	Medium	1
513	Find Bottom Left Tree Value	Medium	1
1609	Even Odd Tree	Medium	1
116	Populating Next Right Pointers in Each Node	Medium	0
117	Populating Next Right Pointers in Each Node II	Medium	0
2415	Reverse Odd Levels of Binary Tree	Medium	0

Tree DFS – Top-Down vs Bottom-Up

THE KERNEL

Two directions of information flow. Top-down passes an accumulated context into the recursion; bottom-up returns a summary out of it. Naming which one you need is 80% of solving a tree problem.

34

 PROBLEMS
MAPPED

5

EASY

27

MEDIUM

2

HARD

The Boundary Rules

- 1 **Top-down:** the function takes extra parameters representing the path so far (`depth`, `running_sum`, `lo/hi bounds`, `prefix`). The answer is updated at the leaves. Signature: `f(node, state)`.
- 2 **Bottom-up:** the function returns a value that summarises the subtree (`height`, `sum`, `is_valid`, `best`). The answer is computed from the children's returns. Signature: `f(node) -> summary`.
- 3 **The hybrid — and this is the key insight.** When the answer is a quantity that the parent also needs (a height, a sum, a longest chain), the function must do two things at once: update a nonlocal best *and* return the value the parent needs. The best and the return value are **different quantities**.
- 4 **Diameter of Binary Tree** (543) is the canonical hybrid: return `1 + max(left_h, right_h)` to the parent, but update `best = max(best, left_h + right_h)` locally. Confusing the two gives the height instead of the diameter — the most common tree bug in existence.

- 5 Binary Tree Maximum Path Sum (124) is 543 with weights: return `node.val + max(0, left, right)` and update `best = max(best, node.val + left + right)`. The `max(0, ...)` clamps negative branches, which is the whole difficulty of the problem.
- 6 Balanced Binary Tree (110) is 543 with a validity flag: return `-1` as a sentinel for 'unbalanced' instead of adding a second pass. The sentinel-return trick turns $O(n \log n)$ into $O(n)$.
- 7 Longest Univalue Path (687) and Longest ZigZag Path (1372) are 543 with a conditional on whether the child can be extended.
- 8 Distribute Coins (979) returns the *excess/deficit* to the parent and adds `abs(excess)` to the moves. Binary Tree Cameras (968) returns a three-state enum. Both are bottom-up with a non-obvious return value — state what the return value *means* before coding.
- 9 Count Good Nodes (1448) is pure top-down: pass the running maximum down, count at the node.
- 10 Path Sum III (437) is top-down *with a prefix-sum hash*: carry the running sum and a dict of seen prefix sums, exactly like chapter 15. Recognising that the tree path problem is a prefix-sum problem is a high-value reduction.
- 11 Sum Root to Leaf Numbers (129) and Path Sum II (113) are top-down with a path list; 113 needs `path.copy()` on append, because the list is mutated on the way back up.
- 12 The back-tracking rule for path lists. Append before recursing, `path.pop()` after. If you forget the pop you get exponential memory and wrong answers.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
366	Find Leaves of Binary Tree †	Medium	5
129	Sum Root to Leaf Numbers	Medium	3
112	Path Sum	Easy	5
113	Path Sum II	Medium	5

#	PROBLEM	LEVEL	ASKED
257	Binary Tree Paths	Easy	5
333	Largest BST Subtree †	Medium	2
1026	Maximum Difference Between Node and Ancestor	Medium	3
298	Binary Tree Longest Consecutive Sequence †	Medium	3
1339	Maximum Product of Splitted Binary Tree	Medium	2
814	Binary Tree Pruning	Medium	2
1372	Longest ZigZag Path in a Binary Tree	Medium	1
1022	Sum of Root To Leaf Binary Numbers	Easy	3
250	Count Univalued Subtrees †	Medium	2
549	Binary Tree Longest Consecutive Sequence II †	Medium	1
687	Longest Univalued Path	Medium	2
1367	Linked List in Binary Tree	Medium	2
508	Most Frequent Subtree Sum	Medium	1
663	Equal Tree Partition †	Medium	1
988	Smallest String Starting From Leaf	Medium	1
1325	Delete Leaves With a Given Value	Medium	1
1457	Pseudo-Palindromic Paths in a Binary Tree	Medium	1
1273	Delete Tree Nodes †	Medium	1
865	Smallest Subtree with all the Deepest Nodes	Medium	0
968	Binary Tree Cameras	Hard	0
1120	Maximum Average Subtree †	Medium	0
1522	Diameter of N-Ary Tree †	Medium	0
1530	Number of Good Leaf Nodes Pairs	Medium	0
2385	Amount of Time for Binary Tree to Be Infected	Medium	0

BST - Exploiting the Ordering

THE KERNEL

A BST gives you binary search for free. The two tools are the (low, high) bounds carried down the recursion, and the inorder traversal that yields sorted order.

37PROBLEMS
MAPPED**8**

EASY

27

MEDIUM

2

HARD

The Boundary Rules

- 1 Bounds, not parent comparison.** `f(node, lo, hi)` with `if not (lo < node.val < hi): return False` then recurse with `(lo, node.val)` and `(node.val, hi)`. Start with `(-inf, +inf)`. Comparing only against the parent is the classic wrong answer.
- 2 Insert (701):** recurse by comparison; when you hit `None`, create the node. Return the (possibly new) root and assign it back: `node.left = insert(node.left, val)`.
- 3 Search (700):** `while node: if val < node.val: node = node.left elif val > node.val: node = node.right else: return node`. Iterative is shorter than recursive here.
- 4 Delete (450)** is the only hard one. Three cases: no child -> return `None`; one child -> return that child; two children -> replace the value with the *inorder successor* (leftmost of the right subtree) and then delete that successor from the right subtree. Say 'inorder successor' out loud — interviewers are listening for it.

- 5 **LCA in a BST** (235): walk down from the root. While both targets are on the same side, move there; the first node where they split is the answer. $O(h)$, no recursion needed.
- 6 **Kth smallest** (230), **mode** (501), **minimum absolute difference** (530), **recover BST** (99), **greater tree** (538) are all 'inorder gives sorted order'. For 99, the two swapped nodes appear as two adjacent inversions in the inorder sequence.
- 7 **Unique BSTs** (96) is Catalan-number DP, not a tree traversal: $dp[i] = \sum(dp[j] \cdot dp[i-1-j])$. **Unique BSTs II** (95) is the recursive enumeration version. Recognise that these are counting and construction* problems wearing a BST costume.
- 8 **Construct from traversal** (105 , 106 , 889 , 1008 , 654): preorder gives the root, inorder gives the split point. For 105, index the inorder array into a dict so each lookup is $O(1)$ — without it you are at $O(n^2)$.
- 9 **Construct BST from Preorder** (1008) needs no inorder because the BST property supplies the split: use bounds, not a lookup.
- 10 **Balance a BST** (1382) is 'inorder to sorted list, then build a balanced tree from the middle' — two templates composed.
- 11 **Closest value** (270) and **inorder successor** (285) are bound walks: keep a `best` and update it as you descend.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
938	Range Sum of BST	Easy	3
173	Binary Search Tree Iterator	Medium	6
99	Recover Binary Search Tree	Medium	6
109	Convert Sorted List to Binary Search Tree	Medium	4
285	Inorder Successor in BST †	Medium	4
270	Closest Binary Search Tree Value †	Easy	1
1008	Construct Binary Search Tree from Preorder Traversal	Medium	4

#	PROBLEM	LEVEL	ASKED
1305	All Elements in Two Binary Search Trees	Medium	4
1382	Balance a Binary Search Tree	Medium	1
108	Convert Sorted Array to Binary Search Tree	Easy	5
669	Trim a Binary Search Tree	Medium	4
333	Largest BST Subtree †	Medium	2
538	Convert BST to Greater Tree	Medium	3
449	Serialize and Deserialize BST	Medium	3
95	Unique Binary Search Trees II	Medium	1
96	Unique Binary Search Trees	Medium	1
783	Minimum Distance Between BST Nodes	Easy	2
1038	Binary Search Tree to Greater Sum Tree	Medium	2
653	Two Sum IV - Input is a BST	Easy	2
530	Minimum Absolute Difference in BST	Easy	1
776	Split BST †	Medium	1
1214	Two Sum BSTs †	Medium	1
1373	Maximum Sum BST in Binary Tree	Hard	1
106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	1
654	Maximum Binary Tree	Medium	1
426	Convert Binary Search Tree to Sorted Doubly Linked List †	Medium	0
501	Find Mode in Binary Search Tree	Easy	0
510	Inorder Successor in BST II †	Medium	0
889	Construct Binary Tree from Preorder and Postorder Traversal	Medium	0
1569	Number of Ways to Reorder Array to Get Same BST	Hard	0

Tree - Path Problems

THE KERNEL

A path is a sequence from an ancestor to a descendant (or through a node, for ‘any path’). Root-to-leaf paths are top-down with a path list; ‘any path’ problems are the hybrid from chapter 41.

22

 PROBLEMS
MAPPED

5

EASY

15

MEDIUM

2

HARD

The Boundary Rules

- 1 **Root-to-leaf with a list.** Append `node.val`, recurse, then `path.pop()`. At a leaf, `result.append(path.copy())` — **`copy()` is mandatory**, otherwise every entry in the result aliases the same mutable list and you finish with `n` copies of the last path.
- 2 **Root-to-leaf with a sum.** Carry the remaining target (`target - node.val`) rather than accumulating, so the leaf test is `target == 0`.
- 3 `Path Sum (112)` is boolean; `Path Sum II (113)` collects the paths; `Sum Root to Leaf Numbers (129)` accumulates `10 cur + node.val`; `Sum of Root To Leaf Binary Numbers (1022)` accumulates `cur * 2 + node.val`. Same walk, different accumulator.
- 4 `Path Sum III (437)` is **not** root-to-leaf: it counts any downward path. The answer is the prefix-sum hash from chapter 15 applied to the DFS path: `seen = {0: 1}`, `cur += node.val`, `ans += seen.get(cur - target, 0)`, recurse, then `seen[cur] -= 1` on the way back. **The decrement on the way back is the whole trick** — it is the DFS equivalent of `path.pop()`.

- 5 **Through-a-node paths.** `Diameter` (543), `Max Path Sum` (124), `Longest Univalued Path` (687), `Longest ZigZag` (1372), `Binary Tree Cameras` (968): return the one-sided value to the parent, update the two-sided answer locally. Re-read chapter 41 if this is not immediate.
- 6 **Leaf-to-leaf paths.** `Number of Good Leaf Nodes Pairs` (1530) and `Smallest Subtree with all the Deepest Nodes` (865) require combining information from *both* children at the parent — a postorder return of a collection, not a scalar.
- 7 **Path with a string accumulator.** `Binary Tree Paths` (257) joins with `'->'`; `Smallest String Starting From Leaf` (988) builds the string bottom-up and must reverse it (or build a list of characters and reverse at the leaf).
- 8 **Pseudo-palindromic paths** (1457) carries a bitmask of odd-count digits down the path; a path is pseudo-palindromic iff the mask has at most one set bit (`mask & (mask - 1) == 0`). That one-line test is worth memorising — it is the standard ‘at most one odd’ check.
- 9 **Distribute Coins** (979) returns `node.val - 1 + left_excess + right_excess` and adds `abs(...)` of each child’s return to the move count. The return value is ‘net coins this subtree needs to send up’.
- 10 If the problem says ‘path’ and does not say ‘root to leaf’, assume it means *any* downward path and reach for the hybrid template.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
543	Diameter of Binary Tree	Easy	6
1448	Count Good Nodes in Binary Tree	Medium	2
979	Distribute Coins in Binary Tree	Medium	4
1026	Maximum Difference Between Node and Ancestor	Medium	3
111	Minimum Depth of Binary Tree	Easy	2
1372	Longest ZigZag Path in a Binary Tree	Medium	1
1022	Sum of Root To Leaf Binary Numbers	Easy	3
687	Longest Univalued Path	Medium	2

#	PROBLEM	LEVEL	ASKED
1367	Linked List in Binary Tree	Medium	2
666	Path Sum IV †	Medium	2
988	Smallest String Starting From Leaf	Medium	1
865	Smallest Subtree with all the Deepest Nodes	Medium	0
968	Binary Tree Cameras	Hard	0
1530	Number of Good Leaf Nodes Pairs	Medium	0
2265	Count Nodes Equal to Average of Subtree	Medium	0

Tree - LCA, Structure, and Serialization

THE KERNEL

Three sub-families that all reduce to ‘compare two trees’ or ‘encode a tree uniquely’. LCA is a postorder search; serialization is a preorder with explicit nulls.

38PROBLEMS
MAPPED**4**

EASY

31

MEDIUM

3

HARD

The Boundary Rules

- 1 **LCA in a binary tree** (236). Postorder: if `node` is `None`, or `node` is `p`, or `node` is `q`, return `node`. Otherwise recurse both sides. If both returns are non-null, the current node is the LCA; otherwise return whichever side was non-null. **This is the single most elegant five-line algorithm in the tree part — memorise it verbatim.**
- 2 **LCA in a BST** (235) is the bound walk from chapter 42, not the postorder search. Using the general algorithm on a BST is correct but wastes the ordering and will be flagged.
- 3 **LCA with parent pointers** (1650) is the intersection of two linked lists — use the two-pointer wraparound from chapter 35, or a visited set.
- 4 **Serialization** (297). Preorder with an explicit `'#'` for nulls is the standard. Deserialization uses an iterator over the tokens and consumes one per call — `next(it)` inside the recursion. Do **not** pass an index around; the iterator makes the code half as long.

- 5 Delimiter choice matters: with multi-digit values, `'1' + '2'` and `'12'` collide unless you join with a separator such as `','`. Always state the separator.
- 6 **Duplicate subtrees** (652). Serialize each subtree bottom-up and hash it into a `Counter`. The key can be the serialized string or a tuple of `(val, left_id, right_id)` with an id-map for $O(1)$ keys. The id-map version is $O(n)$ and is the better answer; the string version is easier to explain.
- 7 **Verify preorder serialization** (331) is a slot-counting problem, not a tree-building problem: start with 1 slot, each node consumes one and adds two, each `'#'` consumes one and adds zero; the sequence is valid iff the count never goes negative and ends at zero.
- 8 **Construct from traversals** (105 , 106 , 889) belongs here too — the pre-order/inorder split is the same ‘find the root, split the rest’ idea that serialization relies on.
- 9 **Flip equivalent** (951) is `Same Tree` with an extra recursive case that tries the unflipped children. Two extra lines.
- 10 **Symmetric** (101) is `Same Tree` with mirrored recursion, and **Subtree** (572) is `Same Tree` nested inside a traversal. Once you have `Same Tree` you have all three.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1650	Lowest Common Ancestor of a Binary Tree III †	Medium	6
101	Symmetric Tree	Easy	8
545	Boundary of Binary Tree †	Medium	4
114	Flatten Binary Tree to Linked List	Medium	4
100	Same Tree	Easy	6
617	Merge Two Binary Trees	Easy	4
1008	Construct Binary Search Tree from Preorder Traversal	Medium	4
333	Largest BST Subtree †	Medium	2
1644	Lowest Common Ancestor of a Binary Tree II †	Medium	1

#	PROBLEM	LEVEL	ASKED
536	Construct Binary Tree from String †	Medium	1
449	Serialize and Deserialize BST	Medium	3
156	Binary Tree Upside Down †	Medium	1
250	Count Univalued Subtrees †	Medium	2
1257	Smallest Common Region †	Medium	1
255	Verify Preorder Sequence in Binary Search Tree †	Medium	2
1367	Linked List in Binary Tree	Medium	2
106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	1
508	Most Frequent Subtree Sum	Medium	1
606	Construct String from Binary Tree	Medium	1
951	Flip Equivalent Binary Trees	Medium	1
331	Verify Preorder Serialization of a Binary Tree	Medium	1
428	Serialize and Deserialize N-ary Tree †	Hard	0
623	Add One Row to Tree	Medium	0
865	Smallest Subtree with all the Deepest Nodes	Medium	0
889	Construct Binary Tree from Preorder and Postorder Traversal	Medium	0
1028	Recover a Tree From Preorder Traversal	Hard	0
1522	Diameter of N-Ary Tree †	Medium	0
1676	Lowest Common Ancestor of a Binary Tree IV †	Medium	0
1740	Find Distance in a Binary Tree †	Medium	0
2096	Step-By-Step Directions From a Binary Tree Node to Another	Medium	0
2196	Create Binary Tree From Descriptions	Medium	0
2385	Amount of Time for Binary Tree to Be Infected	Medium	0

PART X

Backtracking

Choose, recurse, un-choose. Six chapters, one template, and the dedup rules that differ by exactly one character.

Backtracking - The Universal Template

THE KERNEL

Choose, recurse, un-choose. Every backtracking problem in this book is that three-step loop; the only differences are what a ‘choice’ is, how you iterate the choices, and what makes a partial solution invalid.

35PROBLEMS
MAPPED**2**

EASY

32

MEDIUM

1

HARD

The Boundary Rules

- 1 **The template.** `def bt(start, path): if <goal>: result.append(path.copy()); return; for i in <choices from start>: if <invalid>: continue; path.append(choice); bt(i + 1, path); path.pop()`.
- 2 **`path.copy()` on every result append.** Appending `path` itself makes every answer alias the same list. This is the number-one backtracking bug in every language.
- 3 **`path.pop()` must run even on the `continue` path.** Use `continue` (not `return`) for pruning, and never `return` from inside the loop unless the choices are sorted and you are pruning the rest — in which case say why.
- 4 **The three axis questions.** (1) Is the choice set the *remaining* elements (combinations) or *all* elements (permutations)? (2) Can an element be reused? (3) Do duplicate answers need removing? The answers determine the loop bound and the dedup rule, and nothing else changes.

- 5 **Combinations loop from `start`** . `for i in range(start, n)` . This enforces non-decreasing index order and prevents `[1,2]` and `[2,1]` both appearing.
- 6 **Permutations loop from `0` with a `used` array** . `for i in range(n): if used[i]: continue` . The `used` array is what stops an element being picked twice.
- 7 **Reuse allowed -> recurse with `i` ; reuse forbidden -> recurse with `i + 1`** . That single character is the difference between `Combination Sum (39)` and `Combination Sum II (40)` .
- 8 **The dedup rule** . Sort the input first, then skip `if i > start and a[i] == a[i-1]: continue` . Note it is `i > start` , not `i > 0` — at the top of a level you must be allowed to pick the first duplicate.
- 9 **Prune with sorting** . If the choices are sorted, `if a[i] > remaining: break` replaces `continue` and prunes the whole suffix. Always sort when the problem permits it.
- 10 **Complexity** . Subsets and combinations are $O(2^n)$; permutations are $O(n!)$; grid search with a path length `L` is $O(m \cdot n \cdot 4^L)$. State the bound — backtracking is only acceptable when the input is small, and saying why is part of the answer.
- 11 **When to stop backtracking and use DP instead** . If the question asks for a *count* or an *optimum* rather than an *enumeration* , and the subproblems overlap, it is DP . `Target Sum (494)` and `Combination Sum IV (377)` both have backtracking statements and DP answers; only `377` genuinely needs DP for the counting version.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
282	Expression Add Operators	Hard	1
93	Restore IP Addresses	Medium	6
131	Palindrome Partitioning	Medium	5
756	Pyramid Transition Matrix	Medium	3
1079	Letter Tile Possibilities	Medium	2

#	PROBLEM	LEVEL	ASKED
254	Factor Combinations †	Medium	1
291	Word Pattern II †	Medium	2
216	Combination Sum III	Medium	1
267	Palindrome Permutation II †	Medium	1
1258	Synonymous Sentences †	Medium	1
1718	Construct the Lexicographically Largest Valid Sequence	Medium	2
89	Gray Code	Medium	1
306	Additive Number	Medium	1
357	Count Numbers with Unique Digits	Medium	1
784	Letter Case Permutation	Medium	1
1980	Find Unique Binary String	Medium	1
1863	Sum of All Subset XOR Totals	Easy	1
320	Generalized Abbreviation †	Medium	1
401	Binary Watch	Easy	1
1415	The k-th Lexicographical String of All Happy Strings of Length n	Medium	1
842	Split Array into Fibonacci Sequence	Medium	0
1239	Maximum Length of a Concatenated String with Unique Characters	Medium	0
1593	Split a String Into the Max Number of Unique Substrings	Medium	0
2044	Count Number of Maximum Bitwise-OR Subsets	Medium	0
2048	Next Greater Numerically Balanced Number	Medium	0
2375	Construct Smallest Number From DI String	Medium	0
2597	The Number of Beautiful Subsets	Medium	0
2698	Find the Punishment Number of an Integer	Medium	0

Backtracking – Subsets and Combinations

THE KERNEL

Subsets is the ‘include or exclude’ binary tree; combinations is subsets with a fixed size. Both are the `start` -index loop with no invalid state to check.

26

 PROBLEMS
MAPPED

2

EASY

24

MEDIUM

0

HARD

The Boundary Rules

- 1 **Subsets** (78): record the path at *every* node, not just at leaves. The number of answers is 2^n , and each corresponds to a node in the recursion tree.
- 2 There are two equivalent formulations: the index loop (`for i in range(start, n)`) and the include/exclude pair (`bt(i+1, path + [a[i]])`) / `bt(i+1, path)`). The index loop is more efficient because it avoids copying; the include/exclude form is easier to prove correct. Prefer the index loop.
- 3 **Subsets II** (90) adds duplicates: sort, then `if i > start and a[i] == a[i-1]: continue`. That is the entire change.
- 4 **Combinations** (77) adds a size limit: `if len(path) == k: result.append(path.copy()); return`.

- 5 **Prune combinations with an arithmetic bound.** In `range(start, n)` you only need `range(start, n - (k - len(path)) + 1)` — there is no point starting so late that you cannot reach `k` elements. This is a genuine optimisation that interviewers reward.
- 6 **Combination Sum III (216)** fixes both the size (`k`) and the sum (`n`) over digits 1-9. The pruning is `if remaining < 0: break`.
- 7 **Letter Combinations of a Phone Number (17)** is a Cartesian product, not a subset problem: the loop is over the *digits*, and the choice set per digit is fixed by a lookup table. Same template, different `choices`.
- 8 **Generate Parentheses (22)** is a constrained construction: track `open_count` and `close_count` and only recurse when `open < n` or `close < open`. The constraint is the pruning, and it produces exactly the Catalan number of valid strings with no post-filter.
- 9 **Factor Combinations (254)** loops from the last factor onward (`range(start, isqrt(n) + 1)`) and pushes a copy of the path at every step where the remainder is 1. The `isqrt` bound avoids duplicate factorisations.
- 10 **Letter Case Permutation (784)** branches on each character: digits have one branch, letters have two. `path + [c.swapcase()]` is the second branch.
- 11 **Count of Maximum OR subsets (2044)** is a subset enumeration with an aggregate — no path list at all, just a running OR and a count. Recognise when the ‘path’ can be reduced to a scalar.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
39	Combination Sum	Medium	9
93	Restore IP Addresses	Medium	6
40	Combination Sum II	Medium	5
131	Palindrome Partitioning	Medium	5
1079	Letter Tile Possibilities	Medium	2
254	Factor Combinations †	Medium	1

#	PROBLEM	LEVEL	ASKED
89	Gray Code	Medium	1
357	Count Numbers with Unique Digits	Medium	1
784	Letter Case Permutation	Medium	1
1980	Find Unique Binary String	Medium	1
1863	Sum of All Subset XOR Totals	Easy	1
320	Generalized Abbreviation †	Medium	1
401	Binary Watch	Easy	1
842	Split Array into Fibonacci Sequence	Medium	0
1239	Maximum Length of a Concatenated String with Unique Characters	Medium	0
1593	Split a String Into the Max Number of Unique Substrings	Medium	0
2044	Count Number of Maximum Bitwise-OR Subsets	Medium	0
2397	Maximum Rows Covered by Columns	Medium	0
2597	The Number of Beautiful Subsets	Medium	0
2698	Find the Punishment Number of an Integer	Medium	0

Backtracking - Permutations and the Dedup Rule

THE KERNEL

Permutations loop over *all* indices with a `used` mask, because order matters. The dedup rule changes from ‘compare with the previous index’ to ‘compare with the previous *unused* index’.

17PROBLEMS
MAPPED**0**

EASY

14

MEDIUM

3

HARD

The Boundary Rules

- 1** **Permutations (46):** `for i in range(n): if used[i]: continue; used[i] = True; path.append(a[i]); bt(); path.pop(); used[i] = False`. Un-choose the mask as well as the path — forgetting the mask reset is a silent wrong answer.
- 2** **Permutations II (47)** dedup rule: sort, then `if i > 0 and a[i] == a[i-1] and not used[i-1]: continue`. The `not used[i-1]` clause is the crucial part: it means ‘this duplicate’s predecessor is not currently in use, so we are at the top of a level and must take the first duplicate only’.
- 3** **Why `not used[i-1]` and not `used[i-1]`?** Because if the previous duplicate is in use, we are extending a path that already contains it, and that is a legitimate distinct permutation. Get this backwards and you silently drop valid answers. This is the single most subtle line in the backtracking part.

- 4 **Swap-based permutations** are the $O(1)$ -extra-space alternative: swap `a[i]` with `a[start]`, recurse, swap back. But the swap version cannot dedup easily (the array is not sorted after swapping), so for 47 the `used`-mask version is strictly better.
- 5 **Next Permutation** (31) is not backtracking at all — it is a three-step in-place algorithm: find the longest non-increasing suffix, swap the pivot with the smallest larger element in the suffix, then reverse the suffix. Memorise it; it is asked constantly.
- 6 **Permutation Sequence** (60) needs the k th permutation without enumerating: use the factorial number system. `k // (n-1)!` picks the index from the remaining sorted digits. This is $O(n^2)$ and is the answer to ‘can you do better than enumerating?’. 31 plus `k-1` iterations is the fallback.
- 7 **Non-decreasing Subsequences** (491) is a subset problem whose dedup cannot use sorting (the order must be preserved). The fix is a per-level `set` of values already tried at this depth. Recognise that when sorting is forbidden, the dedup mechanism must change.
- 8 **Letter Tile Possibilities** (1079) is ‘count all permutations of all subsets’ — a Counter plus recursion over the distinct letters. The Counter replaces the `used` array and handles duplicates for free.
- 9 **Beautiful Arrangement** (526) and **Count Arrangement** problems are permutation enumeration with a divisibility predicate; the pruning is `if (i+1) % pos == 0 or pos % (i+1) == 0`.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1079	Letter Tile Possibilities	Medium	2
267	Palindrome Permutation II †	Medium	1
1718	Construct the Lexicographically Largest Valid Sequence	Medium	2
484	Find Permutation †	Medium	2
526	Beautiful Arrangement	Medium	1
1508	Range Sum of Sorted Subarray Sums	Medium	1

#	PROBLEM	LEVEL	ASKED
903	Valid Permutations for DI Sequence	Hard	0
996	Number of Squareful Arrays	Hard	0
1238	Circular Permutation in Binary Representation	Medium	0
1850	Minimum Adjacent Swaps to Reach the Kth Smallest Number	Medium	0
1930	Unique Length-3 Palindromic Subsequences	Medium	0
2375	Construct Smallest Number From DI String	Medium	0

Backtracking - Target Sum and Partitioning

THE KERNEL

Same template, but the pruning is arithmetic: sort the choices and break as soon as the remaining target goes negative. The problem becomes a search over a constrained sum.

24

 PROBLEMS
MAPPED

0

EASY

21

MEDIUM

3

HARD

The Boundary Rules

- 1 **Combination Sum (39)**: reuse allowed, so recurse with `i` (not `i + 1`). Prune with `if a[i] > remaining: break` after sorting.
- 2 **Combination Sum II (40)**: each element used once, so recurse with `i + 1`; add the dedup line. Two changes from 39, both one character.
- 3 **The sort-and-break rule**. Sorting is what converts `continue` into `break` and gives you real pruning. Without sorting, `Combination Sum` is still correct but exponentially slower on adversarial inputs.
- 4 **Target Sum (494)** has a backtracking solution (`+` or `-` for each element) and a DP solution. The DP reduction is the important part:
 $\text{sum}(P) - \text{sum}(N) = \text{target}$ and $\text{sum}(P) + \text{sum}(N) = \text{total}$, so $\text{sum}(P) = (\text{target} + \text{total}) / 2$. If $(\text{target} + \text{total})$ is odd or negative, the answer is zero. The problem then becomes 'count subsets summing to $(\text{target} + \text{total}) / 2$ ' — a 0/1 knapsack count. That reduction is the interview.

- 5 **Partition to K Equal Sum Subsets** (698) is the hardest member. Two accelerations matter: (1) sort descending and put the largest elements first, because they are the most constrained; (2) skip a bucket if it is empty and the element failed there (`if bucket[j] == 0: break`). Without both, the exponential blow-up times out.
- 6 **Matchsticks to Square** (473) is 698 with `k = 4` . Same code, smaller constant.
- 7 **Palindrome Partitioning** (131) is a partition search where the validity check is `s[i:j+1] == s[i:j+1][::-1]` . Precomputing a palindrome table with DP turns the check from $O(n)$ to $O(1)$ and is the expected optimisation.
- 8 **Restore IP Addresses** (93) is 131 with a different validity check (each segment 0-255, no leading zeros) and exactly four segments. The ‘exactly four’ constraint means you prune on the segment count.
- 9 **Expression Add Operators** (282) is the hardest: the choices are `+` `-` *at every split, and multiplication needs a `prev_operand` to undo precedence correctly. The trick is to carry the last operand and pass `cur - prev + prev` val on a `*` . If you cannot derive it, at least state the precedence problem out loud.*
- 10 **Word Break II** (140) is a backtracking over a dictionary plus memoisation on the suffix — otherwise it is exponential. Recognise when backtracking needs a memo to survive the constraints.
- 11 **24 Game** (679) is a backtracking over pairwise reductions: pick two numbers, apply an operator, recurse on the reduced list. The choice space is not ‘elements’ but ‘pairs’, which is a different loop shape.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
140	Word Break II	Hard	8
416	Partition Equal Subset Sum	Medium	6
282	Expression Add Operators	Hard	1
377	Combination Sum IV	Medium	4
638	Shopping Offers	Medium	4

#	PROBLEM	LEVEL	ASKED
291	Word Pattern II †	Medium	2
216	Combination Sum III	Medium	1
1718	Construct the Lexicographically Largest Valid Sequence	Medium	2
306	Additive Number	Medium	1
464	Can I Win	Medium	1
473	Matchsticks to Square	Medium	1
526	Beautiful Arrangement	Medium	1
679	24 Game	Hard	1
1593	Split a String Into the Max Number of Unique Substrings	Medium	0
1986	Minimum Number of Work Sessions to Finish the Tasks	Medium	0
2048	Next Greater Numerically Balanced Number	Medium	0
2397	Maximum Rows Covered by Columns	Medium	0
2464	Minimum Subarrays in a Valid Split †	Medium	0

Backtracking - Grid Search

THE KERNEL

The choice set is the four neighbours. Mark the cell before recursing and unmark it after — that is grid backtracking, and it is DFS with a restore step.

20PROBLEMS
MAPPED**0**

EASY

13

MEDIUM

7

HARD

The Boundary Rules

- 1 The four-neighbour loop.** `for dr, dc in ((1,0),(-1,0),(0,1),(0,-1))`. Define it once at the top of the function; writing four explicit recursive calls is error-prone.
- 2 Mark and unmark.** `grid[r][c] = '#'` before the recursion, `grid[r][c] = original` after. This is the grid version of `path.pop()`. Forgetting the restore means the first path wins and every other path is blocked.
- 3 Word Search (79):** match the character, mark, recurse on the four neighbours with `index + 1`, unmark. The bounds check `0 <= nr < R and 0 <= nc < C` must come *before* the character comparison.
- 4 Word Search II (212)** is 79 repeated for every word, which is $O(\text{words cells } 4^L)$. The correct answer is a **Trie plus one DFS**: build a Trie of all words, then do a single grid DFS that walks the Trie in lockstep. When a Trie node is terminal, record the word. Prune by removing matched words from the Trie. This is the single best example in the book of a data structure eliminating a nested loop.

- 5 **Unique Paths III** (980) must visit *every* non-obstacle cell exactly once, so the goal test is `steps == total_non_obstacle` and not just 'reached the end'. Counting the target cells first is required.
- 6 **Path with Maximum Gold** (1219) is mark/unmark with a value accumulator and a `best` update. It never needs to reach a specific cell.
- 7 **Flood-fill problems that do NOT need unmarking.** `Number of Islands` (200), `Max Area of Island` (695), `Surrounded Regions` (130) are *connected component* problems: once a cell is visited it belongs to the component forever, so there is no restore. **The distinction is whether the question is about a path (restore) or a region (do not restore).** That is the key discrimination in this chapter.
- 8 **Surrounded Regions** (130) has a beautiful trick: instead of finding surrounded regions, find the *unsurrounded* ones by flooding inward from the border, then flip everything untouched. Recognising the complement is the whole problem.
- 9 **Pacific Atlantic Water Flow** (417) is the same complement trick: flood *uphill* from both oceans' borders and intersect the reachable sets. Doing it forward from every cell is $O((mn)^2)$; doing it backward from the borders is $O(mn)$.
- 10 **Sudoku Solver** (37) and **N-Queens** (51) are grid backtracking with row/column/box constraint sets. For 37, three sets of `frozenset`/bitmasks (rows, cols, boxes) make the validity check $O(1)$; scanning the grid each time is 27x slower and is the usual reason a solution times out.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
36	Valid Sudoku	Medium	6
37	Sudoku Solver	Hard	11
51	N-Queens	Hard	8
52	N-Queens II	Hard	1
1219	Path with Maximum Gold	Medium	1
1020	Number of Enclaves	Medium	1
1254	Number of Closed Islands	Medium	0

#	PROBLEM	LEVEL	ASKED
1293	Shortest Path in a Grid with Obstacles Elimination	Hard	0
1905	Count Sub Islands	Medium	0
2328	Number of Increasing Paths in a Grid	Hard	0
2596	Check Knight Tour Configuration	Medium	0
2698	Find the Punishment Number of an Integer	Medium	0
2844	Minimum Operations to Make a Special Number	Medium	0
3020	Find the Maximum Number of Elements in Subset	Medium	0

Backtracking - Constraint Satisfaction

THE KERNEL

The choices are constrained by everything already placed. The engineering is in making the validity check $O(1)$ with auxiliary sets, and in choosing the most-constrained variable first.

22

 PROBLEMS
MAPPED

1

EASY

10

MEDIUM

11

HARD

The Boundary Rules

- 1 **N-Queens (51)**: the constraint is not just ‘same column’ but two diagonals. Track three sets: `cols`, `diag1` (where `r - c` is constant), `diag2` (where `r + c` is constant). Adding and removing from three sets is $O(1)$; scanning the board is $O(n)$.
- 2 **The diagonal encoding is the insight**: two cells are on the same `\` diagonal iff `r - c` is equal, and on the same `/` diagonal iff `r + c` is equal. Memorise both.
- 3 **N-Queens II (52)** is `51` with a counter instead of a board — same code, drop the construction.
- 4 **Sudoku Solver (37)**: three arrays of 9 bitmasks (or sets) for rows, columns and 3x3 boxes. The box index is `(r // 3) * 3 + (c // 3)`. Bitmask arithmetic `(mask & (1 << d))` is fastest; sets are more readable. Choose based on how the interviewer reacts.

- 5 **Most-constrained-variable heuristic.** In Sudoku and similar CSPs, choosing the empty cell with the fewest legal values first prunes dramatically. You do not need it to pass LeetCode, but mentioning it is the difference between a good and an excellent answer.
- 6 **Valid Sudoku** (36) is the *checking* version with no backtracking — one pass with the same three mask arrays. Do not over-engineer it.
- 7 **Partition to K Equal Sum Subsets** (698) is a CSP: the variables are elements, the constraints are bucket capacities. Sorting descending and breaking on an empty bucket that already failed are the two heuristics that make it pass.
- 8 **Matchsticks to Square** (473) is 698 with $k = 4$ and a divisibility pre-check ($\text{sum} \% 4 \neq 0 \rightarrow \text{impossible}$).
- 9 **Verbal arithmetic** (1307), **cryptarithm** (2850 -ish) and **Maximum Number of Achievable Transfer Requests** (1601) are constraint problems where the search space must be reduced by an algebraic observation before backtracking becomes viable. 1601 reduces to ‘subset of requests whose net degree change is all zero’.
- 10 The universal CSP checklist to say out loud: (1) what are the variables, (2) what is the domain of each, (3) what are the constraints, (4) which variable is most constrained, (5) can any constraint be encoded as $O(1)$ state.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
79	Word Search	Medium	7
212	Word Search II	Hard	9
1275	Find Winner on a Tic Tac Toe Game	Easy	5
282	Expression Add Operators	Hard	1
980	Unique Paths III	Hard	2
291	Word Pattern II †	Medium	2
267	Palindrome Permutation II †	Medium	1
1718	Construct the Lexicographically Largest Valid Sequence	Medium	2

#	PROBLEM	LEVEL	ASKED
1601	Maximum Number of Achievable Transfer Requests	Hard	2
52	N-Queens II	Hard	1
679	24 Game	Hard	1
1307	Verbal Arithmetic Puzzle	Hard	0
1655	Distribute Repeating Integers	Hard	0
2014	Longest Subsequence Repeated k Times	Hard	0
2305	Fair Distribution of Cookies	Medium	0
2464	Minimum Subarrays in a Valid Split †	Medium	0
2741	Special Permutations	Medium	0

PART XI

Graphs & Grids

Graphs and grids. Flood fill, unweighted shortest path, topological order, Union-Find, and the three weighted-shortest-path algorithms and their conditions.

Grid Flood Fill - The Islands Family

THE KERNEL

A grid is an implicit graph: each cell is a node, each of its four neighbours is an edge. ‘Connected component’ questions are then a plain DFS/BFS with a visited marker — and there is no unmarking.

34

 PROBLEMS
MAPPED

2

EASY

26

MEDIUM

6

HARD

The Boundary Rules

- 1 Mark on entry, not on push.** `grid[r][c] = '0'` at the top of the DFS prevents both infinite recursion and duplicate work. Marking on push works for BFS but not for DFS — pick ‘mark on entry’ and use it for both.
- 2 In-place marking vs a visited set.** For 0/1 grids, mutating the grid is $O(1)$ extra space and is the preferred answer. Mention that you are mutating the input and offer to restore it if the caller needs the original.
- 3 Islands family, one template:** `Number of Islands` (200) counts components; `Max Area of Island` (695) returns the largest component size; `Island Perimeter` (463) adds 4 per cell and subtracts 2 per shared edge; `Number of Closed Islands` (1254) excludes components that touch the border; `Number of Enclaves` (1020) counts the cells in non-border components; `Count Sub Islands` (1905) counts islands in `grid2` fully contained in an island of `grid1`.

- 4 **Flood Fill** (733) is the same traversal with a colour change — the warm-up version of this chapter.
- 5 **The complement trick.** `Surrounded Regions` (130) floods from the *border* inward to find the escaping cells, then flips everything else. `Pacific Atlantic` (417) floods uphill from both ocean borders and intersects. Both avoid the $O((mn)^2)$ forward search. Whenever a problem asks about cells that can reach a boundary, search *from* the boundary.
- 6 **Making A Large Island** (827) requires *labelling* each island with an id and recording its size, then, for each 0 cell, summing the distinct neighbour ids. The distinctness check is essential — two neighbours can belong to the same island and would be double counted.
- 7 **Number of Provinces** (547) is the adjacency-matrix version of 200. The same DFS works, but Union-Find (chapter 55) is usually cleaner.
- 8 **Boundary checks.** Write a helper `in_bounds(r, c)` once. Inlining four comparisons in four places is how index bugs get in.
- 9 **Iterative vs recursive DFS.** A 300x300 grid can have a 90,000-deep recursion path in the worst case (a spiral), which exceeds Python's default limit. Use an explicit stack for grid DFS, or raise the limit. This is a real failure mode, not a theoretical one.
- 10 **Multi-source BFS vs DFS.** DFS answers 'what is connected'; BFS answers 'how far'. If the question contains 'nearest', 'minimum steps', or 'distance', it is chapter 52, not this one.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
339	Nested List Weight Sum †	Medium	3
79	Word Search	Medium	7
286	Walls and Gates †	Medium	7
317	Shortest Distance from All Buildings †	Hard	4
364	Nested List Weight Sum II †	Medium	1
529	Minesweeper	Medium	5

#	PROBLEM	LEVEL	ASKED
323	Number of Connected Components in an Undirected Graph †	Medium	6
490	The Maze †	Medium	7
694	Number of Distinct Islands †	Medium	4
261	Graph Valid Tree †	Medium	4
302	Smallest Rectangle Enclosing Black Pixels †	Hard	1
1267	Count Servers that Communicate	Medium	1
305	Number of Islands II †	Hard	2
1765	Map of Highest Peak	Medium	2
565	Array Nesting	Medium	2
1020	Number of Enclaves	Medium	1
547	Number of Provinces	Medium	0
711	Number of Distinct Islands II †	Hard	0
1034	Coloring A Border	Medium	0
1254	Number of Closed Islands	Medium	0
1584	Min Cost to Connect All Points	Medium	0
1905	Count Sub Islands	Medium	0
1970	Last Day Where You Can Still Cross	Hard	0
2658	Maximum Number of Fish in a Grid	Medium	0
2684	Maximum Number of Moves in a Grid	Medium	0
2812	Find the Safest Path in a Grid	Medium	0
3055	Top Percentile Fraud †	Medium	0

Graph BFS - Unweighted Shortest Path

THE KERNEL

BFS explores in rings of increasing distance, so the first time you reach a node you have reached it by a shortest path. That is the entire justification for using BFS instead of DFS here.

38PROBLEMS
MAPPED**0**

EASY

27

MEDIUM

11

HARD

The Boundary Rules

- 1 Mark visited when you enqueue, not when you dequeue.** Marking on dequeue lets the same node enter the queue multiple times and can blow up the frontier exponentially. This is the most common BFS performance bug.
- 2 Track distance in the queue,** either by storing `(node, dist)` tuples or by the level-size idiom `( for _ in range(len(q)): )` with an outer counter. The level-size idiom is more memory-efficient; the tuple is easier to get right.
- 3 Early exit.** Return as soon as you pop or enqueue the target — there is no need to finish the level, because BFS guarantees the first arrival is optimal. `Word Ladder ( 127 )` is the classic case where failing to exit early is the difference between passing and timing out.
- 4 Building the adjacency list.** `defaultdict(list)`, then one pass over the edge list. For an undirected graph add both directions — forgetting the reverse edge is the most common graph bug in existence.

- 5 **Implicit graphs.** `Word Ladder` (127) has edges between words differing by one character — generating neighbours is $O(26 L)$ with wildcard bucketing (`h t` -> `[hot, hit]`) or $O(n * L)$ by pairwise comparison. The wildcard bucket is the optimisation that makes it pass.
- 6 **Open the Lock** (752) and **Minimum Genetic Mutation** (433) are the same implicit-graph BFS with a small alphabet. Recognise the family: *state space + allowed moves + fewest moves*.
- 7 **Bidirectional BFS** halves the branching factor: expand from both ends and stop when the frontiers intersect. `Word Ladder` is the canonical use case. It is $O(b^{(d/2)})$ instead of $O(b^d)$. Mention it as an optimisation; implement it only if asked.
- 8 **Shortest Path in a Grid with Obstacles Elimination** (1293) adds a resource to the state: the visited set becomes `(r, c, k_remaining)`, and a cell can be revisited with a *larger* remaining budget. Recognise when the visited key must include extra state.
- 9 **Shortest Path Visiting All Nodes** (847) is BFS over `(node, visited_bitmask)` — the state is a bitmask, and the answer is the first state where the mask is full. This is the bridge between BFS and bitmask DP (chapter 79).
- 10 **Bus Routes** (815) is BFS over *routes* rather than stops — the insight is that the nodes of your BFS graph need not be the entities in the problem statement.
- 11 **Snakes and Ladders** (909) is a flat BFS over squares with a coordinate conversion. `Minimum Knight Moves` (1197) is a flat BFS over a board with eight moves and a symmetry optimisation.
- 12 If all edges have the same weight, BFS is optimal. If weights differ, you need Dijkstra (chapter 56) or 0-1 BFS (chapter 53). Saying this sentence in an interview signals that you know why BFS works.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
269	Alien Dictionary †	Hard	9
286	Walls and Gates †	Medium	7

#	PROBLEM	LEVEL	ASKED
126	Word Ladder II	Hard	5
323	Number of Connected Components in an Undirected Graph †	Medium	6
490	The Maze †	Medium	7
542	01 Matrix	Medium	4
261	Graph Valid Tree †	Medium	4
1345	Jump Game IV	Hard	3
690	Employee Importance	Medium	2
675	Cut Off Trees for Golf Event	Hard	2
310	Minimum Height Trees	Medium	2
1466	Reorder Routes to Make All Paths Lead to the City Zero	Medium	1
1926	Nearest Exit from Entrance in Maze	Medium	1
433	Minimum Genetic Mutation	Medium	0
773	Sliding Puzzle	Hard	0
787	Cheapest Flights Within K Stops	Medium	0
815	Bus Routes	Hard	0
863	All Nodes Distance K in Binary Tree	Medium	0
909	Snakes and Ladders	Medium	0
934	Shortest Bridge	Medium	0
1129	Shortest Path with Alternating Colors	Medium	0
1197	Minimum Knight Moves †	Medium	0
1298	Maximum Candies You Can Get from Boxes	Hard	0
1730	Shortest Path to Get Food †	Medium	0
2059	Minimum Operations to Convert Number	Medium	0
2101	Detonate the Maximum Bombs	Medium	0
2115	Find All Possible Recipes from Given Supplies	Medium	0
2146	K Highest Ranked Items Within a Price Range	Medium	0
2258	Escape the Spreading Fire	Hard	0
2812	Find the Safest Path in a Grid	Medium	0

#	PROBLEM	LEVEL	ASKED
2998	Minimum Number of Operations to Make X and Y Equal	Medium	0
3286	Find a Safe Walk Through a Grid	Medium	0

Multi-Source BFS and 0-1 BFS

THE KERNEL

Seed the queue with *all* the starting points at once, and every cell gets the distance to its nearest source in one pass. For edge weights in $\{0, 1\}$, use a deque and push 0-weight edges to the front.

16PROBLEMS
MAPPED**0**

EASY

13

MEDIUM

3

HARD

The Boundary Rules

- 1 **Multi-source template.** `q = deque(all_sources)` ; then a plain BFS. The only change from chapter 52 is the initialisation. But the payoff is huge: it turns ‘for each cell, find the nearest source’ from $O(mn * s)$ into $O(mn)$.
- 2 `Rotting Oranges` (994) seeds with every rotten orange and counts levels; the answer is the number of levels until the queue empties, and you must check that no fresh orange remains. Forgetting that check is the classic wrong answer.
- 3 `01 Matrix` (542) seeds with every `0` and BFSs outward. The DP alternative (two passes, one from the top-left and one from the bottom-right) is $O(mn)$ with no queue and is worth knowing as a cross-check.
- 4 `As Far from Land as Possible` (1162) is `542` plus a maximum over the distances. `Walls and Gates` (286) is the same problem with a different sentinel.
- 5 **0-1 BFS.** When edges have weight 0 or 1, a deque replaces the priority queue: `appendleft` for weight-0 edges and `append` for weight-1 edges. The deque stays sorted by distance, so it is $O(V + E)$ instead of Dijkstra’s $O(E \log V)$.

- 6 The 0-1 BFS correctness argument: at any moment the deque contains at most two distinct distances, `d` and `d + 1`, and both pushes preserve that. State this if asked why a deque suffices.
- 7 **When a node is finalised in 0-1 BFS**, you may need a `visited` check on pop because a node can be pushed twice with different distances. Pop-and-skip-if-visited is the standard guard.
- 8 **Multi-source + grid** is also the right frame for `Shortest Bridge (934)` (label one island, then multi-source BFS to the other) and `Minimum Knight Moves` with several targets.
- 9 **Do not use multi-source BFS when the sources have different costs.** Different starting costs require a priority queue (Dijkstra with all sources pushed at their own initial distance).
- 10 `Nearest Exit from Entrance in a Maze (1926)` is a single-source BFS with a border test instead of a target — recognise that ‘reaches the boundary’ is a goal condition.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1091	Shortest Path in Binary Matrix	Medium	6
1765	Map of Highest Peak	Medium	2
1926	Nearest Exit from Entrance in Maze	Medium	1
1129	Shortest Path with Alternating Colors	Medium	0
1293	Shortest Path in a Grid with Obstacles Elimination	Hard	0
1730	Shortest Path to Get Food †	Medium	0
2045	Second Minimum Time to Reach Destination	Hard	0
2258	Escape the Spreading Fire	Hard	0
2812	Find the Safest Path in a Grid	Medium	0
2998	Minimum Number of Operations to Make X and Y Equal	Medium	0
3286	Find a Safe Walk Through a Grid	Medium	0

Topological Sort

THE KERNEL

Ordering with prerequisites is a DAG. Kahn's algorithm is BFS on in-degrees; the DFS version is postorder reversed. If a cycle exists, the topological order is incomplete — and that incompleteness is usually the actual answer.

25PROBLEMS
MAPPED**3**

EASY

12

MEDIUM

10

HARD

The Boundary Rules

- 1 **Kahn's algorithm.** Build `indegree[]` and the adjacency list; seed the queue with every node of in-degree 0; pop, append to the order, and decrement each neighbour's in-degree, enqueueing at zero. **If the final order is shorter than the node count, there is a cycle.** That check is the answer to `Course Schedule` (207).
- 2 The `indegree` construction is where the bugs live. For each edge `(u, v)`: `adj[u].append(v)`; `indegree[v] += 1`. Getting the direction backwards produces a reversed order that still looks plausible.
- 3 **DFS version.** Postorder DFS, appending a node *after* its descendants, then reverse the result. Cycle detection needs a three-colour state: white (unvisited), grey (on the stack), black (done). A grey neighbour means a cycle. The three colours are mandatory — a plain `visited` set cannot distinguish 'on the current path' from 'finished'.
- 4 `Course Schedule` (207) asks 'is there a cycle'; `Course Schedule II` (210) asks for the order. One algorithm, different return value.

- 5 **Alien Dictionary** (269) is topological sort where you must first *derive the edges* by comparing adjacent words. Two edge cases decide correctness: a prefix word placed after a longer word is invalid, and fewer than `k` distinct letters means you must append the missing ones in arbitrary order.
- 6 **Minimum Height Trees** (310) is *reverse* Kahn: repeatedly peel leaves; the last one or two nodes remaining are the roots. The insight that MHT roots are the tree's centroids is what makes it $O(n)$ instead of $O(n^2)$.
- 7 **Find Eventual Safe States** (802) is reverse topological sort — work backwards from the terminal nodes. Or colour-DFS forward. Both are accepted; the reverse Kahn version is more elegant.
- 8 **Sort Items by Groups** (1203) is two-level topological sort: once over the groups and once within each group, then merge. It is the hardest problem in this chapter.
- 9 **Sequence Reconstruction** (444) and **Parallel Courses** (1136) are direct applications. 444 adds a uniqueness requirement: the order is reconstructible iff the queue never holds more than one node at a time.
- 10 **When to reach for topological sort.** Any problem with 'prerequisite', 'before', 'dependency', 'order', or 'can be finished' language, plus any DAG reachability question.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
953	Verifying an Alien Dictionary	Easy	4
1557	Minimum Number of Vertices to Reach All Nodes	Medium	2
631	Design Excel Sum Formula †	Hard	1
2050	Parallel Courses III	Hard	1
444	Sequence Reconstruction †	Medium	1
1136	Parallel Courses †	Medium	1
1791	Find Center of Star Graph	Easy	1
802	Find Eventual Safe States	Medium	0

TOPOLOGICAL SORT

#	PROBLEM	LEVEL	ASKED
913	Cat and Mouse	Hard	0
1059	All Paths from Source Lead to Destination †	Medium	0
1203	Sort Items by Groups Respecting Dependencies	Hard	0
1462	Course Schedule IV	Medium	0
1632	Rank Transform of a Matrix	Hard	0
1857	Largest Color Value in a Directed Graph	Hard	0
2115	Find All Possible Recipes from Given Supplies	Medium	0
2192	All Ancestors of a Node in a Directed Acyclic Graph	Medium	0
2204	Distance to a Cycle in Undirected Graph †	Hard	0
2360	Longest Cycle in a Graph	Hard	0
2392	Build a Matrix With Conditions	Hard	0
2695	Array Wrapper	Easy	0
2856	Minimum Array Length After Pair Removals	Medium	0

Union-Find (Disjoint Set Union)

THE KERNEL

Two operations: `find(x)` returns a representative; `union(a, b)` merges two sets. With path compression and union by size it is effectively $O(1)$ per operation, and it answers connectivity questions incrementally.

42

 PROBLEMS
MAPPED

0

EASY

24

MEDIUM

18

HARD

The Boundary Rules

- 1 **The implementation, memorised.** `parent = list(range(n))`, `size = [1] * n`; `find` with path halving (`while parent[x] != x: parent[x] = parent[parent[x]]; x = parent[x]`); `union` by attaching the smaller root to the larger and updating `size`.
- 2 **Both optimisations matter.** Path compression alone gives $O(\log n)$ amortised; union by size alone gives $O(\log n)$; together they give $O(\alpha(n))$, which is effectively constant. Interviewers expect both.
- 3 **Use Union-Find when the edges arrive incrementally**, or when the question is ‘are these connected’, or when you need to count components after a sequence of merges. Use DFS/BFS when you have the whole graph and one connectivity question.
- 4 **Redundant Connection** (684): process edges one at a time; the first edge whose endpoints are already in the same set is the redundant one. This is Union-Find’s canonical use — an incremental cycle test.

- 5 **Redundant Connection II** (685) is the directed version and is much harder: handle the two cases (a node with in-degree 2, or a cycle). The in-degree-2 check comes first, and if both exist the answer is the edge that both creates the cycle and is a second parent.
- 6 **Accounts Merge** (721)**: union the emails within each account, then group by root and sort. The `find`-as-dict-key step is the pattern.
- 7 **Satisfiability of Equality Equations** (990): union all `==` first, then check that no `!=` has both sides in the same set. Order matters — two passes, not one.
- 8 **Number of Provinces** (547) and **Number of Connected Components** (323): count roots by `sum(1 for i in range(n) if find(i) == i)`. Remember to `find` first — a node whose parent chain was compressed lazily may not point at the root directly.
- 9 **Largest Component Size by Common Factor** (952): union each number with its prime factors, using the primes as pseudo-nodes. The reduction from ‘shared factor’ to ‘shared prime’ is the insight.
- 10 **Min Cost to Connect All Points** (1584) is Kruskal’s MST: sort all edges by weight and union greedily, skipping edges whose endpoints are already connected. Union-Find is the engine; the greedy is the algorithm.
- 11 **Couples Holding Hands** (765) is a counting problem solvable by Union-Find or by a greedy swap pass. The Union-Find version counts components and answers `n - components`.
- 12 **Union-Find cannot un-merge**. If the problem needs deletion, you need a different structure (offline reverse processing, or a dynamic connectivity structure). Recognising this limitation is itself worth saying.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
128	Longest Consecutive Sequence	Medium	9
323	Number of Connected Components in an Undirected Graph †	Medium	6
130	Surrounded Regions	Medium	4

#	PROBLEM	LEVEL	ASKED
261	Graph Valid Tree †	Medium	4
305	Number of Islands II †	Hard	2
685	Redundant Connection II	Hard	1
737	Sentence Similarity II †	Medium	1
1361	Validate Binary Tree Nodes	Medium	1
839	Similar String Groups	Hard	0
924	Minimize Malware Spread	Hard	0
952	Largest Component Size by Common Factor	Hard	0
1061	Lexicographically Smallest Equivalent String	Medium	0
1101	The Earliest Moment When Everyone Become Friends †	Medium	0
1102	Path With Maximum Minimum Value †	Medium	0
1135	Connecting Cities With Minimum Cost †	Medium	0
1168	Optimize Water Distribution in a Village †	Hard	0
1202	Smallest String With Swaps	Medium	0
1254	Number of Closed Islands	Medium	0
1319	Number of Operations to Make Network Connected	Medium	0
1489	Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree	Hard	0
1579	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	0
1627	Graph Connectivity With Threshold	Hard	0
1631	Path With Minimum Effort	Medium	0
1697	Checking Existence of Edge Length Limited Paths	Hard	0
1722	Minimize Hamming Distance After Swap Operations	Medium	0
1724	Checking Existence of Edge Length Limited Paths II †	Hard	0
1905	Count Sub Islands	Medium	0
1970	Last Day Where You Can Still Cross	Hard	0
1998	GCD Sort of an Array	Hard	0
2092	Find All People With Secret	Hard	0

UNION-FIND (DISJOINT SET UNION)

#	PROBLEM	LEVEL	ASKED
2316	Count Unreachable Pairs of Nodes in an Undirected Graph	Medium	0
2421	Number of Good Paths	Hard	0
2492	Minimum Score of a Path Between Two Cities	Medium	0
2493	Divide Nodes Into the Maximum Number of Groups	Hard	0
2709	Greatest Common Divisor Traversal	Hard	0
2812	Find the Safest Path in a Grid	Medium	0

Weighted Shortest Path - Dijkstra, Bellman-Ford, Floyd

THE KERNEL

Three algorithms, three conditions. All weights non-negative -> Dijkstra. Negative weights allowed -> Bellman-Ford. All-pairs with small n -> Floyd-Warshall.

34

 PROBLEMS
MAPPED

0

EASY

20

MEDIUM

14

HARD

The Boundary Rules

- 1 **Dijkstra, memorised.** `dist = {src: 0}`; a min-heap of `(distance, node)` seeded with `(0, src)`; pop; **if the popped distance exceeds the stored distance, skip** (lazy deletion); relax each edge and push improved distances. The lazy-deletion skip is what makes the `visited` set unnecessary and keeps the code short.
- 2 **Dijkstra fails on negative edges.** It is greedy: once a node is finalised it is never revisited. State this limitation before using it — it is a standard interview question.
- 3 **Complexity** is $O(E \log V)$. Say it out loud.
- 4 **Bellman-Ford** relaxes all edges $V - 1$ times. $O(VE)$. *Use it when edges can be negative, or when the number of hops is limited — `Cheapest Flights Within K Stops` (787) is Bellman-Ford with exactly $K + 1$ relaxation rounds, and each round must use a copy* of the previous distance array so a single round cannot chain. Forgetting the copy is the classic bug.*

- 5 **Network Delay Time** (743) is textbook Dijkstra: run from the source and take the maximum finalised distance. If any node is unreachable, return -1.
- 6 **Path With Minimum Effort** (1631) and **Swim in Rising Water** (778) are Dijkstra on a grid where the 'distance' is a running maximum rather than a sum. The relaxation becomes `new = max(d, |height diff|)` or `new = max(d, grid[nr][nc])`. Recognising that Dijkstra works for any *monotone* combination, not just sums, is the transferable insight.
- 7 **Path with Maximum Probability** (1514) is Dijkstra with a max-heap on probabilities — the same algorithm with the comparison flipped. Dijkstra works on any *order-preserving* semiring.
- 8 **Number of Ways to Arrive at Destination** (1976) adds a counting array alongside `dist`: when a shorter path is found, reset the count; when an equal path is found, add to it. This is the standard 'shortest-path counting' extension.
- 9 **Floyd-Warshall** is three nested loops with `k` outermost — `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`. **The `k` loop must be outermost**; swapping the order gives a wrong answer. $O(V^3)$, so only for `V <= 400`.
- 10 **Evaluate Division** (399) is a graph problem in disguise: `a / b = 2` is a weighted edge. Build the graph, then DFS or Floyd-Warshall over ratios. Recognising the graph structure is the whole problem.
- 11 **Path with Maximum Minimum Value** (1102) is the grid version of 778 and is also solvable by Union-Find with descending edge weights — know both.
- 12 **Choosing the algorithm in an interview.** Say: 'all weights are non-negative, so Dijkstra; if hops are limited I need Bellman-Ford; if I needed all pairs with n under 400, Floyd-Warshall.' That one sentence covers the whole chapter.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
133	Clone Graph	Medium	6
269	Alien Dictionary †	Hard	9
542	01 Matrix	Medium	4

#	PROBLEM	LEVEL	ASKED
403	Frog Jump	Hard	5
505	The Maze II †	Medium	3
1514	Path with Maximum Probability	Medium	1
310	Minimum Height Trees	Medium	2
499	The Maze III †	Hard	1
882	Reachable Nodes In Subdivided Graph	Hard	0
1102	Path With Maximum Minimum Value †	Medium	0
1129	Shortest Path with Alternating Colors	Medium	0
1334	Find the City With the Smallest Number of Neighbors at a Threshold Distance	Medium	0
1368	Minimum Cost to Make at Least One Valid Path in a Grid	Hard	0
1489	Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree	Hard	0
1786	Number of Restricted Paths From First to Last Node	Medium	0
1928	Minimum Cost to Reach Destination in Time	Hard	0
2045	Second Minimum Time to Reach Destination	Hard	0
2093	Minimum Cost to Reach City With Discounts †	Medium	0
2203	Minimum Weighted Subgraph With the Required Paths	Hard	0
2290	Minimum Obstacle Removal to Reach Corner	Hard	0
2473	Minimum Cost to Buy Apples †	Medium	0
2577	Minimum Time to Visit a Cell In a Grid	Hard	0
2642	Design Graph With Shortest Path Calculator	Hard	0
2662	Minimum Cost of a Path With Special Roads	Medium	0
2714	Find Shortest Path with K Hops †	Hard	0
2812	Find the Safest Path in a Grid	Medium	0
2976	Minimum Cost to Convert String I	Medium	0
3112	Minimum Time to Visit Disappearing Nodes	Medium	0

Bipartite Graphs and Colouring

THE KERNEL

Two-colour the graph by BFS/DFS. If you ever need a third colour, the graph is not bipartite. The colour array doubles as the visited set.

13PROBLEMS
MAPPED**0**

EASY

6

MEDIUM

7

HARD

The Boundary Rules

- 1 The two-colour BFS.** `color = [-1] * n`; for each uncoloured node, colour it 0 and BFS, requiring every neighbour to be the opposite colour. A conflict means not bipartite.
- 2 The colour array replaces `visited`.** `-1` means unvisited, `0` and `1` are the colours. Do not maintain a separate visited set — it is redundant and creates a second source of truth.
- 3 Loop over all nodes.** A graph can be disconnected; starting from node 0 only checks one component. The `for i in range(n): if color[i] == -1` wrapper is mandatory.
- 4 Odd cycles are exactly the obstruction.** A graph is bipartite iff it has no odd-length cycle. Being able to say this is the conceptual answer interviewers want.
- 5 Possible Bipartition** (886) is `Is Graph Bipartite` (785) with the graph built from a ‘dislike’ list. The graph construction is the only new work.
- 6 Is Graph Bipartite** (785) is also Union-Find-solvable: for each edge, union the neighbours of each endpoint together; if an edge’s endpoints end up in the same set, it is not bipartite. That is a nice alternative to offer.

- 7 **Flower Planting With No Adjacent** (1042) is *greedy* colouring, not 2-colouring: each node has at most 3 neighbours, so 4 colours always suffice and you can pick any colour not used by a neighbour. Do not confuse it with bipartiteness.
- 8 **K-colouring in general is NP-hard.** Only 2-colouring is polynomial. Saying this stops you from attempting the wrong thing under pressure.
- 9 **Shortest Path with Alternating Colors** (1129) is BFS where the state includes the colour of the last edge. The visited set becomes `visited[node][color]`. It is the same ‘extra state in the key’ idea as chapter 52’s 1293.
- 10 **Bipartite matching** (Hungarian algorithm, Hopcroft-Karp) is beyond most interviews, but **Maximum Students Taking Exam** (1349) and **Maximum Compatibility Score Sum** (1947) are assignment problems that reduce to it. Know the name and the reduction.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
2076	Process Restricted Friend Requests	Hard	1
924	Minimize Malware Spread	Hard	0
1129	Shortest Path with Alternating Colors	Medium	0
1349	Maximum Students Taking Exam	Hard	0
1947	Maximum Compatibility Score Sum	Medium	0
2328	Number of Increasing Paths in a Grid	Hard	0
2508	Add Edges to Make Degrees of All Nodes Even	Hard	0
2608	Shortest Cycle in a Graph	Hard	0
3020	Find the Maximum Number of Elements in Subset	Medium	0
3203	Find Minimum Diameter After Merging Two Trees	Hard	0

Matrix Manipulation - Rotate, Spiral, and In-Place Marks

THE KERNEL

Four index formulas cover every matrix problem: the transpose swap, the four-way rotation cycle, the spiral boundary shrink, and `divmod` for flattening.

63PROBLEMS
MAPPED**24**

EASY

36

MEDIUM

3

HARD

The Boundary Rules

- 1 **Transpose** is `for i in range(n): for j in range(i+1, n): a[i][j], a[j][i] = a[j][i], a[i][j]`. The `i+1` start is what avoids double-swapping.
- 2 **Rotate 90 clockwise** = transpose, then reverse each row. **Rotate 90 counter-clockwise** = transpose, then reverse the row order. **Rotate 180** = reverse rows and reverse each row. Two operations, no index arithmetic — far safer than the four-way cycle swap.
- 3 **The four-way cycle** (if you must do it in one pass): `a[i][j] -> a[j][n-1-i] -> a[n-1-i][n-1-j] -> a[n-1-j][i] -> a[i][j]`. The four cells involved are the corners of a rotating square; you only need to iterate `i` over the top-left quadrant.

- 4 **Spiral traversal** (54) is four boundary pointers `top`, `bottom`, `left`, `right` that shrink after each side. **The re-check before the third and fourth sides is mandatory** — on a single-row or single-column remnant you will otherwise emit duplicates. `if top <= bottom` before the bottom row and `if left <= right` before the left column.
- 5 **Spiral generation** (59) is the same four pointers writing instead of reading. **Spiral Matrix III** (885) is a different construction: walk in increasing step lengths, turning at the right moments.
- 6 **Set Matrix Zeroes** (73) is the flagship in-place marking problem. The $O(1)$ -space solution uses the first row and first column as the marker arrays, with two extra booleans for the first row/column themselves. **You must process the first row and column last**, or the markers overwrite data before it is read.
- 7 **Game of Life** (289) is the same idea: encode the next state in a second bit of the existing cell (`0b10` for ‘was alive, now dead’), then do a second pass shifting right. Using two bits instead of a copy is the expected answer.
- 8 **Search a 2D Matrix** (74) flattens to 1-D with `row, col = divmod(mid, n)`. **Search a 2D Matrix II** (240) is the top-right walk (chapter 10). Two problems, two completely different approaches — do not mix them up.
- 9 **Diagonal traversal** (498) has two directions and a boundary flip. The `if r == 0 / if c == 0` flip logic is fiddly; write it as ‘move up-right, and if you exit, re-enter at the correct edge’.
- 10 **Prefix-sum rectangles** (304 , 1314) belong to chapter 16; **matrix DP** (62 , 64 , 221) belongs to chapter 67. Recognise which of the three matrix families a problem belongs to before writing code.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
498	Diagonal Traverse	Medium	3
74	Search a 2D Matrix	Medium	10
36	Valid Sudoku	Medium	6
766	Toeplitz Matrix	Easy	2

#	PROBLEM	LEVEL	ASKED
240	Search a 2D Matrix II	Medium	6
419	Battleships in a Board	Medium	5
118	Pascal's Triangle	Easy	7
304	Range Sum Query 2D - Immutable	Medium	5
1428	Leftmost Column with at Least a One †	Medium	3
311	Sparse Matrix Multiplication †	Medium	4
723	Candy Crush †	Medium	3
1920	Build Array from Permutation	Easy	4
867	Transpose Matrix	Easy	4
1861	Rotating the Box	Medium	3
2125	Number of Laser Beams in a Bank	Medium	1
1929	Concatenation of Array	Easy	2
807	Max Increase to Keep City Skyline	Medium	2
1389	Create Target Array in the Given Order	Easy	3
566	Reshape the Matrix	Easy	2
755	Pour Water †	Medium	1
1560	Most Visited Sector in a Circular Track	Easy	1
1572	Matrix Diagonal Sum	Easy	2
119	Pascal's Triangle II	Easy	2
422	Valid Word Square †	Easy	2
832	Flipping an Image	Easy	1
840	Magic Squares In Grid	Medium	1
1583	Count Unhappy Friends	Medium	1
661	Image Smoother	Easy	1
861	Score After Flipping Matrix	Medium	1
1260	Shift 2D Grid	Easy	1
1329	Sort the Matrix Diagonally	Medium	1
1380	Lucky Numbers in a Matrix	Easy	1

#	PROBLEM	LEVEL	ASKED
1439	Find the Kth Smallest Sum of a Matrix With Sorted Rows	Hard	1
885	Spiral Matrix III	Medium	1
999	Available Captures for Rook	Easy	1
1582	Special Positions in a Binary Matrix	Easy	1
1672	Richest Customer Wealth	Easy	1
1886	Determine Whether Matrix Can Be Obtained By Rotation	Easy	1
1806	Minimum Number of Operations to Reinitialize a Permutation	Medium	1
2022	Convert 1D Array Into 2D Array	Easy	1
2079	Watering Plants	Medium	1
1072	Flip Columns For Maximum Number of Equal Rows	Medium	0
1292	Maximum Side Length of a Square with Sum Less than or Equal to Threshold	Medium	0
1568	Minimum Number of Days to Disconnect Island	Hard	0
1605	Find Valid Matrix Given Row and Column Sums	Medium	0
1975	Maximum Matrix Sum	Medium	0
2028	Find Missing Observations	Medium	1
2326	Spiral Matrix IV	Medium	0
2352	Equal Row and Column Pairs	Medium	0
2482	Difference Between Ones and Zeros in Row and Column	Medium	0
2500	Delete Greatest Value in Each Row	Easy	0
2643	Row With Maximum Ones	Easy	0
2661	First Completely Painted Row or Column	Medium	0
2732	Find a Good Subset of the Matrix	Hard	0
2906	Construct Product Matrix	Medium	0
3033	Modify the Matrix	Easy	0
3035	Maximum Palindromes After Operations	Medium	0
3142	Check if Grid Satisfies Conditions	Easy	0

PART XII

Dynamic Programming

Sixteen chapters. DP is recursion plus a cache; the state design is the skill, and the loop directions are the traps.

DP - How to Think About It

THE KERNEL

DP is recursion plus a cache. If you can write the brute-force recursion and name its state, the DP is mechanical. Never start from the table — start from the recursion.

4

PROBLEMS
MAPPED

1

EASY

3

MEDIUM

0

HARD

The Method

- 1 **The 4-step method.** (1) Write the brute-force recursion. (2) Identify the state — the arguments that fully determine the answer. (3) Add a cache keyed on the state. (4) Convert to a bottom-up table only if it saves memory or the interviewer asks. Steps 1-3 are what you should do in an interview.
- 2 **State design is the whole skill.** The state must be (a) sufficient — it determines the future, and (b) minimal — it has no redundant components. If your state includes an index into the input *and* a derived quantity, the derived quantity is probably redundant.
- 3 **Two DP idioms, and you should pick one per problem.** *Take-it-or-leave-it* ($f(i) = \text{best}(f(i-1), \text{value}(i) + f(i-2))$) for selection problems; *try-every-choice* ($f(i) = \text{best over } j \text{ of } g(f(j), \text{cost}(i,j))$) for partition problems.
- 4 **The $n + 1$ sizing rule.** Allocate tables of size $n + 1$ (or $m + 1 \times n + 1$) and define `dp[i]` as 'the answer for the first i elements'. The extra row/column is the empty prefix, and it removes every `if i == 0` branch from the loop body.

- 5 **Top-down vs bottom-up.** Top-down is easier to write correctly and only computes reachable states. Bottom-up is faster in Python because it avoids function-call overhead, and it enables the rolling-array optimisation. **Write top-down first to get it correct, then convert if you have time.**
- 6 **Rolling arrays.** If `dp[i]` depends only on `dp[i-1]` (and maybe `dp[i-2]`), you can reduce the table to two variables. This is the standard follow-up question. For 2-D tables, iterate in the direction that lets you overwrite safely — for 0/1 knapsack that means iterating the capacity *backwards*.
- 7 **Initialisation is where most DP answers die.** For minimisation, initialise to `inf`; for maximisation, to `-inf` (or 0 if the empty selection is allowed); for counting, to 0 with `dp[0] = 1`. Getting the sentinel wrong produces plausible-looking wrong answers.
- 8 **Order of evaluation.** Bottom-up loops must be ordered so that every cell is computed after its dependencies. For 1-D that is left to right; for interval DP it is *by length*; for tree DP it is postorder; for bitmask DP it is by increasing mask.
- 9 **Counting mod $10^9 + 7$.** When the problem says ‘modulo $10^9 + 7$ ’, apply `% MOD` at every addition. Applying it only at the end overflows in other languages and is a habit interviewers check.
- 10 **The three-question triage.** (1) Is the answer an optimum, a count, or a yes/no? (2) Does the state need one index, two indices, an interval, or a mask? (3) Is there a greedy that is provably correct? If yes to (3), skip the DP entirely — greedy is shorter and faster.

DP - 1D Linear Recurrence

THE KERNEL

`dp[i]` depends on a constant number of previous entries. Two variables suffice. This is the largest DP family and the one most often asked as a warm-up.

55

 PROBLEMS
MAPPED

7

EASY

37

MEDIUM

11

HARD

The Boundary Rules

- 1 **Climbing Stairs** (70), **Fibonacci** (509), **Tribonacci** (1137), **Min Cost Climbing Stairs** (746) are `dp[i] = dp[i-1] + dp[i-2]` with different base cases and different coefficients. Write the two-variable rolling version directly.
- 2 **House Robber** (198) is `dp[i] = max(dp[i-1], dp[i-2] + nums[i])`. **House Robber II** (213) is 198 run twice — once excluding the first house and once excluding the last — because the array is a circle. **House Robber III** (337) is the tree version and belongs to chapter 70.
- 3 **Delete and Earn** (740) is House Robber after a bucket transform: `points[v] = v * count(v)`, then rob the bucket array. Recognising the transform is the whole problem.
- 4 **Decode Ways** (91) is the flagship 1-D string DP `dp[i] = ways to decode the first i characters`; transitions from `dp[i-1]` (single digit, if non-zero) and `dp[i-2]` (two digits, if in 10..26). **Both guards must be checked**, and `'0'` alone contributes nothing.

- 5 **Integer Break** (343) is `dp[i] = max over j of max(j, dp[j]) * max(i-j, dp[i-j])`. The `max(j, dp[j])` is the subtle part — sometimes not breaking is better.
- 6 **Perfect Squares** (279)** is a coin-change problem in disguise: coins are the squares, the target is `n`. Do not invent a new DP.
- 7 **Coin Change** (322) is `dp[i] = min(dp[i - c] + 1)` over all coins `c <= i`. It is the *unbounded* knapsack (chapter 65) and is the single most-asked DP problem in existence. Initialise `dp = [inf] * (n+1)` with `dp[0] = 0`; the answer is `-1` if `dp[n]` is still `inf`.
- 8 **Counting problems with `dp[0] = 1`**. `Count Ways To Build Good Strings` (2466) and `Student Attendance Record II` (552) both have `dp[0] = 1` and add contributions. The `dp[0] = 1` seed means ‘there is one way to do nothing’.
- 9 **State-machine 1-D DP** `Student Attendance Record II` (552) and `Knight Dialer` (935) have `dp[i][state]` with a small `state` dimension. When you need to remember ‘how many consecutive X’, that is a state dimension, not an index.
- 10 **N-th Tribonacci** and similar are also the standard demonstration of matrix exponentiation for $O(\log n)$ — worth knowing as a follow-up, never as the first answer.
- 11 The recognition signal: ‘how many ways’, ‘minimum cost’, or ‘maximum value’ over a *sequence* where each element is processed once and the choice depends on a bounded number of predecessors.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
5	Longest Palindromic Substring	Medium	9
509	Fibonacci Number	Easy	9
1567	Maximum Length of Subarray With Positive Product	Medium	2
935	Knight Dialer	Medium	5
118	Pascal's Triangle	Easy	7

#	PROBLEM	LEVEL	ASKED
740	Delete and Earn	Medium	3
552	Student Attendance Record II	Hard	1
337	House Robber III	Medium	3
746	Min Cost Climbing Stairs	Easy	3
488	Zuma Game	Hard	3
313	Super Ugly Number	Medium	3
96	Unique Binary Search Trees	Medium	1
1025	Divisor Game	Easy	2
2110	Number of Smooth Descent Periods of a Stock	Medium	1
119	Pascal's Triangle II	Easy	2
1137	N-th Tribonacci Number	Easy	1
264	Ugly Number II	Medium	1
397	Integer Replacement	Medium	1
639	Decode Ways II	Hard	1
1387	Sort Integers by The Power Value	Medium	1
790	Domino and Tromino Tiling	Medium	0
983	Minimum Cost For Tickets	Medium	0
1191	K-Concatenation Maximum Sum	Medium	0
1220	Count Vowels Permutation	Hard	0
1411	Number of Ways to Paint N × 3 Grid	Hard	0
1449	Form Largest Integer With Digits That Add up to Target	Hard	0
1531	String Compression II	Hard	0
1696	Jump Game VI	Medium	0
1857	Largest Color Value in a Directed Graph	Hard	0
1871	Jump Game VII	Medium	0
1911	Maximum Alternating Subsequence Sum	Medium	0
2054	Two Best Non-Overlapping Events	Medium	0
2140	Solving Questions With Brainpower	Medium	0

#	PROBLEM	LEVEL	ASKED
2266	Count Number of Texts	Medium	0
2320	Count Number of Ways to Place Houses	Medium	0
2369	Check if There is a Valid Partition For The Array	Medium	0
2370	Longest Ideal Subsequence	Medium	0
2400	Number of Ways to Reach a Position After Exactly k Steps	Medium	0
2466	Count Ways To Build Good Strings	Medium	0
2522	Partition String Into Substrings With Values at Most K	Medium	0
2547	Minimum Cost to Split an Array	Hard	0
2560	House Robber IV	Medium	0
2767	Partition String Into Minimum Beautiful Substrings	Medium	0
2787	Ways to Express an Integer as Sum of Powers	Medium	0
2999	Count the Number of Powerful Integers	Hard	0
3147	Taking Maximum Energy From the Mystic Dungeon	Medium	0
3193	Count the Number of Inversions	Hard	0
3259	Maximum Energy Boost From Two Drinks	Medium	0

DP - Kadane and the Maximum Subarray Family

THE KERNEL

Kadane's algorithm is one line: `cur = max(x, cur + x); best = max(best, cur)`. It is the maximum-subarray DP, and every variant changes exactly one of those two `max` calls.

20

 PROBLEMS
MAPPED

1

EASY

14

MEDIUM

5

HARD

The Boundary Rules

- 1 The invariant.** `cur` is the best sum of a subarray *ending at the current index*. The `max(x, cur + x)` is the decision 'start fresh or extend'. State that invariant and the algorithm is obvious.
- 2** `Maximum Subarray` (53) is the base form. `Maximum Product Subarray` (152) needs two running values — `cur_max` and `cur_min` — because multiplying a negative by a negative flips the ordering. Forgetting `cur_min` is the classic 152 bug.
- 3** `Maximum Sum Circular Subarray` (918) is `max(kadane(a), total - min_kadane(a))`, with one special case: if all values are negative, the circular answer is invalid (it would be the empty subarray) and you must return the plain Kadane result.
- 4** `Maximum Subarray Sum with One Deletion` (1186) keeps two states: `keep` (no deletion used) and `delete` (deletion used). The transition is `delete = max(delete + x, keep)`.

- 5 Maximum Absolute Sum (1749) is `max(max_kadane, -min_kadane)`.
- 6 K-Concatenation Maximum Sum (1191) splits by `k`: `k == 1` is plain Kadane; `k >= 2` and `total > 0` is the circular-style answer; otherwise it is Kadane on the doubled array.
- 7 Longest Turbulent Subarray (978) and Maximum Length of a Concatenated String -style problems use Kadane's *shape* with a boolean comparison instead of a sum.
- 8 Kadane as a DP is the same as Kadane as a running maximum. Both derivations are correct; present whichever the interviewer seems to prefer. The DP framing generalises better.
- 9 Divide and conquer Kadane. 53 has an $O(n \log n)$ divide-and-conquer solution (best in the left, best in the right, best crossing the middle) which is the same idea as chapter 85's merge-sort-based counting. Know it as the answer to 'can you solve it without DP?'.
 - 10 The recognition signal: 'contiguous subarray' plus 'maximum' or 'minimum'. If the problem says *subsequence* instead, it is chapter 62.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
121	Best Time to Buy and Sell Stock	Easy	24
1004	Max Consecutive Ones III	Medium	7
122	Best Time to Buy and Sell Stock II	Medium	8
188	Best Time to Buy and Sell Stock IV	Hard	5
123	Best Time to Buy and Sell Stock III	Hard	3
714	Best Time to Buy and Sell Stock with Transaction Fee	Medium	2
309	Best Time to Buy and Sell Stock with Cooldown	Medium	0
978	Longest Turbulent Subarray	Medium	0
1031	Maximum Sum of Two Non-Overlapping Subarrays	Medium	0
1191	K-Concatenation Maximum Sum	Medium	0
2272	Substring With Largest Variance	Hard	0

#	PROBLEM	LEVEL	ASKED
2321	Maximum Score Of Spliced Array	Hard	0
2444	Count Subarrays With Fixed Bounds	Hard	0
2606	Find the Substring With Maximum Cost	Medium	0
3026	Maximum Good Subarray Sum	Medium	0

DP – Longest Increasing Subsequence and Friends

THE KERNEL

`dp[i]` = length of the best subsequence *ending at i*. The $O(n^2)$ version is obvious; the $O(n \log n)$ version replaces the table with a `tails` array and a binary search.

27PROBLEMS
MAPPED**1**

EASY

18

MEDIUM

8

HARD

The Boundary Rules

- 1** **$O(n^2)$ form.** `dp[i] = 1 + max(dp[j] for j < i if a[j] < a[i])`. Always start here — it is correct, and it generalises to every variant.
- 2** **$O(n \log n)$ form.** `tails[k]` = the smallest possible tail of an increasing subsequence of length `k+1`. For each `x`, `i = bisect_left(tails, x)`; if `i == len(tails)`: append, else `tails[i] = x`. The answer is `len(tails)`. **The `bisect_left` versus `bisect_right` choice is what switches between strictly increasing and non-decreasing.** For strictly increasing use `bisect_left`.
- 3** **The `tails` array is not the subsequence.** You cannot reconstruct the LIS from `tails` without extra bookkeeping (a parent array plus positions). Do not claim otherwise.

- 4 **Strict vs non-strict.** `Longest Increasing Subsequence` (300) is strict (`bi-sect_left`). `Maximum Length of Pair Chain` (646) is strict on the second coordinate. `Russian Doll Envelopes` (354) is strict on *both* dimensions, which requires sorting width ascending and height *descending* so that equal widths cannot chain — that sort trick is the entire difficulty of 354.
- 5 **Counting the number of LIS** (673) needs the $O(n^2)$ form plus a `count[]` array: when `dp[j] + 1 > dp[i]`, reset `count[i] = count[j]`; when equal, add. The `tails` array cannot do this.
- 6 **Longest String Chain** (1048) is LIS on a DAG of words, sorted by length. The ‘is a predecessor’ test is a single deletion check.
- 7 **Longest Arithmetic Subsequence** (1027) and its ‘given difference’ variant (1218) have `dp[i][diff]` as the state — a second dimension keyed on the difference. When the transition depends on a *value* rather than an index, the state gains a dimension keyed by that value.
- 8 **Length of Longest Fibonacci Subsequence** (873) is the same value-keyed state: `dp[i][j]` = length of the Fibonacci-like sequence ending with `a[i]`, `a[j]`, and you look up `a[j] - a[i]` in a dict.
- 9 **Minimum Number of Removals to Make Mountain Array** (1671) is LIS from the left plus LIS from the right, combined. Two passes, one answer.
- 10 **Maximum Height by Stacking Cuboids** (1691) is Russian Doll in 3-D after sorting each cuboid’s dimensions.
- 11 **Longest Increasing Subsequence** is the parent of all ‘best subsequence with a local comparison’ problems. If the transition condition is `a[j] < a[i]` (or any pairwise predicate), the shape is LIS.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
435	Non-overlapping Intervals	Medium	3
446	Arithmetic Slices II - Subsequence	Hard	4
452	Minimum Number of Arrows to Burst Balloons	Medium	3
334	Increasing Triplet Subsequence	Medium	2

#	PROBLEM	LEVEL	ASKED
1296	Divide Array in Sets of K Consecutive Numbers	Medium	1
646	Maximum Length of Pair Chain	Medium	2
368	Largest Divisible Subset	Medium	1
1626	Best Team With No Conflicts	Medium	2
944	Delete Columns to Make Sorted	Easy	1
873	Length of Longest Fibonacci Subsequence	Medium	0
960	Delete Columns to Make Sorted III	Hard	0
1105	Filling Bookcase Shelves	Medium	0
1218	Longest Arithmetic Subsequence of Given Difference	Medium	0
1671	Minimum Number of Removals to Make Mountain Array	Hard	0
1691	Maximum Height by Stacking Cuboids	Hard	0
1964	Find the Longest Valid Obstacle Course at Each Position	Hard	0
2111	Minimum Operations to Make the Array K-Increasing	Hard	0
2370	Longest Ideal Subsequence	Medium	0
2407	Longest Increasing Subsequence II	Hard	0
2826	Sorting Three Groups	Medium	0
3176	Find the Maximum Length of a Good Subsequence I	Medium	0
3202	Find the Maximum Length of Valid Subsequence II	Medium	0

DP - Two-Sequence DP (LCS and Edit Distance)

THE KERNEL

$dp[i][j]$ = the answer using the first i characters of A and the first j characters of B. Two nested loops, three transitions: match, skip-A, skip-B. Every two-string DP in this book is this table.

19

 PROBLEMS
MAPPED

1

EASY

10

MEDIUM

8

HARD

The Boundary Rules

- 1 **The table is $(m+1) \times (n+1)$.** The extra row and column are the empty prefixes, so $dp[0][j]$ and $dp[i][0]$ are the base cases and the loop body needs no special-casing.
- 2 **LCS (1143):** `if a[i-1] == b[j-1]: dp[i][j] = dp[i-1][j-1] + 1 else max(dp[i-1][j], dp[i][j-1])`. Note the $i-1$ / $j-1$ indices inside the loop — the table is offset by one from the strings, and forgetting the offset is the most common bug.
- 3 **Edit Distance (72):** `if equal: dp[i-1][j-1] else 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])`. The three terms are replace, delete, insert. **Be able to name which is which** — that is the standard follow-up question.

- 4 **The three-term structure is universal.** Once you see $dp[i-1][j-1]$, $dp[i-1][j]$, $dp[i][j-1]$, you can derive: Delete Operation for Two Strings (583) = $m + n - 2 * \text{LCS}$; Minimum ASCII Delete Sum (712) = edit distance with character values as costs; Uncrossed Lines (1035) = LCS exactly; Shortest Common Supersequence (1092) = $m + n - \text{LCS}$ and the string is reconstructed by walking the table backwards.
- 5 **Wildcard Matching (44)** and **Regular Expression Matching (10)** are two-sequence DP with a *that matches any run*. 44 has a one-dimensional greedy solution (two pointers with a backtrack point); 10 does not, because `*` applies to the preceding element. Recognise the difference.
- 6 **Interleaving String (97)** is $dp[i][j]$ = 'the first i of A and j of B interleave to form the first $i+j$ of C'. Two transitions, no `min` / `max`. The state is defined by the two indices, and $i + j$ is implicit — **if you ever find yourself writing $dp[i][j][k]$, check whether k is derivable from i and j .**
- 7 **Distinct Subsequences (115)** counts, so the transition is `+` not `max`: $dp[i][j] = dp[i-1][j]$ plus $dp[i-1][j-1]$ if the characters match.
- 8 **Rolling arrays.** Every table in this chapter can be reduced to two rows, or even one row if you keep the $dp[i-1][j-1]$ value in a temporary before overwriting. For edit distance, the one-row version needs a `prev_diag` variable — write it carefully.
- 9 **Reconstructing the answer.** To output the actual LCS or the actual edit script, walk backwards from $dp[m][n]$: if the characters match and the value is consistent, take the diagonal; otherwise move to the cell that produced the value. Always check which predecessor was used — guessing the direction produces wrong strings.
- 10 **Longest Palindromic Subsequence (516)** is LCS of the string with its reverse. That reduction converts a 1-D interval problem into this table, and it is worth knowing both framings.
- 11 **Maximum Length of Repeated Subarray (718)** is LCS with contiguous matching: $dp[i][j] = dp[i-1][j-1] + 1$ on match, `0` otherwise, and the answer is the table maximum (not $dp[m][n]$). Note that difference — *contiguous* means you reset on mismatch.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
44	Wildcard Matching	Hard	6
161	One Edit Distance †	Medium	3
1925	Count Square Sum Triples	Easy	1
712	Minimum ASCII Delete Sum for Two Strings	Medium	0
718	Maximum Length of Repeated Subarray	Medium	0
1035	Uncrossed Lines	Medium	0
1092	Shortest Common Supersequence	Hard	0
1312	Minimum Insertion Steps to Make a String Palindrome	Hard	0
1458	Max Dot Product of Two Subsequences	Hard	0
1639	Number of Ways to Form a Target String Given a Dictionary	Hard	0
2466	Count Ways To Build Good Strings	Medium	0
2933	High-Access Employees	Medium	0
3303	Find the Occurrence of First Almost Equal Substring	Hard	0

DP – 0/1 Knapsack

THE KERNEL

`dp[i][c]` = the best value using the first `i` items with capacity `c`. Each item is either taken or not. The one-dimensional version iterates capacity *backwards*, and that direction is the whole difference from unbounded knapsack.

29

PROBLEMS
MAPPED

1

EASY

18

MEDIUM

10

HARD

The Boundary Rules

- 1 The 2-D recurrence.** `dp[i][c] = max(dp[i-1][c], dp[i-1][c - w[i]] + v[i])` for `c >= w[i]`. The `i-1` in the second term is what makes each item usable *once*.
- 2 The 1-D rolling version.** `for i: for c in range(C, w[i]-1, -1): dp[c] = max(dp[c], dp[c - w[i]] + v[i])`. **Capacity iterates descending.** Ascending would let item `i` be used multiple times, silently converting 0/1 knapsack into unbounded knapsack. This is the single most important line in the DP part.
- 3 The mnemonic.** Descending = each item once (0/1). Ascending = each item unlimited (unbounded). Same code, opposite loop direction, completely different problem.
- 4 Partition Equal Subset Sum** (416) is 0/1 knapsack with `value == weight` and `target = total // 2`. The pre-check `if total % 2: return False` is required.

- 5 **Target Sum** (494) is 416 after the algebraic reduction from chapter 48: the target subset sum is $(\text{target} + \text{total}) // 2$.
- 6 **Last Stone Weight II** (1049) is 416 phrased as 'minimise the difference', which is the same as 'maximise a subset sum under $\text{total}/2$ '. Recognising the reduction is the problem.
- 7 **Ones and Zeroes** (474) is 2-D 0/1 knapsack — the capacity is a pair (zeros, ones), so the DP table has two capacity dimensions and **both must iterate descending**. Getting one direction wrong is a classic error on this problem.
- 8 **Profitable Schemes** (879) is knapsack where the 'value' dimension is clamped at the minimum required — profits above `minProfit` are equivalent, so the state is `min(profit, minProfit)`. Clamping the value dimension is the trick.
- 9 **Counting rather than maximising**. Replace `max` with `+`. **Target Sum** counts, **Partition Equal Subset Sum** decides, **Last Stone Weight II** minimises. The recurrence skeleton is identical.
- 10 **Profitable Schemes** and **Number of Ways to Form a Target String** (1639) require `% MOD` at every addition.
- 11 **Boolean knapsack** (416 , **Partition to K Equal Sum Subsets**) can use a bit-set instead of a list of booleans: `dp |= dp << w`. In Python that is one line and is dramatically faster. Offer it as an optimisation.
- 12 **The recognition signal**. 'Choose a subset', 'each item once', 'capacity or budget', 'maximise value / minimise difference / count ways'. If items can be reused, it is chapter 65.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
322	Coin Change	Medium	7
377	Combination Sum IV	Medium	4
698	Partition to K Equal Sum Subsets	Medium	3
638	Shopping Offers	Medium	4

#	PROBLEM	LEVEL	ASKED
1774	Closest Dessert Cost	Medium	3
1981	Minimize the Difference Between Target and Chosen Elements	Medium	1
518	Coin Change II	Medium	0
956	Tallest Billboard	Hard	0
1155	Number of Dice Rolls With Target Sum	Medium	0
1230	Toss Strange Coins †	Medium	0
1449	Form Largest Integer With Digits That Add up to Target	Hard	0
1755	Closest Subsequence Sum	Hard	0
1995	Count Special Quadruplets	Easy	0
2035	Partition Array Into Two Arrays to Minimize Sum Difference	Hard	0
2291	Maximum Profit From Trading Stocks †	Medium	0
2518	Number of Great Partitions	Hard	0
2742	Painting the Walls	Hard	0
2787	Ways to Express an Integer as Sum of Powers	Medium	0
2915	Length of the Longest Subsequence That Sums to Target	Medium	0
2920	Maximum Points After Collecting Coins From All Nodes	Hard	0
3007	Maximum Number That Sum of the Prices Is Less Than or Equal to K	Medium	0
3181	Maximum Total Reward Using Operations II	Hard	0
3291	Minimum Number of Valid Strings to Form Target I	Medium	0

DP - Unbounded Knapsack and Coin Change

THE KERNEL

Items can be reused. Capacity iterates *ascending*, and the order of the two loops decides whether you are counting *combinations* or *permutations*.

39PROBLEMS
MAPPED**1**

EASY

26

MEDIUM

12

HARD

The Boundary Rules

- 1 The 1-D recurrence.** `for i: for c in range(w[i], C+1): dp[c] += dp[c - w[i]]`. Capacity ascending. That is the only change from chapter 64, and it is what allows reuse.
- 2 The loop-order rule — memorise this.** *Coins outer, capacity inner* counts **combinations** (order does not matter): `Coin Change II` (518). *Capacity outer, coins inner* counts **permutations** (order matters): `Combination Sum IV` (377). Same recurrence, swapped loops, different answers. This is the single most-asked subtlety in the whole DP part.
- 3 Why it works.** With coins outer, each coin is considered exactly once and only in combination with previously-processed coins, so `[1,2]` and `[2,1]` are the same path. With capacity outer, each capacity can be reached by appending any coin last, so the two orders are different paths.

- 4 **Coin Change** (322) is the *minimum number of coins* version: `dp[c] = min(dp[c - coin] + 1)`. Initialise with `inf`, `dp[0] = 0`. Note that for minimisation the loop order does not matter (both give the same answer), but for counting it does.
- 5 **Perfect Squares** (279) is 322 with the coins being perfect squares. Generate them with `[i*i for i in range(1, isqrt(n)+1)]`.
- 6 **Word Break** (139) is unbounded knapsack over dictionary words where the ‘capacity’ is a string prefix. `dp[i] = 'the first i characters can be segmented'`. The inner loop over words is $O(\text{len}(\text{words}) * L)$ and is usually fast enough; a Trie or a max-word-length bound optimises it.
- 7 **Word Break II** (140) adds reconstruction, which turns it into backtracking with memoisation on the suffix. Recognise when ‘count’ becomes ‘enumerate’.
- 8 **Integer Break** (343), **Minimum Cost For Tickets** (983) and **Count Ways To Build Good Strings** (2466) are all unbounded selections over a set of ‘coins’ that are not literally coins.
- 9 **Maximum Number of Points with Cost** (1937) and **Frog Jump** (403) are DP over a *reachable* set rather than a capacity; the state space is sparse, so a dict or a set beats an array.
- 10 **The recognition signal.** ‘Unlimited supply’, ‘reuse’, ‘any number of times’, or a set of items you may draw from repeatedly. And whenever you see ‘number of ways’, stop and ask: *does order matter?* That question decides your loop order.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
140	Word Break II	Hard	8
472	Concatenated Words	Hard	2
403	Frog Jump	Hard	5
740	Delete and Earn	Medium	3
256	Paint House †	Medium	3
638	Shopping Offers	Medium	4

#	PROBLEM	LEVEL	ASKED
1043	Partition Array for Maximum Sum	Medium	2
265	Paint House II †	Hard	3
343	Integer Break	Medium	1
764	Largest Plus Sign	Medium	1
1014	Best Sightseeing Pair	Medium	1
656	Coin Path †	Hard	1
983	Minimum Cost For Tickets	Medium	0
1155	Number of Dice Rolls With Target Sum	Medium	0
1449	Form Largest Integer With Digits That Add up to Target	Hard	0
1504	Count Submatrices With All Ones	Medium	0
1547	Minimum Cost to Cut a Stick	Hard	0
1937	Maximum Number of Points with Cost	Medium	0
2009	Minimum Number of Operations to Make Array Continuous	Hard	0
2054	Two Best Non-Overlapping Events	Medium	0
2063	Vowels of All Substrings	Medium	0
2140	Solving Questions With Brainpower	Medium	0
2291	Maximum Profit From Trading Stocks †	Medium	0
2547	Minimum Cost to Split an Array	Hard	0
2606	Find the Substring With Maximum Cost	Medium	0
2742	Painting the Walls	Hard	0
2915	Length of the Longest Subsequence That Sums to Target	Medium	0
2920	Maximum Points After Collecting Coins From All Nodes	Hard	0
2944	Minimum Number of Coins for Fruits	Medium	0
2979	Most Expensive Item That Can Not Be Bought †	Medium	0
3007	Maximum Number That Sum of the Prices Is Less Than or Equal to K	Medium	0
3079	Find the Sum of Encrypted Integers	Easy	0
3225	Maximum Score From Grid Operations	Hard	0

DP – Subset Sum and Partitioning

THE KERNEL

A special case of knapsack where value equals weight, so the state is just a set of reachable sums. A bitset makes it one line in Python.

25

PROBLEMS
MAPPED

1

EASY

15

MEDIUM

9

HARD

The Boundary Rules

- 1 **The reachable-sums DP** `dp = [False] * (target + 1); dp[0] = True`, then `for x in nums: for s in range(target, x-1, -1): dp[s] |= dp[s-x]`. Descending, because each number is used once.
- 2 **The bitset version.** `bits = 1; for x in nums: bits |= bits << x`. Then `bits >> target & 1` is the answer for 416. This is $O(n * \text{target} / 64)$ and is dramatically faster in Python — it is the version to write if you want to impress.
- 3 **Partition Equal Subset Sum** (416) is the canonical statement: `total` must be even and `total // 2` must be reachable.
- 4 **Partition to K Equal Sum Subsets** (698) is NP-hard in general, so it is backtracking (chapter 48), not DP. The DP works only when `k = 2`. Knowing why is a good interview moment.
- 5 **Matchsticks to Square** (473) is 698 with `k = 4`. Same backtracking, same heuristics.
- 6 **Minimum Number of Work Sessions** (1986) is subset-sum over bitmasks: `dp[mask] = (sessions, time_in_last_session)` as a tuple, minimised lexicographically. It is the bridge to chapter 73.

- 7 **Tallest Billboard** (956) has a state of ‘the difference between the two sums’, which is a clever re-parameterisation: `dp[d] = max total height with difference d`. When the natural state is too large, re-parameterise by a difference or a ratio.
- 8 **Number of Ways to Split Array** (2270) and **Find the Array Concatenation Value**-style problems are prefix-sum problems, not subset-sum — do not over-fit the pattern.
- 9 **Largest Multiple of Three** (1363) is a greedy modulo problem, not a subset DP. The recognition: the target is a *property of the whole selection* (divisibility), and greedy by remainder class works.
- 10 **The recognition signal.** ‘Can you partition’, ‘split into two equal’, ‘reach exactly’, ‘subset with sum’. If the numbers are large and `n` is small, it is backtracking; if the target is small and `n` is large, it is bitset DP. State which regime you are in — it is the deciding question.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
39	Combination Sum	Medium	9
40	Combination Sum II	Medium	5
698	Partition to K Equal Sum Subsets	Medium	3
473	Matchsticks to Square	Medium	1
1981	Minimize the Difference Between Target and Chosen Elements	Medium	1
1363	Largest Multiple of Three	Hard	0
1402	Reducing Dishes	Hard	0
1755	Closest Subsequence Sum	Hard	0
1995	Count Special Quadruplets	Easy	0
2035	Partition Array Into Two Arrays to Minimize Sum Difference	Hard	0
2172	Maximum AND Sum of Array	Hard	0
2291	Maximum Profit From Trading Stocks †	Medium	0
2305	Fair Distribution of Cookies	Medium	0

#	PROBLEM	LEVEL	ASKED
2518	Number of Great Partitions	Hard	0
2742	Painting the Walls	Hard	0
2787	Ways to Express an Integer as Sum of Powers	Medium	0
2915	Length of the Longest Subsequence That Sums to Target	Medium	0
3007	Maximum Number That Sum of the Prices Is Less Than or Equal to K	Medium	0
3180	Maximum Total Reward Using Operations I	Medium	0
3181	Maximum Total Reward Using Operations II	Hard	0

DP – Grid and Path DP

THE KERNEL

$dp[r][c]$ = the best/count for reaching cell (r, c) . It depends on the top and left neighbours. The extra top row and left column of zeros are the boundary conditions.

30

 PROBLEMS
MAPPED

0

EASY

19

MEDIUM

11

HARD

The Boundary Rules

- 1 **Unique Paths** (62): $dp[r][c] = dp[r-1][c] + dp[r][c-1]$, with the first row and first column all 1. It is binomial coefficients in disguise — $C(m+n-2, m-1)$ — which is the $O(\min(m,n))$ follow-up.
- 2 **Unique Paths II** (63) adds obstacles: set $dp[r][c] = 0$ where $grid[r][c] == 1$. The first-row/first-column initialisation must stop at the first obstacle, not skip it.
- 3 **Minimum Path Sum** (64) changes $+$ to $\min$ and adds $grid[r][c]$. **Triangle** (120) is the same DP on a ragged array, and iterating *bottom-up* avoids all boundary handling — that is the trick worth memorising.
- 4 **Minimum Falling Path Sum** (931) has three predecessors (up-left, up, up-right). **Minimum Falling Path Sum II** (1289) removes the diagonal restriction, which naively becomes $O(n^3)$; the optimisation is to track the two smallest values in the previous row, making it $O(n^2)$. **Tracking the best and second-best to avoid a forbidden choice is a reusable trick.**

- 5 **Maximal Square** (221): $dp[r][c] = 1 + \min(dp[r-1][c], dp[r][c-1], dp[r-1][c-1])$ when the cell is '1'. The `min` of three is the insight — it says ‘the largest square ending here is limited by the smallest of the three neighbours’. **Count Square Submatrices** (1277) is the same table summed instead of maximised.
- 6 **Cherry Pickup** (741) is two walkers moving simultaneously: $dp[steps][r1][r2]$ where $c1 = steps - r1$, $c2 = steps - r2$. **The key reduction is that both walkers take the same number of steps**, so one index is redundant. **Cherry Pickup II** (1463) is the same with two robots in different rows.
- 7 **Dungeon Game** (174) is a *backwards* DP: compute the minimum health needed from each cell to the end, iterating from bottom-right to top-left. When the constraint must hold at every prefix, DP backwards.
- 8 **Out of Boundary Paths** (576) and **Knight Probability** (688) are DP over a state $(steps, r, c)$ — the step count is an explicit dimension because the answer depends on it.
- 9 **Rolling rows**. Any grid DP that only looks one row up can be reduced to two rows. **Maximal Square** needs $dp[r-1][c-1]$, so keep a `prev_diag` variable alongside a single row.
- 10 **Grid DP vs grid DFS**. If the question asks for a *count* or an *optimum* over all paths, and moves are only in fixed directions, it is DP. If moves can go in all four directions, or you must not revisit cells, it is DFS/backtracking (chapter 49). **The direction restriction is what makes DP possible** — say this out loud, because it is exactly the question an interviewer asks.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
174	Dungeon Game	Hard	2
1277	Count Square Submatrices with All Ones	Medium	2
562	Longest Line of Consecutive One in Matrix †	Medium	1
576	Out of Boundary Paths	Medium	2
1289	Minimum Falling Path Sum II	Hard	2

#	PROBLEM	LEVEL	ASKED
688	Knight Probability in Chessboard	Medium	1
361	Bomb Enemy †	Medium	1
568	Maximum Vacation Days †	Hard	1
931	Minimum Falling Path Sum	Medium	1
1301	Number of Paths with Max Score	Hard	0
1444	Number of Ways of Cutting a Pizza	Hard	0
1463	Cherry Pickup II	Hard	0
1594	Maximum Non Negative Product in a Matrix	Medium	0
1595	Minimum Cost to Connect Two Groups of Points	Hard	0
1824	Minimum Sideway Jumps	Medium	0
1937	Maximum Number of Points with Cost	Medium	0
2267	Check if There Is a Valid Parentheses String Path	Hard	0
2304	Minimum Path Cost in a Grid	Medium	0
2328	Number of Increasing Paths in a Grid	Hard	0
2435	Paths in Matrix Whose Sum Is Divisible by K	Hard	0
2510	Check if There is a Path With Equal Number of 0's And 1's †	Medium	0
2684	Maximum Number of Moves in a Grid	Medium	0
3147	Taking Maximum Energy From the Mystic Dungeon	Medium	0
3290	Maximum Multiplication Score	Medium	0

DP - Interval DP

THE KERNEL

`dp[i][j]` = the answer for the subarray `i..j`. You iterate by *length*, and the transition splits at a `k` in between. The classic trick is to think about what happens *last*, not first.

21

PROBLEMS
MAPPED

0

EASY

10

MEDIUM

11

HARD

The Boundary Rules

- The loop skeleton.** `for length in range(2, n+1): for i in range(n-length+1): j = i + length - 1; for k in range(i, j): ...`. **The outer loop is length, not i.** Iterating by `i` computes cells before their dependencies exist. This is the single most important structural fact about interval DP.
- Think about the last operation, not the first.** `Burst Balloons` (312) is impossible if you reason about which balloon to burst first (the subproblems are not independent). If you reason about which balloon is burst *last*, the left and right subarrays become independent and the DP works. **This reframing is the entire problem, and it is the most valuable single idea in the DP part.**
- The sentinel padding.** For 312, pad the array with 1 at both ends so the boundary products are handled without special cases. For `Minimum Cost to Cut a Stick` (1547), pad the cut positions with 0 and `n`.
- Minimum Score Triangulation** (1039) is 312 with a different cost function. `Minimum Cost Tree From Leaf Values` (1130) is the same shape with `max` instead of a sum, and has a *monotonic stack* solution as well — knowing both is a strong answer.

- 5 **Remove Boxes** (546) needs a three-dimensional state `dp[i][j][k]` where `k` is the number of boxes of the same colour as `a[i]` attached to the left. Without `k`, the subproblems are not independent. It is the hardest interval DP in the book.
- 6 **Strange Printer** (664) has the trick: if `s[i] == s[k]`, you can print them in the same stroke, so `dp[i][j] = min(dp[i][j], dp[i][k-1] + dp[k+1][j])`. Recognising that equal endpoints can be merged into one operation is the reduction.
- 7 **Guess Number Higher or Lower II** (375) is minimax interval DP: `dp[i][j] = min over k of (k + max(dp[i][k-1], dp[k+1][j]))`. The `min(max(...))` structure is what makes it a game problem.
- 8 **Palindrome Partitioning II** (132) combines interval DP with a palindrome precomputation. `dp[i]` = minimum cuts for the prefix; the inner loop checks palindromic suffixes. Precomputing `is_pal[i][j]` in $O(n^2)$ is what makes it $O(n^2)$ instead of $O(n^3)$.
- 9 **Scramble String** (87) is interval DP on a 3-D state (two intervals plus a length) with memoisation. It is a good example of the state being richer than `(i, j)`.
- 10 **Complexity** is $O(n^3)$ for the basic shape. With `n <= 100` that is 10^6 operations and fine; with `n <= 500` you need a better idea. Read the constraints to decide whether interval DP is even admissible.
- 11 **The recognition signal.** The input is a sequence, the operation *consumes* or *merges* elements, and the order of operations matters. ‘Burst’, ‘merge’, ‘cut’, ‘remove’, ‘partition with a cost’ — all interval DP.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
647	Palindromic Substrings	Medium	3
516	Longest Palindromic Subsequence	Medium	6
375	Guess Number Higher or Lower II	Medium	1
87	Scramble String	Hard	1

#	PROBLEM	LEVEL	ASKED
1000	Minimum Cost to Merge Stones	Hard	0
1312	Minimum Insertion Steps to Make a String Palindrome	Hard	0
1770	Maximum Score from Performing Multiplication Operations	Hard	0
1850	Minimum Adjacent Swaps to Reach the Kth Smallest Number	Medium	0
1900	The Earliest and Latest Rounds Where Players Compete	Hard	0
2002	Maximum Product of the Length of Two Palindromic Subsequences	Medium	0
2311	Longest Binary Subsequence Less Than or Equal to K	Medium	0
2340	Minimum Adjacent Swaps to Make a Valid Array †	Medium	0
2472	Maximum Number of Non-overlapping Palindrome Substrings	Hard	0
2767	Partition String Into Minimum Beautiful Substrings	Medium	0
3018	Maximum Number of Removal Queries That Can Be Processed I †	Hard	0

DP - State Machine (The Stock Family)

THE KERNEL

At each day the trader is in one of a small number of states (holding or not, and how many transactions used). Write one recurrence per state. The whole stock family is one template with a different number of states.

21

 PROBLEMS
MAPPED

1

EASY

12

MEDIUM

8

HARD

The Boundary Rules

- 1 Two states, unlimited transactions (122):**
`hold = max(hold, cash - price)`, `cash = max(cash, hold + price)`. Iterate the two updates carefully — using the *new* `hold` in the `cash` update lets you buy and sell on the same day. **Use the old values:**
`hold, cash = max(hold, cash - p), max(cash, hold + p)` as a tuple assignment evaluates the right-hand side first, which is exactly what you want.
- 2 One transaction only (121):** `buy = min(buy, price)`, `profit = max(profit, price - buy)`. Two variables, no `dp` array.
- 3 At most two transactions (123):** four variables — `buy1`, `sell1`, `buy2`, `sell2` — with the chain `buy1 = max(buy1, -p)`, `sell1 = max(sell1, buy1 + p)`, `buy2 = max(buy2, sell1 - p)`, `sell2 = max(sell2, buy2 + p)`. **Again, the tuple assignment order matters** and is why the chain works in one pass.

- 4 **At most k transactions (188)**: the same chain generalised to arrays `buy[j]` and `sell[j]` for `j` in `1..k`. If `k >= n // 2` the answer is the unlimited case — that early return is what keeps the complexity at $O(n * \min(k, n/2))$.
- 5 **With a cooldown (309)**: three states — `hold`, `sold`, `rest`. The transition is `hold = max(hold, rest - p)`, `rest = max(rest, sold)`, `sold = hold_old + p`. The extra `rest` state is what encodes the one-day gap.
- 6 **With a transaction fee (714)**: subtract the fee on either the buy or the sell, but **not both**. Subtracting on the sell is the usual choice: `cash = max(cash, hold + price - fee)`.
- 7 **The four-line template for the whole family**. `for p in prices: buy = max(buy, prev_state - p); sell = max(sell, buy + p); prev_state = ...`. Every variant in this chapter is a different definition of `prev_state`.
- 8 **Generalising beyond stocks**. Any problem where you are ‘in a mode’ that changes what moves are legal is a state machine: `Best Time to Buy and Sell Stock with Transaction Fee`, `Student Attendance Record II (552)`, `Number of Ways to Paint N x 3 Grid (1411)`, `Count Vowels Permutation (1220)`. Recognise the shared shape.
- 9 **State machines beat ‘prefix/suffix max’ solutions**. Many stock problems also have a clever prefix-suffix-array solution for the two-transaction case, but it does not generalise to `k`. The state machine generalises, so lead with it.
- 10 **The recognition signal**. ‘Buy’, ‘sell’, ‘hold’, ‘cooldown’, ‘fee’, ‘at most k transactions’, or any problem with an explicit mode that restricts the next action.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
198	House Robber	Medium	9
552	Student Attendance Record II	Hard	1
213	House Robber II	Medium	2
337	House Robber III	Medium	3
2110	Number of Smooth Descent Periods of a Stock	Medium	1
801	Minimum Swaps To Make Sequences Increasing	Hard	0

#	PROBLEM	LEVEL	ASKED
1220	Count Vowels Permutation	Hard	0
1411	Number of Ways to Paint $N \times 3$ Grid	Hard	0
1653	Minimum Deletions to Make String Balanced	Medium	0
1692	Count Ways to Distribute Candies †	Hard	0
1911	Maximum Alternating Subsequence Sum	Medium	0
2400	Number of Ways to Reach a Position After Exactly k Steps	Medium	0
2466	Count Ways To Build Good Strings	Medium	0
2518	Number of Great Partitions	Hard	0
3186	Maximum Total Damage With Spell Casting	Medium	0

DP – Tree DP

THE KERNEL

Postorder returns a small tuple of values summarising the subtree; the parent combines its children's tuples. The state lives in the return value, not in a table.

21PROBLEMS
MAPPED**1**

EASY

12

MEDIUM

8

HARD

The Boundary Rules

- 1 The return value is the state.** For `House Robber III` (337) the return is `(rob_this, skip_this)` — a two-tuple. The parent computes `rob = node.val + left.skip + right.skip` and `skip = max(left.rob, left.skip) + max(right.rob, right.skip)`. Writing two separate recursive functions instead of one returning a tuple is the usual first attempt and is $O(n^2)$.
- 2 Returning a tuple is the general technique.** Whenever a node needs more than one piece of information from its subtree, return a tuple. Common tuples: `(rob, skip)`, `(height, diameter)`, `(min, max, size)`, `(is_bst, min, max, sum)`.
- 3 Binary Tree Cameras** (968) returns a three-state enum: covered-with-camera, covered-without-camera, uncovered. The greedy assignment rule (place a camera at the parent of any uncovered node) falls out of the postorder. It is the hardest tree DP in the book.

- 4 **Largest BST Subtree** (333) returns (`is_bst`, `min_val`, `max_val`, `size`). The `min / max` are needed so the parent can check the BST property — returning only the size is not enough. This is the standard example of why the return value must be *sufficient*.
- 5 **Diameter** (543) and **Max Path Sum** (124) return one value and update a nonlocal best — the hybrid from chapter 41. They are tree DP in the loose sense: the state is one number.
- 6 **Distribute Coins** (979) returns the net excess and accumulates `abs()` into a global. The return value is a *deficit*, which is the insight.
- 7 **Most Frequent Subtree Sum** (508) returns the sum and accumulates a Counter. **Longest ZigZag Path** (1372) returns a two-tuple (left-going, right-going). **Maximum Average Subtree** (1120) returns (`sum`, `count`).
- 8 **Sum of Distances in Tree** (834) is the flagship hard tree DP: compute subtree sizes and the answer for the root in one postorder, then reroot in a second pass. The rerooting formula `ans[child] = ans[parent] + n - 2 * size[child]` is worth memorising — it is the standard ‘all nodes as root’ technique.
- 9 **Count Nodes With the Same Label** (1519) combines tree DP with a per-label running count. **Number of Good Leaf Nodes Pairs** (1530) returns a list of depths.
- 10 **Binary Tree Maximum Path Sum** and **Diameter** are the two tree DP problems you are most likely to be asked. If you only memorise two, memorise those, plus `House Robber III` as the tuple-returning archetype.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
979	Distribute Coins in Binary Tree	Medium	4
1372	Longest ZigZag Path in a Binary Tree	Medium	1
508	Most Frequent Subtree Sum	Medium	1
1120	Maximum Average Subtree †	Medium	0
1245	Tree Diameter †	Medium	0

#	PROBLEM	LEVEL	ASKED
1519	Number of Nodes in the Sub-Tree With the Same Label	Medium	0
1530	Number of Good Leaf Nodes Pairs	Medium	0
2313	Minimum Flips in Binary Tree to Get Result †	Hard	0
2440	Create Components With Same Value	Hard	0
2445	Number of Nodes With Value One †	Medium	0
2538	Difference Between Maximum and Minimum Price Sum	Hard	0
2646	Minimize the Total Price of the Trips	Hard	0
2673	Make Costs of Paths Equal in a Binary Tree	Medium	0
2973	Find Number of Coins to Place in Tree Nodes	Hard	0
3249	Count the Number of Good Nodes	Medium	0

DP – Palindrome DP

THE KERNEL

`dp[i][j]` = whether/how `s[i..j]` is a palindrome, computed by increasing length. Once you have the table, every palindrome problem is a second DP or a greedy on top of it.

27

PROBLEMS
MAPPED

8

EASY

9

MEDIUM

10

HARD

The Boundary Rules

- The palindrome table.** `is_pal[i][j] = s[i] == s[j]` and `(j - i < 2` or `is_pal[i+1][j-1])`. Iterate **i** descending and **j** ascending (or iterate by length) so that `is_pal[i+1][j-1]` is already computed.
- Longest Palindromic Substring** (5) tracks the maximum `j - i + 1` over the table. The expand-around-centre solution is $O(n^2)$ time and $O(1)$ space and is shorter — know both, and note that Manacher’s algorithm gets $O(n)$.
- Palindromic Substrings** (647) counts the `True` cells. Expand-around-centre is again the shorter solution.
- Longest Palindromic Subsequence** (516) is *not* the same as the substring: it is LCS of the string with its reverse, or equivalently `dp[i][j] = dp[i+1][j-1] + 2` if the ends match, else `max(dp[i+1][j], dp[i][j-1])`. Note the difference from the substring version: **subsequence** takes a `max`, **substring** resets to `False`.

- 5 **Palindrome Partitioning II** (132) needs the palindrome table plus a 1-D `dp[i]` = minimum cuts for the prefix. The two-layer structure (table then DP) is the standard composition for palindrome problems.
- 6 **Palindrome Partitioning III** (1278) is 132 with a fixed number of cuts — a two-dimensional DP over `(prefix, cuts_used)` on top of the palindrome table. Three layers, but each is simple.
- 7 **Minimum Insertion Steps to Make a String Palindrome** (1312) is $n - \text{LPS}(s)$. That identity is worth memorising: insertions needed = length minus longest palindromic subsequence.
- 8 **Valid Palindrome III** (1216) is 'can it become a palindrome with at most `k` deletions', which is $n - \text{LPS} \leq k$. Same identity.
- 9 **Shortest Palindrome** (214) needs KMP: the answer is the string prepended with the reverse of the part that is not a palindromic prefix. The KMP failure function on `s + '#' + s[::-1]` gives the longest palindromic prefix. It is a chapter 83 problem that lives here by topic.
- 10 **Construct K Palindrome Strings** (1400) is a counting argument, not a DP: at most `n` palindromes, at least the number of characters with an odd count.
- 11 **The expand-around-centre function, memorised.** `for c in range(2*n - 1): l, r = c // 2, c // 2 + c % 2; while l >= 0 and r < n and s[l] == s[r]: l -= 1; r += 1`. Handling both odd and even centres in one loop is the trick.
- 12 **Recognition signal.** The word 'palindrome' plus 'longest', 'count', 'partition', or 'minimum cuts'. All of them start with the same table.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
680	Valid Palindrome II	Easy	7
125	Valid Palindrome	Easy	6
9	Palindrome Number	Easy	8
131	Palindrome Partitioning	Medium	5

#	PROBLEM	LEVEL	ASKED
266	Palindrome Permutation †	Easy	1
418	Sentence Screen Fitting †	Medium	1
1400	Construct K Palindrome Strings	Medium	1
1332	Remove Palindromic Subsequences	Easy	1
267	Palindrome Permutation II †	Medium	1
409	Longest Palindrome	Easy	1
467	Unique Substrings in Wraparound String	Medium	1
471	Encode String with Shortest Length †	Hard	1
1216	Valid Palindrome III †	Hard	0
1278	Palindrome Partitioning III	Hard	0
1745	Palindrome Partitioning IV	Hard	0
1771	Maximize Palindrome Length From Subsequences	Hard	0
2108	Find First Palindromic String in the Array	Easy	0
2396	Strictly Palindromic Number	Medium	0
2472	Maximum Number of Non-overlapping Palindrome Substrings	Hard	0
2697	Lexicographically Smallest Palindrome	Easy	0
2911	Minimum Changes to Make K Semi-palindromes	Hard	0

DP – Game Theory and Minimax

THE KERNEL

`dp[i][j]` is the *advantage* (current player's score minus the opponent's) obtainable from the interval. The recurrence is `max(take_left - dp[i+1][j], take_right - dp[i][j-1])` — note the minus, which flips the perspective.

23PROBLEMS
MAPPED**4**

EASY

10

MEDIUM

9

HARD

The Boundary Rules

- 1 The advantage formulation is the key trick.** Define `dp[i][j]` as `current_player_score - other_player_score` for the subarray `i..j`. Then `dp[i][j] = max(a[i] - dp[i+1][j], a[j] - dp[i][j-1])`. The subtraction encodes ‘after I take, my opponent faces the same problem and their advantage is my disadvantage’. **Deriving this recurrence is the entire insight** — the naive ‘maximise my total’ formulation requires tracking both scores and is much harder.
- 2 The win condition.** `Predict the Winner` (486) is `dp[0][n-1] >= 0`. `Stone Game` (877) is always `True` for even-length arrays — that observation (the first player can always take all odd-indexed or all even-indexed piles) is the intended ‘solution’ and is worth stating.
- 3 Space optimisation.** The advantage DP only needs two rows, so it can be done with a 1-D array of length `n` iterated correctly. Offer it as the follow-up.

- 4 **Stone Game II** (1140) has state `(index, M)` — the number of piles you may take depends on `M`, which changes. **Stone Game III** (1406) is a 1-D advantage DP. **Stone Game VII** (1690) uses the *sum* of the remaining interval as the gain and the advantage DP as the structure.
- 5 **Can I Win** (464) is minimax over a bitmask of used numbers — it is chapter 73's bitmask DP with a game recurrence. State `(mask) -> can current player win`.
- 6 **Guess Number Higher or Lower II** (375) is minimax where the current player *chooses* the pivot and the adversary chooses the outcome: `dp[i][j] = min over k of (k + max(dp[i][k-1], dp[k+1][j]))`. The `min(max(...))` is the signature of a minimax with adversarial choice.
- 7 **Cat and Mouse** (913) and **Cat and Mouse II** (1728) are game DP over a joint state with three outcomes (win, lose, draw). The draw condition makes them harder than the standard minimax and requires cycle-aware evaluation.
- 8 **Nim-style problems** (292 , 1025 , 1908) are *not* DP: the answer is a parity or a modulo condition. **Nim Game** (292) is `n % 4 != 0`; **Divisor Game** (1025) is `n % 2 == 0`. **Always check for the closed form before writing a DP** — interviewers ask these to see whether you over-engineer.
- 9 **Predict the Winner** **with a greedy is wrong**. There is a tempting 'always take the larger end' greedy and it fails. If you propose it, produce the counter-example.
- 10 **Recognition signal**. Two players, alternating turns, perfect play, optimal strategy, and a question about who wins or by how much. The advantage formulation applies whenever the game is zero-sum and the state is an interval or a mask.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
2038	Remove Colored Pieces if Both Neighbors are the Same Color	Medium	1
292	Nim Game	Easy	2
1025	Divisor Game	Easy	2

#	PROBLEM	LEVEL	ASKED
294	Flip Game II †	Medium	1
295	Find Median from Data Stream	Hard	0
913	Cat and Mouse	Hard	0
1510	Stone Game IV	Hard	0
1563	Stone Game V	Hard	0
1690	Stone Game VII	Medium	0
1728	Cat and Mouse II	Hard	0
1872	Stone Game VIII	Hard	0
1908	Game of Nim †	Medium	0
2029	Stone Game IX	Medium	0
2321	Maximum Score Of Spliced Array	Hard	0
2409	Count Days Spent Together	Easy	0
2973	Find Number of Coins to Place in Tree Nodes	Hard	0
3222	Find the Winning Player in Coin Game	Easy	0

DP – Bitmask DP

THE KERNEL

When $n \leq 20$, the state is a subset, and a subset is an integer.

`dp[mask]` = the answer for the subset `mask`. Iterate masks in increasing order so every subset is computed before its supersets.

26

 PROBLEMS
MAPPED

0

EASY

8

MEDIUM

18

HARD

The Boundary Rules

- 1 **The $n \leq 20$ signal.** 2^{20} is about a million states, which is the upper limit of comfortable bitmask DP. If $n \leq 20$, think bitmask; if $n \leq 12$, an $O(2^n * n^2)$ transition is fine; if $n \leq 25$, you need meet-in-the-middle instead.
- 2 **Enumerating the set bits of a mask.** `while m: b = m & -m; i = b.bit_length() - 1; m ^= b`. Or simply `for i in range(n): if mask >> i & 1`. The `m & -m` idiom extracts the lowest set bit and is worth knowing.
- 3 **Enumerating submasks.** `sub = mask; while sub: ...; sub = (sub - 1) & mask`. This enumerates all submasks in $O(3^n)$ total, which is the standard technique for subset-partition DP (1723, 1601, 2741).
- 4 **Shortest Path Visiting All Nodes (847):** BFS over `(node, visited_mask)`; the answer is the first state where `mask == (1 << n) - 1`. It is BFS, not DP, but the state is a bitmask — recognise the shared representation.

- 5 **Smallest Sufficient Team** (1125): `dp[mask]` = the smallest team covering the skill set `mask`. The transition tries each person and ORs their skill mask. The ‘people as items, skills as the subset’ framing is the reduction.
- 6 **Partition to K Equal Sum Subsets** (698) and **Minimum Number of Work Sessions** (1986) use `dp[mask] = (sessions, time_in_last)` as a tuple minimised lexicographically. Tuple-valued DP states are common in this chapter.
- 7 **Maximum AND Sum of Array** (2172) assigns numbers to slots with capacity 2 — the state is which slots are filled. **Number of Ways to Wear Different Hats** (1434) is subset DP over hats with a people-mask.
- 8 **Stickers to Spell Word** (691) memoises on the *remaining* target string, which is a compressed state. When the subset is a set of *letters* rather than indices, a Counter or a string is the state.
- 9 **Find the Shortest Superstring** (943) is TSP on strings: `dp[mask][i]` = shortest superstring ending at `i`, with an overlap precomputation. It is the canonical ‘bitmask DP for ordering’ problem.
- 10 **Minimum Incompatibility** (1681) and **Maximize Score After N Operations** (1799) are subset-partition DP with a validity check per subset. Precompute the valid subsets first, then DP.
- 11 **Count Number of Maximum Bitwise-OR Subsets** (2044) and **Sum of All Subset XOR Totals** (1863) have closed forms or simple enumerations — check before reaching for DP.
- 12 **The generic shape.** `dp[mask] = best over i not in mask of f(dp[mask | (1 << i)], cost(mask, i))`. If your transition looks like that, you have bitmask DP; the only work left is defining `cost`.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
465	Optimal Account Balancing †	Hard	3
351	Android Unlock Patterns †	Medium	2
1066	Campus Bikes II †	Medium	2
1601	Maximum Number of Achievable Transfer Requests	Hard	2

#	PROBLEM	LEVEL	ASKED
464	Can I Win	Medium	1
526	Beautiful Arrangement	Medium	1
996	Number of Squareful Arrays	Hard	0
1349	Maximum Students Taking Exam	Hard	0
1434	Number of Ways to Wear Different Hats to Each Other	Hard	0
1655	Distribute Repeating Integers	Hard	0
1681	Minimum Incompatibility	Hard	0
1723	Find Minimum Time to Finish All Jobs	Hard	0
1986	Minimum Number of Work Sessions to Finish the Tasks	Medium	0
1994	The Number of Good Subsets	Hard	0
2044	Count Number of Maximum Bitwise-OR Subsets	Medium	0
2172	Maximum AND Sum of Array	Hard	0
2403	Minimum Time to Kill All Monsters †	Hard	0
2741	Special Permutations	Medium	0
2920	Maximum Points After Collecting Coins From All Nodes	Hard	0
3276	Select Cells in Grid With Maximum Score	Hard	0

DP – Digit DP

THE KERNEL

Count numbers in $[0, N]$ satisfying a digit property by processing the digits left to right with a `tight` flag. The state is `(position, tight, ...)`.

18PROBLEMS
MAPPED**0**

EASY

6

MEDIUM

12

HARD

The Boundary Rules

- 1 **The `tight` flag.** `tight` means 'the prefix so far equals the prefix of `N`', so the current digit is capped at `N[pos]`. When `tight` is true the digit may equal `N[pos]`; otherwise it may be 0-9 freely. **This one boolean is the entire idea of digit DP.**
- 2 **The recursive shape.** `f(pos, tight, state)` where `state` carries whatever property you are tracking (last digit, whether a repeat occurred, the digit count). Memoise on `(pos, tight, state)`. The `tight=True` states are never revisited, so you may exclude them from the cache — a common optimisation.
- 3 **The `started` flag.** If leading zeros are not real digits, you need a second boolean for 'has the number started'. `Number of Digit One` (233) and `Count Special Integers` (2376) both need it. Forgetting it produces counts that treat `007` as a three-digit number.
- 4 **`Number of Digit One` (233)** is the flagship: count the occurrences of the digit `1` across all numbers in `[0, n]`. There is also a beautiful $O(\log n)$ closed form counting per digit position, and knowing both is the ideal answer.

- 5 **Numbers With Repeated Digits** (1012) is 'count numbers with no repeated digit' via the complementary count. **Count Special Integers** (2376) is the same problem restated. **Numbers At Most N Given Digit Set** (902) restricts the allowed digits, which removes the `tight`-only branching.
- 6 **Non-negative Integers without Consecutive Ones** (600) tracks the last digit in the state. **Count Numbers with Unique Digits** (357) has a closed form (`9 9 8 * ...`) and is a good warm-up.
- 7 **Number of Beautiful Integers in the Range** (2827) and **Count the Number of Powerful Integers** (2999) are modern digit DP problems with richer states (digit-sum parity, suffix constraints).
- 8 **Range queries are $f(R) - f(L-1)$** . Every 'in the range `[L, R]`' digit problem is two prefix counts. That reduction is always the first line.
- 9 **The iterative alternative.** 233 and 902 can be done by iterating digit positions and counting completions combinatorially, which is $O(\log n)$ and often shorter than the recursive DP. If you can derive the combinatorial count, prefer it — interviewers reward the closed form.
- 10 **Recognition signal.** 'Count numbers in `[0, N]` such that the digits satisfy ...', with `N` up to `10^9` or `10^18`. The constraint size (`N` huge, `n` irrelevant) is the giveaway: the complexity must be in the number of *digits*, not the value.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
650	2 Keys Keyboard	Medium	2
788	Rotated Digits	Medium	2
357	Count Numbers with Unique Digits	Medium	1
651	4 Keys Keyboard †	Medium	1
553	Optimal Division	Medium	1
1067	Digit Count in Range †	Hard	0
1397	Find All Good Strings	Hard	0
2719	Count of Integers	Hard	0

#	PROBLEM	LEVEL	ASKED
2801	Count Stepping Numbers in Range	Hard	0
2827	Number of Beautiful Integers in the Range	Hard	0
2999	Count the Number of Powerful Integers	Hard	0
3153	Sum of Digit Differences of All Pairs	Medium	0
3352	Count K-Reducible Numbers Less Than N	Hard	0

PART XIII

Greedy

Sort by the constraint, not the value. The greedy proof obligation, the retroactive choice, and the heap that turns scheduling into a sweep.

Greedy - Sort First

THE KERNEL

If the optimal solution can be built by repeatedly taking the best remaining item under some order, sort by the right key and sweep once. The whole problem is choosing the sort key.

66PROBLEMS
MAPPED**15**

EASY

43

MEDIUM

8

HARD

The Boundary Rules

- 1 **Choosing the sort key is the entire problem.** Sort by the *constraint* (deadline, end time, capacity), not by the *value*, when the constraint is what runs out.
- 2 **Intervals: sort by end to remove, sort by start to merge.** `Non-overlapping Intervals` (435) and `Minimum Number of Arrows` (452) sort by end and greedily keep the earliest-finishing interval. `Merge Intervals` (56) sorts by start. Getting these backwards produces a plausible but wrong answer, so state your key and your justification explicitly.
- 3 **Assign Cookies** (455): sort both; two pointers; give the smallest cookie that satisfies the least greedy child. The greedy is ‘do not waste a large cookie on a small child’.
- 4 **Array Partition** (561) is ‘sort and sum every second element’. The proof is an exchange argument and is worth being able to sketch.
- 5 **Two City Scheduling** (1029) sorts by the *difference* `costA - costB` and sends the first `n` to A. Sorting by a difference is the standard technique when each item has two competing costs.

- 6 **Maximum Units on a Truck** (1710) and **Boats to Save People** (881) sort by value density and by weight respectively, then sweep with a two-pointer.
- 7 **Task Scheduler** (621) has a closed-form greedy: $(\text{max_freq} - 1) * (n + 1) + \text{count_of_max_freq}$ and then `max(result, len(tasks))`. The **max with len(tasks)** is required for the case where there are enough distinct tasks to fill every gap.
- 8 **Maximum Number of Events That Can Be Attended** (1353) is the flagship sort-plus-heap greedy: sort by start day, and use a min-heap of end days to always attend the event that expires soonest.
- 9 **Minimum Number of Refueling Stops** (871) is a greedy with a max-heap: drive as far as possible, and when stuck, retroactively add the largest fuel station passed. The **'retroactive choice' pattern** — making a decision about the past when you learn new information — is a recurring greedy technique.
- 10 **Course Schedule III** (630) is the same retroactive pattern: add every course, and when the total duration exceeds a deadline, drop the longest course (max-heap).
- 11 **Furthest Building You Can Reach** (1642) is the same: use ladders on the largest gaps, bricks on the rest.
- 12 **Partition Labels** (763) is a two-pass greedy: record each character's last index, then extend the current partition's end to the maximum last index seen. No sorting needed.
- 13 **The proof obligation.** A greedy answer needs an exchange argument: 'suppose an optimal solution differs from the greedy choice at the first point; swapping the greedy choice in does not make it worse.' Being able to state that sentence is often worth more than the code.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1710	Maximum Units on a Truck	Easy	2
179	Largest Number	Medium	9
1046	Last Stone Weight	Easy	4

#	PROBLEM	LEVEL	ASKED
2007	Find Original Array From Doubled Array	Medium	2
1481	Least Number of Unique Integers after K Removals	Medium	4
1642	Furthest Building You Can Reach	Medium	3
757	Set Intersection Size At Least Two	Hard	1
1400	Construct K Palindrome Strings	Medium	1
1353	Maximum Number of Events That Can Be Attended	Medium	2
954	Array of Doubled Pairs	Medium	1
1732	Find the Highest Altitude	Easy	1
1974	Minimum Time to Type Word Using Special Typewriter	Easy	1
280	Wiggle Sort †	Medium	1
1005	Maximize Sum Of Array After K Negations	Easy	1
1338	Reduce Array Size to The Half	Medium	1
1403	Minimum Subsequence in Non-Increasing Order	Easy	1
1561	Maximum Number of Coins You Can Get	Medium	1
1968	Array With Elements Not Equal to Average of Neighbors	Medium	1
1733	Minimum Number of People to Teach	Medium	1
1090	Largest Values From Labels	Medium	1
1637	Widest Vertical Area Between Two Points Containing No Points	Easy	1
2126	Destroying Asteroids	Medium	1
2160	Minimum Sum of Four Digit Number After Splitting Digits	Easy	1
1007	Minimum Domino Rotations For Equal Row	Medium	0
1013	Partition Array Into Three Parts With Equal Sum	Easy	0
1054	Distant Barcodes	Medium	0
1262	Greatest Sum Divisible by Three	Medium	0
1509	Minimum Difference Between Largest and Smallest Value in Three Moves	Medium	0
1665	Minimum Initial Energy to Finish Tasks	Hard	0
1671	Minimum Number of Removals to Make Mountain Array	Hard	0
1798	Maximum Number of Consecutive Values You Can Make	Medium	0

#	PROBLEM	LEVEL	ASKED
1833	Maximum Ice Cream Bars	Medium	0
2037	Minimum Number of Moves to Seat Everyone	Easy	0
2071	Maximum Number of Tasks You Can Assign	Hard	0
2131	Longest Palindrome by Concatenating Two Letter Words	Medium	0
2141	Maximum Running Time of N Computers	Hard	0
2178	Maximum Split of Positive Even Integers	Medium	0
2242	Maximum Score of a Node Sequence	Hard	0
2279	Maximum Bags With Full Capacity of Rocks	Medium	0
2311	Longest Binary Subsequence Less Than or Equal to K	Medium	0
2325	Decode the Message	Easy	0
2340	Minimum Adjacent Swaps to Make a Valid Array †	Medium	0
2357	Make Array Zero by Subtracting Equal Amounts	Easy	0
2363	Merge Similar Items	Easy	0
2405	Optimal Partition of String	Medium	0
2410	Maximum Matching of Players With Trainers	Medium	0
2448	Minimum Cost to Make Array Equal	Hard	0
2456	Most Popular Video Creator	Medium	0
2498	Frog Jump II	Medium	0
2541	Minimum Operations to Make Array Equal II	Medium	0
2561	Rearranging Fruits	Hard	0
2587	Rearrange Array to Maximize Prefix Score	Medium	0
2611	Mice and Cheese	Medium	0
2645	Minimum Additions to Make Valid String	Medium	0
2789	Largest Element in an Array after Merge Operations	Medium	0
2807	Insert Greatest Common Divisors in Linked List	Medium	0
2934	Minimum Operations to Maximize Last Elements in Arrays	Medium	0
2996	Smallest Missing Integer Greater Than Sequential Prefix Sum	Easy	0
3025	Find the Number of Ways to Place People I	Medium	0

GREEDY - SORT FIRST

#	PROBLEM	LEVEL	ASKED
3075	Maximize Happiness of Selected Children	Medium	0
3143	Maximum Points Inside the Square	Medium	0

Greedy - Reachability and Exchange Arguments

THE KERNEL

Track a single scalar — the furthest reach, the current fuel, the current coverage — and update it with a bound. These problems look like DP and are not.

87

 PROBLEMS
MAPPED

14

EASY

57

MEDIUM

16

HARD

The Boundary Rules

- 1 **Jump Game (55)**: keep `reach = max(reach, i + nums[i])`; if `i > reach` at any point, return `False`. Check the failure condition *before* the update** — the loop must verify that the current index is reachable before extending.
- 2 **Jump Game II (45)**: BFS in disguise. Track `cur_end` and `farthest`; when `i == cur_end`, increment the jump count and set `cur_end = farthest`. The ‘level end’ variable is what makes it a BFS** — recognising that is the insight.
- 3 **Jump Game III (1306)** and **Jump Game IV (1345)** are BFS/DFS, not greedy — the jumps can go backwards, which destroys the reachability monotonicity. Recognise when a ‘jump’ problem is actually a graph problem.
- 4 **Gas Station (134)**: if the total gas is at least the total cost, a solution exists; the start is the index after the last point where the running tank went negative. Both facts must be stated** — the existence condition and the start-index rule.

- 5 **Minimum Number of Taps to Open to Water a Garden** (1326) is **Jump Game II** after converting intervals into reach values. The reduction from intervals to a reach array is the whole problem.
- 6 **Video Stitching** (1024) is 1326 with a different framing — again, convert intervals to a `farthest[i]` array and run the jump algorithm.
- 7 **Patching Array** (330): maintain `miss` = the smallest sum you cannot form. If `nums[i] <= miss`, extend `miss += nums[i]`; otherwise patch with `miss` and double it. The invariant `miss` is the smallest unreachable sum** — state it, and the algorithm is obvious.
- 8 **Candy** (135) is a two-pass greedy: left-to-right enforcing the increasing rule, then right-to-left enforcing the decreasing rule, taking the maximum. Two passes because each direction is independent.
- 9 **Trapping Rain Water** (42) has the two-pointer greedy from chapter 5 — track `left_max` and `right_max` and always advance the shorter side. It is a greedy, not a DP, and that is worth saying.
- 10 **Container With Most Water** (11) is the same greedy: always move the shorter wall. The proof is that moving the taller wall can never increase the area, because the shorter wall is the binding constraint.
- 11 **Maximum Subarray** with a greedy is Kadane (chapter 61) — a greedy and a DP that coincide. When a greedy and a DP agree, prefer the greedy for code length and mention the DP for correctness.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
42	Trapping Rain Water	Hard	18
121	Best Time to Buy and Sell Stock	Easy	24
11	Container With Most Water	Medium	9
152	Maximum Product Subarray	Medium	6
605	Can Place Flowers	Easy	5
122	Best Time to Buy and Sell Stock II	Medium	8

#	PROBLEM	LEVEL	ASKED
1423	Maximum Points You Can Obtain from Cards	Medium	3
1386	Cinema Seat Allocation	Medium	2
1029	Two City Scheduling	Medium	2
1345	Jump Game IV	Hard	3
392	Is Subsequence	Easy	3
860	Lemonade Change	Easy	3
763	Partition Labels	Medium	2
334	Increasing Triplet Subsequence	Medium	2
1658	Minimum Operations to Reduce X to Zero	Medium	2
781	Rabbits in Forest	Medium	3
1024	Video Stitching	Medium	2
1282	Group the People Given the Group Size They Belong To	Medium	2
738	Monotone Increasing Digits	Medium	2
376	Wiggle Subsequence	Medium	1
517	Super Washing Machines	Hard	1
630	Course Schedule III	Hard	1
1689	Partitioning Into Minimum Number Of Deci-Binary Numbers	Medium	1
1775	Equal Sum Arrays With Minimum Number of Operations	Medium	1
1827	Minimum Operations to Make the Array Increasing	Easy	1
2078	Two Furthest Houses With Different Colors	Easy	1
2259	Remove Digit From Number to Maximize Result	Easy	1
1663	Smallest String With A Given Numeric Value	Medium	1
555	Split Concatenated Strings †	Medium	1
625	Minimum Factorization †	Medium	1
1558	Minimum Numbers of Function Calls to Make Target Array	Medium	1
1708	Largest Subarray Length K †	Easy	1
1736	Latest Time by Replacing Hidden Digits	Easy	1
1874	Minimize Product Sum of Two Arrays †	Medium	1

#	PROBLEM	LEVEL	ASKED
624	Maximum Distance in Arrays	Medium	1
1785	Minimum Elements to Add to Form a Given Sum	Medium	1
871	Minimum Number of Refueling Stops	Hard	0
1306	Jump Game III	Medium	0
1414	Find the Minimum Number of Fibonacci Numbers Whose Sum Is K	Medium	0
1520	Maximum Number of Non-Overlapping Substrings	Hard	0
1653	Minimum Deletions to Make String Balanced	Medium	0
1665	Minimum Initial Energy to Finish Tasks	Hard	0
1686	Stone Game VI	Medium	0
1794	Count Pairs of Equal Substrings With Minimum Difference †	Medium	0
1838	Frequency of the Most Frequent Element	Medium	0
1899	Merge Triplets to Form Target Triplet	Medium	0
2037	Minimum Number of Moves to Seat Everyone	Easy	0
2086	Minimum Number of Food Buckets to Feed the Hamsters	Medium	0
2136	Earliest Possible Day of Full Bloom	Hard	0
2144	Minimum Cost of Buying Candies With Discount	Easy	0
2178	Maximum Split of Positive Even Integers	Medium	0
2193	Minimum Number of Moves to Make Palindrome	Hard	0
2224	Minimum Number of Operations to Convert Time	Easy	0
2294	Partition Array Such That Maximum Difference Is K	Medium	0
2311	Longest Binary Subsequence Less Than or Equal to K	Medium	0
2335	Minimum Amount of Time to Fill Cups	Easy	0
2340	Minimum Adjacent Swaps to Make a Valid Array †	Medium	0
2350	Shortest Impossible Sequence of Rolls	Hard	0
2366	Minimum Replacements to Sort the Array	Hard	0
2383	Minimum Hours of Training to Win a Competition	Easy	0
2405	Optimal Partition of String	Medium	0
2434	Using a Robot to Print the Lexicographically Smallest String	Medium	0

#	PROBLEM	LEVEL	ASKED
2448	Minimum Cost to Make Array Equal	Hard	0
2457	Minimum Addition to Make Integer Beautiful	Medium	0
2486	Append Characters to String to Make Subsequence	Medium	0
2541	Minimum Operations to Make Array Equal II	Medium	0
2554	Maximum Number of Integers to Choose From a Range I	Medium	0
2565	Subsequence With the Minimum Score	Hard	0
2571	Minimum Operations to Reduce an Integer to 0	Medium	0
2592	Maximize Greatness of an Array	Medium	0
2607	Make K-Subarray Sums Equal	Medium	0
2611	Mice and Cheese	Medium	0
2645	Minimum Additions to Make Valid String	Medium	0
2712	Minimum Cost to Make All Characters Equal	Medium	0
2734	Lexicographically Smallest String After Substring Operation	Medium	0
2789	Largest Element in an Array after Merge Operations	Medium	0
2800	Shortest String That Contains Three Strings	Medium	0
2870	Minimum Number of Operations to Make Array Empty	Medium	0
2934	Minimum Operations to Maximize Last Elements in Arrays	Medium	0
3001	Minimum Moves to Capture The Queen	Medium	0
3075	Maximize Happiness of Selected Children	Medium	0

Greedy - Heap-Assisted Choice

THE KERNEL

When the greedy needs ‘the best available among the currently eligible’, maintain a heap of eligible candidates. This is the bridge between greedy and scheduling.

33

 PROBLEMS
MAPPED

0

EASY

22

MEDIUM

11

HARD

The Boundary Rules

- 1 **The universal shape.** Sort by one key to control eligibility, push eligible items into a heap keyed by the greedy criterion, then pop when you must commit. The sort controls *when*, the heap controls *which*.
- 2 **Task Scheduler** (621) has both a closed form and a heap simulation. Write the closed form; mention the simulation.
- 3 **IP0** (502) is the canonical two-phase version: sort projects by capital, push every now-affordable project into a max-heap by profit, pop the best, repeat. The **while** loop that pushes all newly affordable projects before each pop is the key structure.
- 4 **Reorganize String** (767) and **Longest Happy String** (1405) use a max-heap by remaining count plus a ‘hold the previous item aside for one step’ buffer. The buffer is what prevents an immediate repeat.
- 5 **Course Schedule III** (630) and **Minimum Number of Refueling Stops** (871) are *retroactive* greedy: take everything, then undo the worst choice when the constraint is violated. The heap holds the undo candidates.

- 6 **Furthest Building You Can Reach** (1642) is the same retroactive shape with a min-heap of gaps and a ladder budget.
- 7 **Maximum Performance of a Team** (1383) sorts by efficiency and maintains a min-heap of speeds of size `k`, tracking `efficiency sum_of_top_k_speeds`. *The state you track is a product**, which is what makes the problem non-obvious.
- 8 **Maximum Number of Events That Can Be Attended II** (1751) is *not* a heap greedy — it is DP after sorting by end time with binary search for the previous compatible event. Recognising when the greedy breaks (overlapping events with weights) is important: weighted interval scheduling is DP, not greedy.
- 9 **Meeting Rooms II** (253) is the simplest version: a min-heap of end times, push the new start, and if the earliest end is `<= start`, reuse the room.
- 10 **Divide Intervals Into Minimum Number of Groups** (2406) is 253 with a different phrasing. **Single-Threaded CPU** (1834) and **Number of the Smallest Unoccupied Chair** (1942) are the same eligibility-plus-heap shape with extra state.
- 11 **Recognition signal**. ‘Maximum profit given a budget’, ‘minimum number of X to cover Y’, ‘schedule to maximise Z’, with a resource that is consumed. If the choice set grows over time, sort plus heap.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
253	Meeting Rooms II †	Medium	9
358	Rearrange String k Distance Apart †	Hard	4
254	Factor Combinations †	Medium	1
1705	Maximum Number of Eaten Apples	Medium	2
630	Course Schedule III	Hard	1
1792	Maximum Average Pass Ratio	Medium	1
1090	Largest Values From Labels	Medium	1
857	Minimum Cost to Hire K Workers	Hard	0
1167	Minimum Cost to Connect Sticks †	Medium	0

#	PROBLEM	LEVEL	ASKED
1405	Longest Happy String	Medium	0
1465	Maximum Area of a Piece of Cake After Horizontal and Vertical Cuts	Medium	0
1606	Find Servers That Handled Most Number of Requests	Hard	0
1751	Maximum Number of Events That Can Be Attended II	Hard	0
1834	Single-Threaded CPU	Medium	0
1882	Process Tasks Using Servers	Medium	0
1942	The Number of the Smallest Unoccupied Chair	Medium	0
2054	Two Best Non-Overlapping Events	Medium	0
2183	Count Array Pairs Divisible by K	Hard	0
2336	Smallest Number in Infinite Set	Medium	0
2406	Divide Intervals Into Minimum Number of Groups	Medium	0
2462	Total Cost to Hire K Workers	Medium	0
2503	Maximum Number of Points From Grid Queries	Hard	0
2542	Maximum Subsequence Score	Medium	0
2551	Put Marbles in Bags	Hard	0
2645	Minimum Additions to Make Valid String	Medium	0
2817	Minimum Absolute Difference Between Elements With Constraint	Medium	0
2940	Find Building Where Alice and Bob Can Meet	Hard	0
3092	Most Frequent IDs	Medium	0

PART XIV

Bits, Maths & Strings

Bit tricks, number theory, simulation, and the three string algorithms that are genuinely string-specific.

Bit Manipulation

THE KERNEL

Seven operations cover every bit problem: `x & -x` (lowest set bit), `x & (x-1)` (clear it), `x ^ x = 0`, `x >> i & 1` (test), `x | (1 << i)` (set), `x & ~(1 << i)` (clear), `x ^ (1 << i)` (flip).

52

 PROBLEMS
MAPPED

26

EASY

22

MEDIUM

4

HARD

The Boundary Rules

- 1 **`n & (n - 1)` clears the lowest set bit.** So `n & (n-1) == 0` is the power-of-two test (231), and looping it counts set bits (191). **Power of Two (231) must also check `n > 0`** — zero passes the bit test and is not a power of two.
- 2 **`x & -x` isolates the lowest set bit.** Use it to iterate set bits, and use it in **Single Number III (260)**: XOR everything to get `a ^ b`, isolate any differing bit, then partition the array by that bit and XOR each half separately.
- 3 **XOR properties.** `a ^ a = 0`, `a ^ 0 = a`, XOR is commutative and associative. Therefore XORing an array where every element appears twice except one leaves the single element (136). **This is the most-asked bit trick in interviews.**
- 4 **Single Number II (137)** breaks the XOR trick because elements appear three times. The answer is a bitwise count mod 3: sum the bits at each position across all numbers and take mod 3. The `ones / twos` state machine is the O(1)-space version.

- 5 **Missing Number** (268) is XOR of `range(n+1)` with the array, or the Gauss sum. **Find the Difference** (389) and **Set Mismatch** (645) are the same XOR-all trick.
- 6 **Counting Bits** (338) is `dp[i] = dp[i >> 1] + (i & 1)`, or `dp[i] = dp[i & (i-1)] + 1`. Both are one-liners in a loop.
- 7 **Bitwise AND of Numbers Range** (201)**: find the common prefix of `left` and `right` by right-shifting both until they are equal, then shift back. The answer is that prefix. Recognising that the AND of a range is the common binary prefix is the insight.
- 8 **Sum of Two Integers** (371) without `+`: `while b: carry = (a & b) << 1; a ^= b; b = carry`. In Python this loops forever on negative numbers because of arbitrary precision — you must mask to 32 bits and convert back at the end. **Python-specific gotcha, and a favourite interview question.**
- 9 **Reverse Bits** (190) and **Number of 1 Bits** (191) are the mechanical warm-ups.
- 10 **Maximum XOR of Two Numbers** (421) uses a prefix-trie of bits built greedily from the top bit down. It is a Trie problem (chapter 83's cousin) in bit clothing.
- 11 **Divide Two Integers** (29) is bit-shift long division with a 32-bit clamp. The clamp to `[-231, 231 - 1]` is required and is the part candidates forget.
- 12 **Minimum Bit Flips** (2220) is `bin(a ^ b).count('1')`.
Total Hamming Distance (477) counts, per bit position, `ones * zeros` and sums. The 'count per bit position' reframing is the key technique — it converts a quadratic problem into a linear one.
- 13 **Python specifics.** `bin(x).count('1')` is `popcount`; `x.bit_length()` gives the position of the highest set bit; `int.bit_count()` exists in Python 3.10+ and is the idiomatic answer.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
78	Subsets	Medium	10
287	Find the Duplicate Number	Medium	7

#	PROBLEM	LEVEL	ASKED
268	Missing Number	Easy	8
231	Power of Two	Easy	4
190	Reverse Bits	Easy	5
389	Find the Difference	Easy	3
137	Single Number II	Medium	1
1009	Complement of Base 10 Integer	Easy	2
405	Convert a Number to Hexadecimal	Easy	2
476	Number Complement	Easy	2
1342	Number of Steps to Reduce a Number to Zero	Easy	2
393	UTF-8 Validation	Medium	2
421	Maximum XOR of Two Numbers in an Array	Medium	2
477	Total Hamming Distance	Medium	2
1680	Concatenation of Consecutive Binary Numbers	Medium	1
89	Gray Code	Medium	1
260	Single Number III	Medium	1
318	Maximum Product of Word Lengths	Medium	1
461	Hamming Distance	Easy	1
645	Set Mismatch	Easy	1
1486	XOR Operation in an Array	Easy	1
201	Bitwise AND of Numbers Range	Medium	1
1863	Sum of All Subset XOR Totals	Easy	1
672	Bulb Switcher II	Medium	1
762	Prime Number of Set Bits in Binary Representation	Easy	1
693	Binary Number with Alternating Bits	Easy	1
342	Power of Four	Easy	1
868	Binary Gap	Easy	1
1521	Find a Value of a Mysterious Function Closest to Target	Hard	0
1720	Decode XORed Array	Easy	0

#	PROBLEM	LEVEL	ASKED
1734	Decode XORed Permutation	Medium	0
1738	Find Kth Largest XOR Coordinate Value	Medium	0
1835	Find XOR Sum of All Pairs Bitwise AND	Hard	0
2011	Final Value of Variable After Performing Operations	Easy	0
2220	Minimum Bit Flips to Convert Number	Easy	0
2317	Maximum XOR After Operations	Medium	0
2354	Number of Excellent Pairs	Hard	0
2425	Bitwise XOR of All Pairings	Medium	0
2429	Minimize XOR	Medium	0
2433	Find The Original Array of Prefix Xor	Medium	0
2595	Number of Even and Odd Bits	Easy	0
2657	Find the Prefix Common Array of Two Arrays	Medium	0
2859	Sum of Values at Indices With K Set Bits	Easy	0
2997	Minimum Number of Operations to Make Array XOR Equal to K	Medium	0
3022	Minimize OR of Remaining Elements Using Operations	Hard	0
3110	Score of a String	Easy	0
3158	Find the XOR of Numbers Which Appear Twice	Easy	0
3211	Generate Binary Strings Without Adjacent Zeros	Medium	0

Bitmask Enumeration

THE KERNEL

Enumerate all 2^n subsets with `for mask in range(1 << n)` and test membership with `mask >> i & 1`. This is brute force made tidy, and it is often the intended solution when `n <= 20`.

23

 PROBLEMS
MAPPED

1

EASY

10

MEDIUM

12

HARD

The Boundary Rules

- 1 The loop.**
`for mask in range(1 << n): for i in range(n): if mask >> i & 1: ...`.
 In Python, `[i for i in range(n) if mask >> i & 1]` is a clean inner expression.
- 2 Gray code (89)** generates an ordering where consecutive masks differ by one bit: `i ^ (i >> 1)`. **That single expression is the whole problem**, and it is worth memorising.
- 3 Sum of All Subset XOR Totals (1863)** has a closed form: each bit contributes to `2^(n-1)` subsets, so the answer is `OR_of_all * 2^(n-1)`. **Always look for the closed form before enumerating.**
- 4 Count Number of Maximum Bitwise-OR Subsets (2044)** requires the OR of the full array first, then a count of subsets achieving it. The enumeration is `0(2^n)` and is intended.

- 5 **Maximum Length of a Concatenated String with Unique Characters** (1239) is subset enumeration with a validity pre-check per string and a bitmask of used letters. Encoding ‘letters used’ as a 26-bit mask turns the duplicate check into `mask & other == 0` — that is the trick.
- 6 **Maximum Product of Word Lengths** (318) is the same 26-bit letter mask idea: two words share no letter iff `mask_a & mask_b == 0`.
- 7 **Letter Case Permutation** (784) is $2^{(\text{number of letters})}$ enumerations where bit `i` selects the case of the `i`-th letter. It is cleaner than backtracking here.
- 8 **Number of Ways to Wear Different Hats** (1434), **Smallest Sufficient Team** (1125) and **Maximum AND Sum of Array** (2172) are bitmask DP (chapter 73), not enumeration — the distinction is whether you memoise on the mask.
- 9 **Partition to K Equal Sum Subsets** (698) is enumeration with pruning; **1601** (Maximum Number of Achievable Transfer Requests) is enumeration over request subsets with a net-degree check. Both are $O(2^n)$ with $n \leq 16$.
- 10 **Meet in the middle**. When `n` is around 30-40, 2^n is too large but $2^{(n/2)}$ is fine: enumerate both halves and combine with a hash or a sort. **Closest Subsequence Sum** (1755) and **Partition Array Into Two Arrays to Minimize Sum Difference** (2035) are the canonical meet-in-the-middle problems.
- 11 **Recognition signal**. `n <= 20` in the constraints, plus ‘subset’, ‘choose a group’, or ‘all combinations’. If you see `n <= 20`, bitmask is usually intended.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
204	Count Primes	Medium	5
1601	Maximum Number of Achievable Transfer Requests	Hard	2
784	Letter Case Permutation	Medium	1
996	Number of Squareful Arrays	Hard	0
1125	Smallest Sufficient Team	Hard	0
1255	Maximum Score Words Formed by Letters	Hard	0

#	PROBLEM	LEVEL	ASKED
1434	Number of Ways to Wear Different Hats to Each Other	Hard	0
1655	Distribute Repeating Integers	Hard	0
1681	Minimum Incompatibility	Hard	0
1723	Find Minimum Time to Finish All Jobs	Hard	0
1755	Closest Subsequence Sum	Hard	0
1986	Minimum Number of Work Sessions to Finish the Tasks	Medium	0
1994	The Number of Good Subsets	Hard	0
2035	Partition Array Into Two Arrays to Minimize Sum Difference	Hard	0
2172	Maximum AND Sum of Array	Hard	0
2741	Special Permutations	Medium	0
3205	Maximum Array Hopping Score I †	Medium	0

Number Theory Essentials

THE KERNEL

Four tools: the sieve of Eratosthenes, `gcd / lcm`, modular exponentiation, and factorisation by trial division up to `sqrt(n)`.

98

 PROBLEMS
MAPPED

46

EASY

48

MEDIUM

4

HARD

The Boundary Rules

- 1 **Sieve of Eratosthenes.** `is_prime = [True] (n+1); for i in 2..isqrt(n), if prime, mark multiples from i i off. Start at ii, not 2 i` — smaller multiples were already marked, and starting at `i*i` is the standard optimisation. `Count Primes (204)` is the archetype.
- 2 **Trial division.** `for d in range(2, isqrt(n) + 1)` — the `isqrt` bound is mandatory or you are $O(n)$. Use `math.isqrt` for exact integer square roots; `int(n ** 0.5)` has floating-point errors at large `n`.
- 3 **GCD and LCM.** `math.gcd(a, b)` and `a // gcd(a, b) b`. **Compute LCM as `a // gcd b`, not `a * b // gcd`** — the latter overflows in fixed-width languages and is the standard style point.
- 4 **Euclid's algorithm** should be reproducible by hand: `while b: a, b = b, a % b; return a`. `GCD of Strings (1071)` is `s + t == t + s` plus the string of length `gcd(len(s), len(t))` — a beautiful non-numeric use of the same idea.

- 5 **Fast modular exponentiation.** `pow(base, exp, mod)` is built into Python and is the answer for `Pow(x, n)` (50) in its integer form. For the floating-point version, square-and-multiply by hand: `while n: if n & 1: result = base; base = base * base; n >>= 1`.
- 6 **Ugly Number II** (264) is *not* a number-theory problem — it is three-pointer merging of three generated sequences. **Super Ugly Number** (313) generalises to `k` pointers, and a heap also works. Recognise the merge.
- 7 **Ugly Number III** (1201) is the counting version and uses inclusion-exclusion plus binary search (chapter 14). Three tools stacked.
- 8 **Modular arithmetic.** `(a + b) % m`, `(a * b) % m`, and for division you need the modular inverse via Fermat's little theorem (`pow(a, m-2, m)` when `m` is prime). Problems that say 'modulo 10^9+7 ' and require division need this.
- 9 **Factorial Trailing Zeroes** (172) is a counting problem: the number of factors of 5, computed as `n//5 + n//25 + n//125 + ...`. No factorial is ever computed.
- 10 **Max Points on a Line** (149) is a geometry problem using `gcd` to normalise slope fractions so they can be hashed. **Normalising a ratio by its gcd before hashing is the reusable technique.**
- 11 **Water and Jug Problem** (365) is a number-theory problem in disguise: the answer is `target % gcd(x, y) == 0` and `target <= x + y`. Recognising Bézout's identity is the whole solution.
- 12 **Perfect Number** (507), **Self Dividing Numbers** (728) and **Count Odd Numbers in an Interval Range** (1523) are arithmetic warm-ups — do not over-engineer them.
- 13 **Bit counting and bases.** `bin`, `oct`, `hex` for output; `int(s, base)` for input; `divmod` for digit extraction. **Excel Sheet Column Number** (171) is base-26 with a twist.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
2	Add Two Numbers	Medium	12

#	PROBLEM	LEVEL	ASKED
13	Roman to Integer	Easy	9
415	Add Strings	Easy	7
12	Integer to Roman	Medium	7
1822	Sign of the Product of an Array	Easy	1
202	Happy Number	Easy	6
7	Reverse Integer	Medium	11
9	Palindrome Number	Easy	8
43	Multiply Strings	Medium	6
69	Sqrt(x)	Easy	7
1492	The kth Factor of n	Medium	2
66	Plus One	Easy	6
67	Add Binary	Easy	6
1304	Find N Unique Integers Sum up to Zero	Easy	1
1344	Angle Between Hands of a Clock	Medium	4
231	Power of Two	Easy	4
171	Excel Sheet Column Number	Easy	3
168	Excel Sheet Column Title	Easy	3
390	Elimination Game	Medium	3
1628	Design an Expression Tree With Evaluate Function †	Medium	1
360	Sort Transformed Array †	Medium	1
453	Minimum Moves to Equal Array Elements	Medium	3
313	Super Ugly Number	Medium	3
593	Valid Square	Medium	2
319	Bulb Switcher	Medium	2
1518	Water Bottles	Easy	3
728	Self Dividing Numbers	Easy	3
1523	Count Odd Numbers in an Interval Range	Easy	3
754	Reach a Number	Medium	1

#	PROBLEM	LEVEL	ASKED
1641	Count Sorted Vowel Strings	Medium	2
292	Nim Game	Easy	2
405	Convert a Number to Hexadecimal	Easy	2
633	Sum of Square Numbers	Medium	2
1524	Number of Sub-arrays With Odd Sum	Medium	2
1071	Greatest Common Divisor of Strings	Easy	2
1281	Subtract the Product and Sum of Digits of an Integer	Easy	1
263	Ugly Number	Easy	1
400	Nth Digit	Medium	1
2033	Minimum Operations to Make a Uni-Value Grid	Medium	1
365	Water and Jug Problem	Medium	1
258	Add Digits	Easy	1
326	Power of Three	Easy	1
483	Smallest Good Base	Hard	1
492	Construct the Rectangle	Easy	1
634	Find the Derangement of An Array †	Medium	1
775	Global and Local Inversions	Medium	1
877	Stone Game	Medium	1
914	X of a Kind in a Deck of Cards	Easy	1
970	Powerful Integers	Medium	1
976	Largest Perimeter Triangle	Easy	1
989	Add to Array-Form of Integer	Easy	1
1056	Confusing Number †	Easy	1
1175	Prime Arrangements	Easy	1
1185	Day of the Week	Easy	1
1252	Cells with Odd Values in a Matrix	Easy	1
1295	Find Numbers with Even Number of Digits	Easy	1
1447	Simplified Fractions	Medium	1

#	PROBLEM	LEVEL	ASKED
1551	Minimum Operations to Make Array Equal	Medium	1
1742	Maximum Number of Balls in a Box	Easy	1
1753	Maximum Score From Removing Stones	Medium	1
1780	Check if Number is a Sum of Powers of Three	Medium	1
1814	Count Nice Pairs in an Array	Medium	1
372	Super Pow	Medium	1
507	Perfect Number	Easy	1
779	K-th Symbol in Grammar	Medium	1
1103	Distribute Candies to People	Easy	1
1228	Missing Number In Arithmetic Progression †	Easy	1
1688	Count of Matches in Tournament	Easy	1
537	Complex Number Multiplication	Medium	1
667	Beautiful Arrangement II	Medium	1
762	Prime Number of Set Bits in Binary Representation	Easy	1
789	Escape The Ghosts	Medium	1
800	Similar RGB Color †	Easy	1
1033	Moving Stones Until Consecutive	Medium	1
1085	Sum of Digits in the Minimum Number †	Easy	1
1118	Number of Days in a Month †	Easy	1
1134	Armstrong Number †	Easy	1
1362	Closest Divisors	Medium	1
1860	Incremental Memory Leak	Medium	1
1952	Three Divisors	Easy	1
908	Smallest Range I	Easy	1
342	Power of Four	Easy	1
441	Arranging Coins	Easy	0
470	Implement Rand10() Using Rand7()	Medium	0
829	Consecutive Numbers Sum	Hard	0

#	PROBLEM	LEVEL	ASKED
858	Mirror Reflection	Medium	0
866	Prime Palindrome	Medium	0
1015	Smallest Integer Divisible by K	Medium	0
1017	Convert to Base -2	Medium	0
1154	Day of the Year	Easy	0
1201	Ugly Number III	Medium	0
1250	Check If It Is a Good Array	Hard	0
1276	Number of Burgers with No Waste of Ingredients	Medium	0

Math Simulation and Big Numbers

THE KERNEL

When the numbers exceed the language's integer range, or the problem is a step-by-step simulation, do the arithmetic digit by digit in an array with an explicit carry.

92PROBLEMS
MAPPED**34**

EASY

48

MEDIUM

10

HARD

The Boundary Rules

- 1 **Multiply Strings** (43) and **Add Strings** (415) and **Add Binary** (67) are one template: two pointers from the least significant digit, an accumulator, `divmod` for digit and carry, then reverse. **Python's arbitrary-precision integers mean you can cheat with `str(int(a) * int(b))`** — offer the honest digit-by-digit version first, then mention that Python makes it a one-liner. Interviewers accept both, but the honest version is what is being tested.
- 2 **Plus One** (66) is the carry template on a single array, with the all-nines edge case prepending a `1`.
- 3 **Reverse Integer** (7) and **String to Integer (atoi)** (8) are careful-parsing problems. `7` has an overflow clamp; `8` has a multi-clause specification (whitespace, sign, non-digits, clamp). **For `8`, enumerate the clauses out loud before coding** — it is a specification-reading test, not an algorithm test.
- 4 **Integer to English Words** (273) is a lookup-table recursion over thousands, millions, billions. The three `if` s (hundreds, tens, teens) are where the bugs are.

- 5 **Integer to Roman** (12) and **Roman to Integer** (13) are lookup tables. For 12, the greedy over a descending value list including the subtractive pairs (900, 400, 90, 40, 9, 4) is the clean solution.
- 6 **Zigzag Conversion** (6) is a simulation with a direction flag and a $2 * \text{num-Rows} - 2$ period. Recognising the period removes the need for nested lists of rows.
- 7 **Count and Say** (38) is a pure simulation: run-length encode the previous term. Use `itertools.groupby` or a manual two-pointer count.
- 8 **Pascal's Triangle** (118, 119)**: `row[i] += row[i-1]` right-to-left in place. 119 asks for a single row, which the in-place right-to-left update gives you with $O(k)$ space.
- 9 **Spiral Matrix**, **Rotate Image**, **Set Matrix Zeroes** are simulations covered in chapter 58. **Game of Life** (289) is the in-place bit encoding from that chapter.
- 10 **Fizz Buzz** (412), **Add Digits** (258), **Nim Game** (292), **Bulb Switcher** (319) are warm-ups with closed forms. 319 is 'count the perfect squares up to n ' — the insight is that only perfect squares have an odd number of divisors.
- 11 **Minimum Moves to Equal Array Elements** (453) is $\text{sum}(a) - n * \min(a)$. The reduction 'increment $n-1$ elements' equals 'decrement 1 element' is the whole problem.
- 12 **Distribute Candies to People** (1103) and similar are pure simulations; do not look for a formula unless the constraints force it.
- 13 **Recognition signal**. 'Without using built-in big integers', 'the result may be very large', or a problem with many explicit specification clauses. Read the specification twice and enumerate the cases before coding.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
2	Add Two Numbers	Medium	12
13	Roman to Integer	Easy	9

#	PROBLEM	LEVEL	ASKED
670	Maximum Swap	Medium	4
7	Reverse Integer	Medium	11
9	Palindrome Number	Easy	8
29	Divide Two Integers	Medium	7
539	Minimum Time Difference	Medium	2
66	Plus One	Easy	6
289	Game of Life	Medium	8
189	Rotate Array	Medium	6
118	Pascal's Triangle	Easy	7
38	Count and Say	Medium	6
299	Bulls and Cows	Medium	2
535	Encode and Decode TinyURL	Medium	3
171	Excel Sheet Column Number	Easy	3
796	Rotate String	Easy	5
1395	Count Number of Teams	Medium	3
166	Fraction to Recurring Decimal	Medium	3
168	Excel Sheet Column Title	Easy	3
412	Fizz Buzz	Easy	3
640	Solve the Equation	Medium	2
453	Minimum Moves to Equal Array Elements	Medium	3
396	Rotate Function	Medium	3
788	Rotated Digits	Medium	2
319	Bulb Switcher	Medium	2
728	Self Dividing Numbers	Easy	3
335	Self Crossing	Hard	2
462	Minimum Moves to Equal Array Elements II	Medium	1
1291	Sequential Digits	Medium	2
1360	Number of Days Between Two Dates	Easy	1

#	PROBLEM	LEVEL	ASKED
628	Maximum Product of Three Numbers	Easy	1
799	Champagne Tower	Medium	3
292	Nim Game	Easy	2
475	Heaters	Medium	2
659	Split Array into Consecutive Subsequences	Medium	2
1377	Frog Position After T Seconds	Hard	2
2018	Check if Word Can Be Placed In Crossword	Medium	1
119	Pascal's Triangle II	Easy	2
592	Fraction Addition and Subtraction	Medium	1
836	Rectangle Overlap	Easy	2
1836	Remove Duplicates From an Unsorted Linked List †	Medium	2
400	Nth Digit	Medium	1
406	Queue Reconstruction by Height	Medium	1
504	Base 7	Easy	1
689	Maximum Sum of 3 Non-Overlapping Subarrays	Hard	1
1877	Minimize Maximum Pair Sum in Array	Medium	1
365	Water and Jug Problem	Medium	1
223	Rectangle Area	Medium	1
258	Add Digits	Easy	1
276	Paint Fence †	Medium	1
492	Construct the Rectangle	Easy	1
598	Range Addition II	Easy	1
629	K Inverse Pairs Array	Hard	1
660	Remove 9 †	Hard	1
823	Binary Trees With Factors	Medium	1
870	Advantage Shuffle	Medium	1
1388	Pizza With 3n Slices	Hard	1
1534	Count Good Triplets	Easy	1

#	PROBLEM	LEVEL	ASKED
1630	Arithmetic Subarrays	Medium	1
1748	Sum of Unique Elements	Easy	1
372	Super Pow	Medium	1
507	Perfect Number	Easy	1
1198	Find Smallest Common Element in All Rows †	Medium	1
762	Prime Number of Set Bits in Binary Representation	Easy	1
812	Largest Triangle Area	Easy	1
1133	Largest Unique Number †	Easy	1
1394	Find Lucky Integer in an Array	Easy	1
1409	Queries on a Permutation With Key	Medium	1
1726	Tuple with Same Product	Medium	1
2006	Count Number of Pairs With Absolute Difference K	Easy	1
2150	Find All Lonely Numbers in the Array	Medium	1
2161	Partition Array According to Given Pivot	Medium	1
2190	Most Frequent Number Following Key In an Array	Easy	1
2206	Divide Array Into Equal Pairs	Easy	1
2244	Minimum Rounds to Complete All Tasks	Medium	1
479	Largest Palindrome Product	Hard	1
573	Squirrel Simulation †	Medium	1
1427	Perform String Shifts †	Easy	1
2154	Keep Multiplying Found Values by Two	Easy	1
2248	Intersection of Multiple Arrays	Easy	1
423	Reconstruct Original Digits from English	Medium	0
441	Arranging Coins	Easy	0
556	Next Greater Element III	Medium	0
564	Find the Closest Palindrome	Hard	0
780	Reaching Points	Hard	0

String – Palindromes, Pointers, and Parsing

THE KERNEL

Most string problems are array problems where the elements happen to be characters. The only genuinely string-specific tools are the frequency count, the two-pointer palindrome check, and the character-class predicate.

135PROBLEMS
MAPPED**80**

EASY

48

MEDIUM

7

HARD

The Boundary Rules

- 1 **Palindrome check.** `s == s[::-1]` for a copy, or the two-pointer loop for $O(1)$ space. **Valid Palindrome** (125) requires filtering to alphanumerics first — `c.isalnum()` and `c.lower()`.
- 2 **Valid Palindrome II** (680) allows one deletion: on the first mismatch, try skipping the left character or the right character and check whether either remainder is a palindrome. **Only the first mismatch needs handling** — a second mismatch means both branches fail.
- 3 **Longest Palindrome** (409) is a counting argument:
`2 * sum(count // 2 for count in Counter(s).values())` plus 1 if any count is odd. No construction needed.
- 4 **Reverse String** (344) and **Reverse Words in a String** (151) are pointer exercises. For 151, `' '.join(s.split()[::-1])` is the one-liner — `split()` with no argument handles all whitespace runs.

- 5 **Is Subsequence** (392) is a two-pointer scan, or `bisect` over per-character index lists for the follow-up ‘many queries’ version.
- 6 **String Compression** (443) is the read/write pointer pattern from chapter 6, writing both the character and its count digits in place.
- 7 **Zigzag**, `atoi`, **Count and Say** and the arithmetic string problems are in chapter 81 — string parsing with explicit specifications.
- 8 **Remove All Adjacent Duplicates** (1047 , 1209), **Make The String Great** (1544) and **Backspace String Compare** (844) are stack problems (chapter 29).
- 9 **Word Pattern** (290) and **Isomorphic Strings** (205) are bijection checks (chapter 21).
- 10 **Decode String** (394), **Basic Calculator** (224 , 227) and **Evaluate Reverse Polish Notation** (150) are stack-based parsers (chapter 25).
- 11 **Character-class predicates worth knowing.** `c.isalpha()`, `c.isdigit()`, `c.isalnum()`, `c.isspace()`, `c.isupper()`. And for conversion: `ord(c)`, `chr(i)`, `int(c)`, `str(i)`.
- 12 **Never build a string with `+=` in a loop** — it is $O(n^2)$ in Python because strings are immutable. Use a list and `''.join(parts)`. This is a real performance issue on LeetCode’s larger test cases and a standard style point.
- 13 **`str.translate` and `str.maketrans`** do character-level replacement in one C-level call — useful for **Ransom Note** -style substitutions and worth knowing.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
408	Valid Word Abbreviation †	Easy	2
5	Longest Palindromic Substring	Medium	9
14	Longest Common Prefix	Easy	9
415	Add Strings	Easy	7
791	Custom Sort String	Medium	2
387	First Unique Character in a String	Easy	8

#	PROBLEM	LEVEL	ASKED
65	Valid Number	Hard	2
1152	Analyze User Website Visit Pattern †	Medium	2
68	Text Justification	Hard	7
443	String Compression	Medium	7
647	Palindromic Substrings	Medium	3
344	Reverse String	Easy	6
1507	Reformat Date	Easy	3
1629	Slowest Key	Easy	2
1366	Rank Teams by Votes	Medium	4
243	Shortest Word Distance †	Easy	5
383	Ransom Note	Easy	4
557	Reverse Words in a String III	Easy	5
616	Add Bold Tag in String †	Medium	1
809	Expressive Words	Medium	3
1487	Making File Names Unique	Medium	2
165	Compare Version Numbers	Medium	2
859	Buddy Strings	Easy	2
796	Rotate String	Easy	5
468	Validate IP Address	Medium	3
420	Strong Password Checker	Hard	2
392	Is Subsequence	Easy	3
1221	Split a String in Balanced Strings	Easy	4
771	Jewels and Stones	Easy	3
609	Find Duplicate File in System	Medium	2
1087	Brace Expansion †	Medium	2
2055	Plates Between Candles	Medium	2
2103	Rings and Rods	Easy	1
1604	Alert Using Same Key-Card Three or More Times in a One Hour Period	Medium	1

#	PROBLEM	LEVEL	ASKED
58	Length of Last Word	Easy	2
551	Student Attendance Record I	Easy	1
824	Goat Latin	Easy	1
884	Uncommon Words from Two Sentences	Easy	2
1408	String Matching in an Array	Easy	2
1417	Reformat The String	Easy	2
1446	Consecutive Characters	Easy	2
271	Encode and Decode Strings †	Medium	2
186	Reverse Words in a String II †	Medium	1
459	Repeated Substring Pattern	Easy	2
520	Detect Capital	Easy	2
524	Longest Word in Dictionary through Deleting	Medium	1
821	Shortest Distance to a Character	Easy	2
929	Unique Email Addresses	Easy	1
1002	Find Common Characters	Easy	1
1108	Defanging an IP Address	Easy	2
1451	Rearrange Words in a Sentence	Medium	3
1592	Rearrange Spaces Between Words	Easy	1
1773	Count Items Matching a Rule	Easy	2
1832	Check if the Sentence Is Pangram	Easy	2
245	Shortest Word Distance III †	Medium	2
599	Minimum Index Sum of Two Lists	Easy	2
1859	Sorting the Sentence	Easy	2
466	Count The Repetitions	Hard	1
514	Freedom Trail	Hard	2
734	Sentence Similarity †	Easy	2
830	Positions of Large Groups	Easy	1
1119	Remove Vowels from a String †	Easy	1

#	PROBLEM	LEVEL	ASKED
1662	Check If Two String Arrays are Equivalent	Easy	2
1844	Replace All Digits with Characters	Easy	2
306	Additive Number	Medium	1
345	Reverse Vowels of a String	Easy	1
434	Number of Segments in a String	Easy	1
522	Longest Uncommon Subsequence II	Medium	1
541	Reverse String II	Easy	1
686	Repeated String Match	Medium	1
709	To Lower Case	Easy	1
748	Shortest Completing Word	Easy	1
819	Most Common Word	Easy	1
838	Push Dominoes	Medium	1
890	Find and Replace Pattern	Medium	1
917	Reverse Only Letters	Easy	1
925	Long Pressed Name	Easy	1
942	DI String Match	Easy	1
1138	Alphabet Board Path	Medium	1
1170	Compare Strings by Frequency of the Smallest Character	Medium	1
1374	Generate a String With Characters That Have Odd Counts	Easy	1
1436	Destination City	Easy	1
1496	Path Crossing	Easy	1
1525	Number of Good Ways to Split a String	Medium	1
1528	Shuffle String	Easy	1
1576	Replace All '?'s to Avoid Consecutive Repeating Characters	Easy	1
1624	Largest Substring Between Two Equal Characters	Easy	1
1678	Goal Parser Interpretation	Easy	1
1772	Sort Features by Popularity †	Medium	1
1869	Longer Contiguous Segments of Ones than Zeros	Easy	1

#	PROBLEM	LEVEL	ASKED
2042	Check if Numbers Are Ascending in a Sentence	Easy	1
2047	Number of Valid Words in a Sentence	Easy	1
804	Unique Morse Code Words	Easy	1
1556	Thousand Separator	Easy	1
1758	Minimum Changes To Make Alternating Binary String	Easy	1
1805	Number of Different Integers in a String	Easy	1
293	Flip Game †	Easy	1
411	Minimum Unique Word Abbreviation †	Hard	1
481	Magical String	Medium	1
482	License Key Formatting	Easy	1
521	Longest Uncommon Subsequence I	Easy	1
816	Ambiguous Coordinates	Medium	1
893	Groups of Special-Equivalent Strings	Medium	1
1165	Single-Row Keyboard †	Easy	1
1324	Print Words Vertically	Medium	1
1452	People Whose List of Favorite Companies Is Not a Subset of Another List	Medium	1
1764	Form Array by Concatenating Subarrays of Another Array	Medium	1
1813	Sentence Similarity III	Medium	1
1816	Truncate Sentence	Easy	1
1945	Sum of Digits of String After Convert	Easy	1
2075	Decode the Slanted Ciphertext	Medium	1
2109	Adding Spaces to a String	Medium	1
2114	Maximum Number of Words Found in Sentences	Easy	1
2124	Check if All A's Appears Before All B's	Easy	1
2255	Count Prefixes of a Given String	Easy	1
1410	HTML Entity Parser	Medium	1
1754	Largest Merge Of Two Strings	Medium	1
1784	Check if Binary String Has at Most One Segment of Ones	Easy	1

#	PROBLEM	LEVEL	ASKED
1935	Maximum Number of Words You Can Type	Easy	1
1967	Number of Strings That Appear as Substrings in Word	Easy	1
2243	Calculate Digit Sum of a String	Easy	1
28	Find the Index of the First Occurrence in a String	Easy	0
657	Robot Return to Origin	Easy	0
722	Remove Comments	Medium	0
831	Masking Personal Information	Medium	0
833	Find And Replace in String	Medium	0
936	Stamping The Sequence	Hard	0
966	Vowel Spellchecker	Medium	0
1003	Check If Word Is Valid After Substitutions	Medium	0
1016	Binary String With Substrings Representing 1 To N	Medium	0
1625	Lexicographically Smallest String After Applying Operations	Medium	1

String - KMP and the Z-Algorithm

THE KERNEL

KMP precomputes, for every prefix, the length of its longest proper prefix that is also a suffix. That table makes pattern matching $O(n + m)$ and answers a surprising number of 'repeated structure' questions.

17

PROBLEMS
MAPPED

4

EASY

4

MEDIUM

9

HARD

The Boundary Rules

- 1 **The failure function.** `lps[i]` = the length of the longest proper prefix of `s[:i+1]` that is also a suffix. Build it with two pointers: `i` scanning and `j` the current matched length, with the fallback `j = lps[j-1]` on mismatch. **The fallback is the whole algorithm** — without it you are back to brute force.
- 2 **Matching** uses the same fallback: on a mismatch after `j` matches, set `j = lps[j-1]` and do not advance `i`. That 'do not advance `i`' is what makes it linear.
- 3 **Find the Index of the First Occurrence** (28) is the direct application. In Python, `s.find(t)` is the one-liner and uses a C-optimised search — but the interviewer is asking for KMP, so write it.
- 4 **Repeated Substring Pattern** (459) has a one-line answer: `s in (s + s)[1:-1]`. **The reasoning is that a repeated string appears in its own doubled version.** Knowing this trick is a strong signal; knowing the KMP version too is stronger.

- 5 **Shortest Palindrome** (214) uses `lps` on `s + '#' + s[::-1]` to find the longest palindromic prefix, then prepends the reverse of the rest. The `'#'` separator prevents the prefix from crossing the boundary.
- 6 **Longest Happy Prefix** (1392) is literally `lps[-1]` — the definition of the failure function's last value.
- 7 **Find the Longest Substring Containing Vowels in Even Counts** (1371) uses a bitmask prefix state, not KMP, but lives in the same mental neighbourhood.
- 8 **The Z-array.** `z[i]` = the length of the longest common prefix of `s` and `s[i:]`. It answers the same questions as KMP with a different precomputation. The `[l, r]` window optimisation is what makes it linear.
- 9 **When to use which.** KMP when you need the failure function for `s + '#' + t`; Z when you need all prefix matches at once. Both are $O(n)$; pick whichever you can write faster.
- 10 **Rolling hash is the alternative** (chapter 84). It is shorter to write but has collision risk; KMP is exact. If the interviewer asks for 'linear time string matching', either is acceptable, but say which and why.
- 11 **Recognition signal.** 'Find the pattern in the text', 'longest repeated substring', 'shortest palindrome by prepending', 'the string is a repetition of a substring'. All of them are the failure function.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
5	Longest Palindromic Substring	Medium	9
44	Wildcard Matching	Hard	6
796	Rotate String	Easy	5
214	Shortest Palindrome	Hard	3
459	Repeated Substring Pattern	Easy	2
686	Repeated String Match	Medium	1
1392	Longest Happy Prefix	Hard	1

#	PROBLEM	LEVEL	ASKED
1371	Find the Longest Substring Containing Vowels in Even Counts	Medium	1
2185	Counting Words With a Given Prefix	Easy	1
1316	Distinct Echo Substrings	Hard	0
2223	Sum of Scores of Built Strings	Hard	0
2851	String Transformation	Hard	0
3006	Find Beautiful Indices in the Given Array I	Medium	0
3008	Find Beautiful Indices in the Given Array II	Hard	0
3036	Number of Subarrays That Match a Pattern II	Hard	0
3303	Find the Occurrence of First Almost Equal Substring	Hard	0

String – Rolling Hash and Substring Search

THE KERNEL

Hash every substring in $O(1)$ from the previous one: $h = (h * \text{base} + \text{code}(c)) \% \text{mod}$. With prefix hashes you can compare any two substrings in $O(1)$.

22

 PROBLEMS
MAPPED

3

EASY

10

MEDIUM

9

HARD

The Boundary Rules

- 1 **Prefix-hash construction.** $\text{pre}[i+1] = (\text{pre}[i] * \text{base} + \text{code}(s[i])) \% \text{mod}$, and $\text{pow_base}[i] = \text{base}^i \% \text{mod}$. Then the hash of $s[l:r]$ is $(\text{pre}[r] - \text{pre}[l] * \text{pow_base}[r-l]) \% \text{mod}$.
- 2 **Collisions.** A single 64-bit modulus is usually enough for competitive programming, but for an interview you should mention the collision risk and offer double hashing (two different moduli) or a verification step. **Saying ‘this is probabilistic’ is the point being tested.**
- 3 **Repeated DNA Sequences** (187) is the simplest case: a 10-character window over a 4-letter alphabet, which fits in 20 bits. Use a bit-shift rolling hash and a set of seen hashes.
- 4 **Longest Duplicate Substring** (1044) is binary search on the length plus a rolling-hash set. **The binary search is monotone because if a duplicate substring of length L exists, one of length $L-1$ does too.** This is the flagship rolling-hash problem and it combines chapters 13, 84 and 22.

- 5 **Maximum Length of Repeated Subarray** (718) can be solved with DP (chapter 63) or rolling hash plus binary search. The DP is simpler; the hash is faster for large inputs.
- 6 **Distinct Echo Substrings** (1147) and **Longest Chunked Palindrome Decomposition** (1147 -adjacent) use hashes to compare halves in $O(1)$.
- 7 **Number of Distinct Substrings** variants: the count of distinct substrings is $n(n+1)/2$ minus the sum of the suffix array's LCP values. Suffix arrays are beyond most interviews, but knowing the formula is a nice touch.
- 8 **Base choice.** Use a prime base larger than the alphabet (131, 137, 1000003) and a large prime modulus ($10^9 + 7$, $2^{61} - 1$). Using `base = 26` with a small modulus collides immediately on English text.
- 9 **Find All Anagrams in a String** (438) and **Permutation in String** (567) are solved with a *frequency* rolling window, not a polynomial hash — the 26-element count array is the right tool there. Recognise which rolling technique the alphabet size suggests.
- 10 **Recognition signal.** ‘Longest repeated substring’, ‘count distinct substrings’, ‘check if two substrings are equal many times’, or any problem where the constraints require better than $O(n^2)$ substring comparison.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
438	Find All Anagrams in a String	Medium	10
567	Permutation in String	Medium	7
214	Shortest Palindrome	Hard	3
459	Repeated Substring Pattern	Easy	2
1392	Longest Happy Prefix	Hard	1
1062	Longest Repeating Substring †	Medium	1
1461	Check If a String Contains All Binary Codes of Size K	Medium	1
28	Find the Index of the First Occurrence in a String	Easy	0
1316	Distinct Echo Substrings	Hard	0

#	PROBLEM	LEVEL	ASKED
1554	Strings Differ by One Character †	Medium	0
1698	Number of Distinct Substrings in a String †	Medium	0
1923	Longest Common Subpath	Hard	0
2223	Sum of Scores of Built Strings	Hard	0
2261	K Divisible Elements Subarrays	Medium	0
2430	Maximum Deletions on a String	Hard	0
3023	Find Pattern in Infinite Stream I †	Medium	0
3042	Count Prefix and Suffix Pairs I	Easy	0
3045	Count Prefix and Suffix Pairs II	Hard	0

PART XV

The Hard Core

Divide and conquer, quickselect, randomised algorithms, and the composite design round where naming your structures is the interview.

Divide and Conquer

THE KERNEL

Split the input in half, solve both halves, combine. The combine step is where the algorithm lives — and for counting problems, the combine step is a merge that also counts inversions.

31PROBLEMS
MAPPED**3**

EASY

19

MEDIUM

9

HARD

The Boundary Rules

- 1 The merge-sort-with-counting template.** Sort the array while counting cross-half pairs. `Count of Smaller Numbers After Self` (315), `Reverse Pairs` (493) and `Count of Range Sum` (327) are all the same code with a different comparison in the merge step. **Learn one, get three.**
- 2 The counting must happen before the merge,** using the fact that both halves are already sorted. Two pointers walk the halves and count pairs satisfying the predicate; then the standard merge proceeds. If you count during the merge you lose the sortedness guarantee.
- 3 `Count of Range Sum` (327)** is the hardest of the three because the predicate is a `range` (`lower <= sum <= upper`), which needs two pointers rather than one. The prefix-sum transformation is the first step.
- 4 `Maximum Subarray divide and conquer` (53):** best in the left, best in the right, best crossing the midpoint (computed with a suffix scan and a prefix scan). $O(n \log n)$, and the same shape as 315.

- 5 **Different Ways to Add Parentheses** (241) and **Unique Binary Search Trees II** (95) are divide-and-conquer *enumeration*: split at every operator or root, recurse both sides, combine the results. The memoisation on the interval is what makes 241 polynomial.
- 6 **Median of Two Sorted Arrays** (4) is divide and conquer on the *partition*, not the array (chapter 14). Recognise that ‘divide’ can mean ‘divide the answer space’.
- 7 **Sort List** (148) is merge sort on a linked list (chapter 36). **Sort an Array** (912) should be merge sort or heap sort if the interviewer forbids the built-in — quicksort’s worst case is $O(n^2)$.
- 8 **Beautiful Array** (932) is the classic construction by parity recursion: $A(n) = 2 A(\text{ceil}(n/2)) - 1 + 2 A(\text{floor}(n/2))$. The recursion on the *structure* rather than the data is unusual and worth seeing once.
- 9 **Expression Add Operators** (282) and **Different Ways to Add Parentheses** are both ‘enumerate the splits’. When a problem says ‘add parentheses’, think divide and conquer.
- 10 **Master theorem.** $T(n) = 2T(n/2) + O(n)$ is $O(n \log n)$; $T(n) = 2T(n/2) + O(1)$ is $O(n)$; $T(n) = T(n/2) + O(1)$ is $O(\log n)$. Being able to state the recurrence and its solution is expected.
- 11 **Recursion depth.** Divide and conquer on $n = 10^5$ gives depth $\log_2(10^5) \approx 17$, which is safe. But a *skewed* split (e.g. always splitting at index 0) gives depth n and will raise `RecursionError`. Check that your split is balanced.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
215	Kth Largest Element in an Array	Medium	9
973	K Closest Points to Origin	Medium	5
23	Merge k Sorted Lists	Hard	14
169	Majority Element	Easy	8
240	Search a 2D Matrix II	Medium	6
109	Convert Sorted List to Binary Search Tree	Medium	4

#	PROBLEM	LEVEL	ASKED
282	Expression Add Operators	Hard	1
148	Sort List	Medium	5
105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	6
1008	Construct Binary Search Tree from Preorder Traversal	Medium	4
1382	Balance a Binary Search Tree	Medium	1
108	Convert Sorted Array to Binary Search Tree	Easy	5
229	Majority Element II	Medium	3
218	The Skyline Problem	Hard	3
95	Unique Binary Search Trees II	Medium	1
96	Unique Binary Search Trees	Medium	1
1763	Longest Nice Substring	Easy	1
1985	Find the Kth Largest Integer in the Array	Medium	1
493	Reverse Pairs	Hard	1
106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	1
654	Maximum Binary Tree	Medium	1
327	Count of Range Sum	Hard	0
889	Construct Binary Tree from Preorder and Postorder Traversal	Medium	0
1095	Find in Mountain Array	Hard	0
1569	Number of Ways to Reorder Array to Get Same BST	Hard	0

Quickselect and Partitioning

THE KERNEL

Partition around a pivot, then recurse into only the side that contains your answer. Expected $O(n)$, worst case $O(n^2)$ — which is exactly what the interviewer will ask about.

17

PROBLEMS
MAPPED

4

EASY

13

MEDIUM

0

HARD

The Boundary Rules

- 1 **Lomuto partition.** Choose a pivot, then one pass swapping elements smaller than the pivot to the front. Returns the pivot's final index. Simple but degrades to $O(n^2)$ on sorted input with a first/last pivot.
- 2 **Randomised pivot.** `random.randint(lo, hi)` before partitioning makes the expected complexity $O(n)$ regardless of input order. **Saying 'I randomise the pivot to avoid the adversarial worst case' is the answer to the follow-up question.**
- 3 **Quickselect.** After partitioning, if the pivot index equals `k`, return; if `k < pivot_index`, recurse left; else recurse right. **Recurse into one side only** — that is what makes it $O(n)$ expected rather than $O(n \log n)$.
- 4 **Kth Largest Element in an Array** (215) is the archetype. The index conversion: the kth largest is the `(n - k)` th smallest. Get this wrong and you are off by one.

- 5 **K Closest Points to Origin** (973) and **Top K Frequent Elements** (347) are quickselect with a different key. Both also have heap solutions (chapter 30); the heap is $O(n \log k)$ and the quickselect is $O(n)$ expected. **Offer both and let the interviewer choose.**
- 6 **Wiggle Sort II** (324) needs the *median* via quickselect and then a three-way partition. It is the hardest application in this chapter.
- 7 **Sort Colors** (75) is the three-way (Dutch national flag) partition in one pass: three pointers for 0, 1, 2. **Memorise it** — it is the most-asked partition problem and it is six lines.
- 8 **Move Zeroes** (283) and **Remove Element** (27) are the one-way partition from chapter 6 — a degenerate case worth recognising as such.
- 9 **Three-way partition** (elements `< pivot`, `= pivot`, `> pivot`) is what makes quicksort fast on duplicate-heavy input. Mention it when asked about quicksort's behaviour on many duplicates.
- 10 **Quickselect vs heap: the decision.** If `k` is small relative to `n`, use a heap of size `k` ($O(n \log k)$, and it streams). If `k` is arbitrary and the data is in memory, quickselect is $O(n)$ expected. State the trade-off rather than picking blindly.
- 11 **The worst case.** Quickselect's $O(n^2)$ happens when every partition is maximally unbalanced. With a random pivot the probability is negligible; with a median-of-medians pivot it is $O(n)$ guaranteed, but the constant is large and it is almost never worth implementing. **Knowing that median-of-medians exists is the answer to 'can you guarantee it?'**

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
88	Merge Sorted Array	Easy	9
283	Move Zeroes	Easy	10
692	Top K Frequent Words	Medium	9
658	Find K Closest Elements	Medium	7
169	Majority Element	Easy	8
148	Sort List	Medium	5

#	PROBLEM	LEVEL	ASKED
27	Remove Element	Easy	5
229	Majority Element II	Medium	3
462	Minimum Moves to Equal Array Elements II	Medium	1
912	Sort an Array	Medium	2
1985	Find the Kth Largest Integer in the Array	Medium	1
2343	Query Kth Smallest Trimmed Number	Medium	0

Randomized Algorithms and Reservoir Sampling

THE KERNEL

Reservoir sampling picks a uniformly random element from a stream of unknown length in $O(1)$ space: keep the i -th element with probability $1/i$.

14PROBLEMS
MAPPED**0**

EASY

11

MEDIUM

3

HARD

The Boundary Rules

- 1 **Reservoir sampling, memorised.** `res = first; for i, x in enumerate(stream[1:], 2): if random.randint(1, i) == 1: res = x`. Each element is kept with probability $1/n$, so the result is uniform.
- 2 **Linked List Random Node** (382) is reservoir sampling over a linked list — you cannot know the length in advance, which is exactly the sampling problem's premise.
- 3 **Random Pick Index** (398) is reservoir sampling over the *matching* indices only. The count `i` must increment only when the value matches.
- 4 **Shuffle an Array** (384) is Fisher-Yates: `for i in range(n-1, 0, -1): j = randint(0, i); a[i], a[j] = a[j], a[i]`. **Iterate from the end and swap with an index in `[0, i]`**. Swapping forward, or picking `j` from `[0, n-1]`, produces a non-uniform distribution — that is the standard trap.

- 5 **Random Pick with Weight** (528) is a prefix-sum plus `bisect` on a random draw in `[0, total)`. **`bisect_right` is the correct variant** — it handles the boundary where the draw lands exactly on a prefix sum.
- 6 **Insert Delete GetRandom O(1)** (380) is chapter 24's swap-with-last trick. **Randomized Collection** (381) extends it to duplicates with a dict of sets.
- 7 **Randomized Set** and friends are design problems, not probability problems — the randomness is trivial and the data-structure engineering is the difficulty.
- 8 **Uniformity is the thing being tested.** For every problem in this chapter, be ready to argue that each outcome has equal probability. 'Each element survives with probability $1/i$ at step `i`, so the probability of surviving to the end is $1/n$ ' is the standard argument.
- 9 **Implement Rand10() Using Rand7()** (470) is rejection sampling: generate a uniform number in a range that is a multiple of 10 and reject the rest. The expected number of calls is constant. It is a favourite problem at large companies and is purely about the sampling argument.
- 10 **Python specifics.** Use `random.randint(a, b)` for inclusive integer ranges and `random.random()` for floats. `random.choice` is uniform over a sequence. `random.shuffle` is Fisher-Yates already — do not reimplement it unless asked.
- 11 **Recognition signal.** 'Uniformly at random', 'each element equally likely', 'without knowing the length in advance', or a stream you cannot store. Also: any design problem where the query is 'give me a random one'.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
138	Copy List with Random Pointer	Medium	6
1146	Snapshot Array	Medium	2
381	Insert Delete GetRandom O(1) - Duplicates allowed	Hard	2
497	Random Point in Non-overlapping Rectangles	Medium	0
519	Random Flip Matrix	Medium	0
710	Random Pick with Blacklist	Hard	0

#	PROBLEM	LEVEL	ASKED
2323	Find Minimum Time to Finish All Jobs II †	Medium	0
2503	Maximum Number of Points From Grid Queries	Hard	0

Hard Design Problems - The Composite Round

THE KERNEL

Every hard design problem is two or three earlier templates bolted together. Name the templates out loud before you write a line of code; that naming is the interview.

92PROBLEMS
MAPPED**8**

EASY

60

MEDIUM

24

HARD

The Boundary Rules

- 1 **LRU Cache** (146) = hash map + doubly linked list with sentinels (chapter 24).
LFU Cache (460) = hash map + per-frequency ordered lists + a `min_freq` counter. Both must be $O(1)$ per operation; if your solution walks a structure, it is not finished.
- 2 **Design Twitter** (355) = per-user tweet lists + a follow set + a heap merge for the feed. The feed is `Merge k Sorted Lists` (chapter 32) in a design costume.
- 3 **Range Module** (715) = a sorted list of disjoint intervals plus `bisect` for insertion. Every operation is ‘find the overlapping range, merge, reinsert’. It is chapter 18’s merge applied incrementally.

- 4 **My Calendar I/II/III** (729 , 731 , 732) = a sorted container of intervals, or a sweep line with a count. 732 (max concurrent bookings) is a difference-array / ordered-map problem — a plain dict of deltas plus a running maximum on demand is not $O(1)$ per query, so you need an ordered structure or a sorted list with `bisect`.
- 5 **Snapshot Array** (1146) = per-index (snap_id, value) history plus a global counter, with `bisect` for the lookup (chapter 24).
- 6 **Time Based Key-Value Store** (981) = per-key parallel lists of timestamps and values, with `bisect_right` (chapter 24).
- 7 **Design Authentication Manager** (1797) = dict of token to expiry with lazy expiration. **Lazy beats eager**: never run a timer, just check on access.
- 8 **Design a Food Rating System** (2353) = dict of cuisines to a heap of (-rating, food, version), with version numbers for lazy deletion. **Lazy deletion by version stamp is the reusable trick** whenever the underlying structure cannot be mutated efficiently.
- 9 **Stock Price Fluctuation** (2034) = dict of timestamp to price plus two heaps (max and min) with lazy deletion by comparing against the current dict. Same trick.
- 10 **Design Skiplist** (1206) and **All 0(1) Data Structure** (432) are the hardest in the chapter. 1206 asks you to implement a probabilistic balanced structure — if asked, implement it as a layered linked list with random promotion. 432 needs a doubly linked list of buckets plus two hash maps (key to bucket, count to bucket).
- 11 **Design In-Memory File System** (588) = a nested dict with sorted listing. It is a trie by another name.
- 12 **Encode and Decode TinyURL** (535) and **Design HashMap** (706) are the warm-ups. 535 is a counter plus base-62 encoding; do not invent hashing.
- 13 **Design Hit Counter** (362) = a queue or a ring buffer of 300 buckets. The ring buffer avoids unbounded memory and is the better answer.

- 14 The universal method for this chapter.** (1) List the operations and their required complexities. (2) Choose one data structure per complexity requirement. (3) State the invariant that links them. (4) Write the helpers first, the public methods second. (5) Test the edge cases: empty structure, single element, delete-then-get.
- 15 The most common failure** in a design interview is starting to code before naming the structures. Spend two minutes on the whiteboard drawing the boxes and arrows; it converts a 60-line struggle into a 25-line derivation.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
1570	Dot Product of Two Sparse Vectors †	Medium	5
1396	Design Underground System	Medium	1
244	Shortest Word Distance II †	Medium	1
348	Design Tic-Tac-Toe †	Medium	5
359	Logger Rate Limiter †	Easy	6
1146	Snapshot Array	Medium	2
588	Design In-Memory File System †	Hard	6
716	Max Stack †	Hard	6
362	Design Hit Counter †	Medium	7
706	Design HashMap	Easy	8
2034	Stock Price Fluctuation	Medium	1
981	Time Based Key-Value Store	Medium	6
642	Design Search Autocomplete System †	Hard	6
341	Flatten Nested List Iterator	Medium	8
1656	Design an Ordered Stream	Easy	1
304	Range Sum Query 2D - Immutable	Medium	5
622	Design Circular Queue	Medium	6
1428	Leftmost Column with at Least a One †	Medium	3
1472	Design Browser History	Medium	4

HARD DESIGN PROBLEMS - THE COMPOSITE ROUND

#	PROBLEM	LEVEL	ASKED
284	Peeking Iterator	Medium	4
353	Design Snake Game †	Medium	6
432	All O'one Data Structure	Hard	3
1352	Product of the Last K Numbers	Medium	3
1603	Design Parking System	Easy	1
535	Encode and Decode TinyURL	Medium	3
729	My Calendar I	Medium	3
307	Range Sum Query - Mutable	Medium	2
381	Insert Delete GetRandom O(1) - Duplicates allowed	Hard	2
251	Flatten 2D Vector †	Medium	1
1429	First Unique Number †	Medium	2
281	Zigzag Iterator †	Medium	3
676	Implement Magic Dictionary	Medium	2
705	Design HashSet	Easy	2
271	Encode and Decode Strings †	Medium	2
288	Unique Word Abbreviation †	Medium	2
379	Design Phone Directory †	Medium	2
635	Design Log Storage System †	Medium	1
308	Range Sum Query 2D - Mutable †	Medium	1
631	Design Excel Sum Formula †	Hard	1
900	RLE Iterator	Medium	1
1670	Design Front Middle Back Queue	Medium	1
303	Range Sum Query - Immutable	Easy	1
604	Design Compressed String Iterator †	Easy	1
1804	Implement Trie II (Prefix Tree) †	Medium	1
2166	Design Bitset	Medium	1
1286	Iterator for Combination	Medium	1
1357	Apply Discount Every n Orders	Medium	1

#	PROBLEM	LEVEL	ASKED
1756	Design Most Recently Used Queue †	Medium	1
1797	Design Authentication Manager	Medium	1
1845	Seat Reservation Manager	Medium	1
2043	Simple Bank System	Medium	1
2080	Range Frequency Queries	Medium	1
295	Find Median from Data Stream	Hard	0
641	Design Circular Deque	Medium	0
707	Design Linked List	Medium	0
731	My Calendar II	Medium	0
732	My Calendar III	Hard	0
745	Prefix and Suffix Search	Hard	0
911	Online Election	Medium	0
1032	Stream of Characters	Hard	0
1157	Online Majority Element In Subarray	Hard	0
1166	Design File System †	Medium	0
1206	Design Skiplist	Hard	0
1244	Design A Leaderboard †	Medium	0
1483	Kth Ancestor of a Tree Node	Hard	0
1500	Design a File Sharing System †	Medium	0
1622	Fancy Sequence	Hard	0
1825	Finding MK Average	Hard	0
1865	Finding Pairs With a Certain Sum	Medium	0
1912	Design Movie Rental System	Hard	0
2013	Detect Squares	Medium	0
2069	Walking Robot Simulation II	Medium	0
2102	Sequentially Ordinal Rank Tracker	Hard	0
2227	Encrypt and Decrypt Strings	Hard	0
2254	Design Video Sharing Platform †	Hard	0

HARD DESIGN PROBLEMS – THE COMPOSITE ROUND

#	PROBLEM	LEVEL	ASKED
2276	Count Integers in Intervals	Hard	0
2296	Design a Text Editor	Hard	0
2336	Smallest Number in Infinite Set	Medium	0
2353	Design a Food Rating System	Medium	0
2408	Design SQL	Medium	0
2424	Longest Uploaded Prefix	Medium	0
2502	Design Memory Allocator	Medium	0
2590	Design a Todo List †	Medium	0
2645	Minimum Additions to Make Valid String	Medium	0
2671	Frequency Tracker	Medium	0
2958	Length of Longest Subarray With at Most K Frequency	Medium	0
3062	Winner of the Linked List Game †	Easy	0
3116	Kth Smallest Amount With Single Denomination Combination	Hard	0

PART XVI

Specialist Patterns

Trie, Eulerian paths and bridges, computational geometry, and where concurrency still appears.

Trie (Prefix Tree)

THE KERNEL

A tree where each edge is a character and each node is a prefix. It turns ‘does any word start with this prefix’ from $O(\text{total length})$ into $O(\text{prefix length})$.

36PROBLEMS
MAPPED**3**

EASY

18

MEDIUM

15

HARD

The Boundary Rules

- 1 The node, memorised.** `children = {}` plus `is_end = False`. A dict beats a 26-array when the alphabet is large or sparse; a 26-list beats a dict when the alphabet is exactly lowercase and speed matters. Say which you are choosing and why.
- 2 Three operations.** `insert`: walk/create children, mark `is_end`. `search`: walk, return `is_end` at the end. `startsWith`: walk, return `True` if the walk succeeded. **The only difference between `search` and `startsWith` is the final `is_end` check** — that is the whole reason Trie has two methods.
- 3 Design Add and Search Words** (211) adds `'.'` as a wildcard, which turns the walk into a DFS over all children at that position. The recursion is `any(search(node.child[c], i+1) for c in node.children)`.

- 4 **Word Search II** (212) is the flagship combination: build a Trie of all words, then do a single DFS over the grid that walks the Trie in lockstep. **Prune matched words from the Trie as you go** (or mark them collected) so the same word is not emitted twice. This is the single best example in the book of a data structure eliminating a nested loop: it takes the problem from $O(\text{words cells } 4^L)$ to $O(\text{cells} * 4^L)$.
- 5 **Replace Words** (648) walks the Trie and returns the first `is_end` it hits. The ‘shortest prefix wins’ requirement is satisfied by the walk order — no comparison needed.
- 6 **Map Sum Pairs** (677) stores a running sum at each node, so a prefix query is a single walk. Storing an aggregate per node is the general technique for prefix-sum queries.
- 7 **Search Suggestions System** (1268) needs the top-3 lexicographic matches per prefix. Two options: store a sorted list (or a heap of 3) at each node, or binary-search the sorted word list per prefix. The Trie-with-cached-results version is the intended answer.
- 8 **Longest Word in Dictionary** (720) and **Long Encoding of Words** (820) use the Trie to test ‘is every prefix also a word’.
- 9 **Stream of Characters** (1032) walks the Trie *backwards* from the newest character — the query string grows on the left. Recognising that the insertion point is the left end is the difficulty.
- 10 **The bitwise Trie.** **Maximum XOR of Two Numbers** (421) and **Maximum XOR With an Element From Array** (1707) build a Trie over the 32 bits of each number, most significant bit first, and greedily take the opposite bit at each level. **The greedy is correct because a difference at a higher bit always outweighs all lower bits combined** — state that argument.
- 11 **Palindrome Pairs** (336) combines a Trie with a palindrome check per node. It is the hardest Trie problem in the book.
- 12 **Complexity.** Building is $O(\text{total characters})$. Each query is $O(\text{query length})$. Space is $O(\text{total characters} * \text{branching})$. Compare against a hash set: a hash set answers exact lookup in $O(\text{len})$ but cannot answer ‘starts with’ at all — **that capability is what you are paying space for**, and saying so is the answer to ‘why not a hash set?’.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
139	Word Break	Medium	8
14	Longest Common Prefix	Easy	9
1268	Search Suggestions System	Medium	6
140	Word Break II	Hard	8
472	Concatenated Words	Hard	2
336	Palindrome Pairs	Hard	6
126	Word Ladder II	Hard	5
792	Number of Matching Subsequences	Medium	2
616	Add Bold Tag in String †	Medium	1
720	Longest Word in Dictionary	Medium	1
527	Word Abbreviation †	Hard	2
425	Word Squares †	Hard	1
440	K-th Smallest in Lexicographical Order	Hard	1
758	Bold Words in String †	Medium	1
820	Short Encoding of Words	Medium	1
1804	Implement Trie II (Prefix Tree) †	Medium	1
1065	Index Pairs of a String †	Easy	1
745	Prefix and Suffix Search	Hard	0
1032	Stream of Characters	Hard	0
1698	Number of Distinct Substrings in a String †	Medium	0
1707	Maximum XOR With an Element From Array	Hard	0
1858	Longest Word With All Prefixes †	Medium	0
1938	Maximum Genetic Difference Query	Hard	0
2261	K Divisible Elements Subarrays	Medium	0
2416	Sum of Prefix Scores of Strings	Hard	0
2707	Extra Characters in a String	Medium	0
3042	Count Prefix and Suffix Pairs I	Easy	0

#	PROBLEM	LEVEL	ASKED
3043	Find the Length of the Longest Common Prefix	Medium	0
3045	Count Prefix and Suffix Pairs II	Hard	0
3093	Longest Common Suffix Queries	Hard	0

Eulerian Paths, Bridges, and Advanced Graph Construction

THE KERNEL

Hierholzer's algorithm finds an Eulerian path by DFS-ing edges and appending nodes in postorder, then reversing. It is the answer to 'use every edge exactly once'.

22PROBLEMS
MAPPED**2**

EASY

12

MEDIUM

8

HARD

The Boundary Rules

- 1 **Hierholzer, memorised.** `def dfs(u): while adj[u]: v = adj[u].pop(); dfs(v); route.append(u)`. Then reverse `route`. **The append happens after the recursion** — that postorder placement is the entire algorithm and is the thing candidates get wrong.
- 2 **Existence condition.** An Eulerian *path* exists iff at most two nodes have odd degree (and the graph is connected on its non-isolated nodes). An Eulerian *circuit* exists iff every node has even degree. State this before writing code — it is the conceptual half of the answer.
- 3 **Reconstruct Itinerary** (332) is Hierholzer with two twists: the adjacency lists must be sorted in reverse so that `pop()` yields the lexicographically smallest next hop, and the answer is the reversed postorder. **The reversed-sort trick is the whole difficulty.**

- 4 **Valid Arrangement of Pairs** (2097) is the same algorithm with an explicit start-node computation: find the node with `out_degree - in_degree == 1`, or any node if all degrees balance.
- 5 **Cracking the Safe** (753) is Eulerian on a *de Bruijn graph*: nodes are `(k-1)`-length strings, edges are `k`-length strings. Recognising that the problem is ‘find an Eulerian circuit in a de Bruijn graph’ is the entire solution, and it is a genuinely beautiful reduction.
- 6 **Bridges and articulation points (Tarjan)**. **Critical Connections in a Network** (1192) needs the `disc / low` arrays: an edge `(u, v)` is a bridge iff `low[v] > disc[u]`. The DFS with a `parent` parameter and the back-edge update `low[u] = min(low[u], disc[v])` is the template. **Memorise the `low[v] > disc[u]` test** — it is the bridge criterion.
- 7 **Minimum Number of Days to Disconnect Island** (1568) and **Minimum Number of Vertices to Reach All Nodes** (1557) are connectivity-edge cases; 1568's answer is always 0, 1 or 2, and proving that bound is the interesting part.
- 8 **Strongly connected components** (Kosaraju, Tarjan) rarely appear in interviews but **Critical Connections** uses the same `low`-link idea. Know the name.
- 9 **Minimum Spanning Tree**. Kruskal = sort edges + Union-Find (chapter 55). Prim = grow from a node with a min-heap. **Min Cost to Connect All Points** (1584) is Kruskal; **Connecting Cities With Minimum Cost** (1135) is the same. **1489 Critical and Pseudo-Critical Edges** requires running Kruskal several times with forced/excluded edges — a good test of whether you understand the algorithm rather than just memorised it.
- 10 **Graph construction problems**. **Find if Path Exists in Graph** (1971), **Keys and Rooms** (841), **Find the Town Judge** (997), **Find the Celebrity** (277), **Kill Process** (582) and **Clone Graph** (133) are all ‘build the adjacency structure, then traverse’. **The difficulty is almost always in the construction, not the traversal** — and 277 has an $O(n)$ elimination solution that avoids building anything.
- 11 **Time Needed to Inform All Employees** (1376) is a tree DFS with a parent array instead of child pointers — recognising that a parent array is a tree is a common interview reduction.

12 **Alien Dictionary** (269) and **Sequence Reconstruction** (444) are topological sort with derived edges (chapter 54).

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
126	Word Ladder II	Hard	5
1376	Time Needed to Inform All Employees	Medium	2
582	Kill Process †	Medium	2
997	Find the Town Judge	Easy	4
1971	Find if Path Exists in Graph	Easy	2
841	Keys and Rooms	Medium	1
1557	Minimum Number of Vertices to Reach All Nodes	Medium	2
1466	Reorder Routes to Make All Paths Lead to the City Zero	Medium	1
1135	Connecting Cities With Minimum Cost †	Medium	0
1489	Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree	Hard	0
1568	Minimum Number of Days to Disconnect Island	Hard	0
1584	Min Cost to Connect All Points	Medium	0
2101	Detonate the Maximum Bombs	Medium	0
2316	Count Unreachable Pairs of Nodes in an Undirected Graph	Medium	0
2492	Minimum Score of a Path Between Two Cities	Medium	0
2612	Minimum Reverse Operations	Hard	0

Computational Geometry Basics

THE KERNEL

Three tools: the cross product for orientation, the squared distance for comparisons, and monotone-chain convex hull. Almost every geometry problem is one of those three.

30

 PROBLEMS
MAPPED

8

EASY

14

MEDIUM

8

HARD

The Boundary Rules

- 1 **Cross product orientation.** For points a, b, c ,

$$\text{cross} = (b.x - a.x)(c.y - a.y) - (b.y - a.y)(c.x - a.x)$$
 Positive means counter-clockwise, negative means clockwise, zero means collinear.
Memorise this one formula — it answers ‘is this a straight line’, ‘is this polygon convex’, and ‘do these segments cross’.
- 2 **Check If It Is a Straight Line** (1232) is ‘all cross products zero’. **Valid Boomerang** (1037) is ‘cross product non-zero’. **Convex Polygon** (469) is ‘all cross products the same sign’.
- 3 **Erect the Fence** (587) is the **monotone chain convex hull**: sort points, build the lower hull by popping while the cross product turns clockwise, then the upper hull, then concatenate. The **while pop condition is**

$$\text{cross}(\text{hull}[-2], \text{hull}[-1], p) \leq 0$$
for counter-clockwise hulls, and the $\leq$ versus $<$ decides whether collinear points are kept. LeetCode’s test cases care about collinear points on the boundary, so use $\leq$.

- 4 **Squared distances, never square roots.** `K Closest Points to Origin` (973), `Largest Triangle Area` (812) and every ‘nearest/farthest’ geometry problem compare `dxdx + dydy`. Using `math.hypot` introduces floating-point error and is slower.
- 5 **Max Points on a Line** (149) normalises each slope fraction by `gcd(dy, dx)` so it can be hashed. **The normalisation must handle the sign consistently** (e.g. force `dx > 0`, or `dx == 0` with `dy > 0`) or the same line produces two different keys. That sign rule is the whole problem.
- 6 **Rectangle Area** (223) and `Rectangle Overlap` (836) are inclusion-exclusion on axis-aligned rectangles: the intersection width is `max(0, min(x2) - max(x1))`. **The `max(0, ...)` clamp is what handles non-overlap** and is the standard bug.
- 7 **Rectangle Area II** (850) and `Perfect Rectangle` (391) require coordinate compression or a sweep line with a segment tree — they are the hard end of this chapter.
- 8 **Surface Area of 3D Shapes** (892) and `Projection Area` (883) are counting arguments on a grid, not geometry. Recognise them and skip the machinery.
- 9 **Minimum Area Rectangle** (939) is a hash-set problem: for each pair of points forming a diagonal, test whether the other two corners exist. $O(n^2)$ with a set, no geometry needed.
- 10 **Floating point.** Never test `a == b` for floats. Use `abs(a - b) < 1e-9`. For geometry, prefer integer arithmetic via cross products so you never leave the integers at all — that is the mark of a careful answer.
- 11 **Best Meeting Point** (296) is a median problem: the Manhattan distance is minimised at the median of each coordinate independently. **Separability of the coordinates is the insight** — it applies to any Manhattan-metric problem.
- 12 **Recognition signal.** ‘Points’, ‘lines’, ‘rectangles’, ‘area’, ‘perimeter’, ‘collinear’, ‘convex’. For Manhattan distance, think median or a coordinate transform; for Euclidean, think cross product and squared distance.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
973	K Closest Points to Origin	Medium	5
391	Perfect Rectangle	Hard	2
531	Lonely Pixel I †	Medium	3
447	Number of Boomerangs	Medium	1
836	Rectangle Overlap	Easy	2
1266	Minimum Time Visiting All Points	Easy	1
892	Surface Area of 3D Shapes	Easy	1
1401	Circle and Rectangle Overlapping	Medium	1
533	Lonely Pixel II †	Medium	1
750	Number Of Corner Rectangles †	Medium	1
812	Largest Triangle Area	Easy	1
1037	Valid Boomerang	Easy	1
1828	Queries on Number of Points Inside a Circle	Medium	1
1030	Matrix Cells in Distance Order	Easy	1
478	Generate Random Point in a Circle	Medium	0
497	Random Point in Non-overlapping Rectangles	Medium	0
850	Rectangle Area II	Hard	0
883	Projection Area of 3D Shapes	Easy	0
939	Minimum Area Rectangle	Medium	0
963	Minimum Area Rectangle II	Medium	0
1453	Maximum Number of Darts Inside of a Circular Dartboard	Hard	0
1924	Erect the Fence II †	Hard	0
2613	Beautiful Pairs †	Hard	0
3020	Find the Maximum Number of Elements in Subset	Medium	0

Concurrency (Where It Still Appears)

THE KERNEL

LeetCode's concurrency track is a separate skill from algorithms. In Python interviews it appears rarely, but the primitives are worth knowing by name.

11PROBLEMS
MAPPED**3**

EASY

7

MEDIUM

1

HARD

The Boundary Rules

- 1 **Print in Order** (1114) is a chained-lock problem: `first` releases `second`, `second` releases `third`. Each thread waits on its own lock.
- 2 **Print FooBar Alternately** (1115) is two threads alternating on two locks — the classic hand-off pattern.
- 3 **Print Zero Even Odd** (1116) is a three-way hand-off with a shared counter.
- 4 **Building H2O** (1117) uses a `Barrier` and a `Semaphore` to enforce the 2:1 ratio.
- 5 **Design Bounded Blocking Queue** (1188) is a queue with two semaphores (slots and items) or a `Condition`.
- 6 **The Dining Philosophers** (1226) is the classic deadlock problem; the fix is an ordering rule on lock acquisition (always pick the lower-numbered fork first).

- 7 **Python specifics.** `threading.Lock`, `threading.Semaphore`, `threading.Barrier`, `threading.Condition`, and `queue.Queue` (which is already thread-safe) are the tools. **Python's GIL means threads do not give CPU parallelism** — they give concurrency for I/O. Saying that is often the most valuable sentence in a concurrency discussion.
- 8 **When this chapter matters.** It matters for interviews at companies that run real systems rounds, and it matters for the system-design portion of a loop. It is not part of the standard algorithm loop at most companies — **if you are short on time, read this chapter's boundary rules and move on.**
- 9 **The transferable skill** is not the LeetCode exercises but the habit of asking: what is shared mutable state, what is the lock that protects it, and what is the lock-acquisition order that prevents deadlock.

The Index – What Changes in the Template

#	PROBLEM	LEVEL	ASKED
158	Read N Characters Given read4 II - Call Multiple Times †	Hard	4
157	Read N Characters Given Read4 †	Easy	4
1116	Print Zero Even Odd	Medium	0
1117	Building H2O	Medium	0
1188	Design Bounded Blocking Queue †	Medium	0
1195	Fizz Buzz Multithreaded	Medium	0
1226	The Dining Philosophers	Medium	0
1242	Web Crawler Multithreaded †	Medium	0
1279	Traffic Light Controlled Intersection †	Easy	0

PART XVII

Appendices

The generated complete index: every problem in the dataset, with its difficulty, frequency score and assigned chapter.

The Complete High-Frequency Index

THE KERNEL

All 1151 problems that appear in at least one of the 70 company interview sets, sorted by weighted frequency. The Chapter column is the chapter where the problem is discussed; problems with no chapter number are the long tail of easy warm-ups.

The **Asked** column counts how many of the 70 company sets contain the problem, and **Score** weights tier-one companies triple. A dagger (†) marks LeetCode Premium.

Use this as a checklist rather than a reading list. The question to ask of each row is not *have I solved this* but *can I write its chapter's template from memory without looking*.

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1	Two Sum	Easy	24	1081	20
56	Merge Intervals	Medium	23	1026	18
146	LRU Cache	Medium	23	1002	24
1249	Minimum Remove to Make Valid Parentheses	Medium	7	914	25
200	Number of Islands	Medium	24	912	49
680	Valid Palindrome II	Easy	7	803	5
236	Lowest Common Ancestor of a Binary Tree	Medium	7	703	44
560	Subarray Sum Equals K	Medium	15	639	15
314	Binary Tree Vertical Order Traversal †	Medium	4	609	40
50	Pow(x, n)	Medium	9	608	80

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
215	Kth Largest Element in an Array	Medium	9	594	30
528	Random Pick with Weight	Medium	8	592	87
3	Longest Substring Without Repeating Characters	Medium	18	559	8
973	K Closest Points to Origin	Medium	5	559	30
20	Valid Parentheses	Easy	21	546	25
1650	Lowest Common Ancestor of a Binary Tree III †	Medium	6	515	23
42	Trapping Rain Water	Hard	18	503	5
347	Top K Frequent Elements	Medium	10	503	30
23	Merge k Sorted Lists	Hard	14	489	30
121	Best Time to Buy and Sell Stock	Easy	24	487	61
53	Maximum Subarray	Medium	20	480	15
227	Basic Calculator II	Medium	10	468	25
938	Range Sum of BST	Easy	3	468	38
15	3Sum	Medium	19	461	5
253	Meeting Rooms II †	Medium	9	451	18
33	Search in Rotated Sorted Array	Medium	21	446	10
49	Group Anagrams	Medium	19	423	21
2	Add Two Numbers	Medium	12	407	36
1570	Dot Product of Two Sparse Vectors †	Medium	5	393	88
199	Binary Tree Right Side View	Medium	8	392	40
71	Simplify Path	Medium	4	384	29
339	Nested List Weight Sum †	Medium	3	380	51
4	Median of Two Sorted Arrays	Hard	13	379	10
88	Merge Sorted Array	Easy	9	374	6
394	Decode String	Medium	15	361	25
125	Valid Palindrome	Easy	6	360	5
408	Valid Word Abbreviation †	Easy	2	351	82

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
543	Diameter of Binary Tree	Easy	6	339	41
79	Word Search	Medium	7	328	49
162	Find Peak Element	Medium	8	325	10
921	Minimum Add to Make Parentheses Valid	Medium	4	322	25
987	Vertical Order Traversal of a Binary Tree	Hard	5	319	40
138	Copy List with Random Pointer	Medium	6	309	36
1091	Shortest Path in Binary Matrix	Medium	6	304	52
5	Longest Palindromic Substring	Medium	9	303	60
380	Insert Delete GetRandom O(1)	Medium	8	291	24
212	Word Search II	Hard	9	280	49
139	Word Break	Medium	8	279	65
13	Roman to Integer	Easy	9	277	80
238	Product of Array Except Self	Medium	8	275	15
366	Find Leaves of Binary Tree †	Medium	5	275	41
150	Evaluate Reverse Polish Notation	Medium	6	269	25
17	Letter Combinations of a Phone Number	Medium	8	261	45
210	Course Schedule II	Medium	12	252	54
14	Longest Common Prefix	Easy	9	249	82
239	Sliding Window Maximum	Hard	17	247	28
1268	Search Suggestions System	Medium	6	238	89
124	Binary Tree Maximum Path Sum	Hard	12	236	41
415	Add Strings	Easy	7	236	80
127	Word Ladder	Hard	8	233	52
8	String to Integer (atoi)	Medium	8	224	81
78	Subsets	Medium	10	224	45
103	Binary Tree Zigzag Level Order Traversal	Medium	9	219	40
22	Generate Parentheses	Medium	12	216	25
297	Serialize and Deserialize Binary Tree	Hard	9	216	44

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
54	Spiral Matrix	Medium	7	214	58
523	Continuous Subarray Sum	Medium	7	214	15
173	Binary Search Tree Iterator	Medium	6	213	39
791	Custom Sort String	Medium	2	207	21
387	First Unique Character in a String	Easy	8	201	21
207	Course Schedule	Medium	9	200	54
21	Merge Two Sorted Lists	Easy	9	198	36
1396	Design Underground System	Medium	1	192	88
11	Container With Most Water	Medium	9	191	5
695	Max Area of Island	Medium	8	191	51
76	Minimum Window Substring	Hard	10	190	8
133	Clone Graph	Medium	6	187	56
346	Moving Average from Data Stream †	Easy	4	185	24
234	Palindrome Linked List	Easy	11	182	34
98	Validate Binary Search Tree	Medium	8	181	39
65	Valid Number	Hard	2	179	82
48	Rotate Image	Medium	9	178	58
10	Regular Expression Matching	Hard	9	172	63
62	Unique Paths	Medium	7	170	67
283	Move Zeroes	Easy	10	169	6
273	Integer to English Words	Hard	8	168	81
498	Diagonal Traverse	Medium	3	168	58
140	Word Break II	Hard	8	164	48
269	Alien Dictionary †	Hard	9	162	52
1152	Analyze User Website Visit Pattern †	Medium	2	162	82
39	Combination Sum	Medium	9	160	45
692	Top K Frequent Words	Medium	9	160	21
977	Squares of a Sorted Array	Easy	5	160	5

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
16	3Sum Closest	Medium	7	159	5
41	First Missing Positive	Hard	7	159	23
198	House Robber	Medium	9	159	59
278	First Bad Version	Easy	7	159	10
19	Remove Nth Node From End of List	Medium	8	157	35
26	Remove Duplicates from Sorted Array	Easy	9	157	6
68	Text Justification	Hard	7	153	82
994	Rotting Oranges	Medium	4	153	52
670	Maximum Swap	Medium	4	150	81
1209	Remove All Adjacent Duplicates in String II	Medium	8	150	29
134	Gas Station	Medium	9	149	76
74	Search a 2D Matrix	Medium	10	148	10
249	Group Shifted Strings †	Medium	2	147	22
438	Find All Anagrams in a String	Medium	10	146	7
721	Accounts Merge	Medium	7	143	55
244	Shortest Word Distance II †	Medium	1	142	88
224	Basic Calculator	Hard	9	141	25
155	Min Stack	Medium	8	140	29
217	Contains Duplicate	Easy	8	140	20
399	Evaluate Division	Medium	8	140	56
152	Maximum Product Subarray	Medium	6	137	15
322	Coin Change	Medium	7	137	59
636	Exclusive Time of Functions	Medium	5	137	25
12	Integer to Roman	Medium	7	136	80
1822	Sign of the Product of an Array	Easy	1	135	80
735	Asteroid Collision	Medium	8	133	29
208	Implement Trie (Prefix Tree)	Medium	9	132	89
1047	Remove All Adjacent Duplicates In String	Easy	2	132	29

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
91	Decode Ways	Medium	9	131	60
151	Reverse Words in a String	Medium	6	131	82
102	Binary Tree Level Order Traversal	Medium	9	128	40
202	Happy Number	Easy	6	127	6
348	Design Tic-Tac-Toe †	Medium	5	127	88
7	Reverse Integer	Medium	11	126	80
206	Reverse Linked List	Easy	8	125	34
359	Logger Rate Limiter †	Easy	6	125	24
242	Valid Anagram	Easy	8	124	21
286	Walls and Gates †	Medium	7	124	51
9	Palindrome Number	Easy	8	123	71
25	Reverse Nodes in k-Group	Hard	8	122	34
36	Valid Sudoku	Medium	6	121	49
55	Jump Game	Medium	8	121	76
1275	Find Winner on a Tic Tac Toe Game	Easy	5	121	50
43	Multiply Strings	Medium	6	120	80
1004	Max Consecutive Ones III	Medium	7	120	8
70	Climbing Stairs	Easy	9	119	59
46	Permutations	Medium	9	118	45
287	Find the Duplicate Number	Medium	7	118	6
101	Symmetric Tree	Easy	8	117	38
37	Sudoku Solver	Hard	11	116	49
378	Kth Smallest Element in a Sorted Matrix	Medium	8	116	12
317	Shortest Distance from All Buildings †	Hard	4	115	51
129	Sum Root to Leaf Numbers	Medium	3	114	41
301	Remove Invalid Parentheses	Hard	2	114	25
1146	Snapshot Array	Medium	2	114	10
443	String Compression	Medium	7	111	6

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1539	Kth Missing Positive Number	Easy	5	111	10
658	Find K Closest Elements	Medium	7	110	30
268	Missing Number	Easy	8	108	23
704	Binary Search	Easy	9	107	10
136	Single Number	Easy	9	106	23
588	Design In-Memory File System †	Hard	6	105	24
128	Longest Consecutive Sequence	Medium	9	104	23
739	Daily Temperatures	Medium	5	104	26
767	Reorganize String	Medium	6	104	21
35	Search Insert Position	Easy	9	103	10
605	Can Place Flowers	Easy	5	103	76
716	Max Stack †	Hard	6	103	24
122	Best Time to Buy and Sell Stock II	Medium	8	102	61
724	Find Pivot Index	Easy	9	102	15
430	Flatten a Multilevel Doubly Linked List	Medium	4	100	37
362	Design Hit Counter †	Medium	7	99	24
472	Concatenated Words	Hard	2	99	65
766	Toeplitz Matrix	Easy	2	99	58
29	Divide Two Integers	Medium	7	98	81
45	Jump Game II	Medium	9	98	76
84	Largest Rectangle in Histogram	Hard	8	98	26
160	Intersection of Two Linked Lists	Easy	8	97	35
460	LFU Cache	Hard	9	97	24
364	Nested List Weight Sum II †	Medium	1	96	51
539	Minimum Time Difference	Medium	2	96	81
69	Sqrt(x)	Easy	7	95	10
169	Majority Element	Easy	8	95	85
509	Fibonacci Number	Easy	9	95	60

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
349	Intersection of Two Arrays	Easy	7	94	5
706	Design HashMap	Easy	8	93	24
1891	Cutting Ribbons †	Medium	1	93	12
1567	Maximum Length of Subarray With Positive Product	Medium	2	91	60
1710	Maximum Units on a Truck	Easy	2	91	75
2034	Stock Price Fluctuation	Medium	1	90	24
72	Edit Distance	Medium	7	89	63
24	Swap Nodes in Pairs	Medium	6	88	34
57	Insert Interval	Medium	5	87	18
153	Find Minimum in Rotated Sorted Array	Medium	7	87	10
410	Split Array Largest Sum	Hard	6	87	13
1492	The kth Factor of n	Medium	2	87	80
75	Sort Colors	Medium	9	86	6
1010	Pairs of Songs With Total Durations Divisible by 60	Medium	7	85	20
51	N-Queens	Hard	8	84	49
44	Wildcard Matching	Hard	6	83	63
981	Time Based Key-Value Store	Medium	6	83	24
99	Recover Binary Search Tree	Medium	6	82	39
219	Contains Duplicate II	Easy	7	82	20
235	Lowest Common Ancestor of a Binary Search Tree	Medium	6	81	42
18	4Sum	Medium	8	80	5
647	Palindromic Substrings	Medium	3	80	68
66	Plus One	Easy	6	79	80
104	Maximum Depth of Binary Tree	Easy	8	79	38
953	Verifying an Alien Dictionary	Easy	4	79	54
73	Set Matrix Zeroes	Medium	5	78	58

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
398	Random Pick Index	Medium	3	78	87
545	Boundary of Binary Tree †	Medium	4	78	44
642	Design Search Autocomplete System †	Hard	6	78	88
64	Minimum Path Sum	Medium	8	77	67
230	Kth Smallest Element in a BST	Medium	5	77	39
209	Minimum Size Subarray Sum	Medium	7	76	8
289	Game of Life	Medium	8	76	58
332	Reconstruct Itinerary	Hard	6	76	90
416	Partition Equal Subset Sum	Medium	6	76	48
540	Single Element in a Sorted Array	Medium	6	76	10
1094	Car Pooling	Medium	7	76	17
312	Burst Balloons	Hard	6	75	68
167	Two Sum II - Input Array Is Sorted	Medium	6	74	5
189	Rotate Array	Medium	6	74	81
1438	Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit	Medium	3	73	28
341	Flatten Nested List Iterator	Medium	8	71	88
32	Longest Valid Parentheses	Hard	7	70	25
63	Unique Paths II	Medium	6	70	67
92	Reverse Linked List II	Medium	6	70	34
114	Flatten Binary Tree to Linked List	Medium	4	70	44
240	Search a 2D Matrix II	Medium	6	70	10
109	Convert Sorted List to Binary Search Tree	Medium	4	69	42
221	Maximal Square	Medium	7	69	27
282	Expression Add Operators	Hard	1	69	45
2130	Maximum Twin Sum of a Linked List	Medium	1	69	34
67	Add Binary	Easy	6	68	80
143	Reorder List	Medium	7	68	34
494	Target Sum	Medium	5	67	48

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
935	Knight Dialer	Medium	5	67	60
141	Linked List Cycle	Easy	7	66	35
419	Battleships in a Board	Medium	5	66	58
652	Find Duplicate Subtrees	Medium	3	66	44
1304	Find N Unique Integers Sum up to Zero	Easy	1	66	80
450	Delete Node in a BST	Medium	5	65	39
285	Inorder Successor in BST †	Medium	4	64	42
336	Palindrome Pairs	Hard	6	64	89
370	Range Addition †	Medium	2	64	17
402	Remove K Digits	Medium	6	64	26
1207	Unique Number of Occurrences	Easy	6	64	21
1423	Maximum Points You Can Obtain from Cards	Medium	3	64	7
191	Number of 1 Bits	Easy	6	63	78
270	Closest Binary Search Tree Value †	Easy	1	63	42
1151	Minimum Swaps to Group All 1's Together †	Medium	1	63	7
344	Reverse String	Easy	6	62	5
567	Permutation in String	Medium	7	62	7
678	Valid Parenthesis String	Medium	5	62	25
1507	Reformat Date	Easy	3	62	82
179	Largest Number	Medium	9	61	75
246	Strobogrammatic Number †	Easy	2	61	7
1344	Angle Between Hands of a Clock	Medium	4	61	80
226	Invert Binary Tree	Easy	6	60	38
277	Find the Celebrity †	Medium	6	60	90
1656	Design an Ordered Stream	Easy	1	60	24
112	Path Sum	Easy	5	59	38
126	Word Ladder II	Hard	5	59	52
272	Closest Binary Search Tree Value II †	Hard	2	58	39

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
100	Same Tree	Easy	6	57	38
118	Pascal's Triangle	Easy	7	57	58
304	Range Sum Query 2D - Immutable	Medium	5	57	15
622	Design Circular Queue	Medium	6	57	37
876	Middle of the Linked List	Easy	7	57	35
1386	Cinema Seat Allocation	Medium	2	57	76
1428	Leftmost Column with at Least a One †	Medium	3	56	10
1629	Slowest Key	Easy	2	55	82
515	Find Largest Value in Each Tree Row	Medium	1	54	40
662	Maximum Width of Binary Tree	Medium	5	54	40
844	Backspace String Compare	Easy	7	54	5
852	Peak Index in a Mountain Array	Medium	4	54	10
1046	Last Stone Weight	Easy	4	54	30
424	Longest Repeating Character Replacement	Medium	4	53	8
516	Longest Palindromic Subsequence	Medium	6	53	68
148	Sort List	Medium	5	51	35
895	Maximum Frequency Stack	Hard	7	51	32
1376	Time Needed to Inform All Employees	Medium	2	51	90
311	Sparse Matrix Multiplication †	Medium	4	50	58
529	Minesweeper	Medium	5	50	51
881	Boats to Save People	Medium	7	50	5
1366	Rank Teams by Votes	Medium	4	50	82
1472	Design Browser History	Medium	4	50	24
93	Restore IP Addresses	Medium	6	49	45
105	Construct Binary Tree from Preorder and In-order Traversal	Medium	6	49	42
232	Implement Queue using Stacks	Easy	5	49	29
323	Number of Connected Components in an Undirected Graph †	Medium	6	49	51

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
490	The Maze †	Medium	7	49	51
792	Number of Matching Subsequences	Medium	2	49	89
1110	Delete Nodes And Return Forest	Medium	4	49	38
222	Count Complete Tree Nodes	Medium	3	48	38
284	Peeking Iterator	Medium	4	48	88
328	Odd Even Linked List	Medium	5	48	34
353	Design Snake Game †	Medium	6	48	88
448	Find All Numbers Disappeared in an Array	Easy	4	48	23
572	Subtree of Another Tree	Easy	4	48	38
1029	Two City Scheduling	Medium	2	48	75
1051	Height Checker	Easy	3	48	33
1347	Minimum Number of Steps to Make Two Strings Anagram	Medium	4	48	21
2007	Find Original Array From Doubled Array	Medium	2	48	75
243	Shortest Word Distance †	Easy	5	47	82
432	All O`one Data Structure	Hard	3	47	24
83	Remove Duplicates from Sorted List	Easy	5	46	34
113	Path Sum II	Medium	5	46	41
315	Count of Smaller Numbers After Self	Hard	4	46	85
417	Pacific Atlantic Water Flow	Medium	4	46	49
542	01 Matrix	Medium	4	46	52
694	Number of Distinct Islands †	Medium	4	46	51
274	H-Index	Medium	4	45	33
403	Frog Jump	Hard	5	45	56
463	Island Perimeter	Easy	3	45	51
703	Kth Largest Element in a Stream	Easy	5	45	30
38	Count and Say	Medium	6	44	81
350	Intersection of Two Arrays II	Easy	5	44	23
480	Sliding Window Median	Hard	3	44	31

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
27	Remove Element	Easy	5	43	6
97	Interleaving String	Medium	5	43	63
149	Max Points on a Line	Hard	4	43	20
442	Find All Duplicates in an Array	Medium	5	43	23
445	Add Two Numbers II	Medium	4	43	34
740	Delete and Earn	Medium	3	43	60
77	Combinations	Medium	4	42	45
205	Isomorphic Strings	Easy	3	42	21
299	Bulls and Cows	Medium	2	42	81
383	Ransom Note	Easy	4	42	21
451	Sort Characters By Frequency	Medium	5	42	21
525	Contiguous Array	Medium	4	42	15
552	Student Attendance Record II	Hard	1	42	60
896	Monotonic Array	Easy	2	42	18
40	Combination Sum II	Medium	5	40	45
158	Read N Characters Given read4 II - Call Multiple Times †	Hard	4	40	92
161	One Edit Distance †	Medium	3	40	63
204	Count Primes	Medium	5	40	79
231	Power of Two	Easy	4	40	78
557	Reverse Words in a String III	Easy	5	40	82
110	Balanced Binary Tree	Easy	5	39	41
131	Palindrome Partitioning	Medium	5	39	45
213	House Robber II	Medium	2	39	60
325	Maximum Size Subarray Sum Equals k †	Medium	2	39	15
337	House Robber III	Medium	3	39	60
435	Non-overlapping Intervals	Medium	3	39	18
437	Path Sum III	Medium	4	39	41
616	Add Bold Tag in String †	Medium	1	39	18

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
856	Score of Parentheses	Medium	4	39	25
1381	Design a Stack With Increment Operation	Medium	1	39	29
94	Binary Tree Inorder Traversal	Easy	4	37	38
130	Surrounded Regions	Medium	4	37	49
188	Best Time to Buy and Sell Stock IV	Hard	5	37	61
261	Graph Valid Tree †	Medium	4	37	51
454	4Sum II	Medium	4	37	20
617	Merge Two Binary Trees	Easy	4	37	38
809	Expressive Words	Medium	3	37	82
1143	Longest Common Subsequence	Medium	3	37	63
1448	Count Good Nodes in Binary Tree	Medium	2	37	38
59	Spiral Matrix II	Medium	3	36	58
190	Reverse Bits	Easy	5	36	78
257	Binary Tree Paths	Easy	5	36	38
266	Palindrome Permutation †	Easy	1	36	71
1352	Product of the Last K Numbers	Medium	3	36	24
1603	Design Parking System	Easy	1	36	88
81	Search in Rotated Sorted Array II	Medium	4	35	10
373	Find K Pairs with Smallest Sums	Medium	4	35	32
465	Optimal Account Balancing †	Hard	3	34	73
532	K-diff Pairs in an Array	Medium	7	34	5
1487	Making File Names Unique	Medium	2	34	82
30	Substring with Concatenation of All Words	Hard	5	33	7
123	Best Time to Buy and Sell Stock III	Hard	3	33	61
165	Compare Version Numbers	Medium	2	33	82
377	Combination Sum IV	Medium	4	33	48
389	Find the Difference	Easy	3	33	78
535	Encode and Decode TinyURL	Medium	3	33	81

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
637	Average of Levels in Binary Tree	Easy	3	33	40
743	Network Delay Time	Medium	2	33	56
849	Maximize Distance to Closest Person	Medium	3	33	18
958	Check Completeness of a Binary Tree	Medium	1	33	40
1008	Construct Binary Search Tree from Preorder Traversal	Medium	4	33	42
1305	All Elements in Two Binary Search Trees	Medium	4	33	38
1345	Jump Game IV	Hard	3	33	52
1382	Balance a Binary Search Tree	Medium	1	33	38
1481	Least Number of Unique Integers after K Removals	Medium	4	33	21
698	Partition to K Equal Sum Subsets	Medium	3	32	48
859	Buddy Strings	Easy	2	32	82
108	Convert Sorted Array to Binary Search Tree	Easy	5	31	42
157	Read N Characters Given Read4 †	Easy	4	31	92
203	Remove Linked List Elements	Easy	4	31	37
723	Candy Crush †	Medium	3	31	58
1920	Build Array from Permutation	Easy	4	31	58
82	Remove Duplicates from Sorted List II	Medium	3	30	34
171	Excel Sheet Column Number	Easy	3	30	80
174	Dungeon Game	Hard	2	30	67
582	Kill Process †	Medium	2	30	90
697	Degree of an Array	Easy	5	30	20
796	Rotate String	Easy	5	30	81
1277	Count Square Submatrices with All Ones	Medium	2	30	16
1642	Furthest Building You Can Reach	Medium	3	30	75
1996	The Number of Weak Characters in the Game	Medium	3	30	26
729	My Calendar I	Medium	3	29	18

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
746	Min Cost Climbing Stairs	Easy	3	29	60
1395	Count Number of Teams	Medium	3	29	81
220	Contains Duplicate III	Hard	4	28	20
316	Remove Duplicate Letters	Medium	4	28	26
468	Validate IP Address	Medium	3	28	82
621	Task Scheduler	Medium	4	28	32
669	Trim a Binary Search Tree	Medium	4	28	42
901	Online Stock Span	Medium	3	28	26
979	Distribute Coins in Binary Tree	Medium	4	28	41
6	Zigzag Conversion	Medium	3	27	81
85	Maximal Rectangle	Hard	3	27	26
142	Linked List Cycle II	Medium	2	27	35
256	Paint House †	Medium	3	27	65
307	Range Sum Query - Mutable	Medium	2	27	88
354	Russian Doll Envelopes	Hard	2	27	62
388	Longest Absolute File Path	Medium	2	27	29
420	Strong Password Checker	Hard	2	27	82
488	Zuma Game	Hard	3	27	60
638	Shopping Offers	Medium	4	27	48
918	Maximum Sum Circular Subarray	Medium	2	27	61
1636	Sort Array by Increasing Frequency	Easy	3	27	30
90	Subsets II	Medium	4	25	46
229	Majority Element II	Medium	3	25	85
333	Largest BST Subtree †	Medium	2	25	41
358	Rearrange String k Distance Apart †	Hard	4	25	21
392	Is Subsequence	Easy	3	25	5
867	Transpose Matrix	Easy	4	25	58
135	Candy	Hard	1	24	76

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
163	Missing Ranges †	Easy	1	24	18
166	Fraction to Recurring Decimal	Medium	3	24	81
168	Excel Sheet Column Title	Easy	3	24	80
214	Shortest Palindrome	Hard	3	24	71
233	Number of Digit One	Hard	2	24	74
247	Strobogrammatic Number II †	Medium	3	24	25
279	Perfect Squares	Medium	4	24	60
390	Elimination Game	Medium	3	24	80
412	Fizz Buzz	Easy	3	24	81
418	Sentence Screen Fitting †	Medium	1	24	71
538	Convert BST to Greater Tree	Medium	3	24	39
581	Shortest Unsorted Continuous Subarray	Medium	2	24	26
690	Employee Importance	Medium	2	24	52
946	Validate Stack Sequences	Medium	3	24	29
974	Subarray Sums Divisible by K	Medium	3	24	15
1026	Maximum Difference Between Node and Ancestor	Medium	3	24	41
1086	High Five †	Easy	1	24	32
1541	Minimum Insertions to Balance a Parentheses String	Medium	2	24	25
1644	Lowest Common Ancestor of a Binary Tree II †	Medium	1	24	44
1721	Swapping Nodes in a Linked List	Medium	3	24	34
1861	Rotating the Box	Medium	3	24	58
381	Insert Delete GetRandom O(1) - Duplicates allowed	Hard	2	23	24
860	Lemonade Change	Easy	3	23	76
218	The Skyline Problem	Hard	3	22	19
241	Different Ways to Add Parentheses	Medium	3	22	85
671	Second Minimum Node In a Binary Tree	Easy	1	22	38

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
713	Subarray Product Less Than K	Medium	3	22	8
997	Find the Town Judge	Easy	4	22	90
1512	Number of Good Pairs	Easy	3	22	9
47	Permutations II	Medium	2	21	47
61	Rotate List	Medium	2	21	34
120	Triangle	Medium	2	21	67
298	Binary Tree Longest Consecutive Sequence †	Medium	3	21	41
302	Smallest Rectangle Enclosing Black Pixels †	Hard	1	21	51
338	Counting Bits	Easy	3	21	78
536	Construct Binary Tree from String †	Medium	1	21	44
640	Solve the Equation	Medium	2	21	81
643	Maximum Average Subarray I	Easy	1	21	7
650	2 Keys Keyboard	Medium	2	21	74
993	Cousins in Binary Tree	Easy	3	21	38
1019	Next Greater Node In Linked List	Medium	2	21	26
1287	Element Appearing More Than 25% In Sorted Array	Easy	2	21	10
1614	Maximum Nesting Depth of the Parentheses	Easy	2	21	25
1628	Design an Expression Tree With Evaluate Function †	Medium	1	21	80
1774	Closest Dessert Cost	Medium	3	21	64
1910	Remove All Occurrences of a Substring	Medium	3	21	21
1971	Find if Path Exists in Graph	Easy	2	21	90
290	Word Pattern	Easy	3	20	21
296	Best Meeting Point †	Hard	3	20	91
360	Sort Transformed Array †	Medium	1	20	80
453	Minimum Moves to Equal Array Elements	Medium	3	20	80
744	Find Smallest Letter Greater Than Target	Easy	3	20	10

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1221	Split a String in Balanced Strings	Easy	4	20	82
1339	Maximum Product of Splitted Binary Tree	Medium	2	20	41
1817	Finding the Users Active Minutes	Medium	1	20	20
355	Design Twitter	Medium	3	19	24
449	Serialize and Deserialize BST	Medium	3	19	42
771	Jewels and Stones	Easy	3	19	82
111	Minimum Depth of Binary Tree	Easy	2	18	38
115	Distinct Subsequences	Hard	2	18	63
156	Binary Tree Upside Down †	Medium	1	18	44
159	Longest Substring with At Most Two Distinct Characters †	Medium	2	18	8
251	Flatten 2D Vector †	Medium	1	18	88
252	Meeting Rooms †	Easy	2	18	18
313	Super Ugly Number	Medium	3	18	32
391	Perfect Rectangle	Hard	2	18	19
396	Rotate Function	Medium	3	18	81
414	Third Maximum Number	Easy	3	18	18
505	The Maze II †	Medium	3	18	56
531	Lonely Pixel I †	Medium	3	18	91
593	Valid Square	Medium	2	18	80
609	Find Duplicate File in System	Medium	2	18	82
720	Longest Word in Dictionary	Medium	1	18	89
763	Partition Labels	Medium	2	18	75
788	Rotated Digits	Medium	2	18	74
814	Binary Tree Pruning	Medium	2	18	41
841	Keys and Rooms	Medium	1	18	90
1043	Partition Array for Maximum Sum	Medium	2	18	65
1087	Brace Expansion †	Medium	2	18	82
1099	Two Sum Less Than K †	Easy	2	18	5

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1267	Count Servers that Communicate	Medium	1	18	51
1372	Longest ZigZag Path in a Binary Tree	Medium	1	18	41
1429	First Unique Number †	Medium	2	18	24
1588	Sum of All Odd Length Subarrays	Easy	3	18	15
2016	Maximum Difference Between Increasing Elements	Easy	2	18	18
2038	Remove Colored Pieces if Both Neighbors are the Same Color	Medium	1	18	72
2055	Plates Between Candles	Medium	2	18	82
2089	Find Target Indices After Sorting Array	Easy	2	18	10
2103	Rings and Rods	Easy	1	18	82
2125	Number of Laser Beams in a Bank	Medium	1	18	58
319	Bulb Switcher	Medium	2	17	80
446	Arithmetic Slices II - Subsequence	Hard	4	17	62
664	Strange Printer	Hard	2	17	68
1518	Water Bottles	Easy	3	17	80
172	Factorial Trailing Zeroes	Medium	2	16	80
187	Repeated DNA Sequences	Medium	2	16	7
305	Number of Islands II †	Hard	2	16	51
452	Minimum Number of Arrows to Burst Balloons	Medium	3	16	18
611	Valid Triangle Number	Medium	3	16	5
701	Insert into a Binary Search Tree	Medium	3	16	39
725	Split Linked List in Parts	Medium	3	16	34
728	Self Dividing Numbers	Easy	3	16	80
1022	Sum of Root To Leaf Binary Numbers	Easy	3	16	38
1523	Count Odd Numbers in an Interval Range	Easy	3	16	80
1574	Shortest Subarray to be Removed to Make Array Sorted	Medium	2	16	26

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1604	Alert Using Same Key-Card Three or More Times in a One Hour Period	Medium	1	16	82
1854	Maximum Population Year	Easy	3	16	15
1929	Concatenation of Array	Easy	2	16	58
58	Length of Last Word	Easy	2	15	82
137	Single Number II	Medium	1	15	23
237	Delete Node in a Linked List	Medium	2	15	37
250	Count Unival Subtrees †	Medium	2	15	41
281	Zigzag Iterator †	Medium	3	15	88
334	Increasing Triplet Subsequence	Medium	2	15	62
335	Self Crossing	Hard	2	15	81
367	Valid Perfect Square	Easy	2	15	10
382	Linked List Random Node	Medium	3	15	87
395	Longest Substring with At Least K Repeating Characters	Medium	2	15	8
447	Number of Boomerangs	Medium	1	15	91
462	Minimum Moves to Equal Array Elements II	Medium	1	15	81
487	Max Consecutive Ones II †	Medium	2	15	8
551	Student Attendance Record I	Easy	1	15	82
562	Longest Line of Consecutive One in Matrix †	Medium	1	15	67
675	Cut Off Trees for Golf Event	Hard	2	15	52
676	Implement Magic Dictionary	Medium	2	15	88
705	Design HashSet	Easy	2	15	24
714	Best Time to Buy and Sell Stock with Transaction Fee	Medium	2	15	61
754	Reach a Number	Medium	1	15	80
807	Max Increase to Keep City Skyline	Medium	2	15	58
824	Goat Latin	Easy	1	15	82
884	Uncommon Words from Two Sentences	Easy	2	15	82

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
912	Sort an Array	Medium	2	15	33
980	Unique Paths III	Hard	2	15	49
1291	Sequential Digits	Medium	2	15	81
1313	Decompress Run-Length Encoded List	Easy	2	15	18
1331	Rank Transform of an Array	Easy	2	15	20
1360	Number of Days Between Two Dates	Easy	1	15	81
1408	String Matching in an Array	Easy	2	15	82
1417	Reformat The String	Easy	2	15	82
1446	Consecutive Characters	Easy	2	15	82
1470	Shuffle the Array	Easy	2	15	18
1514	Path with Maximum Probability	Medium	1	15	56
1658	Minimum Operations to Reduce X to Zero	Medium	2	15	8
2076	Process Restricted Friend Requests	Hard	1	15	57
374	Guess Number Higher or Lower	Easy	2	14	10
628	Maximum Product of Three Numbers	Easy	1	14	81
756	Pyramid Transition Matrix	Medium	3	14	45
757	Set Intersection Size At Least Two	Hard	1	14	75
781	Rabbits in Forest	Medium	3	14	76
799	Champagne Tower	Medium	3	14	81
1009	Complement of Base 10 Integer	Easy	2	14	78
1257	Smallest Common Region †	Medium	1	14	44
1389	Create Target Array in the Given Order	Easy	3	14	58
1400	Construct K Palindrome Strings	Medium	1	14	71
1700	Number of Students Unable to Eat Lunch	Easy	1	14	29
1823	Find the Winner of the Circular Game	Medium	2	14	24
265	Paint House II †	Hard	3	13	65
271	Encode and Decode Strings †	Medium	2	13	82
1353	Maximum Number of Events That Can Be Attended	Medium	2	13	75

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1641	Count Sorted Vowel Strings	Medium	2	13	80
95	Unique Binary Search Trees II	Medium	1	12	42
96	Unique Binary Search Trees	Medium	1	12	42
164	Maximum Gap	Medium	2	12	33
186	Reverse Words in a String II †	Medium	1	12	82
248	Strobogrammatic Number III †	Hard	1	12	25
255	Verify Preorder Sequence in Binary Search Tree †	Medium	2	12	44
288	Unique Word Abbreviation †	Medium	2	12	88
292	Nim Game	Easy	2	12	72
340	Longest Substring with At Most K Distinct Characters †	Medium	2	12	8
351	Android Unlock Patterns †	Medium	2	12	73
405	Convert a Number to Hexadecimal	Easy	2	12	78
407	Trapping Rain Water II	Hard	1	12	32
413	Arithmetic Slices	Medium	2	12	7
456	132 Pattern	Medium	1	12	26
459	Repeated Substring Pattern	Easy	2	12	82
475	Heaters	Medium	2	12	81
476	Number Complement	Easy	2	12	78
485	Max Consecutive Ones	Easy	2	12	18
520	Detect Capital	Easy	2	12	82
524	Longest Word in Dictionary through Deleting	Medium	1	12	82
549	Binary Tree Longest Consecutive Sequence II †	Medium	1	12	41
566	Reshape the Matrix	Easy	2	12	58
576	Out of Boundary Paths	Medium	2	12	67
633	Sum of Square Numbers	Medium	2	12	80
655	Print Binary Tree	Medium	2	12	38

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
659	Split Array into Consecutive Subsequences	Medium	2	12	81
673	Number of Longest Increasing Sub-sequence	Medium	2	12	62
674	Longest Continuous Increasing Sub-sequence	Easy	1	12	18
684	Redundant Connection	Medium	2	12	55
687	Longest Univalve Path	Medium	2	12	41
755	Pour Water †	Medium	1	12	58
783	Minimum Distance Between BST Nodes	Easy	2	12	39
821	Shortest Distance to a Character	Easy	2	12	82
904	Fruit Into Baskets	Medium	1	12	8
929	Unique Email Addresses	Easy	1	12	82
1002	Find Common Characters	Easy	1	12	21
1024	Video Stitching	Medium	2	12	76
1025	Divisor Game	Easy	2	12	60
1038	Binary Search Tree to Greater Sum Tree	Medium	2	12	39
1066	Campus Bikes II †	Medium	2	12	73
1079	Letter Tile Possibilities	Medium	2	12	45
1100	Find K-Length Substrings With No Repeated Characters †	Medium	1	12	7
1108	Defanging an IP Address	Easy	2	12	82
1160	Find Words That Can Be Formed by Characters	Easy	2	12	21
1200	Minimum Absolute Difference	Easy	2	12	18
1282	Group the People Given the Group Size They Belong To	Medium	2	12	76
1296	Divide Array in Sets of K Consecutive Numbers	Medium	1	12	62
1332	Remove Palindromic Subsequences	Easy	1	12	71
1342	Number of Steps to Reduce a Number to Zero	Easy	2	12	78

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1348	Tweet Counts Per Frequency	Medium	1	12	32
1358	Number of Substrings Containing All Three Characters	Medium	3	12	9
1367	Linked List in Binary Tree	Medium	2	12	41
1377	Frog Position After T Seconds	Hard	2	12	81
1451	Rearrange Words in a Sentence	Medium	3	12	82
1464	Maximum Product of Two Elements in an Array	Easy	3	12	32
1524	Number of Sub-arrays With Odd Sum	Medium	2	12	80
1526	Minimum Number of Increments on Subarrays to Form a Target Array	Hard	1	12	17
1557	Minimum Number of Vertices to Reach All Nodes	Medium	2	12	54
1560	Most Visited Sector in a Circular Track	Easy	1	12	58
1572	Matrix Diagonal Sum	Easy	2	12	58
1592	Rearrange Spaces Between Words	Easy	1	12	82
1711	Count Good Meals	Medium	3	12	20
1763	Longest Nice Substring	Easy	1	12	85
1765	Map of Highest Peak	Medium	2	12	51
1773	Count Items Matching a Rule	Easy	2	12	82
1832	Check if the Sentence Is Pangram	Easy	2	12	82
1985	Find the Kth Largest Integer in the Array	Medium	1	12	30
2018	Check if Word Can Be Placed In Crossword	Medium	1	12	81
2110	Number of Smooth Descent Periods of a Stock	Medium	1	12	60
527	Word Abbreviation †	Hard	2	11	89
1289	Minimum Falling Path Sum II	Hard	2	11	67
60	Permutation Sequence	Hard	2	10	47
119	Pascal's Triangle II	Easy	2	10	58
144	Binary Tree Preorder Traversal	Easy	2	10	38

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
245	Shortest Word Distance III †	Medium	2	10	82
254	Factor Combinations †	Medium	1	10	32
291	Word Pattern II †	Medium	2	10	45
310	Minimum Height Trees	Medium	2	10	52
379	Design Phone Directory †	Medium	2	10	24
393	UTF-8 Validation	Medium	2	10	78
421	Maximum XOR of Two Numbers in an Array	Medium	2	10	78
477	Total Hamming Distance	Medium	2	10	78
486	Predict the Winner	Medium	2	10	72
502	IPO	Hard	1	10	31
591	Tag Validator	Hard	2	10	25
592	Fraction Addition and Subtraction	Medium	1	10	81
599	Minimum Index Sum of Two Lists	Easy	2	10	82
635	Design Log Storage System †	Medium	1	10	24
646	Maximum Length of Pair Chain	Medium	2	10	62
653	Two Sum IV - Input is a BST	Easy	2	10	20
836	Rectangle Overlap	Easy	2	10	19
888	Fair Candy Swap	Easy	2	10	10
923	3Sum With Multiplicity	Medium	2	10	5
1071	Greatest Common Divisor of Strings	Easy	2	10	80
1190	Reverse Substrings Between Each Pair of Parentheses	Medium	2	10	25
1281	Subtract the Product and Sum of Digits of an Integer	Easy	1	10	80
1283	Find the Smallest Divisor Given a Threshold	Medium	2	10	12
1290	Convert Binary Number in a Linked List to Integer	Easy	2	10	34
1365	How Many Numbers Are Smaller Than the Current Number	Easy	2	10	33
1749	Maximum Absolute Sum of Any Subarray	Medium	2	10	61

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1836	Remove Duplicates From an Unsorted Linked List †	Medium	2	10	81
1859	Sorting the Sentence	Easy	2	10	82
216	Combination Sum III	Medium	1	9	45
225	Implement Stack using Queues	Easy	1	9	29
263	Ugly Number	Easy	1	9	80
267	Palindrome Permutation II †	Medium	1	9	45
308	Range Sum Query 2D - Mutable †	Medium	1	9	16
321	Create Maximum Number	Hard	2	9	26
343	Integer Break	Medium	1	9	60
400	Nth Digit	Medium	1	9	80
406	Queue Reconstruction by Height	Medium	1	9	81
422	Valid Word Square †	Easy	2	9	58
425	Word Squares †	Hard	1	9	89
429	N-ary Tree Level Order Traversal	Medium	1	9	38
466	Count The Repetitions	Hard	1	9	82
493	Reverse Pairs	Hard	1	9	85
504	Base 7	Easy	1	9	81
514	Freedom Trail	Hard	2	9	82
530	Minimum Absolute Difference in BST	Easy	1	9	39
548	Split Array with Equal Sum †	Hard	1	9	15
565	Array Nesting	Medium	2	9	51
589	N-ary Tree Preorder Traversal	Easy	1	9	38
631	Design Excel Sum Formula †	Hard	1	9	24
665	Non-decreasing Array	Medium	1	9	18
666	Path Sum IV †	Medium	2	9	43
688	Knight Probability in Chessboard	Medium	1	9	67
689	Maximum Sum of 3 Non-Overlapping Subarrays	Hard	1	9	81

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
691	Stickers to Spell Word	Hard	1	9	73
719	Find K-th Smallest Pair Distance	Hard	1	9	14
734	Sentence Similarity †	Easy	2	9	82
738	Monotone Increasing Digits	Medium	2	9	76
742	Closest Leaf in a Binary Tree †	Medium	1	9	38
776	Split BST †	Medium	1	9	42
830	Positions of Large Groups	Easy	1	9	82
832	Flipping an Image	Easy	1	9	58
840	Magic Squares In Grid	Medium	1	9	58
900	RLE Iterator	Medium	1	9	88
954	Array of Doubled Pairs	Medium	1	9	75
965	Univalued Binary Tree	Easy	1	9	38
1119	Remove Vowels from a String †	Easy	1	9	82
1137	N-th Tribonacci Number	Easy	1	9	60
1161	Maximum Level Sum of a Binary Tree	Medium	1	9	38
1213	Intersection of Three Sorted Arrays †	Easy	1	9	20
1214	Two Sum BSTs †	Medium	1	9	42
1258	Synonymous Sentences †	Medium	1	9	45
1266	Minimum Time Visiting All Points	Easy	1	9	91
1373	Maximum Sum BST in Binary Tree	Hard	1	9	42
1431	Kids With the Greatest Number of Candies	Easy	1	9	18
1466	Reorder Routes to Make All Paths Lead to the City Zero	Medium	1	9	52
1583	Count Unhappy Friends	Medium	1	9	58
1662	Check If Two String Arrays are Equivalent	Easy	2	9	82
1670	Design Front Middle Back Queue	Medium	1	9	24
1680	Concatenation of Consecutive Binary Numbers	Medium	1	9	78

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1718	Construct the Lexicographically Largest Valid Sequence	Medium	2	9	45
1732	Find the Highest Altitude	Easy	1	9	15
1877	Minimize Maximum Pair Sum in Array	Medium	1	9	81
1974	Minimum Time to Type Word Using Special Typewriter	Easy	1	9	75
1991	Find the Middle Index in Array	Easy	1	9	15
2033	Minimum Operations to Make a Uni-Value Grid	Medium	1	9	80
2090	K Radius Subarray Averages	Medium	1	9	7
361	Bomb Enemy †	Medium	1	8	67
365	Water and Jug Problem	Medium	1	8	80
436	Find Right Interval	Medium	2	8	18
811	Subdomain Visit Count	Medium	2	8	21
1150	Check If a Number Is Majority Element in a Sorted Array †	Easy	2	8	12
1232	Check If It Is a Straight Line	Easy	2	8	91
1705	Maximum Number of Eaten Apples	Medium	2	8	32
2050	Parallel Courses III	Hard	1	8	54
484	Find Permutation †	Medium	2	7	47
683	K Empty Slots †	Hard	2	7	24
1128	Number of Equivalent Domino Pairs	Easy	2	7	20
1544	Make The String Great	Easy	2	7	29
1601	Maximum Number of Achievable Transfer Requests	Hard	2	7	50
1844	Replace All Digits with Characters	Easy	2	7	82
52	N-Queens II	Hard	1	6	49
80	Remove Duplicates from Sorted Array II	Medium	1	6	6
89	Gray Code	Medium	1	6	45
106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	1	6	42

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
107	Binary Tree Level Order Traversal II	Medium	1	6	40
132	Palindrome Partitioning II	Hard	1	6	71
145	Binary Tree Postorder Traversal	Easy	1	6	38
147	Insertion Sort List	Medium	1	6	36
154	Find Minimum in Rotated Sorted Array II	Hard	1	6	10
223	Rectangle Area	Medium	1	6	19
258	Add Digits	Easy	1	6	80
260	Single Number III	Medium	1	6	23
264	Ugly Number II	Medium	1	6	32
276	Paint Fence †	Medium	1	6	81
280	Wiggle Sort †	Medium	1	6	75
303	Range Sum Query - Immutable	Easy	1	6	15
306	Additive Number	Medium	1	6	45
318	Maximum Product of Word Lengths	Medium	1	6	78
326	Power of Three	Easy	1	6	80
330	Patching Array	Hard	1	6	76
345	Reverse Vowels of a String	Easy	1	6	5
352	Data Stream as Disjoint Intervals	Hard	1	6	18
357	Count Numbers with Unique Digits	Medium	1	6	45
368	Largest Divisible Subset	Medium	1	6	62
371	Sum of Two Integers	Medium	1	6	78
375	Guess Number Higher or Lower II	Medium	1	6	68
376	Wiggle Subsequence	Medium	1	6	76
397	Integer Replacement	Medium	1	6	60
409	Longest Palindrome	Easy	1	6	71
434	Number of Segments in a String	Easy	1	6	82
440	K-th Smallest in Lexicographical Order	Hard	1	6	89
444	Sequence Reconstruction †	Medium	1	6	54

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
455	Assign Cookies	Easy	1	6	75
461	Hamming Distance	Easy	1	6	78
464	Can I Win	Medium	1	6	48
467	Unique Substrings in Wraparound String	Medium	1	6	71
471	Encode String with Shortest Length †	Hard	1	6	71
473	Matchsticks to Square	Medium	1	6	48
474	Ones and Zeroes	Medium	1	6	64
483	Smallest Good Base	Hard	1	6	80
492	Construct the Rectangle	Easy	1	6	80
499	The Maze III †	Hard	1	6	56
508	Most Frequent Subtree Sum	Medium	1	6	38
517	Super Washing Machines	Hard	1	6	76
522	Longest Uncommon Subsequence II	Medium	1	6	82
526	Beautiful Arrangement	Medium	1	6	47
541	Reverse String II	Easy	1	6	82
554	Brick Wall	Medium	1	6	20
559	Maximum Depth of N-ary Tree	Easy	1	6	38
568	Maximum Vacation Days †	Hard	1	6	67
583	Delete Operation for Two Strings	Medium	1	6	63
590	N-ary Tree Postorder Traversal	Easy	1	6	38
598	Range Addition II	Easy	1	6	17
604	Design Compressed String Iterator †	Easy	1	6	24
606	Construct String from Binary Tree	Medium	1	6	38
629	K Inverse Pairs Array	Hard	1	6	81
630	Course Schedule III	Hard	1	6	76
634	Find the Derangement of An Array †	Medium	1	6	80
639	Decode Ways II	Hard	1	6	60
645	Set Mismatch	Easy	1	6	23

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
648	Replace Words	Medium	1	6	89
651	4 Keys Keyboard †	Medium	1	6	74
654	Maximum Binary Tree	Medium	1	6	26
660	Remove 9 †	Hard	1	6	81
661	Image Smoother	Easy	1	6	16
663	Equal Tree Partition †	Medium	1	6	41
677	Map Sum Pairs	Medium	1	6	89
679	24 Game	Hard	1	6	48
685	Redundant Connection II	Hard	1	6	55
686	Repeated String Match	Medium	1	6	82
700	Search in a Binary Search Tree	Easy	1	6	38
709	To Lower Case	Easy	1	6	82
717	1-bit and 2-bit Characters	Easy	1	6	18
727	Minimum Window Subsequence †	Hard	1	6	8
737	Sentence Similarity II †	Medium	1	6	55
748	Shortest Completing Word	Easy	1	6	82
758	Bold Words in String †	Medium	1	6	18
764	Largest Plus Sign	Medium	1	6	65
765	Couples Holding Hands	Hard	1	6	55
769	Max Chunks To Make Sorted	Medium	1	6	26
775	Global and Local Inversions	Medium	1	6	80
784	Letter Case Permutation	Medium	1	6	45
819	Most Common Word	Easy	1	6	82
820	Short Encoding of Words	Medium	1	6	89
823	Binary Trees With Factors	Medium	1	6	81
838	Push Dominoes	Medium	1	6	82
861	Score After Flipping Matrix	Medium	1	6	58
870	Advantage Shuffle	Medium	1	6	81

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
872	Leaf-Similar Trees	Easy	1	6	38
877	Stone Game	Medium	1	6	72
880	Decoded String at Index	Medium	1	6	29
890	Find and Replace Pattern	Medium	1	6	22
892	Surface Area of 3D Shapes	Easy	1	6	91
897	Increasing Order Search Tree	Easy	1	6	38
914	X of a Kind in a Deck of Cards	Easy	1	6	80
917	Reverse Only Letters	Easy	1	6	82
922	Sort Array By Parity II	Easy	1	6	6
925	Long Pressed Name	Easy	1	6	82
931	Minimum Falling Path Sum	Medium	1	6	67
941	Valid Mountain Array	Easy	1	6	18
942	DI String Match	Easy	1	6	82
951	Flip Equivalent Binary Trees	Medium	1	6	44
970	Powerful Integers	Medium	1	6	80
976	Largest Perimeter Triangle	Easy	1	6	80
988	Smallest String Starting From Leaf	Medium	1	6	41
989	Add to Array-Form of Integer	Easy	1	6	80
1005	Maximize Sum Of Array After K Negations	Easy	1	6	75
1014	Best Sightseeing Pair	Medium	1	6	65
1021	Remove Outermost Parentheses	Easy	1	6	25
1056	Confusing Number †	Easy	1	6	80
1057	Campus Bikes †	Medium	1	6	33
1114	Print in Order	Easy	1	6	92
1122	Relative Sort Array	Easy	1	6	33
1136	Parallel Courses †	Medium	1	6	54
1138	Alphabet Board Path	Medium	1	6	82
1170	Compare Strings by Frequency of the Smallest Character	Medium	1	6	82

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1171	Remove Zero Sum Consecutive Nodes from Linked List	Medium	1	6	15
1175	Prime Arrangements	Easy	1	6	80
1185	Day of the Week	Easy	1	6	80
1219	Path with Maximum Gold	Medium	1	6	49
1237	Find Positive Integer Solution for a Given Equation	Medium	1	6	10
1252	Cells with Odd Values in a Matrix	Easy	1	6	80
1260	Shift 2D Grid	Easy	1	6	58
1295	Find Numbers with Even Number of Digits	Easy	1	6	80
1297	Maximum Number of Occurrences of a Substring	Medium	1	6	21
1299	Replace Elements with Greatest Element on Right Side	Easy	1	6	18
1300	Sum of Mutated Array Closest to Target	Medium	1	6	12
1302	Deepest Leaves Sum	Medium	1	6	38
1314	Matrix Block Sum	Medium	1	6	15
1315	Sum of Nodes with Even-Valued Grandparent	Medium	1	6	38
1325	Delete Leaves With a Given Value	Medium	1	6	41
1329	Sort the Matrix Diagonally	Medium	1	6	58
1338	Reduce Array Size to The Half	Medium	1	6	33
1346	Check If N and Its Double Exist	Easy	1	6	10
1351	Count Negative Numbers in a Sorted Matrix	Easy	1	6	10
1354	Construct Target Array With Multiple Sums	Hard	1	6	32
1361	Validate Binary Tree Nodes	Medium	1	6	55
1370	Increasing Decreasing String	Easy	1	6	21
1374	Generate a String With Characters That Have Odd Counts	Easy	1	6	82

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1379	Find a Corresponding Node of a Binary Tree in a Clone of That Tree	Easy	1	6	38
1380	Lucky Numbers in a Matrix	Easy	1	6	58
1387	Sort Integers by The Power Value	Medium	1	6	60
1388	Pizza With 3n Slices	Hard	1	6	81
1392	Longest Happy Prefix	Hard	1	6	83
1403	Minimum Subsequence in Non-Increasing Order	Easy	1	6	75
1413	Minimum Value to Get Positive Step by Step Sum	Easy	1	6	15
1436	Destination City	Easy	1	6	82
1439	Find the Kth Smallest Sum of a Matrix With Sorted Rows	Hard	1	6	14
1441	Build an Array With Stack Operations	Medium	1	6	29
1447	Simplified Fractions	Medium	1	6	80
1455	Check If a Word Occurs As a Prefix of Any Word in a Sentence	Easy	1	6	15
1457	Pseudo-Palindromic Paths in a Binary Tree	Medium	1	6	41
1475	Final Prices With a Special Discount in a Shop	Easy	1	6	26
1480	Running Sum of 1d Array	Easy	1	6	15
1482	Minimum Number of Days to Make m Bouquets	Medium	1	6	13
1485	Clone Binary Tree With Random Pointer †	Medium	1	6	38
1486	XOR Operation in an Array	Easy	1	6	78
1490	Clone N-ary Tree †	Medium	1	6	38
1491	Average Salary Excluding the Minimum and Maximum Salary	Easy	1	6	18
1496	Path Crossing	Easy	1	6	82
1525	Number of Good Ways to Split a String	Medium	1	6	82
1528	Shuffle String	Easy	1	6	82

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1534	Count Good Triplets	Easy	1	6	81
1551	Minimum Operations to Make Array Equal	Medium	1	6	80
1561	Maximum Number of Coins You Can Get	Medium	1	6	75
1576	Replace All ?'s to Avoid Consecutive Repeating Characters	Easy	1	6	82
1624	Largest Substring Between Two Equal Characters	Easy	1	6	82
1626	Best Team With No Conflicts	Medium	2	6	62
1630	Arithmetic Subarrays	Medium	1	6	81
1640	Check Array Formation Through Concatenation	Easy	1	6	20
1664	Ways to Make a Fair Array	Medium	1	6	15
1673	Find the Most Competitive Subsequence	Medium	1	6	26
1675	Minimize Deviation in Array	Hard	1	6	32
1678	Goal Parser Interpretation	Easy	1	6	82
1679	Max Number of K-Sum Pairs	Medium	1	6	20
1689	Partitioning Into Minimum Number Of Deci-Binary Numbers	Medium	1	6	76
1742	Maximum Number of Balls in a Box	Easy	1	6	80
1748	Sum of Unique Elements	Easy	1	6	81
1752	Check if Array Is Sorted and Rotated	Easy	1	6	18
1753	Maximum Score From Removing Stones	Medium	1	6	80
1768	Merge Strings Alternately	Easy	1	6	6
1769	Minimum Number of Operations to Move All Balls to Each Box	Medium	1	6	15
1772	Sort Features by Popularity †	Medium	1	6	82
1775	Equal Sum Arrays With Minimum Number of Operations	Medium	1	6	76
1779	Find Nearest Point That Has the Same X or Y Coordinate	Easy	1	6	18

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1780	Check if Number is a Sum of Powers of Three	Medium	1	6	80
1790	Check if One String Swap Can Make Strings Equal	Easy	1	6	23
1792	Maximum Average Pass Ratio	Medium	1	6	32
1804	Implement Trie II (Prefix Tree) †	Medium	1	6	24
1807	Evaluate the Bracket Pairs of a String	Medium	1	6	25
1814	Count Nice Pairs in an Array	Medium	1	6	80
1827	Minimum Operations to Make the Array Increasing	Easy	1	6	76
1869	Longer Contiguous Segments of Ones than Zeros	Easy	1	6	82
1909	Remove One Element to Make the Array Strictly Increasing	Easy	1	6	18
1925	Count Square Sum Triples	Easy	1	6	63
1926	Nearest Exit from Entrance in Maze	Medium	1	6	52
1968	Array With Elements Not Equal to Average of Neighbors	Medium	1	6	75
1980	Find Unique Binary String	Medium	1	6	45
2023	Number of Pairs of Strings With Concatenation Equal to Target	Medium	1	6	21
2024	Maximize the Confusion of an Exam	Medium	1	6	8
2042	Check if Numbers Are Ascending in a Sentence	Easy	1	6	82
2047	Number of Valid Words in a Sentence	Easy	1	6	82
2078	Two Furthest Houses With Different Colors	Easy	1	6	76
2116	Check if a Parentheses String Can Be Valid	Medium	1	6	25
2134	Minimum Swaps to Group All 1's Together II	Medium	1	6	7
2166	Design Bitset	Medium	1	6	88
2259	Remove Digit From Number to Maximize Result	Easy	1	6	76

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
86	Partition List	Medium	1	4	34
87	Scramble String	Hard	1	4	68
170	Two Sum III - Data structure design †	Easy	1	4	20
201	Bitwise AND of Numbers Range	Medium	1	4	78
275	H-Index II	Medium	1	4	10
324	Wiggle Sort II	Medium	1	4	86
372	Super Pow	Medium	1	4	80
404	Sum of Left Leaves	Easy	1	4	38
457	Circular Array Loop	Medium	1	4	6
495	Teemo Attacking	Easy	1	4	18
507	Perfect Number	Easy	1	4	80
513	Find Bottom Left Tree Value	Medium	1	4	40
546	Remove Boxes	Hard	1	4	68
682	Baseball Game	Easy	1	4	29
779	K-th Symbol in Grammar	Medium	1	4	80
804	Unique Morse Code Words	Easy	1	4	82
848	Shifting Letters	Medium	1	4	15
885	Spiral Matrix III	Medium	1	4	58
950	Reveal Cards In Increasing Order	Medium	1	4	24
999	Available Captures for Rook	Easy	1	4	58
1062	Longest Repeating Substring †	Medium	1	4	84
1103	Distribute Candies to People	Easy	1	4	80
1182	Shortest Distance to Target Color †	Medium	1	4	12
1189	Maximum Number of Balloons	Easy	1	4	21
1198	Find Smallest Common Element in All Rows †	Medium	1	4	81
1228	Missing Number In Arithmetic Progression †	Easy	1	4	80
1286	Iterator for Combination	Medium	1	4	88

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1385	Find the Distance Value Between Two Arrays	Easy	1	4	6
1401	Circle and Rectangle Overlapping	Medium	1	4	91
1556	Thousand Separator	Easy	1	4	82
1582	Special Positions in a Binary Matrix	Easy	1	4	58
1663	Smallest String With A Given Numeric Value	Medium	1	4	76
1672	Richest Customer Wealth	Easy	1	4	58
1688	Count of Matches in Tournament	Easy	1	4	80
1733	Minimum Number of People to Teach	Medium	1	4	75
1758	Minimum Changes To Make Alternating Binary String	Easy	1	4	82
1805	Number of Different Integers in a String	Easy	1	4	82
1863	Sum of All Subset XOR Totals	Easy	1	4	45
1886	Determine Whether Matrix Can Be Obtained By Rotation	Easy	1	4	58
2073	Time Needed to Buy Tickets	Easy	1	4	24
293	Flip Game †	Easy	1	3	82
294	Flip Game II †	Medium	1	3	72
320	Generalized Abbreviation †	Medium	1	3	45
331	Verify Preorder Serialization of a Binary Tree	Medium	1	3	44
369	Plus One Linked List †	Medium	1	3	34
401	Binary Watch	Easy	1	3	45
411	Minimum Unique Word Abbreviation †	Hard	1	3	82
469	Convex Polygon †	Medium	1	3	91
481	Magical String	Medium	1	3	82
482	License Key Formatting	Easy	1	3	82
506	Relative Ranks	Easy	1	3	32
521	Longest Uncommon Subsequence I	Easy	1	3	82
533	Lonely Pixel II †	Medium	1	3	91

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
537	Complex Number Multiplication	Medium	1	3	80
553	Optimal Division	Medium	1	3	74
555	Split Concatenated Strings †	Medium	1	3	76
587	Erect the Fence	Hard	1	3	91
625	Minimum Factorization †	Medium	1	3	76
644	Maximum Average Subarray II †	Hard	1	3	13
656	Coin Path †	Hard	1	3	65
667	Beautiful Arrangement II	Medium	1	3	80
672	Bulb Switcher II	Medium	1	3	78
750	Number Of Corner Rectangles †	Medium	1	3	91
760	Find Anagram Mappings †	Easy	1	3	21
762	Prime Number of Set Bits in Binary Representation	Easy	1	3	78
789	Escape The Ghosts	Medium	1	3	80
800	Similar RGB Color †	Easy	1	3	80
812	Largest Triangle Area	Easy	1	3	81
816	Ambiguous Coordinates	Medium	1	3	82
893	Groups of Special-Equivalent Strings	Medium	1	3	82
933	Number of Recent Calls	Easy	1	3	24
944	Delete Columns to Make Sorted	Easy	1	3	62
1020	Number of Enclaves	Medium	1	3	49
1033	Moving Stones Until Consecutive	Medium	1	3	80
1037	Valid Boomerang	Easy	1	3	91
1065	Index Pairs of a String †	Easy	1	3	89
1078	Occurrences After Bigram	Easy	1	3	21
1085	Sum of Digits in the Minimum Number †	Easy	1	3	80
1090	Largest Values From Labels	Medium	1	3	75
1118	Number of Days in a Month †	Easy	1	3	80
1133	Largest Unique Number †	Easy	1	3	81

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1134	Armstrong Number †	Easy	1	3	80
1165	Single-Row Keyboard †	Easy	1	3	82
1176	Diet Plan Performance †	Easy	1	3	7
1184	Distance Between Bus Stops	Easy	1	3	18
1261	Find Elements in a Contaminated Binary Tree	Medium	1	3	38
1273	Delete Tree Nodes †	Medium	1	3	41
1324	Print Words Vertically	Medium	1	3	82
1337	The K Weakest Rows in a Matrix	Easy	1	3	10
1357	Apply Discount Every n Orders	Medium	1	3	88
1362	Closest Divisors	Medium	1	3	80
1371	Find the Longest Substring Containing Vowels in Even Counts	Medium	1	3	15
1394	Find Lucky Integer in an Array	Easy	1	3	81
1409	Queries on a Permutation With Key	Medium	1	3	81
1415	The k-th Lexicographical String of All Happy Strings of Length n	Medium	1	3	45
1422	Maximum Score After Splitting a String	Easy	1	3	15
1452	People Whose List of Favorite Companies Is Not a Subset of Another List	Medium	1	3	82
1456	Maximum Number of Vowels in a Substring of Given Length	Medium	1	3	7
1461	Check If a String Contains All Binary Codes of Size K	Medium	1	3	84
1469	Find All The Lonely Nodes †	Easy	1	3	38
1471	The k Strongest Values in an Array	Medium	1	3	5
1474	Delete N Nodes After M Nodes of a Linked List †	Easy	1	3	37
1502	Can Make Arithmetic Progression From Sequence	Easy	1	3	18
1508	Range Sum of Sorted Subarray Sums	Medium	1	3	14

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1558	Minimum Numbers of Function Calls to Make Target Array	Medium	1	3	76
1608	Special Array With X Elements Greater Than or Equal X	Easy	1	3	10
1609	Even Odd Tree	Medium	1	3	40
1619	Mean of Array After Removing Some Elements	Easy	1	3	18
1637	Widest Vertical Area Between Two Points Containing No Points	Easy	1	3	75
1708	Largest Subarray Length K †	Easy	1	3	76
1717	Maximum Score From Removing Substrings	Medium	1	3	26
1726	Tuple with Same Product	Medium	1	3	81
1736	Latest Time by Replacing Hidden Digits	Easy	1	3	76
1756	Design Most Recently Used Queue †	Medium	1	3	24
1764	Form Array by Concatenating Subarrays of Another Array	Medium	1	3	82
1781	Sum of Beauty of All Substrings	Medium	1	3	21
1791	Find Center of Star Graph	Easy	1	3	54
1806	Minimum Number of Operations to Reinitialize a Permutation	Medium	1	3	58
1813	Sentence Similarity III	Medium	1	3	82
1816	Truncate Sentence	Easy	1	3	82
1826	Faulty Sensor †	Easy	1	3	6
1828	Queries on Number of Points Inside a Circle	Medium	1	3	91
1860	Incremental Memory Leak	Medium	1	3	80
1874	Minimize Product Sum of Two Arrays †	Medium	1	3	76
1893	Check if All the Integers in a Range Are Covered	Easy	1	3	17
1945	Sum of Digits of String After Convert	Easy	1	3	82
1952	Three Divisors	Easy	1	3	80

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1966	Binary Searchable Numbers in an Unsorted Array †	Medium	1	3	26
1981	Minimize the Difference Between Target and Chosen Elements	Medium	1	3	64
2006	Count Number of Pairs With Absolute Difference K	Easy	1	3	81
2012	Sum of Beauty in the Array	Medium	1	3	18
2022	Convert 1D Array Into 2D Array	Easy	1	3	58
2053	Kth Distinct String in an Array	Easy	1	3	21
2057	Smallest Index With Equal Value	Easy	1	3	18
2075	Decode the Slanted Ciphertext	Medium	1	3	82
2079	Watering Plants	Medium	1	3	58
2109	Adding Spaces to a String	Medium	1	3	82
2114	Maximum Number of Words Found in Sentences	Easy	1	3	82
2124	Check if All A's Appears Before All B's	Easy	1	3	82
2126	Destroying Asteroids	Medium	1	3	75
2149	Rearrange Array Elements by Sign	Medium	1	3	5
2150	Find All Lonely Numbers in the Array	Medium	1	3	81
2155	All Divisions With the Highest Score of a Binary Array	Medium	1	3	18
2160	Minimum Sum of Four Digit Number After Splitting Digits	Easy	1	3	75
2161	Partition Array According to Given Pivot	Medium	1	3	81
2181	Merge Nodes in Between Zeros	Medium	1	3	34
2185	Counting Words With a Given Prefix	Easy	1	3	83
2190	Most Frequent Number Following Key In an Array	Easy	1	3	81
2200	Find All K-Distant Indices in an Array	Easy	1	3	6
2206	Divide Array Into Equal Pairs	Easy	1	3	81
2208	Minimum Operations to Halve Array Sum	Medium	1	3	30

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
2210	Count Hills and Valleys in an Array	Easy	1	3	18
2215	Find the Difference of Two Arrays	Easy	1	3	23
2244	Minimum Rounds to Complete All Tasks	Medium	1	3	81
2255	Count Prefixes of a Given String	Easy	1	3	82
2256	Minimum Average Difference	Medium	1	3	15
2260	Minimum Consecutive Cards to Pick Up	Medium	1	3	23
385	Mini Parser	Medium	1	2	25
439	Ternary Expression Parser †	Medium	1	2	25
479	Largest Palindrome Product	Hard	1	2	81
573	Squirrel Simulation †	Medium	1	2	81
624	Maximum Distance in Arrays	Medium	1	2	76
693	Binary Number with Alternating Bits	Easy	1	2	78
908	Smallest Range I	Easy	1	2	80
1030	Matrix Cells in Distance Order	Easy	1	2	91
1343	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Medium	1	2	7
1410	HTML Entity Parser	Medium	1	2	82
1427	Perform String Shifts †	Easy	1	2	81
1754	Largest Merge Of Two Strings	Medium	1	2	82
1784	Check if Binary String Has at Most One Segment of Ones	Easy	1	2	82
1785	Minimum Elements to Add to Form a Given Sum	Medium	1	2	76
1797	Design Authentication Manager	Medium	1	2	24
1829	Maximum XOR for Each Query	Medium	1	2	15
1845	Seat Reservation Manager	Medium	1	2	88
1876	Substrings of Size Three with Distinct Characters	Easy	1	2	7
1935	Maximum Number of Words You Can Type	Easy	1	2	82
1961	Check If String Is a Prefix of Array	Easy	1	2	15

#	PROBLEM	LEVEL	ASKED	SCORE	CHAPTER
1967	Number of Strings That Appear as Sub-strings in Word	Easy	1	2	82
2032	Two Out of Three	Easy	1	2	23
2043	Simple Bank System	Medium	1	2	88
2068	Check Whether Two Strings are Almost Equivalent	Easy	1	2	21
2080	Range Frequency Queries	Medium	1	2	10
2085	Count Common Words With One Occurrence	Easy	1	2	21
2154	Keep Multiplying Found Values by Two	Easy	1	2	81
2243	Calculate Digit Sum of a String	Easy	1	2	82
2248	Intersection of Multiple Arrays	Easy	1	2	81